



アンチパターンで学ぶ データ分析

作家の判別を題材に

久保山 哲二

学習院大学

Version 1.2.1

まえがき

本書は、著者が15年あまりさまざまな授業で使ってきたデータ分析の教材を1冊のテキストにまとめ直したものです。はじめは全学に向けたデータサイエンス科目(選択)のための教材でした。専攻を問わず受講する授業だったので、人文系の受講者にも自分のこととして興味を持ってもらえる題材をと考えて作りました。その後は他大学の集中講義など理系向けの授業でも使い、数理的な内容を足して育ててきました。入門の語り口のまま、ところどころ発展的な内容へ深入りするのは、二つの流れが合わさっているからです。

もともとなった教材は、授業ごとに独立したスライドと実習用の Jupyter Notebook でした(本書に付属するノートブックは marimo で作り直してあり、ブラウザで図や計算の条件を操作できます。使い方は付録 A を参照)。学期が変わるたびに手を入れ、受講者がつまづいた場所に合わせて例を差し替えてきました。それを繰り返すうちに、ばらばらの断片のあいだにいつも同じ筋が通っていることに気づきました。分析は数えるところから始まって、出てきた数字を疑うところで終わります。コロナ禍の時期を中心に書き溜めた大量の教材も、今回まとめて見直しました。生成 AI の助けも借りて、これまでに作成した教材全体を見直し、内容の重複を整理して、1冊の教科書として読める順序にまとめました。

本書のアンチパターンと定石の多くは、著者自身がさまざまな応用データの分析で学んだものです。SNS のデータ、Web 小説のデータ、遺伝子のデータ、公共事業入札のデータ、公文書の移管・廃棄のデータ、結晶の表面構造と安定性のデータ。性質のまるで違うデータを扱ってきたなかで型として身についたことを書きました。

もう一つの源は、授業で受講者の分析につきあってきた経験です。演習や卒業研究の相談を受けていると、同じつまづきが何度も現れます。しかもその多くは、手順の説明からはこぼれ落ちてしまう、ごく初歩的な取り違えです。本書にときどき細かすぎるように見える注意が入っているのは、そのためです。

まとめるにあたって題材を一つに固定しました。文章の特徴から作家を判別するという問題です。作家の判別という題材自体、以前から授業で使ってきたものです。読点の前にどの平仮名が来るか、つまり「～て、」で切るか「～が、」で切るかといった癖を数えるだけで、著者ごとの塊がきれいに分かります。何を書いているかは一切見ていないのに、です。受講者がまず驚くのは、ここです。

もう一つは、その裏返しです。同じデータでも、数える対象を変えれば結論が変わります。本書の第9章では、内容語まで拾う特徴量に変えたとたん、モデルが文体ではなく物語の固有名詞を覚えてしまう様子を実際に見せます。**何を数えるかを決めた時点で、結論はほとんど決まっている。**この教訓を体験してもらえることが、題材としての持ち味でした。どの章も同じデータを使うので、第3章の前処理の判断が第8章の評価をどう変えるかを1冊の中で追いかけられます。一つの分析が始まって終わるまでの判断の連なりとして読んでもらえるように書きました。

分量は多めです。授業で使うときに聞き手に合わせて取捨選択できるだけの厚みを持たせたため

で、順にすべて読む必要はありません。灰色の枠の「補足」と、濃紺の左罫の「発展」は、飛ばしても本筋がつながるように書いてあります。逆に、独学で読む人は気になった箱にこそ寄り道してください。著者自身、この教材を通じて初めて腑に落ちた事柄が(組成データの扱いをはじめ)いくつもあります。

いまは生成 AI がコードを書き、分析を実行し、説明までしてくれる時代です。それでも本書を書いたのは、分析の実行を任せられるようになった一方で、出てきた数字を疑い引き受ける仕事は人の側に残っているからです。AI は、適切な文脈と検査の観点を与えれば、見せかけの相関にも評価の漏洩にも気づいてくれます。ただ、何を文脈として渡し、どの観点で検査させるかを決めるのは使い手です。合計が 1 に固定されたデータでは相関が歪むことを知らなければ、それを確認するよう指示は出せませんし、差し出された「興味深い発見」に追従の匂いを嗅ぎ取ることもできません(いずれも本書の後半で実演します)。本書が扱うのは、そうした判断のほうです。何を問うかを決め、どの前処理が結論を左右するかを見分け、出てきた数字のどこを疑うかを見当づける仕事は、実行の手間が消えていくほど、かえって人の側に濃く残ります。本書は、その検分の目を作るために書きました。

この教材を長年鍛えてくれた、教室で出会った受講者の皆さんに感謝します。

2026 年 9 月

久保山 哲二

目次

まえがき	i
第 1 章 はじめに	1
1.1 この本が答えようとする問い	1
1.2 一つの題材を最後まで追う	1
1.3 アンチパターンから学ぶ	2
1.4 必要な準備	3
1.5 行と列の呼び方	3
1.6 データについて	4
1.7 各章の内容	6
第 2 章 テキストを数える	8
2.1 文章を数値にすること	8
2.2 Zipf の法則	8
2.3 頻度の偏りが、使える語と使えない語を分ける	11
2.4 機能語はどの区間にもむらなく現れる	12
2.5 日本語には語の区切りがない	17
2.6 章末の課題	19
第 3 章 特徴を設計する	20
3.1 何を数えるかを定める	20
3.2 前処理 — 情報の出どころで捨てるものを定める	20
3.3 特徴設計 1 — 文字種の比率	21
3.4 単体 — 3 つの比率が住む場所	24
3.5 特徴設計 2 — 文の長さ	28
3.6 特徴設計 3 — 読点の打ち方	32
3.7 組成データ — 比率を扱うときの落とし穴	35
3.8 できあがった表	43
3.9 章末の課題	44
第 4 章 距離を測る	45
4.1 「似ている」を数式にする	45
4.2 ミンコフスキー距離	45
4.3 コサイン距離 — 向きだけを見る	47

4.4	標準化 — 文長の 2 列が距離のほとんどを占めてしまう	49
4.5	距離行列を眺める	56
4.6	同著者と異著者を最もよく分けたのはコサイン距離	57
4.7	発展 — 距離の数学的性質	59
4.8	相関係数は偏差ベクトルのコサインである	61
4.9	相関は「伝わらない」	64
4.10	高次元では距離が壊れる	70
4.11	章末の課題	74
第 5 章	まとまりを見つける	76
5.1	正解を見せずに分ける	76
5.2	階層クラスタリング	76
5.3	クラスタ間距離の 4 つの定義	78
5.4	切る高さはデンドログラムだけでは決まらない	80
5.5	答え合わせ — 2 種類の評価	80
5.6	距離と併合方法の相性	83
5.7	k-means	88
5.8	階層クラスタリングと k-means の使い分け	91
5.9	章末の課題	92
第 6 章	次元を減らす	93
6.1	見えないものを見る	93
6.2	主成分分析 — いちばん見晴らしのよい方向から眺める	93
6.3	バイプロット — 点と変数を同時に描く	101
6.4	多次元尺度構成法 (MDS)	102
6.5	非線形の次元削減 — 近傍のつながりを保つ	103
6.6	手法は 1 つに絞らず、並べて描く	121
6.7	章末の課題	122
第 7 章	著者を当てる	123
7.1	正解を与えて学ばせる	123
7.2	まず当てずっぽうの成績を知る	123
7.3	k 近傍法 — 最も単純な分類器	124
7.4	線形モデル — 1 本の境界で分ける	124
7.5	決定木 — 質問を繰り返す	130
7.6	ランダムフォレスト — 木をたくさん作る	131
7.7	勾配ブースティング — 間違いを順に埋める	132
7.8	訓練データで測った正解率は当てにならない	138
7.9	分割のときの注意 — 層別化	140
7.10	モデルを比べる	140

7.11	判別を支えている特徴を突き止める	144
7.12	同じ長編の巻が訓練とテストに分かれている	148
7.13	章末の課題	148
第 8 章	正しく測る	150
8.1	1 回の分割では足りない	150
8.2	交差検証	151
8.3	正解率だけでは足りない	152
8.4	指標は「どの誤りが高くつくか」で選ぶ	163
8.5	しきい値を動かす — ROC 曲線と AUC	167
8.6	較正 — 出力された確率が当たっているかを測る	170
8.7	過学習を目で見る	173
8.8	学習曲線 — 作品を増やせばまだ伸びる	174
8.9	評価の手順をまとめる	175
8.10	章末の課題	175
第 9 章	罨を避ける	176
9.1	出来すぎた結果を疑う	176
9.2	データ漏洩 — 本番では手に入らない情報が訓練に混ざる	176
9.3	前処理の漏れ	177
9.4	特徴選択の漏れ	180
9.5	グループの漏れ	182
9.6	特徴の作り方が漏れを招く — n-gram の罨	185
9.7	ハイパーパラメータ選択の漏れ	187
9.8	多重テストの漏れ	189
9.9	そのほかの漏れ — ターゲットと時系列	191
9.10	訓練で見た範囲の外に出られないモデルがある	192
9.11	COVID-19 診断モデルの教訓	194
9.12	誤差を 3 つに分ける — 偏りとばらつき	195
9.13	次元の呪い	198
9.14	生成 AI に分析を手伝わせるときの罨	200
9.15	手順のまとめ	201
9.16	章末の課題	202
第 10 章	学習された表現と比べる	203
10.1	手作りの特徴を埋め込みに置き換える	203
10.2	入力を変えずにモデルだけ深層学習に取り替える	209
10.3	手作りの特徴が残った理由	221
第 11 章	ジャンルを分ける	222

11.1	問いを変えてみる	222
11.2	まず眺める	222
11.3	分類してみる	223
11.4	喜ぶ前に	223
11.5	著者判別に戻ると	226
11.6	候補の外にいる著者を棄却する	227
11.7	2次元で眺める	228
11.8	当てているものと、当てたいものは違うことがある	229
11.9	この本のまとめ	229
11.10	章末の課題	230
付録 A	marimo 教材の使い方	231
A.1	教材の構成	231
A.2	自分のパソコンで実行する	231
A.3	操作するときの注意	232
A.4	困ったときは	232
付録 B	高次元空間の幾何	233
B.1	球の体積は消える	233
B.2	体積は薄い殻に集中する	234
B.3	距離の集中	235
B.4	高次元ではベクトルがほぼ直交する	237
B.5	呪いと祝福	238
B.6	実務での構え方	239
B.7	章末の課題	239
付録 C	不偏性をめぐる補足	240
C.1	二乗誤差を最小にする割る数は $n + 1$ である	240
C.2	標準偏差を不偏にする補正	240
参考文献		242
参考文献		242
アンチパターンと定石の一覧		247

第 1 章

はじめに

1.1 この本が答えようとする問い

誰が書いたのか分からない文章の書き手を、その書きぶりから推し量る。こうした試みは古くからある。『源氏物語』の最後の十帖は、それまでと文章の調子が違うと言われ、紫式部が書いたのかどうか長く議論されてきた。

決着がついた例もある。**ザ・フェデラリスト**は、1787 年から 1788 年にかけてアメリカ合衆国憲法の批准を訴えるために書かれた 85 編の論文で、3 人が匿名で分担執筆した。そのうち 12 編は、ハミルトンが書いたのかマディソンが書いたのかが分からないまま 150 年以上残っていた。1964 年、モステラーとウォレスがこれに統計で答えを出す [1]。彼らが数えたのは主張の中身ではなく、by、to、upon といった、文の骨組みを作るだけで意味をほとんど持たない単語の使用頻度だった。2 人はそうした語の使い方に一貫した癖を持っていて、12 編はすべてマディソンの側に寄った。この仕事が、文体を数えて調べる分野である**計量文体論** (stylometry) の出発点になった。

意味を運ばない語にこそ書き手の癖が出るという発見は、本書の第 2 章と第 3 章でそのまま使う。

本書ではこの問いを、8 名の近代文学作家、74 作品を使って追いかける。夏目漱石、泉鏡花、島崎藤村、太宰治、夢野久作、谷崎潤一郎、柳田国男、吉川英治。すべて青空文庫から入手できるものである。

最後の第 11 章では、ここにライトノベル 60 区間を加えて、問いそのものを立て直す。

本書でやる作業は、次の二つからなる。

1. 作品ごとの文体を数値の並びにする
2. その数値から、作品をまとめる/著者を当てる

並べてしまえば単純だが、この二つの間に、データ分析でつまづくことのほとんどが詰まっている。まえがきでも述べたとおり、何を数えるかを決めた段階で結論はほぼ決まってしまうし、出た数値が正しく測られたものかどうか、測り方しだいで簡単に狂う。本書はその両方を、実際に狂わせて見せながら進む。

1.2 一つの題材を最後まで追う

データ分析の入門書は、章ごとに別のデータを使うことが多い。アヤメの花で分類を学び、住宅価格で回帰を学び、手書き数字でニューラルネットワークを学ぶ。手法を一つずつ独立に見せるには、この作りが分かりやすい。

本書は逆に、題材を一つに固定した。最初から最後まで同じ 74 作品を使うので、**前の章での判断が、後の章の結果を左右する**ようすをそのまま追いかけられる。

特徴の作り方を換えれば距離が変わる。距離が変われば、クラスタも変わり、分類の精度も変わる。評価の設計を誤れば、それまでの手間がすべて無意味な数字になる。第3章で作った特徴量が、第9章で罣にはまる原因になる。

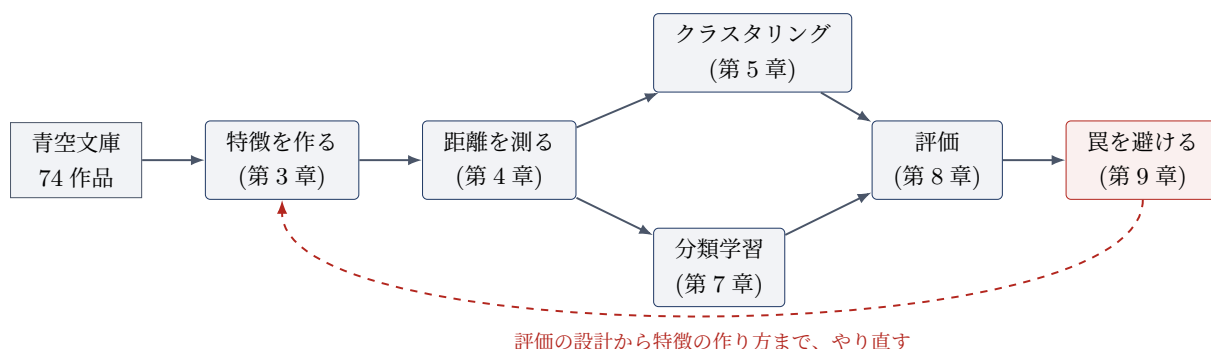


図 1.1 本書の流れ。最後に前へ戻ることになる。

図 1.1 の点線が本書の主張である。分析は一度では終わらない。第9章で評価の甘さに気づけば、特徴の作り方まで戻ることになる。

1.3 アンチパターンから学ぶ

本書は、ありがちな間違いを実際に実行し、**どれだけ数字が変わるかを測ってから正しいやり方**を示す、という順序で書いてある。数字が動くところを一度見ておけば、後で似た場面に出会ったときに思い出しやすいだろうと考えた。もちろん、学び方はこれ一つではない。

間違いは赤い箱 (**アンチパターン**) に、それに対する確立した正しい手筋は緑の箱 (**定石**) にまとめてあり、巻末に全箱の一覧を付けた (一覧では × と ○ で区別する)。分析の途中で不安になったら、そこから逆引きできる。

ただし、緑の箱も**規則ではない**。定石は「多くの場合にうまくいくと確かめられてきた方法」であり、本書の定石も、本書のデータと本書が想定した目的の上で成り立っている。手元のデータがその条件を満たしているかを確かめるのは読む側の仕事で、定石どおりにやったのに結果がおかしいときは、自分の操作より先に、その定石の前提を疑うのが早い。

箱はもう2種類ある。「補足」の灰色の枠は本筋から少し外れる話、濃紺の左罫を付けた「発展」の欄はやや進んだ内容である。見出しの末尾に * が付いた節も同じ扱いで、どれも初読では飛ばして構わない。灰色の「手を動かす」の箱は章末の課題で、一部にはヒントを付けた。

補足 赤い箱だけを拾い読みしても身につかない

間違いが害になる理由は、その手前の議論の中にある。箱の結論だけ覚えても、少し形の違う罣は避けられない。

たとえば第9章では、次のことをやってみせる。著者と何の関係もない**乱数の列**を2000本用意し、その中から著者とよく対応する30本を選び、その30本だけを使って74作品の著者を層別5分割交差検証で当てる。元が乱数なのだから、当たるはずがない。ところが、この手順で測った正解率は

0.57 になる。どの作品にも吉川英治だと答え続けるだけの戦略の正解率が 0.284 なので、その 2 倍である。

からくりは、30 本を選ぶ操作とデータを分ける操作の**順番**にある。順番を正しく入れ替えて同じ乱数で測り直すと、正解率は 0.16 まで下がる。分析の中身は 1 文字も変えていない。順番を入れ替えただけで正解率が 0.41 動いたことになる。0.57 という数字を自分の目で見ておくと、同じ間違いはしにくくなる。

1.4 必要な準備

数学は高校までの内容を前提にする。平均、分散、対数、簡単な確率。行列やベクトルは使う前に説明する。

プログラムは Python を使うが、読めれば十分である。付属の marimo ノートブックでは、条件を変えながら図と計算結果を比較できる。使い方は付録 A にまとめた。

1.5 行と列の呼び方

本書の作業は、最後には 1 枚の表になる。行が作品、列が測った量、マスがその値である。機械学習も統計も、たいていは同じ形の表から始まる。

ところが、表の各部分の呼び方が分野ごとに違う。同じものを指しているのに用語が食い違くと、それだけで読めなくなるので、先に整理しておく。

1 列=1 つの測り方。本書では**特徴量**

著者	題名	平仮名	漢字	文長 μ
太宰治	斜陽	0.731	0.247	3.17
夏目漱石	こころ	0.701	0.297	3.34
吉川英治	三国志 (桃園)	0.673	0.325	3.25

1 行=1 件のデータ
本書では**作品**

当てたい列。本書では**正解ラベル**

図 1.2 表の呼び方。同じ場所が分野ごとに違う名前と呼ばれる。

呼び方の対応を表 1.1 にまとめる。横に並んだものは、どれも同じ場所を指している。

とくに**説明変数**と**目的変数**は対で使う語である。「説明変数で目的変数を説明する」という言い方をするので、片方だけを別の分野の語に置き換えると文がねじれる。「特徴量で目的変数を説明する」は落ち着かない。本書では「特徴量」と「正解ラベル」を対にして使う。

第 4 章からは、表を**幾何の対象**としても見る。1 行に並んだ 52 個の数を、そのまま**座標**として読む。学校で習う平面の点 (x, y) は 2 つの数の組だったが、数が 52 個あれば 52 次元空間の 1 点になる、と考える。絵には描けないが、数の扱い方は同じである。

こう読むと、74 行 $\times$ 52 列の表は、52 次元空間に散らばった 74 個の点になる。2 つの作品が似ているかどうかは、2 つの点が近いかどうかの問題に翻訳される。表として見るか点の集まりとして見るかは、同じものへの 2 通りの視線であって、別のデータになるわけではない。

表 1.1 分野による呼び方の違い

	1 行 (1 件のデータ)	入力に使う列	当てたい列
本書	作品 (第 11 章では区間)	特徴量	正解ラベル
統計学	観測・ケース	説明変数・独立変数	目的変数・従属変数・被説明変数
機械学習	サンプル・インスタンス・事例	特徴量・素性 (そせい)	ラベル・ターゲット・教師信号
データベース	レコード	フィールド・属性	—
幾何として見るとき	点	次元・座標軸	—

補足 「サンプル」まわりの語は取り違えやすい

表 1.1 に「サンプル」が出てくるが、この語の周辺には紛らわしいものが集まっている。統計学で**標本** (sample) といえば、母集団から取り出した**データの集まり全体**を指す。そこから 2 つの語が出る。

- **サンプルサイズ** (標本の大きさ) — その標本に入っているデータの件数。よく n と書く。本書なら $n = 74$
- **標本数** — 標本そのものの個数。調査を 1 回しか行っていないければ 1 である

「サンプル数 100」と書かれた文章をよく見かけますが、統計の語法では標本数は普通 1 なので、これはたいてい**サンプルサイズが 100**の意味である。

ややこしいことに、機械学習の実務では**1 件のデータ**を「サンプル」と呼ぶ (scikit-learn の `n_samples` は行数である)。機械学習の現場では「サンプル数=行数」で通じてしまう。

同じ語が「全体」と「1 件」の両方を指すので、議論の途中でどちらの意味が分からなくなる。本書では 1 件を指して「サンプル」とは呼ばず、行数のことは**作品数**または**データの件数**と書く。他人の文章を読むときは、その人がどちらの意味で使っているかを先に確かめておくとう安全である。

1.6 データについて

使う 74 作品の内訳を表 1.2 に示す。

表 1.2 をよく見てほしい。2 つの点で偏っている。

1 つは**作品数の開き**である。柳田国男の 5 作品に対し、吉川英治は 21 作品と、4 倍以上ある。このとき、何も考えずにどの作品にも「吉川英治」と答え続けるだけで、74 作品に対する正解率が 0.284 になる。3 割近くが当たってしまう。モデルの成績を見るときは、この基準線 0.284 を超えているかどうかを毎回確かめる必要がある (第 7 章)。少数の著者を丸ごと取りこぼしていても、全体の正解率にはほとんど響かないという問題もある (第 8 章)。

もう 1 つは**作品どうしの独立性**である。吉川英治の 21 作品のうち 18 は、『三国志』と『宮本武蔵』という 2 つの長編を巻ごとに分けたものである。同じ長編の巻どうしは、当然よく似ている。訓練データとテストデータに 1 巻ずつ分かれて入れば、モデルは「文体」ではなく「その長編」を覚える

表 1.2 コーパスの内訳

著者	作品数	主な作品
吉川英治	21	『三国志』全 11 巻、『宮本武蔵』全 7 巻
夏目漱石	10	『こころ』『三四郎』『それから』『道草』
島崎藤村	9	『夜明け前』『破戒』『新生』『嵐』
泉鏡花	9	『高野聖』『紅玉』『化銀杏』『陽炎座』
太宰治	7	『人間失格』『斜陽』『津軽』『走れメロス』
夢野久作	7	『ドグラ・マグラ』『少女地獄』
谷崎潤一郎	6	『細雪』『陰翳礼讃』『吉野葛』『盲目物語』
柳田国男	5	『山の人生』『雪国の春』『木綿以前の事』『海上の道』
合計	74	

だけで正解できてしまう。成績は水増しされる (第 9 章)。

扱いにくいデータである。だが本書ではあえてこのまま使う。きれいに整ったデータでは、第 8 章と第 9 章で扱う問題が起きないからである。現実のデータはたいてい似た形をしている。

補足 このコーパスは二重の選抜を通った後のものである

偏りは表の中だけにあるのではない。そもそも表に載る前の段階で、2 回の選抜が起きている。

1 回目は**著作権**である。青空文庫に収録されるのは保護期間の切れた作品なので、自動的に古い時代のものに限られる。本書の 8 名がいずれも 1960 年代までに没した作家なのは、その時代を選んだからではなく、選べる範囲がそこしかなかったからである。

2 回目は**電子化**である。青空文庫の入力と校正は有志の手で行われている。読まれ続けている作品ほど入力されやすいので、本書の 74 作品も**数十年のあいだ読み継がれてきた側**に寄っている。

したがって本書のモデルが学ぶのは、「近代の日本語作家の文体」ではなく、「読み継がれ、電子化された作家の文体」である。当時の平均的な文章がどうだったかは、このデータからは分からない。残らなかった書き手のほうが、はるかに多いからである。

このように、**選抜を通ったものだけを見て全体を語る誤りを生存者バイアス**という。第二次大戦中、帰還した爆撃機の被弾箇所を調べてそこを補強しようとした話が有名である。本当に補強すべきなのは、**被弾すると帰還できなくなる箇所**、つまり帰ってきた機体に穴のない場所だった。

手元のデータを見るとき、「ここに来られなかったものは何か」を一度考えてみるとよい。

1.6.1 もう一つのデータ

第 11 章では、まったく別のデータを足す。Web 小説投稿サイト「小説家になろう」で連載された 6 シリーズ、60 区間である。

連載小説には青空文庫の「作品」に相当する切れ目がない。そこで各シリーズを、**章の区切りを尊重しながら 8 万～16 万字ずつに区切ったもの**を 1 単位とした。これを本書では**区間**と呼ぶ。区切り目は必ず章か話の境界に置く。青空文庫の作品はルビと注記を除いて約 1 万～58 万字 (中央値 12 万字) なので、1 区間はほぼ 1 作品ぶんの分量になる。

表 1.3 ライトノベルの内訳 (第 11 章で使う)

著者	シリーズ	区間数	元の話数
香月美夜	『本好きの下剋上』	10	186
日向夏	『薬屋のひとりごと』	10	344
長月達平	『Re: ゼロから始める異世界生活』	10	148
理不尽な孫の手	『無職転生』	10	129
佐賀崎しげる	『片田舎のおっさん、剣聖になる』	10	295
依空まつり	『サイレント・ウィッチ』	10	216
合計		60	1318

表 1.2 と見比べてほしい。点の数だけを見れば近代文学と大差ない。だが**構造がまったく違う**。

ライトノベルは、著者一人につきシリーズ一つである。「10 区間」といっても、独立した 10 作品ではなく、一つの長編を 10 個に切ったものである。

構造の違いが何をもたらすかは、第 9 章と第 11 章で見る。

補足 配布できるデータとできないデータ

近代文学は青空文庫にあり、著作権が切れているのでテキストごと配布できる。

ライトノベルは著作権が生きているので、テキストは配れない。配れるのは第 3 章で作る**特徴量** (feature)、つまり 52 個の数値だけである。

これは制約であると同時に、実務でよくある状況でもある。個人情報を含むデータを共有できないとき、集約統計量や特徴量だけを渡す。第 11 章で改めて扱う。

1.7 各章の内容

第 2 章 テキストを数える

単語の出現頻度に現れる規則。Zipf の法則とポアソン分布。文体を測るのに使える量とそうでない量の見分け方。

第 3 章 特徴を設計する

文字種の比率、文の長さの分布、読点の打ち方。書き手が意識していない癖を数値にする。合計が 1 に固定された比率 (組成データ) の扱いにも踏み込む。

第 4 章 距離を測る

「似ている」を数式にする。ユークリッド距離、マンハッタン距離、コサイン距離の違いと、標準化が必要になる理由。

第 5 章 まとまりを見つける

正解を見せずにグループを作る。階層クラスタリングと k-means、作った結果を評価するしかた。

第 6 章 次元を減らす

52 次元は見られない。主成分分析と多次元尺度構成法で、情報をなるべく落とさずに 2 次元に写す。

第7章 著者を当てる

正解を与えて予測を学ばせる。k 近傍法、線形モデル、決定木、ランダムフォレスト、勾配ブースティング。どの特徴が判別を支えているかも突き止める。

第8章 正しく測る

交差検証、層別化、適合率と再現率、ROC 曲線、混同行列、確率の較正。「正解率が高い」だけでは何も分からない理由。

第9章 罣を避ける

データ漏洩の6つの型、過学習、偏りとばらつき、次元の呪い。出来すぎた結果を疑う習慣。

第10章 学習された表現と比べる

手で組み立てた52列を、大規模言語モデルの埋め込みや深層学習のモデルと比べる。測り方がそろってからでないと、この比較はできない。

第11章 ジャンルを分ける

ライトノベルを加えて問いを立て直す。1つの特徴で解けてしまう問題に、機械学習が要るかどうか。候補の外にいる著者に「該当なし」と答える方法。

付録Aに環境の作り方、付録Bに高次元空間の幾何をまとめた。付録Bでは数式を省略していない。次元の呪いがなぜ起きるのかを、球の体積と距離の集中から説明する。

第2章

テキストを数える

2.1 文章を数値にすること

コンピュータは文章をそのまま比べられない。「この二つの文章は似ているか」を判定させるには、まず文章を数値の並びに変換しなければならない。

どんな数値にすればよいかを考える前に、そもそも文章の中で数えられる量にどんな性質があるのかを知っておきたい。

本章では英語の小説を1冊使って、単語の出現頻度のふるまいを調べる。『赤毛のアン』(Lucy Maud Montgomery, 1908)を題材にする。日本語ではなく英語を使うのは、単語の区切りが空白で分かるからである。日本語には空白がないので、まず文を単語に切り分ける処理が要る。これを**形態素解析**という^{*1}。本書では形態素解析を使わずに進む。理由は第3章で述べる。

2.2 Zipfの法則

単語を出現頻度の高い順に並べ、順位を横軸、頻度を縦軸に取って何が起きるかを確かめる。

```
import re
from collections import Counter

text = open("anne.txt", encoding="utf-8").read()
words = re.sub(r"[.,?!:~\";()'\"]", " ", text).lower().split()
freq = Counter(words).most_common()
```

以下、本章で単語の「出現回数」というときは、『赤毛のアン』の全文から記号を除き、空白で切り出した延べ語について、同じ綴りが何回現れたかを数えた値を指す。

結果を図2.1に示す。左は普通の日盛り、右は両対数日盛りである。

左の図はほとんど何も見えない。出現回数は1位の単語(the)が4079回、2位が3402回と極端に多く、それ以外がすべて底に張り付いている。

こういうときは、縦軸と横軸の両方を対数日盛にしてみる。桁の違う量を1枚に収める常套手段である。そうしたのが右の図で、点がきれいに直線に並んだ。

右下の様子にも目を留めてほしい。点が横一列に並んだ帯が、何段も重なっている。1回しか出ない語、2回しか出ない語、3回しか出ない語が、それぞれ大量にあるということである^{*2}。線でつないでしまうとこの段差が潰れて、なめらかな曲線に見えてしまう。

直線に並ぶということは、順位と頻度のあいだに単純な関係があるということである。その関係が

^{*1} 「形態素」は意味を持つ最小の単位のこと、おおむね単語だと思ってよい。

^{*2} 1回しか出ない語を**ハパックス** (hapax legomenon) という。どの文書でも語彙の半分近くを占めることが多い。

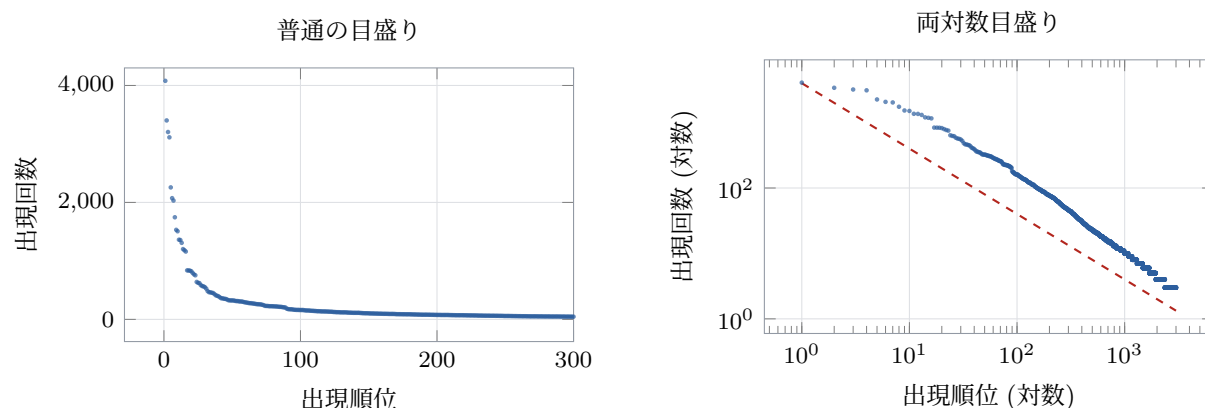


図 2.1 『赤毛のアン』の単語頻度。1 語が 1 点である。右図の破線は傾き -1 の直線。右下で点が横に並んでいるのは、同じ回数しか出ない語が大量にあるためである。

Zipf の法則で、1932 年に言語学者ジョージ・キングズリー・ジップが報告した [6]。

$$\text{出現順位} \times \text{出現頻度} \approx \text{定数} \quad (2.1)$$

法則どおりなら、2 位の単語は 1 位のおよそ半分、3 位は 3 分の 1 になる。実際の『赤毛のアン』では 1 位 4079 回に対して 2 位が 3402 回で、半分の 2040 回より 6 割以上多い。Zipf の法則は正確な等式ではなく、広い範囲でおおよそ成り立つ関係として読むものである。

一般にはべき乗則 (power law) の形で書く。

$$f(r) \approx C \cdot r^\alpha \quad (2.2)$$

r が順位、 $f(r)$ がその順位の単語の頻度である。 $\alpha = -1$ とすると $f(r) \approx C/r$ 、つまり $r \cdot f(r) \approx C$ となって最初の式に戻る。

両辺の対数を取ると

$$\log f(r) = \log C + \alpha \log r \quad (2.3)$$

となる。 $\log r$ を横軸、 $\log f(r)$ を縦軸に取れば、これは傾き α 、切片 $\log C$ の直線の式である。両対数グラフで直線に見えたのは、そういうことだった。

補足 べき乗則はどこにでもある

都市の人口、地震の規模、ウェブサイトの被リンク数、所得の分布。べき乗則は言語に限らず広く現れる。共通するのは「極端に大きいものが少数あり、小さいものが無数にある」構造である。

こうした分布では、平均値があまり役に立たない。裾の減り方が特に緩い場合 (べき指数がおおむね 2 以下) には、標本を集めるほど巨大な値が紛れ込んで平均を引き上げ、何件集めても平均が落ち着かない。そこまで極端でなくても、少数の高額所得者が全体を引っ張って、平均が中央値から大きく離れる。「平均年収」が実感とずれるのはこの歪みのためである。分布が偏っているときは、平均を 1 つ出すだけでは足りない。中央値と分布の形もあわせて見る。この話は第 3 章でもう一度扱う。

2.2.1 同じ直線が乱数列からも出る

図 2.1 の右で点がきれいに直線に並んだ。これを見て「言語には法則がある」と言いたくなるところだが、その前に対照を置いておきたい。言語でないものを同じ手続きにかけたときに直線が出るかどうかを、先に確かめる。

やり方は素朴で、『赤毛のアン』の本文の文字を無作為に並べ替える。空白も混ぜたまま並べ替えるので、語の平均長は保たれるが(長さの分布そのものは幾何分布に近い形に変わる)、語も文法も意味も残らない。残るのはどの文字が出やすいかと空白の割合だけである。これを原文とまったく同じ手続き(空白で切り、頻度順に並べる)にかけると、

もう 1 系列、26 文字と空白を等確率で打ち続けた列も作った。空白の確率は原文の語の平均長 4.11 字に合わせて 0.196 とした。サルがタイプライターを打ち続けるという有名な思考実験を、そのまま実行したものである。

結果が図 2.2 である。

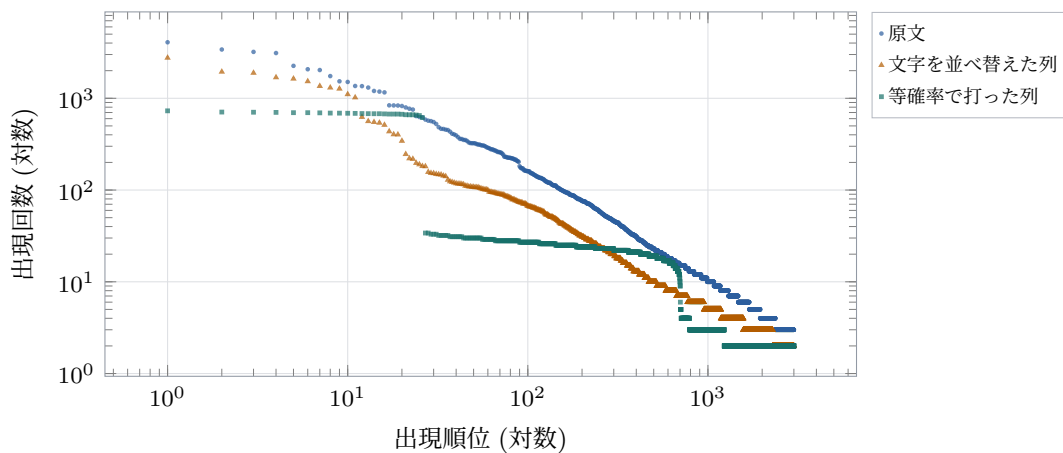


図 2.2 同じ手続きで数えた 3 系列の順位-頻度。上位 3000 語まで描いている。文字を並べ替えただけの列も、原文と同じように直線に乗る。原文より下を通るのは、異なり語数が原文の 6 倍あって 1 語あたりの出現回数が薄くなるためである。等確率で打った列は階段状になる。段の高さが変わるのは語の長さが変わるところで、1 文字の語 26 個、2 文字の語 676 個が、それぞれ同じ確率で出る。

3 系列とも同じ手続きで数え、両対数の上で順位 10 位から 1000 位までに最小二乗で直線を当てて、その傾きと決定係数を測る。原文は傾きが -1.172 、決定係数が 0.996 である。文字を並べ替えた列は傾きが -1.087 、決定係数が 0.991 である。語も意味も持たない列が、直線に乗るという点でも、傾きが -1 の近くにあるという点でも、原文と変わらない。図で並べ替えた列のほうが下を通っているのは語彙の大きさの違いで、直線かどうかとは別の話である。

等確率で打った列だけは、階段状になって直線から外れる。傾きは -0.921 で -1 からそう遠くないが、決定係数は 0.652 しかなく、原文の 0.996 を大きく下回る。1 文字の語が 26 個あってどれも同じ確率、2 文字の語が 676 個あってどれも同じ確率、という作りなので、順位を上げても頻度が段でしか下がらない。文字の出やすさに偏りを入れると、この段がならされて直線になる。それが並べ替えた列である。

なぜ直線になるのかは、語を数えていけば見当がつく。1 文字の語は 26 個しかないが、2 文字の語

は 26×26 で 676 個、3 文字の語は 17576 個ある。文字が 1 つ増えるごとに、語の種類は 26 倍に増えていく。一方で、長い語ほど出にくい。文字をもう 1 つ引き当てなければならないので、1 文字増えるごとに、その語 1 つが出る確率は 26 分の 1 に薄まる (正確には空白を引かない確率も掛かるので、傾きは -1 からわずかにずれる)。順位のほうは 26 倍ずつ増え、頻度のほうは 26 分の 1 ずつ減る。片方が増えるのと同じ勢いでもう片方が減るのだから、両方の対数を取れば、傾きがほぼ -1 の直線になる。言語の性質は 1 つも使っていない。

これは Miller が 1957 年に指摘し [9]、Li が 1992 年に無作為な文字列で確かめた [10]。

× アンチパターン 2.1 — 両対数の直線を言語の証拠として使う —

順位-頻度が両対数で直線に乗ることは、**分布が極端に偏っている**という以上のことを言わない。この直線を出すのに、語の意味も、文法も、書き手も要らない。文字の出やすさに偏りがあって、語の長さが幾何分布に近ければ、それだけで直線は出る。

「Zipf の法則によく当てはまるので、このデータには言語らしい構造がある」「べき乗則が成り立つので、背後に何らかの生成機構がある」という書き方は、どちらも直線から導けない。べき乗則は生成機構を 1 つに絞り込まない。まったく違う仕組みからも同じ直線が出る。

原文と乱数列の違いは、直線の形ではなく語彙の大きさと中身に出る。延べ語数がほぼ同じなのに、異なり語数は原文が 8,509 語、文字を並べ替えた列が 52,716 語で、後者が 6 倍以上多い。異なり語のうち一度きりしか出ない語 (ハパックス) が占める割合も、原文が 0.492、並べ替えた列が 0.920 で、倍近く違う。上位 5 語を並べれば違いはもっとはつきりする。原文は the, and, i, to, a、並べ替えた列は e, t, a, o, i である。後者は 1 文字の語が並んでいるだけで、機能語の顔ぶれではない。

本書が Zipf の法則から先に進むときに使うのも、直線の傾きではない。偏っているという事実と、上位に何が並ぶかという中身のほうである。

2.3 頻度の偏りが、使える語と使えない語を分ける

Zipf の法則が教えてくれるのは、**単語の頻度は極端に偏っている**ということである。この偏りが、どの語なら作品をまたいで比べられるかを決めてしまう。

上位を占めるのは the, and, of, to といった**機能語** (function word) である。それ自体では何も指さず、文の骨組みを作る役目だけを負う語で、日本語なら「は」「が」「を」「ので」がこれにあたる。何を書いているかと必ず出てくる。

対になるのが**内容語** (content word) である。「探偵」「馬車」「アリバイ」のように、それ自体が具体的な何かを指す語で、何を書いているかによって出たり出なかったりする。Zipf の分布の下位に大量にあるのは、こちらである。

文体の判別に向いているのはどちらか、性質を並べて比べる。

- 頻度の高い機能語 — どの文章にも出るので比較できる。書き手が意識せずに使うので癖が出やすい。
- 頻度の低い内容語 — 出たり出なかったりするので比較しにくい。そして**題材**を反映してしまう。

「探偵」「犯人」「アリバイ」という単語で分類すれば、探偵小説を書く作家はきれいに分かれるだ

ろう。だがそれは文体ではなくジャンルを見ているだけである。

× アンチパターン 2.2 — 頻出する内容語で著者を判別する

著者判別で「主人公の名前」や「作品固有の地名」が分類の決め手になってしまうことがある。精度は上がるが、その分類器は**未知の作品には役に立たない**。新しい作品には別の主人公が出てくるからである。

2.4 機能語はどの区間にもむらなく現れる

機能語が文体の手がかりになるなら、その現れ方をもう少し詳しく見ておきたい。同じ現象を二通りに観察する。**回数**を数える見方と、**間隔**を測る見方である。

具体的に思い浮かべるには、新聞がよい。紙面を同じ大きさの区画に等分し、区画ごとに接続詞「しかし」が何回現れるかを数える。0回の区画がいくつ、1回の区画がいくつ、とヒストグラムにすれば、これが**回数**の見方である。同じ紙面で、「しかし」が出てから次の「しかし」が出るまでの字数を測って並べれば、こちらは**間隔**の見方になる。図 2.3 に、2つの見方の関係を示す。

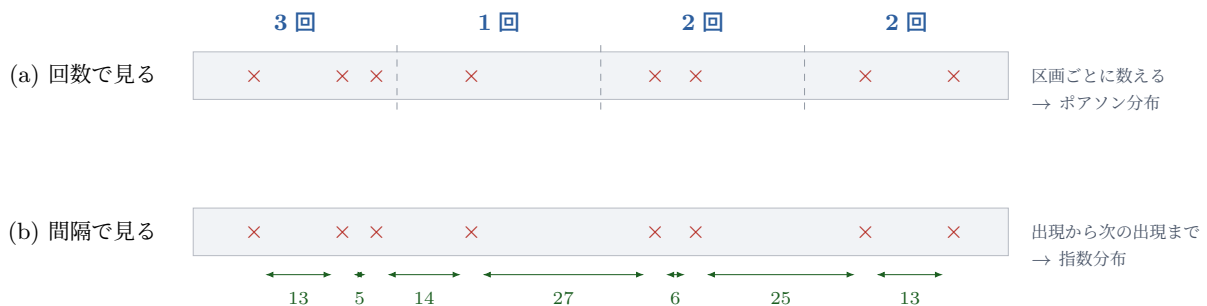


図 2.3 同じ紙面の二通りの見方。× が「しかし」の出現位置。(a) は区画に区切って回数を数え、(b) は出現から次の出現までの間隔を字数で測る (緑の数字)。よく混ざった出現なら、(a) はポアソン分布に、(b) は指数分布に従う。

2.4.1 回数で見える — ポアソン分布

パン生地レーズンを混ぜ込み、よく捏ねてから焼いて、等分に切り分けるとする。一切れに入るレーズンの数は、平均のまわりでばらつく。誰も狙って配置していないのだから、5 個の切れもあるかもしれない。0 個の切れもある。

このばらつき方には決まった形がある。「**全体としては一定の割合で、個々の場所はでたらめ**」という条件さえ満たせば、レーズンでも何でも同じ形の分布になることが知られている。これを**ポアソン分布** (Poisson distribution) という。

『赤毛のアン』の全文を 500 語ずつの塊に分け、各塊の中で特定の単語が何回出るかを数える。単語がレーズンで、500 語の塊が「一切れ」である。書き手がその単語を意識せず、文章のどこにでも満遍なく撒いているなら、回数はポアソン分布に従うはずである。

$$P(X = k) = \frac{\lambda^k e^{-\lambda}}{k!} \quad (2.4)$$

λ は 1 区間あたりの平均出現回数である。式の細部より、次の 2 つの性質を覚えてほしい。

1 つは、山が平均 λ の近くに来ることである。もう 1 つは、**平均と分散が等しい**ことである。

後者が本節の要になる。なぜ等しくなるかは式から出てくることだが、意味のほうを押さえておきたい。ポアソン分布は「でたらめに撒いた」状態を表す分布だった。その状態でのばらつきの大きさが、ちょうど λ になる。つまり**平均と分散が等しい**というのは、**でたらめに撒いたときの標準的なばらつき方**を指している。

だから、実際に数えたデータの分散が平均を大きく上回れば、でたらめではない何かが起きていることになる。ある場所に固まって出ている。本節ではこの物差しを使って、機能語と内容語を分ける。

交通事故の件数、1 年間に落下する隕石の数、そして機能語の出現回数。「試行の機会是非常に多いが、めったに起きない事象」がポアソン分布になる。

図 2.4 に二つの単語の分布を示す。

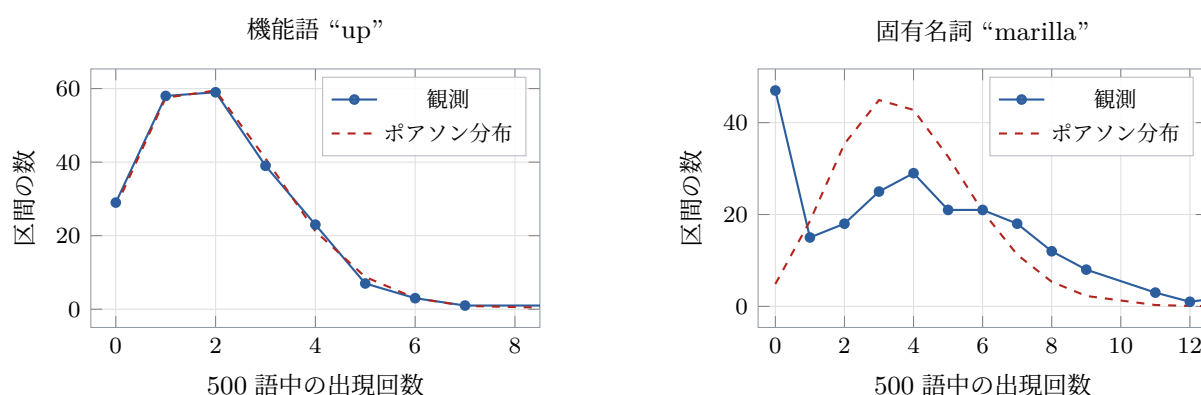


図 2.4 機能語と固有名詞の出現分布。左はポアソン分布によく合うが、右は合わない。

左の “up” は理論曲線によく乗っている。右の “marilla”(登場人物の名前) は大きく外れる。

外れ方に意味がある。マリラが出てくる場面ではまとめて何度も出て、出てこない場面では一度も出ない。**文章の内容に結びついた単語ほど、ポアソン分布から外れる。**

この観察から**残差 IDF** という単語の重要度指標が提案されている [11]*3。ポアソン分布からのずれが大きい単語ほど、内容語らしいと判定する。

○ 定石 2.1 — 著者を当てるなら文書全体に一樣に散らばる単語を選ぶ —

書いた人を当てたい場合の話である。

ポアソン分布によく従う単語 — つまり文章のどこにでも一樣に散らばっている単語 — を選ぶ。これらは書き手の癖を反映し、題材には依存しにくい。

当てたいものが話題やジャンルなら、基準は逆になる。ポアソンから外れて特定の箇所に固まる単語こそが、その文書が何について書かれたかを示している。

*3 IDF(inverse document frequency、逆文書頻度) は情報検索で使う重み付けで、その単語が現れる文書の割合の逆数の対数、つまり**少数の文書にしか出ない語ほど大きくなる**量である。残差 IDF は、実際に観測された IDF から、ポアソン分布を仮定したときに期待される IDF を引いた差を指す。

補足 TF-IDF は「何を当てたいか」を決めてくれない

テキストを数値にするとき、最もよく使われる重みが **TF-IDF** である。文書中の出現回数 (TF) に、その語が何割の文書に出るかの逆数の対数 (IDF) を掛ける。

IDF の働きは、**どの文書にも出る語を軽くすること**である。「は」「が」「の」のような機能語は、すべての文書に出るので IDF がほぼ 0 になり、重みが消える。残るのは少数の文書にしか出ない語、つまり内容語である。

情報検索では、これが正しい設計である。「猫」で検索した人が探しているのは猫の文書であって、「は」を含む文書ではない。

だが本書の目的からすると、**TF-IDF は残すべき語を消してしまう**。著者の癖が出るのは機能語の側だからである。第9章で見ると、内容語を残したまま著者判別をすると、モデルは文体ではなく作品の固有名詞を覚える。

TF-IDF が悪いのではない。検索という目的のために作られた重みを、別の目的にそのまま持ってきたことが問題になる。**既定の重み付けにも、それが想定している目的がある**。

× アンチパターン 2.3 — ワードクラウドや共起ネットワークを分析の結論として出す —

テキスト分析の報告でよく見かける 2 つの図には、共通した弱点がある。

ワードクラウドは、語の大きさに頻度を表す。だが面積が頻度に比例しているとは限らず、語の配置や向きにも意味がない。2 つの語の大小を目で比べることも、2 つの図を並べて比べることもできない。何より、**どの語を除外したか**で見た目が大きく変わる。ストップワードの一覧を少し変えれば、まるで違う雲が出てくる。

共起ネットワークも同じ性質を持つ。辺を張る基準 (共起の回数か、相関か、対数尤度比か)、その閾値、描画のアルゴリズム。どれを選ぶかで図の形が変わるのに、出来上がった図はどれも「それらしく」見える。

どちらも、**見た人が意味を読み取れてしまう**ところが厄介である。出てきた図から物語を作ることはいくらでもできるが、別の設定で作り直せば別の物語ができる。占いの絵解きに近い。

頻度を比べたいなら棒グラフを使う。語どうしの関係を見たいなら、どの基準でどの閾値を使ったかを明記したうえで、その選択が結論を変えないかを確認する。探索の途中で眺めるぶんには構わないが、**結論の証拠としては弱い**。

2.4.2 間隔で見る — 指数分布

回数を数える代わりに、**同じ単語が次に現れるまでの語数**を記録する方法もある。以下、間隔はすべて『赤毛のアン』の全文を通して語数で測った値である。“up” は全文で 455 回現れるので、隣り合う出現の間の間隔が 455 - 1 個得られる。今度は間隔の分布を見る。

時刻表のないバス停を思い浮かべてほしい。バスは平均すれば 1 時間に 4 本来るが、いつ来るかはでたらめだとする。このとき、**あるバスが出てから次のバスが来るまでの間隔が指数分布 (exponential distribution) に従う**。いま測ろうとしている単語の間隔と、同じ量である。

指数分布の形は、短い間隔がいちばん多く、そこから長いほうへ向かって減っていく。減り方には特徴があって、**間隔が 10 分伸びるごとに確率が同じ割合ずつ減る**。10 分から 20 分になるときも、60 分から 70 分になるときも、減る比率は同じである。

この形から、少し意外な性質が出てくる。バス停で 10 分待ったところで、そこから先の待ち時間の分布は、着いた瞬間とまったく同じになる。「もう 10 分も待ったから、そろそろ来るはずだ」は成り立たない。どれだけ待ったかという過去を、分布が覚えていない。この性質を**無記憶性** (memorylessness) という。

$$p(x) = \lambda e^{-\lambda x} \quad (2.5)$$

λ は単位長さあたりの出現率で、間隔の平均は $1/\lambda$ になる。

実は、ポアソン分布と指数分布は**同じコインの裏表**である。「よく混ざった出現」を、区間で区切って回数を数えればポアソン分布になり、出現から次の出現までの間隔を測れば指数分布になる。よく混ざった出現のもとでは、どちらか一方が成り立てば、もう一方も自動的に成り立つ。なぜそう言えるのかは、次の発展で導く。種明かしをすれば、「**間隔が x より長い**」ことと「**長さ x の区間に一度も出ない**」ことが**同じ事象**だからである。

発展 ポアソン分布から指数分布を導く

出現率を λ (1 語進むあたり平均 λ 回) とする。よく混ざった出現では、長さ x の区間に入る回数 X は平均 λx のポアソン分布に従う。

$$P(X = k) = \frac{(\lambda x)^k e^{-\lambda x}}{k!} \quad (2.6)$$

本文のポアソンの式で λ と書いた「1 区間あたりの平均回数」は、いまの記法では $\lambda \times 500$ (区間幅 500 語) にあたる。ポアソン側の λ と指数側の λ は、区間幅の掛け算で結ばれた同じ「率」である。

いま、ある出現の直後から数えて、**次の出現までの間隔 T** を考える。「 T が x より長い」とは、「直後の長さ x の区間に一度も出ない」ということである。同じ事象なので確率も等しく、上の式に $k = 0$ を入れるだけで

$$P(T > x) = P(X = 0) = e^{-\lambda x} \quad (2.7)$$

が出る。累積分布は $F(x) = 1 - e^{-\lambda x}$ 、微分すれば密度

$$p(x) = F'(x) = \lambda e^{-\lambda x} \quad (2.8)$$

となって、指数分布そのものである。ポアソン分布の $k = 0$ の項だけから、間隔の分布が丸ごと決まってしまう。

無記憶性も同じ式から 1 行で出る。 s 語待って出なかったという条件の下で、さらに t 語出ない確率は

$$P(T > s + t \mid T > s) = \frac{e^{-\lambda(s+t)}}{e^{-\lambda s}} = e^{-\lambda t} = P(T > t) \quad (2.9)$$

となり、すでに待った長さ s が消える。

逆向きも成り立つ。間隔が互いに独立に同じ指数分布に従うなら、区間内の回数はポアソン分布になる。つまり「回数がポアソン」と「間隔が指数」は、同じ確率モデル (**ポアソン過程** (Poisson process)) の二つの断面である。おまけとして、次の 1 回ではなく k 回目の出現までの待ち時間を考えると、指数分布を k 個足したことになる、ガンマ分布 (アーラン分布) が現れる。指数分布はその $k = 1$ の特別な場合である。

指数分布には、目で確かめやすいという利点がある。縦軸を対数にすると $\ln p(x) = \ln \lambda - \lambda x$ となって直線になるからである。「片対数のグラフで直線に乗るか」が、よく混ざっているかどうかの簡便な検査になる。

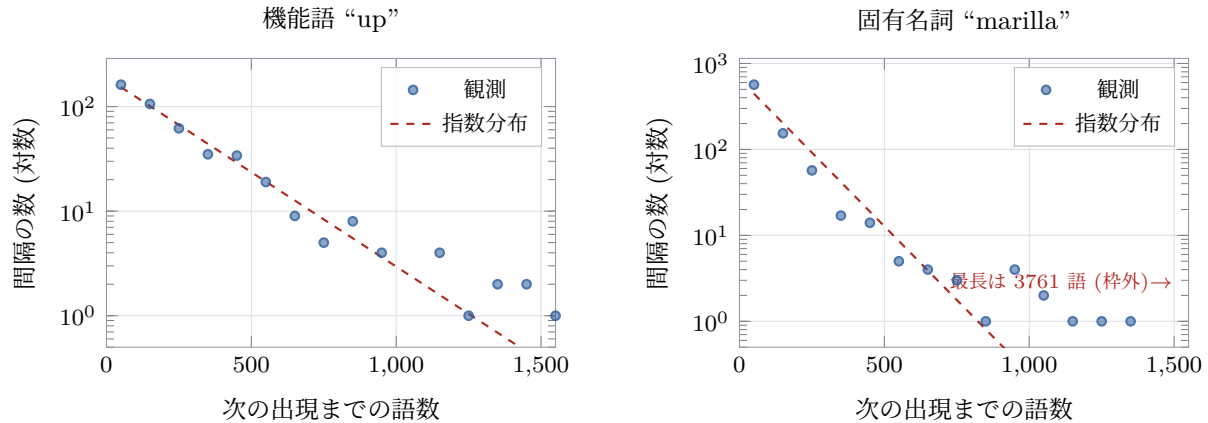


図 2.5 同じ2つの単語を「間隔」で見る。縦軸が対数なので、指数分布は直線になる。“up”は乗るが、“marilla”は短い間隔が多すぎ、長い間隔も多すぎる。

図 2.5 の左、“up”はおおむね直線に乗る。間隔の平均は 240 語、**変動係数** (標準偏差 ÷ 平均) は 1.06 である。指数分布では平均も標準偏差もどちらも $1/\lambda$ になるので、変動係数の理論値はちょうど 1 である。実測の 1.06 は、その理論値を 0.06 上回るだけである。

右の“marilla”は**両端**で直線から外れる。短い間隔が多すぎ(左端の点が直線より上)、そのくせ最長の間隔は 3761 語に達する。マリラは“up”の倍近く登場する(838 回対 455 回)のだから、よく混ざっていれば沈黙はむしろ短くなるはずである。実際には、間隔の中央値は 64 語しかないのに平均は 127 語で、変動係数は 2.07 になる。指数分布の理論値 1 の 2 倍を超えていて、“up”の 1.06 とは大きく違う。出るときは続けて出て、出ないときは延々と出ない。この性質を**バースト性** (burstiness) という。マリラの登場する場面と登場しない場面がある、という物語の構造そのものが数値に写っている。

ところで、“up”の側でも、図 2.5 の当てはまりは図 2.4 ほどぴたりとは見えない。これには次の事情がある。

- 縦軸が対数だから。裾のビンの度数は 1 や 2 しかなく、そこでの ± 1 個の揺らぎは対数軸の上では大きな上下に見える。図 2.4 は線形軸で、しかも度数の大きい山の近くしか目に入らない。
- 集計の量が違うから。回数の見方は数百の区間を 10 個ほどの棒に集めるので偶然が均されるが、間隔の見方は 454 個の間隔を 15 ほどのビンに散らすので、1 本あたりの標本が少なく、ばらつきがそのまま残る。
- “up”自体が完全なポアソンではないから(変動係数 1.06 はわずかに 1 を超えている)。ずれは、集計で均される回数の図では隠れ、集計しない間隔の図ではそのまま見える。

当てはまりが粗く見えるのは、間隔という見方が偶然のゆらぎにも実際のずれにも敏感であることの裏返しで、検査の道具としては、このずれへの敏感さこそ望ましい。

2.4.3 回数と間隔は同じ現象を別の側から測っている

図 2.4 と図 2.5 は同じ現象の写し方を変えただけだが、実務上の得失がある。

回数方式の弱点は、**区間幅を自分で決めなければならない**ことである。本節では 500 語にしたが、これに根拠はない。幅を変えると図の形も変わり、粗すぎればバーストが平均されて隠れ、細かすぎればほとんどの区間が 0 回になってしまう。また、区間の境界をまたぐ近接出現は別々の区間に分かれて、「まとまって出た」という情報が消える。代わりに、集計するぶん個々の値は安定していて、複数のテキストを区間数を揃えて比べるのも簡単である。

間隔方式には区間幅という決めごとがない。バーストは「短い間隔の過剰」として直接目に見える。その代わり、標本の数 は出現回数で頭打ちになるので、まれな語では分布の形が定まらない。裾も重いので、平均が少数の長い間隔に引っ張られやすい。

○ 定石 2.2 — 刻み幅を自分で決めたら刻み幅のいらない見方でも確かめる

区間幅、ビンの幅、集計の単位。分析者が決めた刻みは、結果の見え方を左右する。片方だけでは、見えたものが現象なのか、自分の決めた刻み幅の産物なのか区別がつかない。本節でいえば、回数の見方は区間幅に依存し、間隔の見方は依存しない。両方描いて同じ結論になるかを確かめるのが安全である。とくに「ポアソン分布に合っている」と主張したいときは、区間幅に依存しない間隔の図を添えると強い。

発展 離散と連続とバースト性の指標

語数は整数なので、間隔の分布は厳密には指数分布にならない。各語の位置で**独立に、一定の確率**で出現すると仮定すれば、間隔は指数分布の離散版である**幾何分布**に従う。出現率 λ が小さければ両者はほとんど区別できないので、本節では連続の指数分布で通した。なお、観測した度数分布が幾何分布や指数分布に合っている、それだけで出現の独立性まで確かめたことにはならない。

バースト性の大きさは、変動係数 CV から作った $B = (CV - 1)/(CV + 1)$ という指標で測ることが多い [13]。ポアソンのなら $B \approx 0$ 、バーストが強いほど 1 に近づく。“up” は $B \approx 0.03$ 、“marilla” は $B \approx 0.35$ である。単語の再帰間隔の分布は、指数分布よりワイブル分布のほうがよく当てはまるという報告もある [12]。内容語ほど形状パラメータが 1 から離れる、つまりバーストが強いという結果で、本節の観察と同じことをより精密に測っている。

2.5 日本語には語の区切りがない

ここまでの議論は英語を前提にしていた。日本語には空白による語の区切りがなく、単語を数えるには本章の冒頭で触れた形態素解析が必要になる。

形態素解析は有用な道具だが、辞書を含むソフトウェアの導入に手間がかかり、固有名詞や古い言い回しでは切り方を誤ることもある。本書は環境構築でつまずかずに進めることを優先して、別の道を選ぶ。**単語を数えず、文字を数える。**

日本語には英語にない性質がある。平仮名・片仮名・漢字という 3 種類の文字体系が混在していることである。どの語をどの文字種で書くかには、書き手ごとの選択がある。

- 「わたし」か「私」か

- 「たくさん」か「沢山」か
- 「コンピュータ」か「計算機」か

どちらを選ぶかは、内容とはほとんど無関係に、書き手の習慣で決まる。機能語に求めた「内容に依存せず、書き手が意識しない」という条件を、単語ではなく文字のレベルで満たしているわけである。

2.5.1 単語の代わりに数える単位 — n-gram

「単語を数えず、文字を数える」という方針を道具にしたものが **n-gram**(n-グラム) である。連続する n 個の並びを1つの単位として数える。何を1個と見るかで2種類ある。

- **単語 n-gram** — 語を単位にする。the cat sat から 2-gram を取ると the cat と cat sat の2つになる
- **文字 n-gram** — 文字を単位にする。「吾輩は猫である」から 3-gram を取ると「吾輩は」「輩は猫」「は猫で」「猫であ」「である」の5つになる

日本語で文字 n-gram がよく使われるのは、語の区切りを決めずに済むからである。形態素解析を通さなくても、 n 文字の窓を1文字ずつずらして数えるだけで、表の形になる。 $n = 1$ なら文字そのものの頻度、 n を上げるほど長い言い回しを捉える。

代償として、 n を上げると列の種類が急に増える。常用漢字だけで2136字あるので、漢字2文字の並びだけで約456万通りある。実際に現れるのはその一部だが、それでも数千から数万の列になり、第9章で扱う次元の問題が顔を出す。

本書は文字 n-gram を主役にしない。第9.6節で確かめるように、文字 n-gram は著者ではなく**固有名詞**を拾ってしまうからである。ただし本書が数えるものも、n-gram の一種として書き直せる。字種の比率は $n = 1$ の文字 n-gram を3つの集合にまとめたものであり、読点の直前に来る平仮名は、**2文字目を読点に固定した文字 2-gram** にあたる。n-gram をそのまま全部使うのではなく、条件を付けて絞り込んだものが本書の特徴である、と見ることができる。

発展 n-gram の統計から語彙を取り出す — 分岐エントロピー

n-gram の数え上げだけを使って、辞書も学習データも使わずに語の切れ目を見つける方法がある。

考え方は Harris にさかのぼる [3]。語の切れ目では、**次に来る文字の種類が急に増える**。「自然言」まで読んだ時点では次の文字はほぼ「語」に決まっているが、「自然言語」まで読むと、次には「処」「の」「を」「は」など多くの文字が来る。語の内部では次の文字が絞られ、語の終わりで一気に広がるわけである。

この広がりをエントロピーで測ったものを**分岐エントロピー** (branching entropy) という。ある文字列の後ろに続く文字の分布を n-gram の頻度から求め、その分布のエントロピーを取る。文字列を1文字ずつ伸ばしながらこの値を追えば、値が減少から増加に転じた位置を語の境界とみなす。Jin と Tanaka-Ishii は、この方法だけで中国語の文を語に分割できることを示した [4]。

こうして得られるのは語の一覧、つまり**語彙**である。数えるだけで、そのテキストに現れる語をある程度まで取り出せる。

ただし本書の課題には向かない。この方法が取り出すのは内容語を含む語彙全体であって、本章で機能語に求めた「内容に依存しない」という条件から外れる。著者ではなく話題を当てる特徴になってしまう。

次章では、文字を数えるという方針で 3 種類の特徴を設計する。

2.6 章末の課題

1. 図 2.1 の両対数グラフから、傾き α を目分量で読み取れ。 -1 にどれだけ近いかな。
2. 日本語のテキストで同じことをするには何が必要か。単語ではなく文字単位で Zipf の法則を調べると、どうなると予想するか。
3. 図 2.2 の「文字を並べ替えた列」を自分で作り、順位 10 位から 1000 位の傾きを測れ。次に、文字ではなく**単語の順序**だけを並べ替えたら順位-頻度がどうなるかを、測る前に予想せよ。予想が当たった理由、あるいは外れた理由を 1 行で書け。
4. “the” と “said” について、回数 (ポアソン分布) と間隔 (指数分布) の両方で適合を調べよ。どちらの単語が機能語らしいふるまいをするか。2 つの見方の結論は一致するか。
5. 図 2.4 の区間幅 500 語を、100 語と 2000 語に変えて描き直せ。“marilla” の見え方はどう変わるか。区間幅によらない結論を引き出すにはどうすればよいか。
6. 手元にある好きな小説の 1 章分で、助詞「と」の出現間隔を字数で測り、片対数のグラフに描け。直線に乗るか。乗らないなら、どちら側にどうずれているか。(ヒント: 「～と言った」のような会話の受けの「と」は、会話場面にまとまって現れやすい。図 2.5 の “marilla” と見比べるとよい。)

第3章

特徴を設計する

3.1 何を数えるかを定める

前章で方針は決まった。内容に依存せず、書き手が意識しない量を数える。本章ではそれを3種類作る。作業の全体像を図3.1に示す。

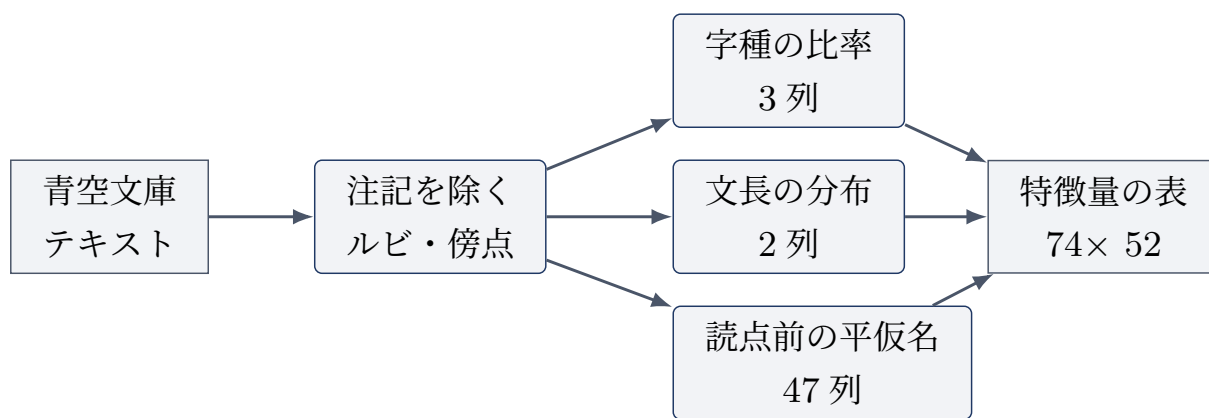


図3.1 特徴量を作る手順。3種類の特徴を横に並べて1枚の表にする。

3.2 前処理 — 情報の出どころで捨てるものを決める

青空文庫のテキストには、本文以外の情報が混ざっている。

斜陽
太宰治

【テキスト中に現れる記号について】
《》：ルビ
(例) 生国《しょうごく》

朝、食堂でスープを一さじ、すっと吸って...

本文だけを取り出す。捨てるものを順に挙げる。

1. ヘッダとフッタ — 罫線より前、「底本」以降
2. ルビ — 生国《しょうごく》の《》の中
3. 注記 — [#改ページ] のような編集記号


```
RUBY = re.compile(r"《[^》]+》")
ANNOTATION = re.compile(r" [# [^ ]+ ] ")

def strip_annotations(text: str) -> str:
    text = RUBY.sub("", text.replace(" | ", ""))
    return ANNOTATION.sub("", text)
```

× アンチパターン 3.1 — ルビや注記を取り除かずに文字を数える

ルビを残したまま文字種を数えると、平仮名の比率が跳ね上がる。しかもルビの量は出版社の方針で決まるので、著者の文体とは関係のない差が入り込む。同じ作家の作品が、収録された全集ごとに別のクラスタに分かれる、ということが起こりうる。

○ 定石 3.1 — 捨てるものはその情報が誰に由来するかで決める

前処理とは、要するに何を捨てるかを定める作業である。「よく分からないので全部残す」も「念のため全部消す」も、判断ではない。

物差しの一つが由来である。ルビの量を決めているのは著者ではなく編集者や出版社だから、著者を当てたいなら捨てる。読点の位置は著者が決めているので残す。当てたい相手が出版社なら、判断は逆になる。残すか捨てるかは、データの性質ではなく問いの側が決める。

× アンチパターン 3.2 — 分析に使う CSV を表計算ソフトで開く

捨てるつもりがないのに捨ててしまうこともある。

CSV を表計算ソフトで開くと、中身を見た瞬間に型を推測して書き換える機能が働く。書き換えたという記録は残らないので、以降のすべての分析が汚染されたまま進む。

生物学ではよく知られた事故がある。遺伝子名の SEPT2 が日付の「2-Sep」に、MARCH1 が「1-Mar」に変換されてしまう。2016 年の調査では遺伝子リストを載せた論文の約 2 割 [70]、2021 年に 1 万本以上を再調査したときには 3 割以上がこの変換で汚染されていた [71]。

日本語のデータでも起きる。先頭が 0 の郵便番号や電話番号は 0 が落ち、1E5 のような文字列は指数表記の数値になり、長い数値は勝手に丸められる。

分析に使うファイルは、表計算ソフトで開かない。中身を見たいなら、テキストエディタか、`pl.read_csv` で読んでから確かめる。どうしても開くなら、書き換えられていないかを開く前後で照合する。

3.3 特徴設計 1 — 文字種の比率

日本語は平仮名・片仮名・漢字が混ざる。その比率を数える。

```
import regex

def count_char_types(text: str) -> dict[str, int]:
    return {
```

```

    "平仮名": len(regex.findall(r"\p{Hiragana}", text)),
    "片仮名": len(regex.findall(r"\p{Katakana}", text)),
    "漢字":   len(regex.findall(r"\p{Han}", text)),
}

```

このままでは使えない。長い作品ほど数が大きくなるからである。『三国志』は『走れメロス』の何十倍もの文字数がある。数えた回数をそのまま並べれば、文体の違いではなく作品の長さの違いを見ることになってしまう。

素直な対処は、**総文字数で割って割合にすること**である。「漢字が12万字」ではなく「漢字が全体の33%」と言えば、長さの違いは消える。このように、作品ごとの大きさの違いをそろえる操作を**正規化 (normalization)** という。

3.3.1 正規化は大きさだけを揃えて向きを残す

「総文字数で割る」は正規化の一例でしかない。何で割るかには選択肢があり、選び方で結果が変わる。そこを見るために、いったん幾何の話にする。

2次元で考えると分かりやすい。点 (x, y) を、原点から見た**方向はそのままに、大きさだけを揃える**。点を原点から見た矢印だと思えば、向きを変えずに長さだけを1にそろえる操作にあたる。

その「長さ」の測り方に、2つの流儀がある^{*1}。

$$\ell_1 \text{ 正規化: } (x, y) \mapsto \left(\frac{x}{x+y}, \frac{y}{x+y} \right) \quad (3.1)$$

$$\ell_2 \text{ 正規化: } (x, y) \mapsto \left(\frac{x}{\sqrt{x^2+y^2}}, \frac{y}{\sqrt{x^2+y^2}} \right) \quad (3.2)$$

どちらも「点を原点方向に動かして、ある図形の上に載せる」操作で、違うのは載せる先である。

図3.2で、点 P と Q は原点から見て別の方向にある。正規化しても**方向は変わらない**。破線の上を滑って、直線または円の上に落ちるだけである。

残るのは方向だけで、 P と Q の「大きさ」の違い、つまり原点からの遠さは消える。

この遠さが、いま消したいものにあたる。文字数の多い作品ほど、どの成分の数も大きくなるので、点は原点から遠いところに置かれる。短い作品は原点の近くに来る。正規化はその差を潰して、**どの字種をどんな配分で使うか**という方向の情報だけを残す。

^{*1} ℓ という字は、数学では**数列の空間**を指す記号である。測度空間の上の関数を集めた空間を L^p と書き、そのノルムは積分 $(\int |f|^p d\mu)^{1/p}$ で定める。このうち、自然数に「1つ数えたら1」という測度を入れた場合、関数は数列になり、積分は和になって $(\sum_i |x_i|^p)^{1/p}$ という形をとる。この特別な場合を ℓ^p と書き分ける。連続を L 、離散を ℓ と覚えておけばよい。本書が扱うのは52個の数の並びなので、数学の流儀に従えば ℓ のほうである。ただし機械学習では、正規化(第4.4.2節)を指すときだけ大文字で「L1 正規化」「L2 正規化」と書くのが通例になっている。本書もその慣習に合わせ、ノルムと正規化には ℓ 、正規化の呼び名には **L** を使う。記号の由来なので、この脚注は飛ばして構わない。

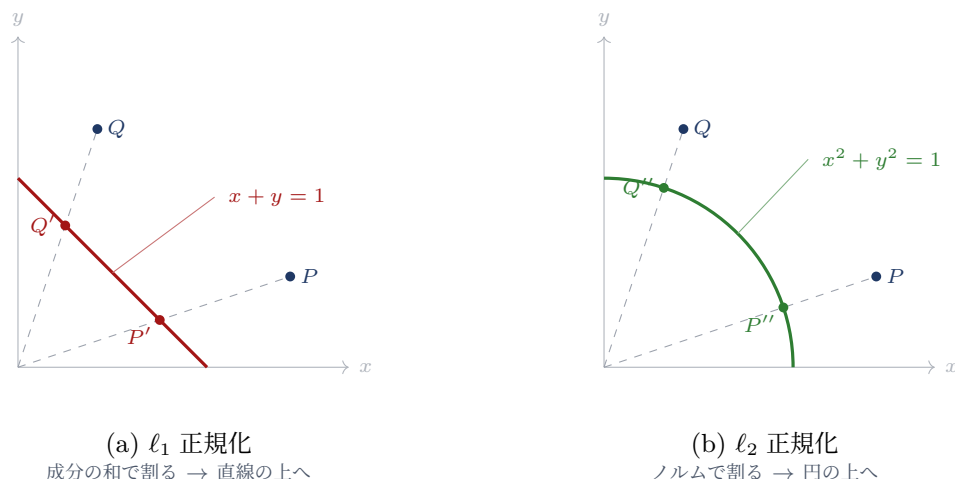


図 3.2 正規化は「原点からの方向を保ったまま、決まった図形の上に乗せる」操作。載せる先が ℓ_1 では直線、 ℓ_2 では円になる。

補足 ℓ_1 と ℓ_2 のどちらで正規化するか

ℓ_1 は「割合」の解釈ができる。(0.66, 0.34) なら「66% と 34%」と読める。合計が 1 になるからである。

ℓ_2 は距離との相性がよい。すべての点が円 (高次元では球) の上に乗るので、2 点の隔たりを直線距離で測っても、角度の開きで測っても、同じ順序になる。第 4 章で見るユークリッド距離とコサイン距離が一致する、というのはこの意味である。

本書では字種比率と読点の頻度に ℓ_1 を使う。「割合」として読めるほうが、意味を追いやすいからである。

× アンチパターン 3.3 — 正規化と標準化を混同する —

名前が似ているが、揃える向きが違う。

	何を揃えるか	いつ使うか
正規化	行 (サンプル) ごとの大きさ	作品の長短を消したい
標準化 (standardization)	列 (特徴) ごとのばらつき	特徴どうしの単位を揃えたい

混同すると、2 つの形で害が出る。

1 つはどちらかで済ませてしまうことである。「正規化しておいたから単位はそろっている」と考えると、列ごとの単位が違うまま距離を測ることになる (第 4 章)。逆に「標準化したから作品の長短は消えた」と考えるのも誤りで、列を揃えても行の大きさは残っている。

もう 1 つは順番を取り違えることである。本書では両方使う。第 3 章で行方向の正規化、第 4 章で列方向の標準化である。正規化が先で、標準化が後になる。逆順にすると何が壊れるかは、第 4.4.1 節で確かめる。

3.3.2 3 成分の場合

字種は平仮名・片仮名・漢字の 3 つなので、式 (3.1) は 3 成分になる。

$$r_{\text{平仮名}} = \frac{n_{\text{平仮名}}}{n_{\text{平仮名}} + n_{\text{片仮名}} + n_{\text{漢字}}} \quad (3.3)$$

図 3.2(a) の直線 $x + y = 1$ が、3 次元では平面 $x + y + z = 1$ になる。次節で詳しく見る。結果を図 3.3 に示す。

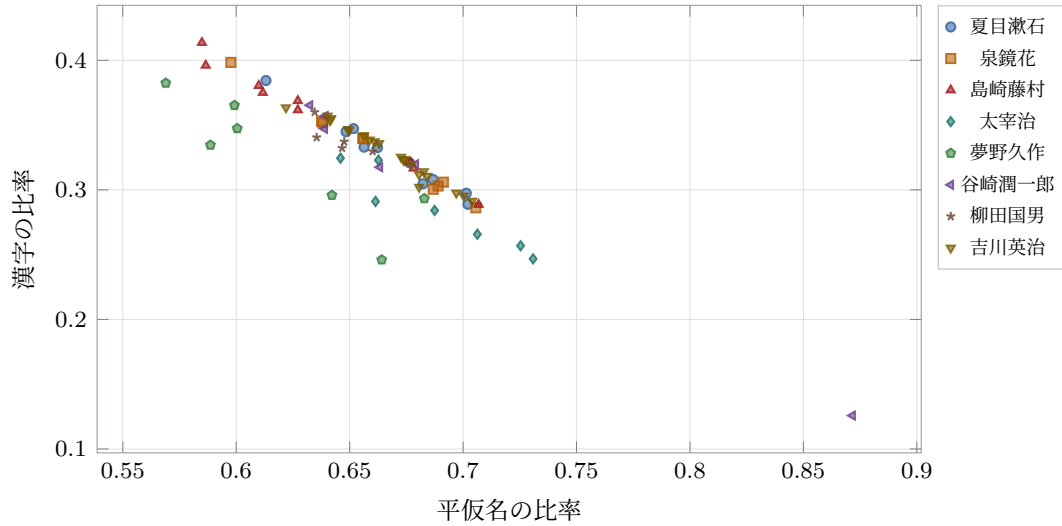


図 3.3 文字種の比率。点は 1 作品。

いくつか読み取れることがある。

- 点がほぼ一直線に並んでいる
- 吉川英治 (右下寄り) がまとまっている
- 泉鏡花は漢字が多い側に散らばる
- 重なっている著者も多い

一直線に並ぶのには理由がある。3 つの比率の合計が 1 なので、

$$r_{\text{平仮名}} + r_{\text{片仮名}} + r_{\text{漢字}} = 1 \quad (3.4)$$

が常に成り立つ。片仮名の比率は近代文学ではどれも小さいので、実質的に $r_{\text{漢字}} \approx 1 - r_{\text{平仮名}}$ となり、直線に乗る。

3.4 単体 — 3 つの比率が住む場所

式 (3.4) は、よく見れば 3 次元空間の中の平面の方程式である。

$$x + y + z = 1, \quad x, y, z \geq 0 \quad (3.5)$$

さらに 3 つとも非負という条件がつくので、正規化した点が存在できるのは平面のうち 3 つの座標がすべて 0 以上になる部分、つまり三角形の内側 (辺の上を含む) だけである。

正規化は、この三角形の面の上に点を落とす操作である。

3つの頂点は $(1, 0, 0)$ 、 $(0, 1, 0)$ 、 $(0, 0, 1)$ 。それぞれ「全部が平仮名」「全部が片仮名」「全部が漢字」という極端な状態を表す。

この三角形を **2次元単体** (2-simplex) という。成分は3つあるのに「2次元」と呼ぶのは、合計が1に固定されているぶん、自由に動けるのが2つ分だからである。2つ決まれば3つ目は引き算で決まってしまう。

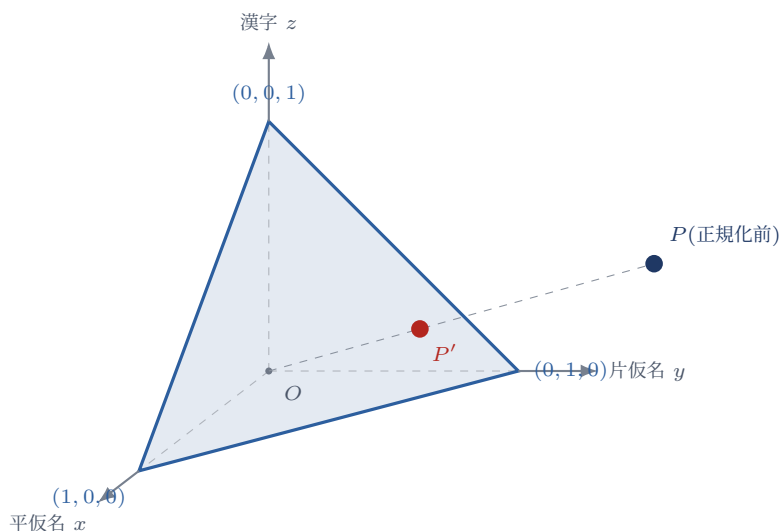


図 3.4 正規化前の点 P は、原点への破線に沿って三角形の面上の P' に落ちる。図 3.2(a) の 3次元版である。

補足 合計が1なら3つの数でも自由に決められるのは2つ分

3つの数を使っているのに、実際に自由に動けるのは2つ分しかない。これを「自由度が1つ減っている」という。

3次元空間に点を打っているつもりでも、点は平面上にしか存在しない。第6章で扱う主成分分析は、**見かけ倒しの次元**を自動的に見つけ出す技術でもある。実際、字種の3列にPCAをかけると第3主成分の寄与率は**厳密に0**になる。合計1の平面と直交する方向には、分散が一切ないからである。第6章では組み合わせを変えて(平仮名・漢字・文長 μ) 同じ現象のゆるい版を見る。そちらは合計の制約がないので、第3主成分の寄与率が2.8%だけ残る(図6.4)。

3.4.1 三角図で描く

点が三角形の上にしかないのなら、その三角形を正面から見れば、情報を失わずに2次元で描ける。

これが**三角図** (ternary plot、三角ダイアグラム、三成分図) である。地質学で岩石の鉱物組成を表すのに古くから使われてきた。

読み方は次のとおりである。

- 各頂点は「その成分が100%」を表す
- ある成分の値は、**その成分の頂点に向かい合う辺からの距離**で読む
- 頂点に近いほど、その成分が多い

実際のデータを重ねたのが図3.6である。

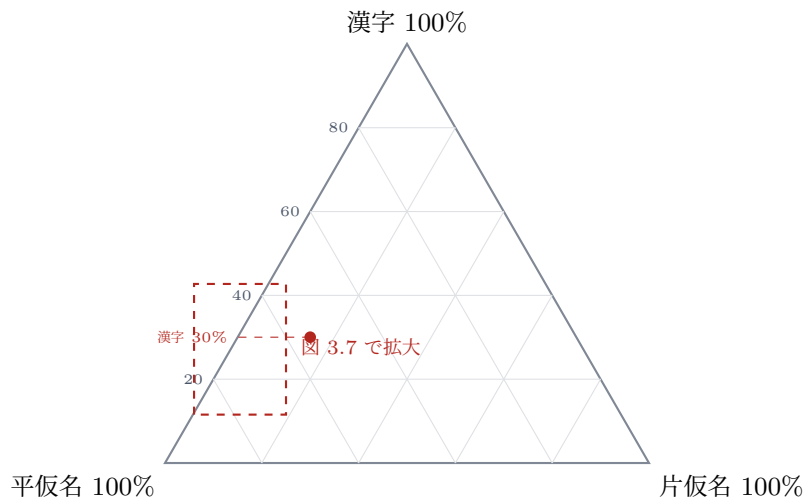


図 3.5 三角図の読み方。目盛りは漢字の比率。

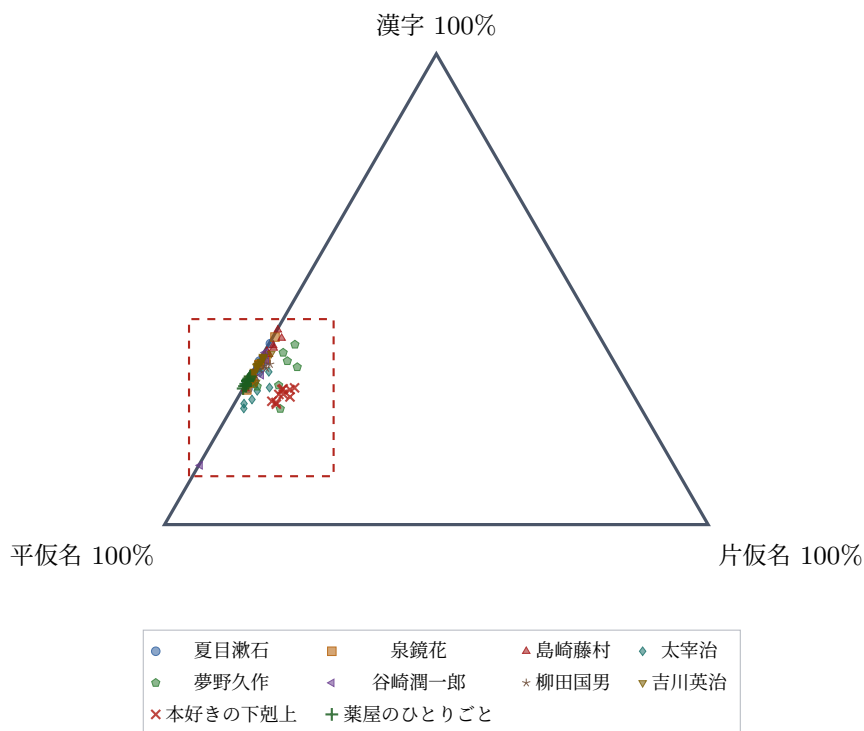


図 3.6 三角図に載せた近代文学 74 作品と、ライトノベル 2 シリーズ。全体としては左下の隅に固まっている。

図 3.6 を見ると、点が三角形の左下、つまり**平仮名の頂点の近く**に集中している。日本語の文章は平仮名が 6 割前後を占めるので当然ではある。

比較のため、ライトノベル 2 シリーズも重ねてある。『本好きの下剋上』（赤い ×）と『薬屋のひとりごと』（緑の +）である。2 作を選んだ理由は、次の拡大図で分かる。

だがこれでは著者ごとの違いが見えない。破線の枠を拡大する。

拡大すると様子が変わる。吉川英治と柳田国男がそれぞれまとり、泉鏡花は漢字の多い側に散らばっている。

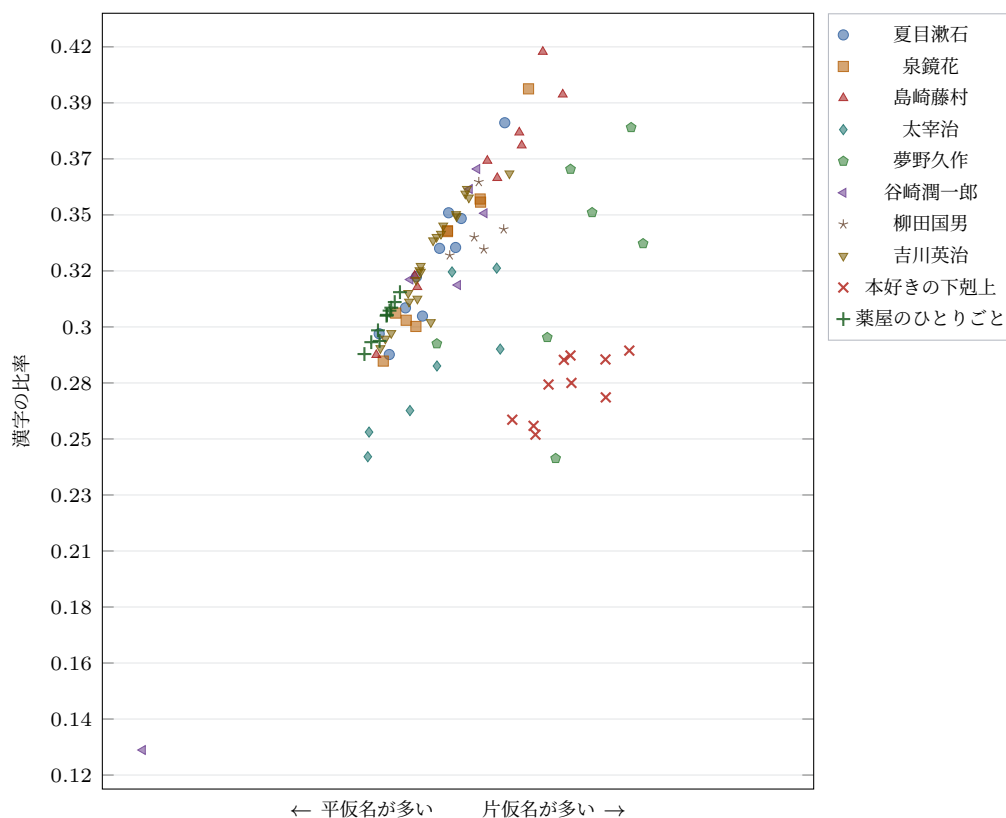


図 3.7 図 3.6 の破線部を拡大。著者ごとのまとまりが見えてくる。縦軸の目盛りは漢字の比率に換算した。横軸方向は特定の成分の比率に対応しないので、目盛りを付けていない。

そしてライトノベル 2 作の位置が対照的である。

表 3.1 字種比率の平均。近代文学は 74 作品、ライトノベルは各シリーズ 10 区間で平均した。片仮名の比率はライトノベルの中でも大きく違う。各シリーズの内訳は第 1 章の表 1.3 を参照

作品群	平仮名	片仮名	漢字
近代文学 (74 作品)	0.660	0.013	0.326
サイレント・ウィッチ	0.581	0.135	0.285
無職転生	0.640	0.093	0.267
リゼロ	0.631	0.080	0.289
本好きの下剋上	0.645	0.079	0.275
片田舎のおっさん	0.654	0.061	0.285
葉屋のひとりごと	0.697	0.000	0.303

以下に挙げる比率は、すべて表 3.1 の平均値である。『サイレント・ウィッチ』10 区間の片仮名比率の平均は 0.135 で、近代文学 74 作品の平均 0.013 の約 10 倍である。異世界の固有名詞、「モニカ」「フェリクス」「シ ril」といった片仮名の人名・地名が絶えず出てくるためである。『本好きの下剋上』の「マイン」「トゥーリ」「ルッツ」も同じである。

一方『葉屋のひとりごと』10区間の片仮名比率の平均は0.000である。表では0.000に丸まっているが、厳密なゼロではない。1万字に3文字という水準で、近代文学74作品の平均0.013の40分の1以下しかない。10区間のうち1つは、10万字を超える分量で片仮名が1文字も出てこない。同じライトノベルでありながら、ここまで違う。

理由は舞台設定にある。中国の後宮を思わせる世界が舞台で、登場人物名も「猫猫」「壬氏」「羅漢」と漢字である。外来語が入り込む余地がない。

補足 三角図は3成分の関係を1枚で見せる

表3.1の数字を眺めるだけでも違いは分かる。だが図3.7を見ると、**葉屋が近代文学の集団の中に混ざっているのが一目で分かる。**

字種比率という特徴だけでは、『葉屋のひとりごと』と近代文学は区別できない。第11章で見のように、ジャンル判別は「時代の違い」を手がかりにしている面があるが、その手がかりも万能ではないということである。

散布図なら2変数しか見られない。3成分の関係を1枚で見せられるのが三角図の利点である。

○ 定石 3.2 — 散布図は全体図と拡大図の2枚を描く —

全体像を示す図と、注目部分を拡大した図。2枚を並べるだけで、読者が受け取る情報は大きく増える。

全体図だけでは「どこに何があるか」しか分からない。拡大図だけでは「それが全体のどこなのか」が分からない。

この特徴だけでは著者を分けきれない。次の特徴を足す。

3.5 特徴設計2 — 文の長さ

文を長く書く作家と短く書く作家がいる。句点で区切って文字数を数える。

一つの作品(太宰治『斜陽』)について、文長の分布を見たのが図3.8である。

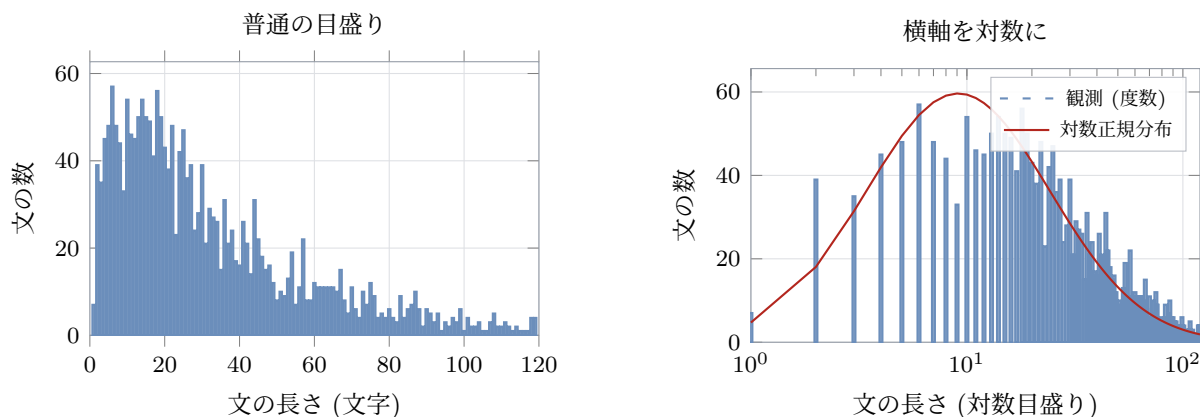


図3.8 太宰治『斜陽』の文長分布。横軸を対数にすると左右対称の山になる。

左の図は右に長い裾を引いている。裾の長い分布の「代表値」に平均を使うと、少数の長い文に

引っ張られる。

横軸を対数にすると、左右対称の山になった。つまり**文長の対数が正規分布に近い**。これを**対数正規分布** (log-normal distribution) という。

$$p(x) = \frac{1}{x\sigma\sqrt{2\pi}} \exp\left(-\frac{(\ln x - \mu)^2}{2\sigma^2}\right), \quad x > 0 \quad (3.6)$$

パラメータは μ と σ の 2 つである。

- μ — 対数をとった文長の平均。文の長さの代表値
- σ — 対数をとった文長の標準偏差。長さのばらつき

何千という文長の値が、たった 2 つの数にまとまる。これが**分布を仮定することの利点**である。

```
lengths = np.array([len(s) for s in sentences])
log_lengths = np.log(lengths)
mu, sigma = log_lengths.mean(), log_lengths.std()
```

発展 $n-1$ で割るべきか — 不偏性と最尤法

上のコードは `std()` をそのまま呼んでいる。NumPy の既定は偏差平方和を n で割る版で、統計の教科書が「不偏分散」と呼ぶ $n-1$ で割る版ではない。どちらを使うべきかを考える。

まず**不偏性** (unbiasedness) とは何かをはっきりさせておく。手元にあるのはデータ 1 組だけだが、同じ条件でデータを取り直せば、推定値は毎回違う値になる。その**取り直しを無限に繰り返したときの平均**が真の値に一致することを、推定量が不偏であるという。真の値を θ 、推定量を $\hat{\theta}$ と書けば

$$E[\hat{\theta}] = \theta \quad (3.7)$$

である。ここで注意がいる。不偏性は**推定量という手続きの性質**であって、いま手元に出た 1 つの値の性質ではない。不偏な推定量でも、1 回の値は真の値から大きく外れうる。「不偏だから正確」とは言えない。

σ^2 の推定では、 $n-1$ で割ると等式が成り立つ。 n で割ると平均は真の値より小さくなる。標本平均 $\bar{x}$ が手元のデータに引き寄せられているぶん、偏差平方和が本来より小さく出るためである。

では、平均が合っていれば良い推定量と言えるのかを確かめる。標準正規分布から n 個取る操作を 40 万回繰り返し、偏差平方和を $n-1$ 、 n 、 $n+1$ で割った 3 通りについて、真の値からの偏り ($E[\hat{\theta}] - \theta$) と、1 回ごとの外れ具合 ($E[(\hat{\theta} - \theta)^2]$ 、二乗誤差) を測った。真の分散は 1 なので、表の偏りは推定値の平均が 1 からどれだけずれたか、二乗誤差は推定値が 1 から外れた量の 2 乗を平均したものである。

発展 (続き)

n	割る数	偏り	二乗誤差
5	$n-1$ (不偏)	-0.0004	0.5029
	n (最尤)	-0.2003	0.3620
	$n+1$	-0.3336	0.3348
10	$n-1$ (不偏)	-0.0007	0.2221
	n (最尤)	-0.1006	0.1901
	$n+1$	-0.1824	0.1820
30	$n-1$ (不偏)	-0.0007	0.0688
	n (最尤)	-0.0340	0.0655
	$n+1$	-0.0651	0.0645

$n-1$ で割ると偏りは消えるが、二乗誤差は n が 5、10、30 のどれでも **3 通りの中で最も大きい**。つまり不偏性とは、精度をいくらか手放して偏りをゼロにする選択である。第 9.12 節で扱う偏りとばらつきの分解でいえば、偏りを 0 にするために、ばらつきの分を引き受けている。二乗誤差だけを見れば $c = n+1$ で割るのが最良になるが、それは正規分布を仮定したこの形の推定量に限った話である (導出は付録 C)。

n で割る版は、正規分布を仮定したときの**最尤推定量**である。最尤推定は「観測されたデータが最も出やすくなるパラメータ」を選ぶ方法で、不偏とは限らない。実際 σ^2 では下に偏る。それでも標準的に使われるのは、 n が増えれば偏りが消え (一致性)、ばらつきも理論上の下限に近づく (漸近有効性) からである。本書でも第 7.4 節のロジスティック回帰は係数を最尤法で決めている。

ここまで σ^2 の話をしてきたが、本章で特徴にしたのは σ 、つまり**標準偏差**である。ここで s^2 が**不偏**でも、 $s = \sqrt{s^2}$ は**不偏にならない**。不偏という性質は平方根を取ると失われる。平方根が上に凸な関数だからで、イェンセンの不等式から $E[s] = E[\sqrt{s^2}] < \sqrt{E[s^2]} = \sigma$ になる。標準偏差まで不偏にする補正はあるが、実務で使う場面は限られるので付録 C に回す。本書の文長 σ は、作品の文数が 234 から数千 (中央値 2507) である。74 作品それぞれについて、 $n-1$ で割った σ と n で割った σ の相対差を測ると、最大でも 0.21%、中央値で 0.020% であった。平方根の偏りも、文数 234 で $c_4 = 0.999$ 、中央値の 2507 で $c_4 = 0.9999$ と、 $E[s]/\sigma$ が 1 からほとんど離れない。どちらでも結論は変わらない。

× アンチパターン 3.4 — 分散は必ず $n-1$ で割ると覚える —

$n-1$ で割ると、**データを取り直して推定をやり直すことを無限に繰り返したときの平均**が真の分散に一致する。これが不偏性である。 $n-1$ は「正しい割り方」ではなく、**この性質がほしいときの割り方にすぎない**。しかも、その性質は分散にしか付かない。標準偏差 (平方根) は不偏にならない。現に、1 回ごとの外れ具合 (二乗誤差) で測れば、 $n-1$ は 3 通りの中で最も悪かった。

「必ず $n-1$ 」と覚えていると、害が 2 つの形で出る。

1 つは**他人の計算を誤りと決めつけること**である。NumPy の `std()` は既定で n で割る。それを見て「バグだ」「直すべきだ」と指摘してしまう。 n で割るのは最尤推定量で、それ自体は誤りではない。何を良しとするかを決めなければ、どちらが良いかは決まらない。

もう 1 つは**割り方の違う数値を突き合わせる**ことである。既定はライブラリごとに違う。同じ 4 つの数から標準偏差を計算しても、NumPy の `std()` は n で割って 1.118 を返し、Polars

の `std()` は $n - 1$ で割って 1.291 を返す。後者は前者より 15% 大きい。自分の計算、引用した論文の数値、手元の集計表。比べる前に、どれがどちらで割ったものかを確認しておく。

差が問題になるのは n が小さいときだけである。2つの割る数の違いは比にして $1/(2n)$ ほどなので、 n が 500 を超えれば標準偏差の相対差は 0.1% より小さくなる。本書の文長 σ では、74 作品を通して相対差は最大でも 0.21% であった。どちらで割るかで迷う時間があるなら、 n が 10 前後まで小さくなる集計がどこかに紛れ込んでいないかを探すほうがよい。

全作品について (μ, σ) を計算したのが図 3.9 である。

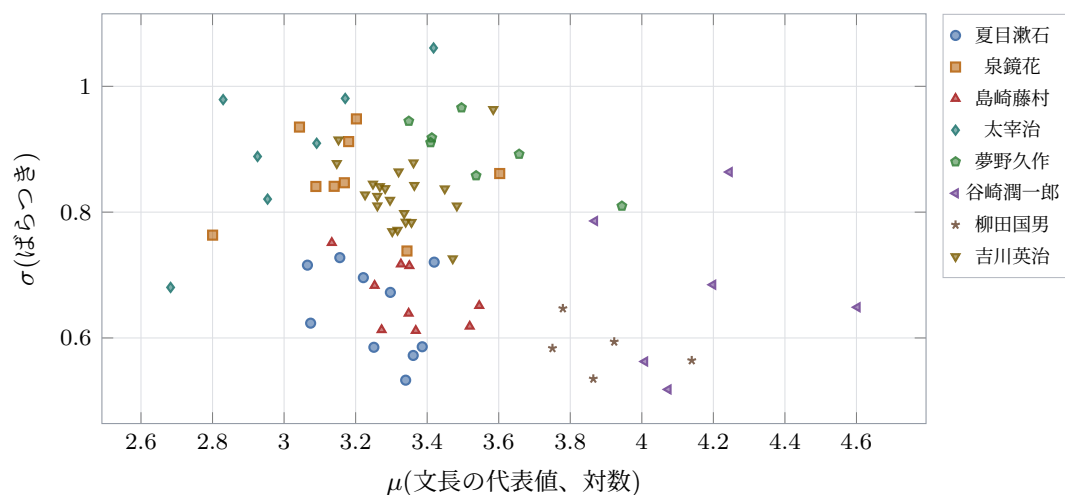


図 3.9 文長分布のパラメータ。右へ行くほど長い文、上へ行くほど長さがばらつく。

吉川英治が左下、つまり短くて揃った文を書いている。講談調の語り口を思えば納得がいく。谷崎潤一郎は右寄り、長い文を書く。

× アンチパターン 3.5 — 分布を見ずに平均だけを取る —

文長の平均を特徴にすると、外れ値に弱くなる。1つだけ 500 文字の文があれば、平均はそれだけで大きく引き上げられる。分布の形を見てから代表値を決める。対数正規なら、対数の平均のほうが安定する。

3.5.1 「長い/短い」に潰さない

もう 1 つ、やりがちな加工がある。連続した値を、ある値を境に **2つのカテゴリに潰す** ことである。文長 μ を中央値で切って「長い作品/短い作品」の 2 値にする、といった操作にあたる。

読みやすくはなる。だが情報は確実に減る。中央値のすぐ両側にある 2 作品は、値がほとんど同じなのに別のカテゴリに入り、逆に片側の端と中央値の隣は、大きく違うのに同じカテゴリになる。

どれだけ減るかを測った。字種 3 列と文長 2 列の計 5 列をランダムフォレストに渡し、74 作品の著者を 5 分割の層別交差検証で判別したときの正解率を比べる。値をそのまま渡したときの正解率は 0.635 である。同じ 5 列を、それぞれ 74 作品の中央値で 2 値に潰してから渡すと、正解率は 0.500 に下がる。0.635 を 0.135 下回っている。

境目をどこに置くかをデータを見てから決めると、事情はさらに込み入る。いちばん差が出る位置を探して切れば、その差は「本当にある差」ではなく「探したから見つかった差」になる。第9.8節の多重テストと同じ構造である。

× アンチパターン 3.6 — 連続した値を中央値で2つに割ってから分析する

「高い群」と「低い群」に分けると解釈しやすくなるが、そのために捨てた情報は戻らない。とくに、切る位置をデータから選ぶのは避ける。どうしても2値にしたいなら、分野の知識で決まる基準（法定速度、診断基準、合格点）があるときに限る。

3.6 特徴設計3 — 読点の打ち方

3つ目が本書で最も判別力の高い特徴である。

「吾輩は猫である」と書くか、「吾輩は、猫である」と書くか。読点をどこに打つかは、文法で決まっているわけではない。書き手の呼吸である。

これは本書の思いつきではない。日本語の計量文体論では、読点の打ち方に書き手の個性が出ることが1990年代に報告されている[2]。とくに読点の直前に来る文字に着目する作り方は、その後の日本語の著者識別で標準的な特徴量の一つになった[5]。本書もこれに倣い、読点の直前に来る平仮名を数える。

```
pattern = regex.compile(r"(\p{Hiragana}),")
counter = Counter(pattern.findall(text))
```

「～は、」「～が、」「～て、」「～で、」など、どの助詞や活用語尾の後で区切るかの癖が数値になる。これも作品ごとに合計1で正規化する。

そのうえで、74作品を通した平均の割合が0.1%に満たない文字は落とす。まれにしか現れない文字は、その作品にたまたま1回か2回出たかどうかで値が大きく振れる。書き手の癖ではなく偶然を測ることになるので、外しておく。0.1%という線に理論的な根拠はない。分布を見て、そこから下が明らかに偶然の領域だと判断して引いた線である。結果として47文字が残った。

もっとも、「落とす」という操作は見た目より重い。何をしていることになるかは、第3.7節で改めて扱う。

表 3.2 読点前の平仮名（上位10文字と、74作品での平均割合）

文字	平均割合	文字	平均割合
「て、」	0.175	「で、」	0.064
「は、」	0.121	「ら、」	0.057
「が、」	0.089	「も、」	0.053
「と、」	0.076	「り、」	0.023
「に、」	0.069	「し、」	0.021

「て」の後で区切るというのが最も多い。だがその割合は作家によって違う。それが判別の手がかりになる。

平均を眺めるだけでは、その「違う」がどれくらいのものか分からない。74 作品すべてを 1 枚に並べたのが図 3.10 である。

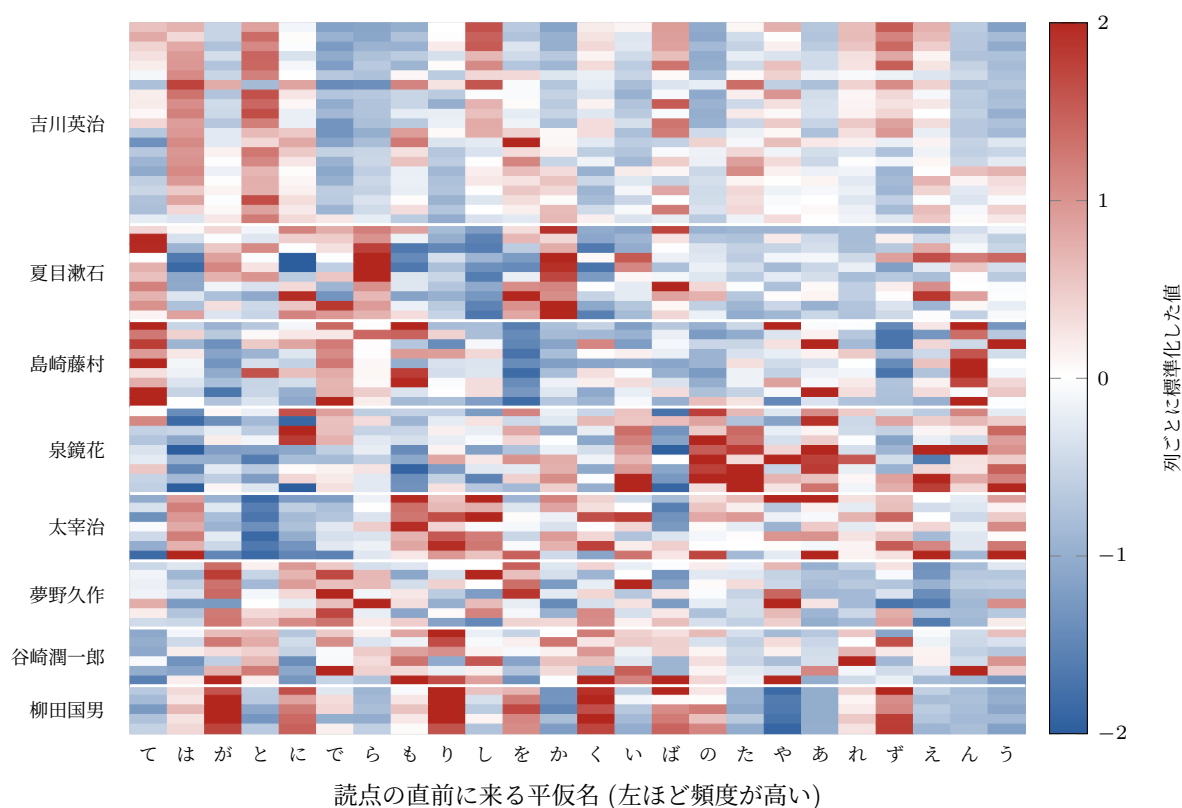


図 3.10 読点の直前に来る平仮名。1 行が 1 作品、1 列が 1 文字で、横は左ほど頻度の高い文字である (上位 24 文字だけを描いた)。色は列ごとに標準化した値で、赤はその文字をよく使う作品、青は使わない作品を表す。著者ごとに横縞が見える。

補足 この図の色は文字ごとの物差しで塗ってある

色にしたのは割合そのものではない。割合のままだと桁が違いすぎるからである。表 3.2 と同じ 74 作品の平均割合で見ると、「て、」は読点 6 回に 1 回ほど現れるのに、図の右端の「う、」は 150 回に 1 回しか現れない。この 25 倍の開きがあるものを 1 本の物差しで塗ると、「て」の列だけが真っ赤になり、残りの 23 列はどれも白くなってしまう。作家の違いが無いのではなく、「て」に合わせた目盛りでは細かすぎて見えない。

そこで、文字ごとに別の物差しを用意した。どの列でも 74 作品の平均を真ん中 (白) に置き、そこからどれだけ離れているかを赤と青の濃さにしている。学校の点数を科目ごとの平均と散らばりで測り直す偏差値と、考え方は同じである。数学の 70 点と国語の 70 点を並べても比べたことにならないのと同じで、「て、」の平均割合 0.17 と「う、」の平均割合 0.007 を並べても比べたことにならない。だから赤は「この作品はこの文字を平均よりよく使う」という意味であって、違う列どうしで濃さを比べるためのものではない。この操作を列ごとの標準化という (第 4 章で改めて扱う)。

引っかかるところがある。1 行は「読点の前に来た文字の内訳」なので、47 文字ぶんを足すと (ほぼ) 1 になる (図に描いたのは、そのうち頻度の高い 24 文字である)。どれか 1 文字の割合が上がれば、必ずどこか別の文字の割合が下がる。列ごとに平均を引く操作は、この引っ張り合いを無かったことにして、列

補足（続き）

を1本ずつ別々に扱っている。標準化した後の行は、もう内訳ではない。行と列のどちらを揃えるべきかは、第4.4.1項で正面から扱う。

この図に関しては、塗り方を変えても読み方が変わらないことを確かめてある。内訳という性質を保ったまま比べる別のやり方 (clr 変換、第3.7.3項) で塗り直すと、マス目1つ1つの色は変わる。図に描いた74作品×24文字の全マスについて、いまの図の値と clr で塗り直した図の値を突き合わせると、順位相関 (スピアマン) は0.751、赤か青かの符号が一致するマスの割合は0.775である。それでも、この後に挙げる濃い薄いの向きはすべて残り、著者ごとに横縞が出るという読みも変わらない。24列のそれぞれについて、著者の違いで説明できる分散がその列の分散全体に占める割合を求め、24列で平均すると、いまの図で0.527、clr で塗り直した図で0.554である。

同じ著者の作品が**横縞**になっているのが見どころである。吉川英治は「と、」が濃く「で、」が薄い。島崎藤村は「て、」が濃く「を、」が薄い。泉鏡花は「の、」が濃く「は、」が薄い。柳田国男は「り、」「が、」「く、」の3列が濃い (図の中では離れた位置にある)。著者が変わると縞の模様が変わり、同じ著者の中では似た模様が続く。模様の違いが、第5章以降で距離やクラスタになる。

模様がどの列で出るかは著者ごとに違う点にも注意したい。「て、」を多用するのは藤村で、太宰治や谷崎潤一郎はむしろ避けている。1つの列だけを見て「この文字が多いから誰それ」とは言えず、47列の並び全体が署名として働いている。

読点を打つ位置は文法で決まっているわけではないので、書き手はここを意識せずに選んでいる。その無意識の選択が、47個の数値の並びとして残る。

○ 定石 3.3 — 著者を当てる特徴に求める3条件 —

次の3条件は**書いた人を当てる**という課題に限ったもので、何を当てたいかが変われば条件も変わる。

1. 内容に依存しない — どんな題材を書いても同じように出る
2. どの文書にも現れる — 比較できる
3. 書き手が意識していない — 意図的に変えにくい

読点の癖は、3つを単語の頻度よりよく満たす。ただし完全ではない。たとえば「あっ、」のような感動詞がらみの読点は、驚きや悲鳴の多い場面、つまり**内容**に引きずられるし、会話の多い文体かどうかにも左右される。3条件は満たすか満たさないかの二択ではなく程度の問題で、候補どうしの満たし方を比べて特徴を選ぶ。

目的が変われば条件も変わる。ジャンルを当てたいなら1は逆になり、題材によって変わる特徴のほうがよい (第11章)。3が重要になるのは、**当てられる側に隠す動機がある**課題 (著者判定、生体認証、不正検知) に限られる。一般の形に直せば、特徴に求めるものは「当てたいラベルとともに変わり、それ以外の要因では変わらず、どの行でも測れること」である。上の3条件は、その著者判定版にとどまる。何が邪魔者かは目的が決める—残すか捨てるかを問いの側が決めるという、前処理の節で見た原理と同じである。

3種類の特徴がそろった。表にまとめる前に、比率という数値そのものが持つ厄介な性質を見てお

きたい。いま作った 52 列のうち 50 列が、厄介な性質を抱えている。

3.7 組成データ — 比率を扱うときの落とし穴

合計が一定になるデータを**組成データ** (compositional data) という。比率、割合、パーセント、シェア、得票率、予算の配分。身の回りの数値の多くが組成データである。

本書の特徴量も例外ではない。字種比率の 3 列は合計 1、読点前平仮名の 47 列もほぼ合計 1 になる (「ほぼ」なのは、まれな文字を落としたぶん行の合計が 0.97 から 1.00 の間になるためである)。52 列のうち **50 列が組成データ** である。残る 2 列は文長の μ と σ で、これらは合計の制約を持たない。組成データには、普通の数値と同じつもりで扱うと足をすくわれる性質がある。

3.7.1 合計が固定されると、無関係な成分にも負の相関が出る

74 作品それぞれについて平仮名の比率と漢字の比率を並べ、両者の相関係数を測ると $r = -0.915$ になる。ほとんど直線に乗る強い負の相関である。

素直に読めば「漢字を多用する作家は平仮名を控える」となる。文体の発見に見える。

だが、この負の相関だけでは、**文体の性質を示したことになる**。

平仮名 + 片仮名 + 漢字 = 1 だから、漢字 = 1 - 平仮名 - 片仮名 である。片仮名の比率は小さく、変動も小さい (近代文学 74 作品の平均は 0.013)。すると漢字はほぼ「1 - 平仮名」に等しく、相関が -1 に近づくのは**足し算の帰結**にすぎない。

3 成分すべての相関を見ると、偏りがはっきりする。

表 3.3 74 作品で測った字種比率どうしの相関係数。非対角の 3 つはすべて負である

	平仮名	片仮名	漢字
平仮名	+1.000	-0.250	-0.915
片仮名	-0.250	+1.000	-0.163
漢字	-0.915	-0.163	+1.000

表 3.3 の非対角にある 3 つの相関係数の和は -1.328 である。偶然ではない。

発展 合計が固定された比率どうしには必ず負の相関が出る

$x_1 + \dots + x_D = c$ (定数) とする。任意の j について共分散は第 2 引数について線形だから、

$$\sum_{k=1}^D \text{Cov}(x_j, x_k) = \text{Cov}\left(x_j, \sum_k x_k\right) = \text{Cov}(x_j, c) = 0. \quad (3.8)$$

定数との共分散は 0 なので、**共分散行列の各行の和はちょうど 0** になる。74 作品の字種 3 列から作った共分散行列で確かめると、3 つの行和の絶対値の最大は 2.2×10^{-19} であった。理論値 0 との差は浮動小数点の誤差の範囲にとどまる。

$k = j$ の項を分けると

$$\sum_{k \neq j} \text{Cov}(x_j, x_k) = -\text{Var}(x_j) < 0. \quad (3.9)$$

発展 (続き)

つまり各成分は、他の成分との共分散の和が必ず負になる。どんなデータであろうと、合計が固定されている限り成り立つ。

とくに $D = 2$ なら和の項が 1 つしかないので $\text{Cov}(x_1, x_2) = -\text{Var}(x_1)$ 、すなわち $r = -1$ が恒等的に成り立つ。2 成分の組成データの相関を測るのは、何も測っていないに等しい。

表 3.3 の非対角の和 -1.328 にも根拠がある。もし 3 成分の分散がすべて等しければ、上の行和ゼロの式を相関で書き直して、非対角の相関の和 (3 つ) はちょうど $-D/2 = -1.5$ になる。74 作品での実際の標準偏差は平仮名 0.042、片仮名 0.017、漢字 0.041 と完全には揃っていないので、 -1.5 を 0.172 上回る -1.328 が観測される。

× アンチパターン 3.7 — 比率どうしの相関をそのまま解釈する

「A の比率と B の比率に強い負の相関があった」という報告は、それだけでは文体や行動の性質を示したことになる。合計が固定されていれば、制約だけで負の共分散が生まれるからである (3 成分以上なら、どの 2 成分も必ず負になるわけではない。負の向きに引かれる、というのが正確な言い方である)。

売上構成比にも、時間の使い方の割合にも、支持率にも同じ罫がある。

3.7.2 一部だけ取り出すと答えが変わる

もっと不気味な性質がある。

字種から片仮名を捨て、平仮名と漢字だけの 2 成分にして、合計が 1 になるよう割り直してみる。元のデータは 1 文字も変えていない。ただ片仮名を見ないだけである。

平仮名と漢字の相関係数は -0.915 から -1.000 に変わる。2 成分の組成データでは相関係数が理論上 -1 になるので、74 作品の点は完全な直線に乗ってしまう。

これを部分組成の非整合性 (subcompositional incoherence) という。組成データでは、どの成分を分析に含めたかによって、成分どうしの関係が変わってしまう。

補足 まれな列を落とすのも「部分組成」である

第 3.6 節で読点前の平仮名を数えたとき、74 作品を通した平均の割合が 0.1% に満たない文字を落として 47 列にした。あれも部分組成を取る操作である。落とした後は再正規化していないので、47 列の行和は厳密には 1 でなく、作品によって 0.97 から 1.00 の間になる。

これは手抜きではなく判断である。落とした列の割合の合計は、どの作品でも 3.3% 以下にとどまり、距離や分類への影響は小さい。一方、比率どうしの相関を論じる場面では、どの列を落としたかで数値が変わることを本項で見たとおりである。自分がいまどちらの使い方をしているかを意識しておきたい。なお、成分どうしの比は列を落としても変わらない。対数比 $\ln(x_i/x_j)$ を直接見る分析は、列を落としても揺らがない。clr や ilr の座標値そのものは、どの成分を含めたかで変わる (次項の「clr 変換の性質と限界」を参照)。

図 3.11 の (1) と (2) を見比べてほしい。片仮名という、比率にして数パーセントの成分を外しただけで、散らばりが直線につぶれる。

3 枚を貫いて、赤い丸の 1 点を目で追ってみてほしい。谷崎潤一郎『盲目物語』である。盲目の按

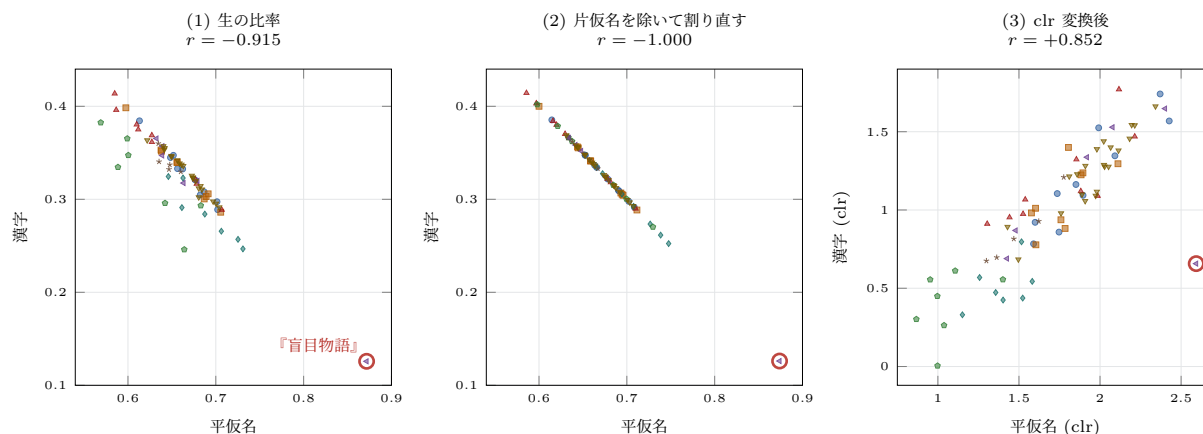


図 3.11 同じ 74 作品を 3 通りに描いた。点の位置も相関も変わるが、元の文章は何一つ変えていない。(2) は完全な直線、(3) では相関の符号が逆転している。色とマーカーは図 3.3 と同じ。赤い丸は谷崎潤一郎『盲目物語』で、3 枚とも同じ作品を指している。

摩の語りをそのまま写すために、谷崎が意図して漢字を抑え仮名書きに寄せた作品で、平仮名比率が 0.8717 と 74 作品の中で最も高く、表 3.1 の平均 0.660 から離れた飛び地になっている。(1) で外れていた点は、(2) でも (3) でも外れたままである。変換は個々の作品の個性を消さない。消えたり現れたりしているのは、点どうしの見かけの関係のほうである。

3.7.3 clr 変換 — 比を見る

組成データの扱い方は Aitchison が体系化した [14]。もともと岩石の化学組成を扱う地質学で発展し、いまではマイクロバイーム (細菌叢) 解析の標準になっている [17]。細菌の組成はシーケンサの読み取り総数で割った比率でしか得られないから、本節の問題がそっくり現れる。考え方は単純で、**比率そのものではなく成分どうしの比を見る**。

「比率が 0.1 増えた」という言い方は、元が 0.05 のときと 0.5 のときで意味がまるで違ってしまうが、「2 倍になった」ならどちらの場合でも同じことを指す。だから差ではなく比で見たい。

そこで比の**対数をとると**、割り算が引き算に変わり ($\ln(a/b) = \ln a - \ln b$)、普通の距離や相関や平均のように引き算しかできない道具を、比を見る目的にそのまま使えるようになる。

比を取る相手、つまり基準も決めなければならない。特定の 1 成分を基準にすると、どれを選ぶかで結果が変わってしまう。そこで全成分を平等に扱うため、**幾何平均** (全部を掛け合わせて成分数の根を取った値) を基準にする。こうして作る変換を **clr 変換** (centred log-ratio transformation) という。

$$\text{clr}(x)_j = \ln \frac{x_j}{g(x)}, \quad g(x) = \left(\prod_{k=1}^D x_k \right)^{1/D} \quad (3.10)$$

$g(x)$ が幾何平均、 D が成分の数である。式全体は「その成分が、全体の平均的な水準に比べて何倍あるか」の対数を表す。変換後は合計 1 の制約から解放され、普通のユークリッド空間の点として扱える。clr 変換したあとのユークリッド距離を **Aitchison 距離**という。

図 3.11(3) がその結果である。平仮名と漢字の相関係数は -0.915 から $+0.852$ へ、符号ごと逆転した。

逆転の理由は、共通の分母 $g(x)$ にある。値の小さい成分は、対数スケールで見た変動が大きくなりがちである。本書の 3 成分では片仮名の対数スケールでの変動が桁違いに大きいので、 $\ln g(x)$ の変動は片仮名の項に支配される。つまり $g(x)$ は事実上、片仮名の多寡を映す量になっている。

そうすると、平仮名も漢字も「片仮名に比べて何倍か」を見ていることになる。片仮名の少ない作品では、平仮名も漢字も相対的に多い。だから両者は同じ向きに動き、正の相関になる。合計 1 という縛りの下では負に見えていた関係が、比で見ると逆の姿を見せる。

発展 clr 変換の性質と限界

clr には次の性質がある。

組成の等価性: x とその定数倍 λx は clr で同じ点に写る。生の個数から比率に直す操作が不要になる。

部分組成の整合性: 成分を一部除いても、残した成分どうしの比は変わらない。clr でも、2 成分の座標の差 $\ln(x_i/g) - \ln(x_j/g) = \ln(x_i/x_j)$ はその対数比に厳密に一致する。ただし、各座標値は全成分の幾何平均 $g(x)$ を基準にしているため、成分を除けば $g(x)$ が変わり、座標値も変わる。座標間の相関や、次元を減らした後の距離まで保存されるわけではない。実際、平仮名と漢字の相関は、3 成分の clr では $+0.852$ だが、片仮名を除いた 2 成分の clr では -1.000 になる (2 成分の clr は $\pm \frac{1}{2} \ln(x_1/x_2)$ で、符号が逆なだけの 1 本の量だからである)。一方、対数比 $\ln(x_{\text{平仮名}}/x_{\text{漢字}})$ 自体は 3 成分でも 2 成分でも同じ値である。ilr は、同じ組成を直交座標で表す方法であり (次項)、座標値が部分組成で保存される点で clr に勝るわけではない。

制約: clr 変換後のベクトルは成分の和が必ず 0 になる。 $\sum_j \ln(x_j/g) = \ln(\prod_j x_j/g^D) = 0$ だからである。 D 次元に見えて実質は $D-1$ 次元で、共分散行列は特異になる。線形回帰など、逆行列を必要とする手法にそのまま入れると破綻する。これを避けたければ、次に述べる ilr 変換を使う。

3.7.4 ilr 変換 — バランスとして読む*

clr の「和が 0」という制約を取り除くには、 $D-1$ 本の直交した対数比を座標に選べばよい。これを **ilr 変換** (isometric log-ratio transformation) という。

座標の選び方は一つではないが、成分を 2 つのグループに割っていく方法が読みやすい。字種の 3 成分なら、まず「かな漢字のバランス」、次に「片仮名とそれ以外のバランス」と割ると

$$b_1 = \frac{1}{\sqrt{2}} \ln \frac{x_{\text{平仮名}}}{x_{\text{漢字}}}, \quad b_2 = \sqrt{\frac{2}{3}} \ln \frac{x_{\text{片仮名}}}{\sqrt{x_{\text{平仮名}} x_{\text{漢字}}}} \quad (3.11)$$

となる。 b_1 は平仮名と漢字のどちらに寄った文体か、 b_2 は片仮名を地の文に対してどれだけ使うかを表す。 b_1 や b_2 のような座標を **バランス** と呼ぶ。平方根の係数は座標系の直交性を保つためのもので、これがあるおかげで **ilr 座標上のユークリッド距離は Aitchison 距離にちょうど一致する**。74 作品で両者を突き合わせると、差は 10^{-15} の桁にとどまり、浮動小数点の誤差しかない。

図 3.12 を図 3.3 と見比べよう。生の比率では片仮名は数パーセントに押し込められて、著者差はほとんど見えなかった。対数比にすると、「小さいが情報を持つ」成分が主役になる。夢野久作がひとかたまりで上に浮いているのは、擬音や台詞の片仮名書きを多用する文体のためである。片仮名比率の差でいえば数パーセントの違いが、比で見れば 10 倍以上の開きだったことになる。

三角図 (図 3.7) で対照的だったライトノベル 2 シリーズは、ここでは上下に引き裂かれる。『本好

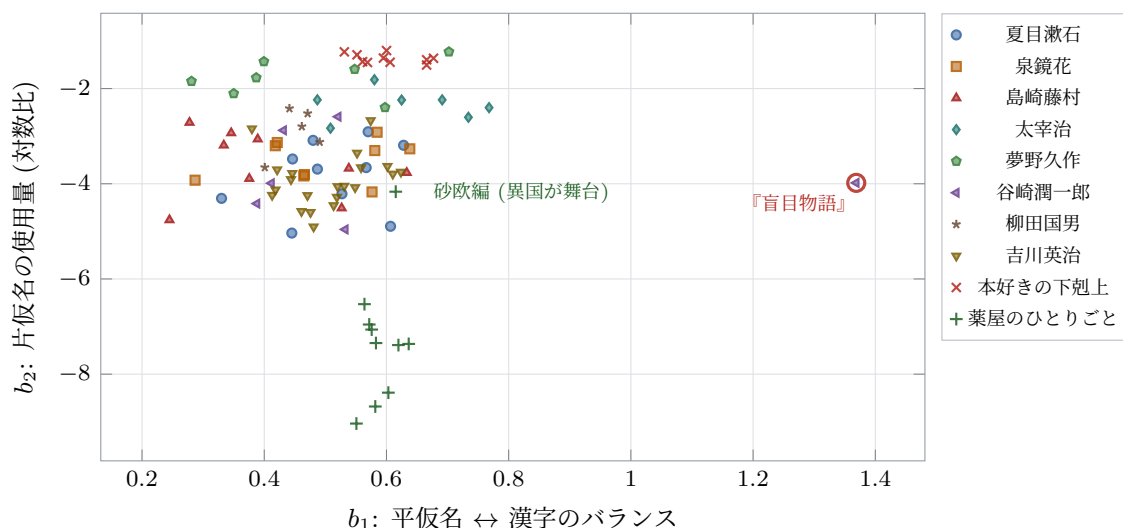


図 3.12 ilr のバランス座標に載せた 74 作品とライトノベル 2 シリーズ。生の比率 (図 3.3) では潰れて見えなかった片仮名の使い方の差が、縦軸いっぱい広がる。

きの下剋上』は夢野久作のさらに上、『薬屋のひとりごと』は全著者のはるか下である。三角図では「近代文学の集団に混ざって」見えた薬屋が、対数比では一目で異質だと分かる。微量成分の癖は、比で見て初めて特徴になる。

薬屋の 10 点だけでも縦に大きくばらついている。上に飛び出した 1 点は異国 (砂欧) を舞台にした区間で、地名と人名の片仮名がここだけ増える。比で見ると、舞台の違いが他の著者どうしの差より大きく出る。

もう一步進むと、対数が取れなくなる。図の各点は連載を 8 万～16 万字ずつに区切った区間だが、これを 1 話ずつまで細かくすると、『薬屋のひとりごと』の 395 話のうち 370 話は片仮名が **1 文字も出てこない**。厳密にゼロである。同じ作品でも、どの粒度で数えるかによってゼロが現れる。それをどう扱うかが次項の主題である。

補足 小さい成分ほど比で見ると情報が出る

生の比率のままだと、大きい成分 (平仮名) の増減が図の大半を占め、小さい成分 (片仮名) の増減はスケールに埋もれる。対数比はどの成分の「2 倍」も同じ距離として扱うので、小さい成分の癖を拾える。微量成分こそが決め手になる分析 (地球化学の微量元素、細菌叢の希少種) で対数比変換が標準になっているのは、小さい成分の癖を拾えるからである。

3.7.5 ゼロがあると対数が取れない

式 (3.10) には弱点がある。 $x_j = 0$ だと $\ln 0 = -\infty$ になる。

そして実データにゼロは頻繁に現れる。組成データ 50 列 $\times$ 74 作品 = 3700 マスのうち、255 マス (6.9%) がゼロだった。ゼロはすべて読点前平仮名の列に出る。たとえば「っ、」は 74 作品中 42 作品で一度も現れない。「あっ、と叫んだ」のような書き方をしない作家のほうが多い。74 作品のうち 55 作品が、どこかにゼロを含んでいる。

字種比率でも起きる。『葉屋のひとりごと』を1話ずつ数えると、395話のうち370話で片仮名比率が**完全なゼロ**になる。中国風の後宮が舞台なので、外来語が一語も出てこない話のほうが多い。

3.7.6 ゼロには数え足りないものと、構造として出ないものがある

ゼロを埋める話に入る前に、片付けておくことがある。**そのゼロが何を意味しているか**である。

「っ、」に戻る。「っ、」が一度も現れなかった作品の読点の本数は、中央値で3026本である。これだけの読点がありながら1回も出ないのが、数え足りないせいなのか、使わないという癖なのかを分けたい。「っ、」を実際に使う著者では、読点408本に1回ほどの割合で現れる。同じ割合なら、読点3026本のあいだに1回も現れない確率は 6×10^{-4} である。小さいが、ありえない話ではない。

ところが「っ、」がゼロの42作品には、読点を15485本含むものがある。同じ割合なら、15485本の読点のあいだに1回も現れない確率は 3×10^{-17} 。事実上ゼロである。ゼロの由来は、数え足りたかどうかではない。**この著者は「あっ、」という書き方をしないという事実の反映である。**

同じ0という数字でも、由来が2通りある。

- **数え上げゼロ** (count zero) — 本当は正の割合で起きるが、観測した量が足りずに1回も出なかった。量を増やせば(粒度を粗くすれば)ゼロは消える
- **構造ゼロ** (essential zero) — その書き手・その条件では原理的に起きない。いくら量を増やしてもゼロのまま

見分け方は素朴である。**粒度を粗くしてゼロが消えるかどうか**を見る。『葉屋のひとりごと』の片仮名比率は、1話ずつ数えると395話のうち370話がゼロだが、10区間にまとめると9区間でゼロが消える。数え上げゼロである*²。「っ、」は、15485本の読点を集めてもゼロのままだった。構造ゼロに近い。

区別が要る理由は、扱いが逆になるからである。数え上げゼロは「観測できなかった小さな正の値」なので、小さな値で埋めるのが理にかなう。構造ゼロを同じように埋めると、**存在しないものを存在することにしてしまう**。しかもゼロであること自体が著者の癖なのだから、埋めた瞬間にその情報は δ という共通の値に均される。

どれくらいの情報かは測れる。読点前47列を「ゼロか否か」の0/1に潰して比率の値を全部捨て、74作品を階層的クラスタリングにかけて著者数と同じ8クラスタに切る。できたクラスタと正解の著者の分かれ方との一致をARI*³で測ると0.355になる。でたらめに分けたときの期待値0よりは高い。数値を捨てても著者がある程度分かるのだから、ゼロの位置は無視できる情報ではない。

○ 定石 3.4 — ゼロを埋める前にそのゼロの由来を確かめる

ゼロの置換は、対数比変換を使うために必要になる操作であって、それ自体が望ましい処理ではない。埋める前に3つ確かめる。

1. **そもそも埋める必要があるか。**ゼロを埋めるのは、対数を取るときだけである。生の比率

*² 残る1区間だけは、10万字を超える分量でも片仮名が1文字も出なかった。粒度をもう一段粗くすれば消えるともてよい。

*³ Adjusted Rand Index(調整ランド指数)の略。作られたクラスタが正解の分かれ方とどれだけ一致するかを表す指標である。完全に一致すれば1、でたらめに分けたときの期待値が0になるよう調整してある。定義は第5.5節で述べる。

のまま距離を測るなら、決定木やランダムフォレスト (第 7 章) に渡すなら、ゼロは 0 のままでよい

2. 粒度を粗くしてゼロが消えるか。消えるなら数え上げゼロで、小さな値で埋めてよい。消えないなら構造ゼロを疑う
3. ゼロの位置に情報があるか。「ゼロか否か」の 0/1 だけでどれだけ分けられるかを測っておく。分けられるなら、埋めることでその情報が失われる

構造ゼロが多い列は、埋めるより次のどれかを選ぶ。

- その列を落とし、残りで**部分組成**を作り直す
- 「ゼロか否か」を別の 2 値の列として残し、値の列は正の行だけで扱う
- ゼロの多い列をまとめて 1 つの成分に合算する

本書では、読点前 47 列に出た 255 個のゼロを乗法的置換で埋めた。粒度を粗くすると大半が消えるので、多くは数え上げゼロだと判断したためである。埋める扱いと、ゼロを含む列を落として部分組成にする扱いの両方について、clr 変換したうえで 74 作品を階層的クラスタリング (Ward 法) にかけて、8 クラスに切ったときの ARI を比べた。ゼロを含まない 17 列だけの部分組成では 0.756、47 列すべてを埋めた場合は 0.740 で、差は 0.016 である。どちらを選んでも結論は変わらなかったという確認まで含めて報告するのが安全である。

発展 乗法的置換の式

数え上げゼロだと判断したうえで埋めるなら、**残りの成分を縮めて合計 1 を保つ**のが要点である。

$$\tilde{x}_j = \begin{cases} \delta & (x_j = 0) \\ x_j \left(1 - \frac{z\delta}{\sum_{k: x_k > 0} x_k} \right) & (x_j > 0) \end{cases} \quad (3.12)$$

z はその行のゼロの個数である。単に δ を足すだけでは合計が 1 を超え、組成データでなくなってしまう。

δ の決め方は、ゼロの出方によって変わる。化学分析のように**測定器の下限**があってゼロになる場合 (丸めゼロ) は、検出限界の 0.65 倍程度が使われる [15]。本書のように数え上げでゼロになる場合は検出限界という概念がないので、観測された最小の正の値の 0.65 倍を代用した。行ごとの観測数が違うなら、数え上げゼロ専用のベイズ的な置換法もある [16]。

いずれにしても δ は恣意的な値である。ゼロの割合が大きいときは、置換の値が結果を左右する。 δ を 10 倍・1/10 倍して結論が変わらないか確かめるとよい。

× アンチパターン 3.8 — 対数比を保ちたいのに、比率へ一律に 1 を足す —

$\ln(1+x)$ そのものが悪いわけではない。問題になるのは、**比率どうしの対数比を使いたいのに、ゼロを避けるため一律に 1 を足しても、元の対数比が保たれると考える場合**である。

例えば 0.001 と 0.01 は 10 倍違う。対数の差は $\ln(0.01/0.001) \approx 2.303$ だが、1 を足してから差は $\ln(1.01/1.001) \approx 0.009$ に縮む。小さな x では $\ln(1+x) \approx x$ なので、対数比を測る

つもりが、ほぼ元の比率の差を測ることになる。

一方、カウントデータにゼロが含まれ、大きな値と小さな値の差を圧縮したい場合には、 $\ln(1+n)$ を使える。この用途では、元の対数比を保たないこと自体は問題にならない。 n が十分大きければ $\ln(1+n)$ と $\ln n$ は近いが、小さなカウントでは加えた 1 の影響が残る。

ただし、 $\ln(1+x)$ では、単位を変えると加えた 1 の影響も変わる。例えば割合の 0.01 を百分率の 1 に直してから変換すると、1 を足す前後の値の比が変わる。比率の対数比を調べる処理と、カウントの大きな値を圧縮する処理を区別して使う。

3.7.7 clr 変換で効果量もクラスタも正解率も上がる

理屈は分かった。残るのは、実際に精度が上がるかどうかである。

50 列の組成データに clr 変換を施し、本書のあとの章で使う 3 つの評価をそのまま実行した。どれも対象は 74 作品である。

表 3.4 clr 変換の効果。74 作品の組成データ 50 列で測った。距離の効果量はユークリッド距離で測った値、ARI は Ward 法の階層的クラスタリングを 8 クラスタに切ったときの値で、この 2 つは全作品を一度に変換して測った (探索なので構わない)。正解率はランダムフォレストを 5 分割の層別交差検証にかけたときの値で、ゼロ置換量 δ と標準化の平均・標準偏差は各分割の訓練側だけから決めた。最下行は、字種 3 列と読点前 47 列を別々の組成として変換した場合である

前処理	距離の効果量	ARI	正解率
標準化のみ	1.354	0.595	0.918
clr 変換	1.389	0.655	0.960
clr 変換 + 標準化	1.418	0.696	0.960
群ごとの clr 変換 + 標準化	1.447	0.637	0.932

「距離の効果量」は、同じ著者どうしの距離と別の著者どうしの距離がどれだけ離れているかを標準偏差を単位に測ったものである (第 4 章)。ARI はクラスタリングが著者の分かれ方とどれだけ一致するかの指標 (第 5 章)、正解率はランダムフォレストによる著者判定を 5 分割の層別交差検証で測った値 (第 7 章) である。3 つとも値が大きいほどよい。

3 つとも改善している。標準化のみの行と clr 変換 + 標準化の行を比べると、距離の効果量は 1.354 から 1.418 へ 0.064、クラスタリングの ARI は 0.595 から 0.696 へ 0.101、著者判定の正解率は 0.918 から 0.960 へ 0.042、それぞれ上がった。正解率は clr 変換だけで 0.960 になり、標準化を重ねても同じ値である。標準化を重ねて上がるのは、距離の効果量と ARI のほうである。

50 列をひとまとめに変換するか、群ごとに変換するかは判断の分かれる点である。字種 3 列と読点前 47 列は分母が違う (全文字数と読点の数) ので、別々の組成として扱うほうが筋は通る。ところが最下行のとおり、群ごとに変換すると距離の効果量は 1.447 へさらに上がるが、ARI は 0.637、正解率は 0.932 へ下がる。ひとまとめに変換したときの幾何平均は、字種と読点前の両方をまたいだ基準になり、作品全体の「地の文の密度」のようなものを吸収する。どちらが正しいというより、何を基準にした対数比を見たいかで決まる。本書では、下流の 3 つの評価がそろって上がったひとまとめ

の変換を採用した。

補足 本書が `clr` ではなく生の比率を使い続ける理由

表 3.4 を見れば `clr` のほうが良い。にもかかわらず、第 4 章以降では標準化した生の比率を使い続ける。理由は 2 つある。一つは、ゼロ置換という恣意的な操作が入ること。 δ の選び方で結果がわずかに変わるので、手法そのものの比較をしたい場面では余計な変数になる。しかも δ は最小の正值から決めるので、データを見て決める量である。未知の作品への性能を測るときは、 δ も訓練側だけから決めなければならない (表 3.4 の正解率はそうしてある。第 9 章の前処理の漏れを参照)。

もう一つは、読み手が生の値を頭の中で追えること。「平仮名の比率が 0.68」は文章のイメージと結びつくが、「平仮名の `clr` 変換後の値が 1.42」はそうではない。

生の比率を選ぶことは正しさと分かりやすさの取り引きである。現実の分析でも同じ判断を迫られる。取り引きをしていると自覚したうえで選べばよい。避けたいのは、黙って生の比率を使ったまま、負の相関を文体の発見として報告してしまうことである。

3.8 できあがった表

3 種類の特徴を横に並べると、74 行 $\times$ 52 列の表になる。

表 3.5 特徴量テーブルの一部

著者	題名	平仮名	漢字	文長 μ	文長 σ	て	が
太宰治	斜陽	0.731	0.247	3.17	0.98	0.172	0.058
夏目漱石	こころ	0.701	0.297	3.34	0.53	0.190	0.100
吉川英治	三国志_桃園の巻	0.673	0.325	3.25	0.84	0.170	0.072

できあがった表が本書の残りすべての出発点である。以後、テキストそのものには戻らない。74 個の点が 52 次元の空間に散らばっているという見方だけで進む。

補足 本章でやってきた作業の名称 — 特徴量エンジニアリング

本章の作業には名前がある。特徴量エンジニアリング (feature engineering)、生のデータをモデルが扱える数値の表に加工する工程である。振り返ると、特徴量エンジニアリングの定番の道具を一通り使っている。数える対象を領域の知識で選ぶ (読点前の平仮名)、大きさを比率に直す (正規化)、分布の形を見てから少数の数に要約する (対数正規分布の μ と σ)、まれすぎる列を落とす。第 2 章の対数目盛りも、第 3.7 節の対数比も、同じ仲間である。

深層学習は特徴を生データから自動で学ぶ (表現学習) ので、特徴量エンジニアリングはもう不要になった、と言われることがある。画像や音声ではおおむねそのとおりである。だが本書のような表形式・小標本のデータでは、いまま「手作りの特徴 + 木のモデル」が強いことが多い。モデルを取り替えるより、良い特徴を 1 つ足すほうが成績を大きく引き上げるという場面は、実務にも多い。

これは言い切りにできる話ではない。本章の 52 列と、大規模言語モデルが作る表現とを実際に突き合わせるのは第 10.1 節である。比べるには分類器と交差検証が要るので、そこまで待つことになる。

補足 本書が形態素解析を使わない理由

単語の頻度を使えば、もっと精度は上がるかもしれない。だが、次の理由で本書では使わない。

- 環境構築が要る。辞書のインストールでつまづく人が必ず出る
- 解析には誤りが混ざる。古い文語調のテキストでは特にそうである
- 単語を使うと**内容語が紛れ込む危険**が高まる

文字レベルの特徴だけで、著者判別は十分な精度に達する。それを本書で確かめる。

3.9 章末の課題

1. 図 3.3 で、片仮名の比率を縦軸にすると何が見えるか。近代文学とライトノベルを比べるとどうなるか予想せよ。
2. 文長を「句点で区切った文字数」と定義した。会話文の「」をどう扱うべきか。定義を変えると結果は変わるか。
3. 読点前の平仮名のほかに、著者の癖が出そうな量を 3 つ挙げよ。それぞれが上記の 3 条件を満たすか検討せよ。
4. 特徴の数を 47 から 10 に減らすとしたら、どの文字を残すか。判断の基準を述べよ。
5. 表 3.5 の『斜陽』の行 (平仮名 0.731、漢字 0.247。片仮名は残りとする) に `clr` 変換を手計算で適用し、3 成分の和が 0 になることを確かめよ。(ヒント: 幾何平均の対数は、対数の算術平均に等しい。先に 3 成分の $\ln$ を取ってしまうのが早い。)

第 4 章

距離を測る

4.1 「似ている」を数式にする

74 作品が 52 次元の空間に散らばっている。ここから「似た作品」を見つけたい。

そのためには、2 つの作品がどれだけ離れているかを 1 つの数で表す必要がある。その数を距離という。

距離は好きに定義してよいわけではない。次の 4 条件を満たすものを距離と呼ぶ。

$$d(x, y) \geq 0 \quad \text{非負性} \quad (4.1)$$

$$d(x, y) = 0 \iff x = y \quad \text{同一性} \quad (4.2)$$

$$d(x, y) = d(y, x) \quad \text{対称性} \quad (4.3)$$

$$d(x, z) \leq d(x, y) + d(y, z) \quad \text{三角不等式} \quad (4.4)$$

最後の三角不等式は「A と B が近く、B と C が近ければ、A と C もそれなりに近い」ことを保証する。近傍探索の高速化 (候補の枝刈り) や、一部のアルゴリズムの理論保証はこの性質に頼っている。ただし、4 条件を守らない指標が実務で使えないわけではない。本章の実験で最もよく著者を分けるのは、実は三角不等式を満たさない指標である。この折り合いは第 4.7.2 項で付ける。

4.2 ミンコフスキー距離

最もよく使われる距離の族である。

$$d_p(x, y) = \left(\sum_{j=1}^n |x_j - y_j|^p \right)^{1/p} \quad (4.5)$$

p を変えると性格が変わる。

言葉で説明するより、**原点から等距離にある点の集合** (単位球) を描くほうが分かりやすい。図 4.1 に示す。

$p = 1$ のとき、原点から距離 1 の点はひし形の上に乗る。(1, 0) も (0.5, 0.5) も同じ距離 1 である。 $p = 2$ では円になり、(0.5, 0.5) の原点からの距離は $\sqrt{0.5} \approx 0.71$ になる。(1, 0) の距離 1 より 0.29 小さい。

つまり p が小さいほど、複数の次元に分散した差を重く見る。 p が大きいと、一つの次元の大きな差が支配的になる。

表 4.1 p の値による違い

p	名前	性格
1	マンハッタン距離	各次元の差の絶対値を足すだけ。外れ値の影響が小さい
2	ユークリッド距離	直線距離。最もよく使われる
4	—	大きい差を強調する
∞	チェビシェフ距離	最大の差だけで決まる

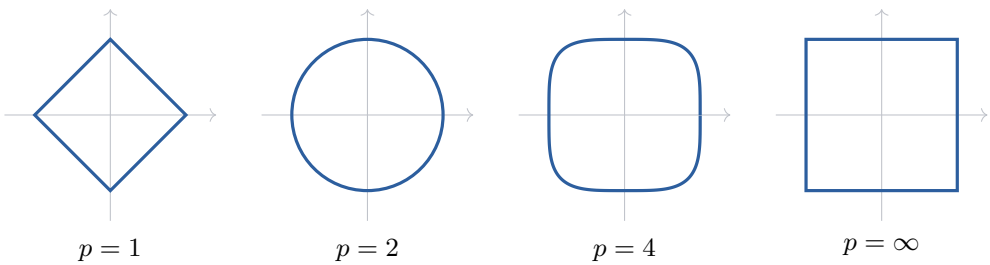


図 4.1 原点からの距離が 1 になる点の集合。 p が大きくなるほど正方形に近づく。

4.2.1 マンハッタン距離の直感

$p = 1$ を「マンハッタン」と呼ぶのは、碁盤の目の街路を歩くとき、斜めには進めないからである。

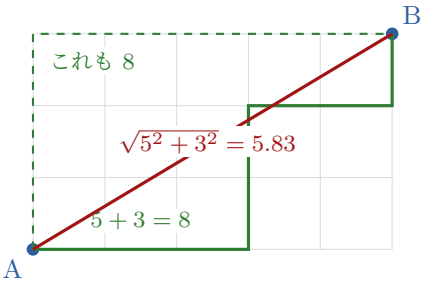


図 4.2 マンハッタン距離。街路をどう曲がっても合計は同じ 8 になる。

図 4.2 で大事なのは、経路が違っても距離が同じという点である。右に 5、上に 3 進むという内訳は変わらない。ユークリッド距離が「一直線の長さ」なのに対し、マンハッタン距離は「各方向にどれだけ動いたかの合計」である。

2 つの距離のもう 1 つの違いは、差を 2 乗するかどうかである。そして、この違いが外れ値への強さを分ける。

いま 2 つの作品を 52 次元で比べているとする。51 個の特徴はほぼ同じで、1 個だけ極端にずれているとしよう。その大きなずれを δ と書く。

$$d_1 \approx (\text{小さな差の和}) + \delta \quad (4.6)$$

$$d_2 \approx \sqrt{(\text{小さな差の2乗和}) + \delta^2} \approx \delta \quad (4.7)$$

2つ目の式の後半の $\approx$ を確かめておく。 δ が他の差より桁違いに大きければ、それを2乗した δ^2 はさらに桁が離れる。根号の中身はほとんど δ^2 で占められ、残りの51項は誤差のようなものになる。だから d_2 は δ そのものにほぼ等しくなる。

つまりユークリッド距離では、たった1つの大きなずれが他の51次元の情報を押し流してしまう。マンハッタン距離のほうは δ を2乗しないので、あくまで52項のうちの1項として足されるだけで済む。

補足 1 文字の誤植が作品を孤立点に変えることがある

テキスト分析では、1文字の誤植や記号の混入で、1つの特徴だけが極端な値になることがある。

そのとき、ユークリッド距離だとその作品が「どの作品からも遠い孤立点」になってしまう。マンハッタン距離ならその影響は限定的である。

第4.6節で $p=1$ が $p=2$ よりよく分けている理由は、本書では測り分けていない。誤植への強さも候補の1つだが、高次元では1つの列にたまたま大きな差が出やすいこと(第9.13節)も同じ向きに働くので、どちらがどれだけ寄与したかは分からない。

4.3 コサイン距離 — 向きだけを見る

ミンコフスキー距離とは発想の違う測り方がある。ベクトルの**向き**だけを比べ、長さを無視する。

$$\cos \theta = \frac{x \cdot y}{\|x\| \|y\|} = \frac{\sum_j x_j y_j}{\sqrt{\sum_j x_j^2} \sqrt{\sum_j y_j^2}} \quad (4.8)$$

これは2つのベクトルのなす角の余弦である。同じ向きなら1、直交していれば0、正反対なら-1になる。距離にするには1から引く。

$$d_{\cos}(x, y) = 1 - \cos \theta \quad (4.9)$$

4.3.1 長さを残すと、別の作家のほうが近くなる

図4.3で違いを見る。

AとBは同じ作家が書いた長編と短編だとする。すべての単語の出現回数が、長編では単純に多くなる。向きは同じで、長さだけが違う。

図4.3の座標をそのまま使ってユークリッド距離を測ると、同じ作家の2作品が $d_2(A, B) = 1.79$ 、別の作家をはさんだ2作品が $d_2(B, C) = 1.43$ になる。後者のほうが0.36小さく、**別の作家Cのほうが近い**という結果になってしまう。

コサイン距離なら $d_{\cos}(A, B) = 0$ である。向きが同じだからである。

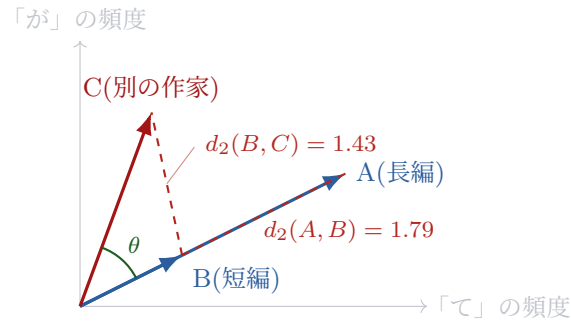


図 4.3 A と B は同じ作家の長編と短編。ユークリッド距離では B と C のほうが近い。

○ 定石 4.1 — 長さの違いを消したいときにコサイン距離を選ぶ —

- 文書の長さがばらばら
- 特徴が「回数」で、全体の量に比例して増える
- 高次元でスパース (ほとんどの成分が 0)

情報検索や文書分類でコサイン類似度が定番なもの、長さを消せるからである。

4.3.2 ℓ_2 正規化すればコサイン距離と順位が一致する

本書の特徴量は第3章ですでに ℓ_1 正規化してある。それでもコサイン距離がユークリッド距離よりよく分けるのはなぜか。

ℓ_1 正規化は「成分の和を 1 にする」操作で、コサイン距離が無視する ℓ_2 ノルム (ベクトルの長さ) を 1 にするわけではない。だから差は残る。

もし ℓ_2 正規化 ($\|x\| = 1$) してあれば、両者は完全に対応する。

$$\|x - y\|^2 = \|x\|^2 + \|y\|^2 - 2x \cdot y = 2 - 2\cos\theta = 2d_{\cos}(x, y) \quad (4.10)$$

つまり単位球面上では $d_2 = \sqrt{2d_{\cos}}$ となり、片方が小さいときもう片方も小さい。順位付けの目的なら、どちらを使っても同じ結果になる。

補足 コサイン距離と ℓ_2 正規化の使い分け

ℓ_2 正規化してからユークリッド距離を使うのと、生のベクトルにコサイン距離を使うのは、実質的に同じことである。

どちらで書くかは、後段のアルゴリズムが何を要求するかで決める。Ward 法や k-means はユークリッド距離を前提とするので、「 ℓ_2 正規化 → ユークリッド距離」の形にしておく使いやすい。

補足 コサイン距離が定番になった由来とその限界

コサイン距離の地位は、情報検索のベクトル空間モデル (Salton らの SMART システム) に由来する [18]。文書を単語頻度のベクトルで表す世界では、文書の長さを自動的に打ち消せること、ほとんどの成分が 0 の疎なベクトルなら内積が「両方に現れた単語」だけの和になり計算が軽いこと、値が -1 から

補足（続き）

1 に収まり解釈しやすいこと。実務的な利点の束が、定番の座を作った。

現代的な理由も一つある。単語や文の埋め込みベクトルは、学習の目的関数そのものが内積（またはコサイン）で類似度を測る形をしているので、**学習時に使った距離で結果も測るためにコサインが選ばれる**。裏を返せば、コサインが「意味の類似度の正しい定義」だから使われているわけではない。実際、学習済み埋め込みのコサイン類似度は、学習時の正則化の自由度しだいで恣意的な値を取りうるが示されている [20]。

ただし、注意すべき点もある。コサインは**次元の呪いの対策ではない**。高次元ではランダムなベクトル対の角度が 90° に集中し（付録 B）、角度もまた「みな同じ」へ向かう。表 4.3 でコサインが勝つのは、本書のデータでは長さの情報が判別の邪魔だからであって、呪いを退治したからではない。ノルムを捨てるので、量の絶対値が意味を持つ課題でも損になる。そして万能でもない。確率分布どうしの近さを測る尺度を体系的に比べた計算言語学の古典的な研究では、コサインやユークリッド距離といった幾何的な尺度より、Jensen-Shannon divergence のように**分布のずれそのもの**を測る尺度のほうが良い成績を出した [19]。和が 1 に固定された分布どうしを比べるなら、そちらのほうが向いている。

× アンチパターン 4.1 — 定番だからという理由で距離を選ぶ

ある距離が定番の座にあるのは、それが生まれた分野の問題に都合がよかったからにすぎない（コサイン距離については直前の箱で見た）。手元のデータが、その分野と同じ単位や分布の形を持っているとは限らない。

チェビシェフ距離が表 4.3 で最下位に沈むのも、その設計が「最悪の 1 項目で決める」場面向けだからで、距離そのものの優劣ではない。

距離はデータの構造と、（学習器を使うなら）その目的関数に合わせて選ぶ。候補を 2 つか 3 つ実際に走らせ、効果量なり正解率なりの数字で決める。表 4.3 は、そのための比較を 1 枚にしたものである。

4.4 標準化 — 文長の 2 列が距離のほとんどを占めてしまう

距離を計算する前に、必ず確認すべきことがある。**列ごとの値の大きさが揃っているか**である。図 4.4 に、52 列それぞれの標準偏差を大きい順に並べた。

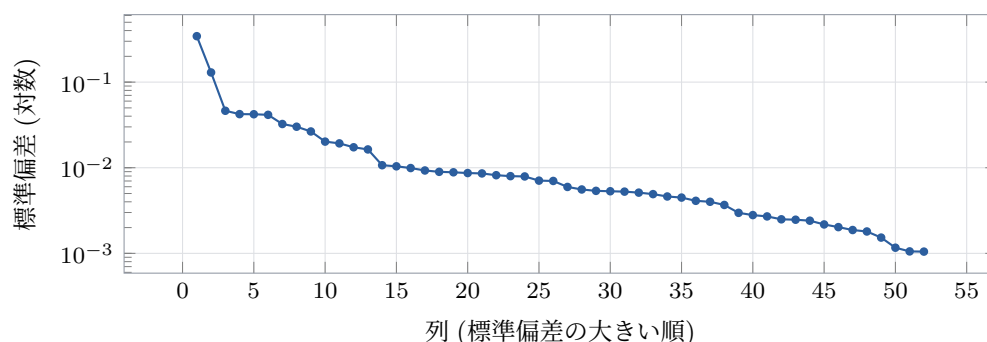


図 4.4 列ごとの標準偏差。最大と最小で 300 倍以上の開きがある。

標準偏差が最大の列（文長 μ ）と最小の列で、300 倍以上の開きがある。

式 (4.5) を見返すと、各項が同じ重みで足されている。ばらつきの大きい列は差も大きくなるので、その列だけで距離がほぼ決まってしまう。

文長 μ の差が 0.5 あったとして、「あ」の割合の差 0.005 など距離にはまったく現れない。47 個の読点の特徴を苦労して作った意味がなくなる。

本当にそうなるか、74 作品を生のまま (標準化せずに) 測ってみた。全 2701 ペアの二乗ユークリッド距離を列ごとの寄与に分解すると、文長 μ が 80.0%、文長 σ が 11.4% を占め、字種 3 列は合わせて 2.6%、残る読点 47 列は全部合わせても 6.1% しかない。文長の 2 列だけで 9 割を超えている。52 次元の距離を測っているつもりで、**実質は 2 次元**だった。同じ生のままのユークリッド距離で、同著者ペアの距離と異著者ペアの距離がどれだけ分かれているかを効果量 (第 4.6 節で定義する) として測ると、値は 1.24 である。

対処が**標準化**である*¹。

$$z_j = \frac{x_j - \mu_j}{\sigma_j} \quad (4.11)$$

各列を平均 0、標準偏差 1 に揃える。これで全列が対等になる。本書の 74 作品で同じ効果量 (ユークリッド距離で測った、同著者ペアと異著者ペアの分離の度合い) を測り直すと、生のままの 1.24 に対して標準化後は 1.42 になった。0.18 の増加である。

× アンチパターン 4.2 — 単位の違う列を混ぜたまま距離を測る

単位の違う量を混ぜれば、この偏りは必ず起きる。身長 (cm) と体重 (kg) と年収 (円) を並べて距離を測れば、年収だけで結果が決まる。

「なぜか 1 つの変数でしか分類されない」という症状が出たら、まずスケールを疑ってみる。

4.4.1 行の標準化と列の標準化とその両方*

読点前平仮名の 47 列は、第 3 章で**行ごと**に合計 1 へ正規化した。いま行った標準化は**列ごと**の操作である (名前の似た 2 つの操作の違いは、第 3 章のアンチパターン「正規化と標準化を混同する」にまとめた)。ここで一度、行と列の操作を整理しておきたい。「行を z 化したらどうなるのか」「両方やったらどうなるのか」は、実際の分析で必ず一度は迷う点だからである。

行を z 化したときに測っているもの z 変換を列でなく行に施す、つまり各作品のベクトルから自分の行内平均を引き、行内標準偏差で割ってみる。

こうすると、どの行も平均 0・標準偏差 1 になる。標準偏差が 1 ということは、 n 個の成分の 2 乗和が n に固定されるということである (n は列数)。つまり**どの作品のベクトルも、原点から同じ距離のところに並ぶ**。長さがそろってしまうので、残る違いは向きだけになる。

この状態で 2 つのベクトルのユークリッド距離を測ると、式 (4.10) と同じ展開ができて

*¹ 標準化して得た値を **z 得点** (z-score、標準得点) と呼び、この操作を **z 変換** や「 z 化」と言うことがある。文字 z は、平均 0・標準偏差 1 の正規分布 (標準正規分布) に従う変数を z と書く慣習に由来する。統計の教科書の巻末にある正規分布表の見出しが z になっているのも同じ理由である。同じ名前の別物が 2 つあるので、検索するときは注意がいる。信号処理の **Z 変換** は離散時間の信号を複素数の級数に直す操作で、これとは無関係である。統計にも **Fisher の z 変換** があるが、そちらは相関係数 r を $\frac{1}{2} \ln \frac{1+r}{1-r}$ に直して分布を正規分布に近づける操作で、本節の標準化とは別物である。

$$d_2(\tilde{x}, \tilde{y})^2 = \|\tilde{x}\|^2 + \|\tilde{y}\|^2 - 2\tilde{x} \cdot \tilde{y} = 2n(1 - r(x, y)) \quad (4.12)$$

となる (r は行どうしの相関係数)。 $\|\tilde{x}\|^2 = \|\tilde{y}\|^2 = n$ を代入し、内積 $\tilde{x} \cdot \tilde{y}$ が nr に等しいことを使った。平均を引いて標準偏差で割ったベクトルどうしの内積が相関係数になる、というのは相関係数の定義そのものである。

結論として、**行の標準化 + ユークリッド距離は、この章の後半 (第 4.8 節) で扱う相関距離と同じものになる**。「行を z 化する」という前処理のつもりの操作が、距離の選択として再登場することになる。本書の 74 作品を行ごとに z 化してからユークリッド距離を測ると、同著者ペアと異著者ペアの分離の効果量は 1.72 になった。列ごとに標準化したときの 1.42 より 0.30 高い。後述の表 4.3 で相関距離とコサイン距離が最上位に来ることの、別の言い方である (効果量の数値そのものが表と一致しないのは、 $1 - r$ と $\sqrt{2n(1 - r)}$ のように単調変換で結ばれた距離どうしても、効果量の値は変わるからである。効果量は平均の差を標準偏差で割った比なので、分子と分母が別々に伸び縮みすれば比も変わる。順位は同じでも、指標の値は距離の定義に依存する)。

ただし意味には注意が要る。行内平均は「52 個の特徴の平均」であって、文長 μ と「て」の割合を混せて平均した量に単位はない。単位のない量を引いたり割ったりしているのだから、本来なら意味を問われてもおかしくない。

それでもこの操作が働くのは、次のように読めるからである。1 作品の 52 個の数値を、左から順に並べた折れ線だと思ってほしい。行の z 化は、その折れ線の**高さ**と**振れ幅**を**そろえる**操作にあたる。そろえたあとに残るのは折れ線の**形**だけで、2 つの作品の形がどれだけ似ているかを表すのが相関係数である。つまり測っているのは「文体の全体的な濃さ」ではなく、「どの特徴が相対的に高く、どれが低いか」というでこぼこの一致度である。

行と列の両方を z 化する 行も z 化し列も z 化した「完全に揃った」行列は、一度ずつの操作では作れない。列を z 化すると各行の平均と分散が崩れ、行を z 化すると各列のそれが崩れる。互い違いに繰り返すとどうなるかというと、これは**収束することが知られている** (逐次標準化。Olshen と Rajaratnam が 2010 年に収束を証明した [23])。本書のデータでは 33 回の反復で、全行・全列が機械精度で平均 0・分散 1 になった。この行列でユークリッド距離を測ると、同著者ペアと異著者ペアの分離の効果量は 1.86 である。列だけの標準化 (1.42) より 0.44、行だけの z 化 (1.72) より 0.14 高い。

面白い性質だが、使いどころは選ぶ。収束後の各値は「行から見ても列から見ても相対的な位置」としか言いようがなく、元の単位との対応をたどれない。探索でクラスタを浮かび上がらせる道具としては強力だが、「この値は何か」と問われて答えられない前処理を、報告書に黙って入れるのは避けたい。

× アンチパターン 4.3 — 順序を考えずに正規化と標準化を混ぜる

第 3 章の正規化 (行) → 本章の標準化 (列) という順序には意味がある。逆に、**列を z 化したあとで行を合計 1 に正規化すると破綻する**。 z 化した値には負の数が混ざるので、行の合計は 0 の前後に散らばる。合計が 0 に近い行では、それで割った瞬間に値が発散し、符号も反転する。「とりあえず両方かけておく」前処理をパイプラインに積むときは、順序を入れ替えても

同じだとは限らない点に気をつける。

× アンチパターン 4.4 — 標準化すれば組成データの制約が消えると思う

第3章で見た「合計1の縛り」は、列の標準化では消えない。合計が1なら、 z 化後も $\sum_j \sigma_j z_j = 0$ という線形の縛りがそのまま残る(超平面の法線が $(1, \dots, 1)$ から $(\sigma_1, \dots, \sigma_D)$ に変わるだけである)。実際、字種3列を z 化して共分散行列の3つの固有値を計算すると、最小の固有値は 1.0×10^{-16} で、計算誤差の範囲で0である。負の相関も、部分組成の非整合も、標準化後の世界にそのまま持ち込まれる。制約を外したければ、clr や ilr のような**組成のための変換**(第3章)を使うしかない。

4.4.2 列をまたいで足し合わせる手法には標準化が要る

標準化は儀式ではないので、要る手法と要らない手法がある。さらに「要らない」の中にも、**やっても結果が変わらない場合と、やらないほうがよい場合がある**。

なお、本項には第5章から第7章で扱う手法の名前が先に出てくる。中身を知らなくても、**列をまたいで足すかどうか**という一つの見方だけで要否が決まるという点が読めれば足りる。個々の手法は、それぞれの章で改めて説明する。

見分ける鍵は、その手法が**複数の列を足し合わせて1つの数を作るか**である。距離も内積も正則化項も分散も、すべて列をまたいだ和になっている。和を取る前に単位を揃えていなければ、大きい単位の列が和を独り占めしてしまう。逆に、列を1つずつ見て大小を比べるだけの手法なら、どの列に何を掛けても順序が変わらないので影響を受けない。和を取るかどうかで分けたのが図4.5である。

理屈だけでは心もとないので、本書の74作品・52列で実測した。第7章で登場する5つの分類器を先取りして、74作品の著者を5分割の層別交差検証で判別したときの正解率を、標準化なしと標準化ありで並べたのが表4.2である。比べる相手は、最頻クラスを答え続ける基準線0.284である。

表 4.2 標準化の有無による正解率の変化(5分割の層別交差検証)。標準化は交差検証の内側で行っている(理由は第9章)。参考: 基準線(最頻クラス)は0.284

手法	なし	あり	何が起きたか
ランダムフォレスト	0.987	0.987	完全に同じ
k近傍法($k=5$)	0.650	0.877	距離が文長 μ に支配される
SVM(RBFカーネル)	0.284	0.959	基準線と同じ=何も学べない
ロジスティック回帰(L2正則化)	0.351	0.986	正則化が係数を単位ごと罰する
ロジスティック回帰(L1正則化)	0.351	0.946	使う特徴が2列だけになる

木のモデルに標準化は要らない ランダムフォレストの交差検証正解率は、標準化なしでも標準化ありでも0.987で、小数第3位まで**完全に一致**する。偶然ではない。決定木の分割は「文長 $\mu \leq 3.4$ か?」のように**1列の大小関係だけ**を見るので、列ごとにどんな単調変換(標準化、対数、順位化)をしても、選べる分割の集合が変わらない。決定木・ランダムフォレスト・勾配ブースティングは、す

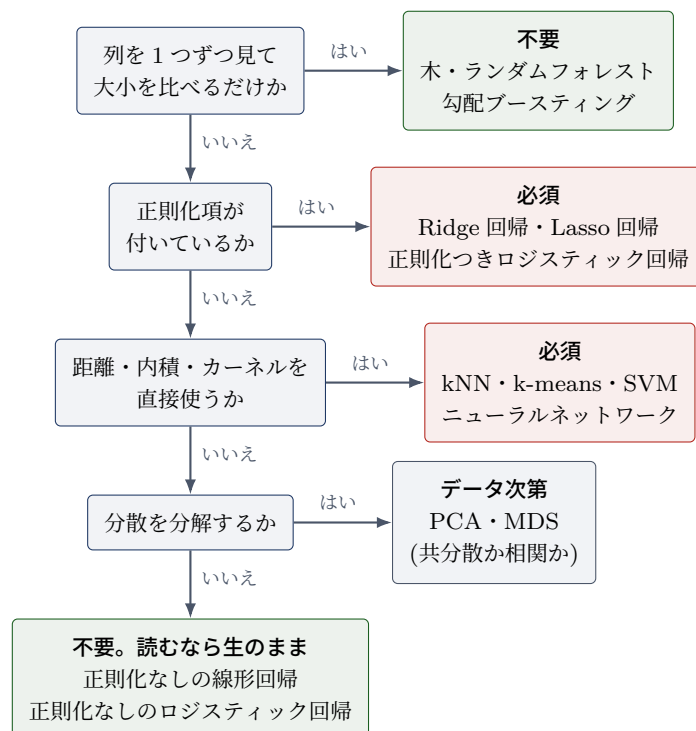


図 4.5 標準化の要否。赤が必須、緑が不要、灰色はデータで決まる。

べてこの単調変換への不変性を持つ。

正則化項が付いたモデルには標準化が要る ここが最も大事な分かれ目なので、理由から述べる。Ridge 回帰は予測誤差に $\lambda \sum_j \beta_j^2$ を、Lasso 回帰は $\lambda \sum_j |\beta_j|$ を足して学習する。この足す項を**正則化項**という。足しているのは、**係数を並べたベクトルのノルム**である。Lasso が ℓ_1 ノルム、Ridge が ℓ_2 ノルムの 2 乗を使うので、それぞれ **L1 正則化・L2 正則化**とも呼ぶ (第 3 章の脚注で断ったとおり、正則化の名前だけ大文字で書く慣習に従う。scikit-learn の引数も `penalty="l1"` である)。係数が大きいほど値が増える項なので、**大きい係数に罰を与える働きをする***2。正則化項が見ているのは**係数の大きさ**である。ところが係数の大きさは、対応する列を何の単位で測ったかで決まる。長さを cm で測れば m で測るより係数は 1/100 になり、正則化項も L2 なら 1/10000、L1 なら 1/100 になる。

つまり単位を揃えないと、**どの係数を強く縮めるかを、列の単位の選び方が決めてしまう**。分析者が意図していない重み付けが、測り方の都合から入り込むことになる。正則化つきのモデルで標準化が必須なのは、**正則化項と係数の単位を揃えるため**である。

実測でもそのとおりになった。表 4.2 の L1 正則化つきロジスティック回帰について、係数が 0 にならなかった列を数えると、標準化なしでは 52 列のうち **2 列しか使わない**モデルになっていた (標準化ありでは 34 列)。交差検証正解率も、標準化なしの 0.351 に対して標準化ありは 0.946 で、0.595 の開きがある。第 7 章で使うロジスティック回帰や SVM は、既定で正則化が入っている。「ロジス

*2 罰を与えるという見方から**罰則項** (penalty term) とも呼ぶ。本書では手法を分類するときは正則化、項の働きを説明するときは罰という言い方をする。係数を 0 に向かって縮めるので、訓練データに合わせすぎる過学習 (第 7 章) を抑えられる。とくに Lasso 回帰は係数を厳密に 0 にできるため変数選択を兼ねる。

ティック回帰だから標準化は不要」と覚えていると、ここで足をすくわれる。

正則化の強さ α を大きくしていくと係数がどう変わるかを描いたのが図 4.6 である。「太宰治かどうか」を 52 列から当てる問題で、74 作品を標準化してから当てはめてある。 α は 0.004 から 7.9 まで動かした。

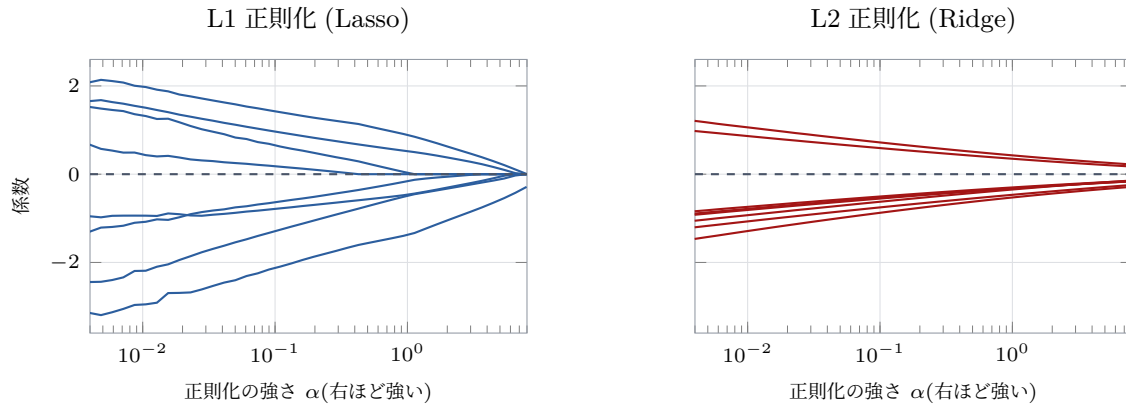


図 4.6 正則化を強くしたときの係数の動き (値の大きい 8 列を描いた)。L1 では係数が順に 0 へ落ちて、そこから戻らない。L2 はどれも 0 に近づくが、0 にはならない。

左右で形が違う。L1 では、係数が 1 本ずつ 0 に到達して、そのまま 0 に留まる。係数が 0 でない列の数は、正則化が最も弱い $\alpha = 0.004$ のときで 15 列、最も強い $\alpha = 7.9$ のときで 3 列 (と・漢字・は) である。どの列を使うかを決める働きを兼ねているので、Lasso 回帰は変数選択の道具としても使われる。L2 では、すべての係数が 0 へ向かって縮むが、 α が最も強いところでも係数が 0 でない列は 52 列、つまり 52 列すべてである。

L1 だけが係数を厳密に 0 にできる理由 (動ける範囲の角) と、関連した列があるときの読み方の注意は、第 7 章の第 7.11 節で図解する。

ここで標準化の話に戻る。正則化項が見ているのは、図 4.6 の縦軸の高さである。列の単位を変えれば縦軸の目盛りが変わるので、どの線が先に 0 へ落ちるかも変わってしまう。単位を揃えずに α を選ぶことは、測り方の都合で変数選択をするのと同じである。

距離とマージンを使う手法にも標準化が要る k 近傍法は距離そのものを使うので、本節で見たとおり標準化が要る。サポートベクトルマシン (SVM) も同じで、マージン (境界からの距離) を最大化する手法であるうえ、RBF カーネルは $\exp(-\gamma\|x - x'\|^2)$ とユークリッド距離を直接使う。表 4.2 の SVM の交差検証正解率は、標準化なしでは基準線と小数第 3 位まで同じ 0.284 で、**最頻クラスを答え続けるだけの箱**になっている。標準化すると 0.959 になり、基準線を 0.675 上回る。k-means などの距離ベースのクラスタリングも同様である。ニューラルネットワークも入力の線形結合から始まるので同じ側にある。

正則化のない回帰に標準化は要らない 正則化項を外すと話が変わる。正則化なしの線形回帰やロジスティック回帰は、予測に関して**スケール不変**である。ある列を 100 倍したとする。同じ予測値を出すには、その列の係数が $1/100$ になればよい。掛け算の相手が 100 倍になったのだから、係数がその分だけ小さくなれば積は元のままである。最適化はその係数を見つけるので、当てはめる直線

もあてはめ値も1つも変わらない。本書の74作品で、読点前の47列から文長 μ を予測する正則化なしの線形回帰を組み、5分割交差検証の**決定係数**^{*3}を測ると、標準化なしと標準化ありのどちらも -0.934 で、**小数第3位まで一致**した。値が負なので、読点から文長を当てる課題では平均を答えるより悪い。確かめたかったのは当たるかどうかではなく、2つが一致することのほうである。同じ列に正則化つきのRidge回帰($\alpha = 1$)をかけて同じ決定係数を測ると、標準化なしで -0.051 、標準化ありで $+0.005$ になり、今度は一致しない。

予測が変わらないなら、標準化するかどうかは**何を読みたいか**で決めてよい。係数を「この変数が1単位増えると何が起きるか」として読みたいなら、標準化しないほうが解釈しやすい。「1標準偏差あたりの効果」として変数間で比べたいなら標準化する。どちらも正しく、目的が違うだけである。

なお、スケール不変性は理屈の上での話で、実装では崩れることがある。本書の74作品・52列に正則化なしのロジスティック回帰を当てはめ、著者を判別したときの交差検証正解率を測ると、標準化なしで 0.839 、標準化ありで 0.972 になる。理屈のうえでは一致するはずの2つが、 0.133 ずれている。

理由は、52列に対して点が74個しかないことにある。これだけ次元が多いと、訓練データを1つも間違えずに分ける直線が引けてしまう。完全に分けられると、係数を大きくすればするほど「自信のある正解」になって尤度が上がり続ける。上がり続けるので最大値がない。最大値がなければ、最適化はどこかで打ち切るしかなく、打ち切る位置が列のスケールに左右される。そもそも正則化を外した線形モデルを高次元で使うこと自体に無理がある。

PCAでは、どちらの行列を分解するかを選択になる 第6章で扱う主成分分析も同じ影響を受ける。本書の74作品・52列を生のまま(標準化せずに)PCAにかけると、第1主成分の寄与率(その軸だけでデータ全体の散らばりの何割を説明できるかという値。第6章で定義する)は 0.821 で、散らばりの8割をこの1軸が占める。第1主成分に対する文長 μ の負荷は 0.986 で、上限の1にはほぼ等しい。「主成分」と呼んではいるが、**いちばん分散の大きい1列を言い換えただけ**になる。同じデータを標準化してからPCAにかけると、第1主成分の寄与率は 0.191 まで下がり、多数の列に寄与が散る。

「PCAの標準化はケースバイケース」と言われるのは、これが**共分散行列を分解するか、相関行列を分解するか**の選択だからである。標準化してからのPCAは相関行列のPCAに等しい。全列が同じ単位で測られ、分散の大小そのものに意味がある場合(同一条件で測ったスペクトル強度、同じ満点のテスト得点など)は、共分散のまま分解するのが正当である。単位の違う列が混ざるなら、相関(=標準化)を選ぶしかない。

なお、**PCAは正規分布を仮定しない**。PCAの計算に出てくるのは、分散と内積だけである。どちらもデータの形がどうであれ計算できるので、どんな分布のデータにも当てはめられる^{*4}。

正規分布が要るのは、PCAに確率モデルを重ねたいときだけである。たとえば主成分どうしが**無相関**であることは計算から必ず言えるが、これを**独立**だと読み替えられるのは、データが多変量正規

^{*3} R^2 と書く、回帰の当てはまりの指標である。1なら完全な予測、0なら「つねに平均を答える」と同じ、負ならそれより悪い。

^{*4} 「分散が最大の向きを選ぶ」と「元のデータにいちばん近い低次元の近似を作る」ことが一致するのも、分布によらず成り立つ(Eckart-Youngの定理[26])。

分布に従う場合である。一般には、無相関から独立は出てこない。「PCA は正規分布が前提」という言い回しは、この2つを混ぜた俗説である。

○ 定石 4.2 — 標準化の要否を1行で判定する —

その手法は、距離・内積・正則化項・分散のどれかを列をまたいで足し合わせるか。足し合わせるなら標準化する (kNN、k-means、SVM、正則化つき回帰、PCA、ニューラルネットワーク)。1列ずつの大小しか見ないなら不要である (木とそのアンサンブル)。迷ったら標準化しておく。木のモデルは標準化しても何も壊れない。

細かい注意を2つ。ほとんどの要素が0であるようなデータは、0でない場所だけを覚えておく形式で保存する。ところが平均を引くと、0だった要素がすべて0でない値に変わり、その形式が使えなくなってメモリが溢れる。こういうデータでは、平均を引かずに割り算だけを行う。外れ値が強いデータでは、平均と標準偏差の代わりに中央値と四分位範囲で揃える方法 (ロバストスケールリング) がある。そして標準化は必ず交差検証の内側で行う (第9章)。

4.5 距離行列を眺める

標準化した74作品について、すべての組み合わせのユークリッド距離を計算する。74×74の行列になる。これを著者順に並べて色で示したのが図4.7である。

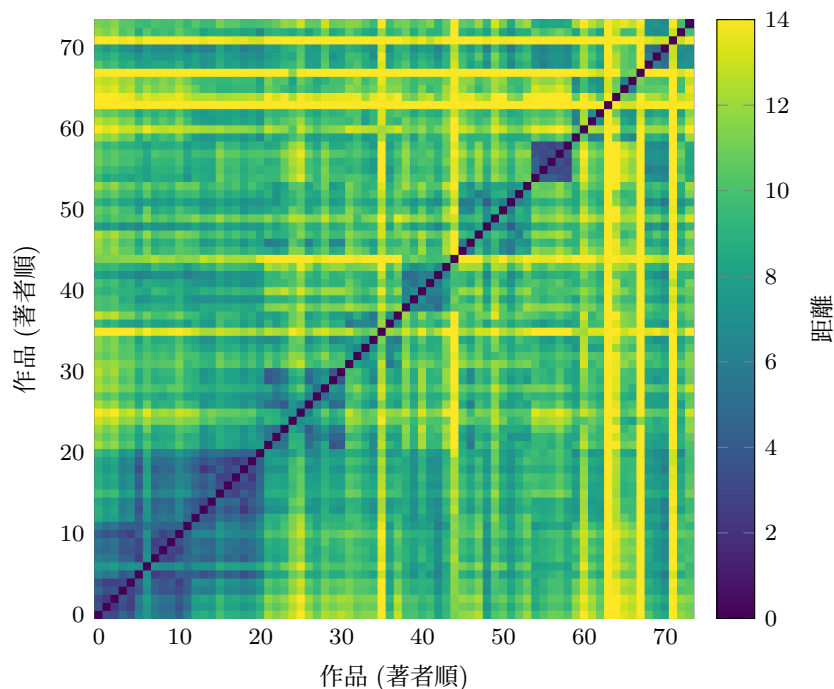


図 4.7 距離行列。暗いほど近い。左上の大きな暗いブロックが吉川英治 21 作品。

対角線に沿って暗い正方形が並んでいれば、同じ著者の作品が互いに近いということである。左上の大きなブロックが吉川英治で、中でもとりわけ暗い。これは後の章で重要になる。

4.6 同著者と異著者を最もよく分けたのはコサイン距離

距離の種類を変えると結果は変わる。選ぶときの目安の一つは、**同じ著者の作品どうしの距離と違う著者の作品どうしの距離**が、どれだけ分かれているかである。2つの分布が離れているほど、著者の識別に向いている。

分離の度合いを**効果量** (effect size) で測る。2つの平均の差を、両群の分散を平均した標準偏差で割った値である (Cohen's d の通例と違い、群の大きさで重み付けしない版を使う)。

$$\text{効果量} = \frac{\bar{d}_{\text{異著者}} - \bar{d}_{\text{同著者}}}{\sqrt{(s_{\text{同}}^2 + s_{\text{異}}^2)/2}} \quad (4.13)$$

1つの数にする前に、もとの2つの分布を見ておきたい。74作品から作れるペアは全部で2701組あり、うち394組が同じ著者、2,307組が違う著者である。標準化した74作品の全2701ペアについてコサイン距離を測り、同著者と異著者に分けて分布を描いたのが図4.8である。

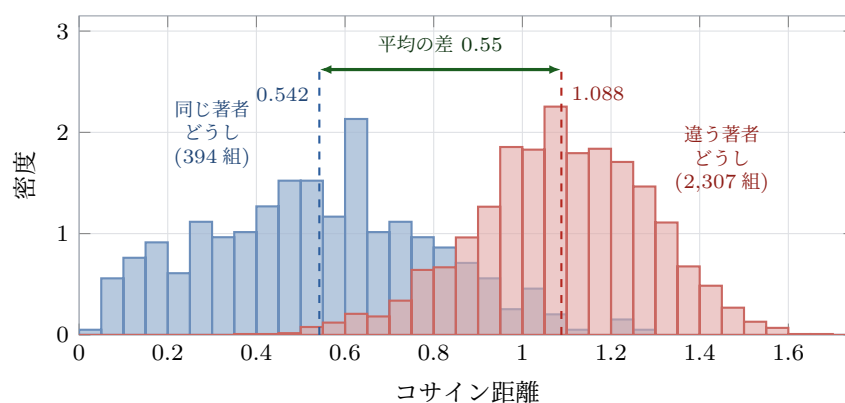


図 4.8 同じ著者どうしの距離と、違う著者どうしの距離。効果量は、2つの山の平均の差を、山の幅 (標準偏差) で割った値である。

コサイン距離の平均は、同著者 394 組が 0.542、異著者 2,307 組が 1.088 で、差は $1.088 - 0.542 = 0.55$ である。標準偏差は同著者側が 0.255、異著者側が 0.195 である。差を共通の幅で割ると 2.402 になる。これが後述の表 4.3 に載る効果量 2.40 の中身である。山どうしが離れているほど、幅が狭いほど大きくなる量だと読める。

2つの山は重なりを残している。かりにコサイン距離ひとつで「同じ著者か否か」を判定するなら、しきい値を 0.66 に置くのが最良である。それでも同著者 394 組のうち 120 組を遠いと判定し、異著者 2,307 組のうち 53 組を近いと判定してしまう。誤りは全 2701 組のうち 173 組、割合にして 0.064 である。効果量 2.40 は**完全に分かれた**という意味ではない。

同じ 74 作品・全 2701 ペアについて、標準化した値のまま距離の種類だけを取り替えて、同じ効果量を測り直した。結果を表 4.3 に示す。

はっきりした傾向がある。

- コサイン距離が最も良く分ける (効果量 2.40。既定値として使われるユークリッド距離の 1.42 より 0.98 高い)

表 4.3 距離の種類による分離度 (標準化済みデータ)

距離	同著者の平均	異著者の平均	効果量
マンハッタン ($p = 1$)	35.60	56.91	1.59
ユークリッド ($p = 2$)	6.68	10.41	1.42
$p = 4$	3.56	5.37	1.18
チェビシェフ ($p = \infty$)	2.81	4.15	0.97
コサイン	0.54	1.09	2.40
相関	0.54	1.09	2.40

- ミンコフスキー距離では p が小さいほど効果量が高い ($p = 1$ で 1.59、 $p = 2$ で 1.42、 $p = 4$ で 1.18)
- チェビシェフ距離が最も悪い (0.97)

p が大きいほど効果量が下がるのは、52 次元という高次元では「一つの次元の大きな差」が偶然に生じやすいためだと考えられる。著者の癖は多数の特徴に薄く広がっているので、それらを均等に足し合わせる $p = 1$ やコサイン距離のほうが拾える。ただし、どの機構がどれだけ寄与したかまでは本書では測り分けていない。

補足 チェビシェフ距離の使いどころ

表 4.3 で最下位に沈んだ距離だが、使いどころはある。最悪の 1 項目がすべてを決める場面である。

元祖はチェス盤である。王将がマス目 (x_1, y_1) から (x_2, y_2) へ移動するのに必要な手数は、縦の差と横の差の**大きいほう**に等しい (斜めに動けるので、小さいほうの差は道中で消化できる)。チェビシェフ距離が**チェス盤距離**とも呼ばれる理由である。

同じ構造は機械の駆動にある。プロッタや倉庫のクレーンのように **X 軸と Y 軸が独立に同時に動く**装置では、移動にかかる時間は各軸の所要時間の最大値で決まる。つまり「時間で測った距離」がチェビシェフ距離になる。最近傍の巡回順を考えると、チェビシェフ距離で最適化するのが正しい。

製造の検査も同じ型である。「全寸法が公差内なら合格」は、基準点からのチェビシェフ距離が ε 以下、と言い換えられる。図 4.1 の $p = \infty$ の単位球が正方形になるのは、「どの軸も ± 1 以内」という条件の形そのものである。画像処理で画素の周囲 8 マスを近傍とする (ムーア近傍) のも、チェビシェフ距離 1 以内という意味になる。

数値計算では、誤差の**最大値**を最小化する近似 (ミニマックス近似) で ∞ ノルムが主役になる。これを最初に体系化したのがチェビシェフその人で、名前はここに由来する。

共通するのは、平均的な違いではなく**最悪ケースを管理したい**という動機である。本書の著者判別のように「癖が多数の特徴に薄く広がる」データとは正反対の状況だから、ここで最下位になるのは欠陥ではなく設計どおりである。

なお効果量は、単調変換で結ばれた距離どうし (ユークリッドと二乗ユークリッドなど) でも値が変わる (第 4.4.1 項)。この表の順位は目安であって、距離の族をまたぐ厳密な優劣ではない。

補足 計量文献学の定番 — Burrows の Delta

文体の統計分析 (計量文献学) には、著者判別のための距離の定番がある。Burrows が 2002 年に提案した [24] **Delta** で、高頻度語の出現率を列ごとに z 化し、その差の絶対値の平均を取る。つまり **標準化 + マンハッタン距離** (のスケール違い) であり、表 4.3 の 1 行目と同じ構成である。

その後の検証で、 z 化した頻度に **コサイン距離** を使う変種 (Cosine Delta) が最も頑健だと報告されている [25]。表 4.3 でコサイン距離が最上位に来たのは、計量文献学の 20 年分の経験則を、手元の 74 作品で追試した格好である。

ユークリッド距離は既定値としてよく使われるが、それが最良である保証はどこにもない。本書のデータでは、効果量がユークリッド距離の 1.42 に対してコサイン距離は 2.40 で、1.7 倍の開きがある。本章のはじめのアンチパターン「定番だからという理由で距離を選ぶ」で書いたとおり、候補を 2 つか 3 つ実際に走らせて数字で決める。

ただし、その「数字で決める」にも落とし穴がある。

× アンチパターン 4.5 — 距離を正解ラベルで選ぶ

表 4.3 を作るには **著者名を使っている**。だがクラスタリングは本来、正解を知らない場面で使う手法である。

正解を見て距離を選び、その距離でクラスタリングして「よく分かれました」と報告するのは循環論法である。

正解を持っているのは、本書が教材だからである。実務では、距離の選択は **列の単位と分布の形、そして何を「近い」と呼びたいかから決める**。

4.7 発展 — 距離の数学的性質

ここまで「距離の 4 条件」を前提に話を進めてきた。実際にそれが満たされているか、確かめておく。

発展 ミンコフスキー距離が距離であること

$p \geq 1$ のとき、式 (4.5) が三角不等式を満たすことは **ミンコフスキーの不等式** から従う。

定理 4.1 (ミンコフスキーの不等式) $p \geq 1$ のとき、任意のベクトル u, v に対して

$$\left(\sum_j |u_j + v_j|^p \right)^{1/p} \leq \left(\sum_j |u_j|^p \right)^{1/p} + \left(\sum_j |v_j|^p \right)^{1/p} \quad (4.14)$$

$u = x - y, v = y - z$ と置けば $u + v = x - z$ なので、そのまま $d_p(x, z) \leq d_p(x, y) + d_p(y, z)$ が得られる。

証明の骨子はヘルダーの不等式による。 $q = p/(p-1)$ として

$$\sum_j |u_j + v_j|^p \leq \sum_j |u_j| |u_j + v_j|^{p-1} + \sum_j |v_j| |u_j + v_j|^{p-1} \quad (4.15)$$

と分け、各項にヘルダーの不等式 $\sum |a_j b_j| \leq (\sum |a_j|^p)^{1/p} (\sum |b_j|^q)^{1/q}$ を適用する。 $p \geq 1$ が必要なのは、ここで共役指数 $q = p/(p-1) \geq 1$ が存在することを要求するためである。

4.7.1 $p < 1$ では距離にならない

$p = 0.5$ を試してみるとどうなるか。2次元で $x = (0, 0)$, $y = (1, 0)$, $z = (1, 1)$ とすると

$$d_{0.5}(x, y) = (1^{0.5})^2 = 1 \quad (4.16)$$

$$d_{0.5}(y, z) = (1^{0.5})^2 = 1 \quad (4.17)$$

$$d_{0.5}(x, z) = (1^{0.5} + 1^{0.5})^2 = 4 \quad (4.18)$$

$4 > 1 + 1$ なので三角不等式が破れている。

図 4.1 を $p < 1$ まで延長すると、単位球が内側にへこんだ星型になる。凸でない図形が現れたら、三角不等式は成り立たないと思ってよい。

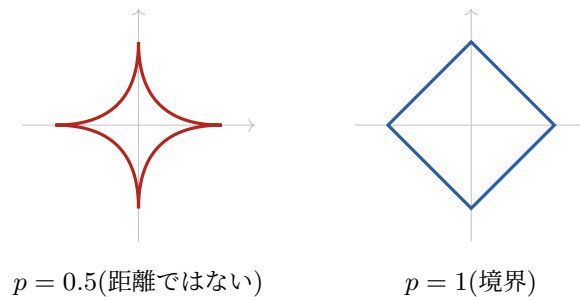


図 4.9 $p < 1$ の単位球はへこむ。凸でなければ三角不等式は成り立たない。

補足 距離の公理を満たさなくても $p < 1$ を使うことがある

高次元では $p < 1$ のほうが「近い点」と「遠い点」を区別しやすい、という報告がある [22]。距離の公理を満たさなくても、近傍探索の指標としては機能する。

ただし三角不等式に依存するアルゴリズム (多くの高速化手法、一部のクラスタリング) は正しい答えを返さなくなる。使うなら、何が壊れるかを把握したうえで使う。

4.7.2 コサイン距離は三角不等式を満たさない

$d_{\cos} = 1 - \cos \theta$ も三角不等式を満たさない。反例を作るのは簡単で、 $x = (1, 0)$, $y = (1, 1)/\sqrt{2}$, $z = (0, 1)$ とすると

$$d_{\cos}(x, y) = 1 - \frac{1}{\sqrt{2}} \approx 0.293 \quad (4.19)$$

$$d_{\cos}(y, z) = 1 - \frac{1}{\sqrt{2}} \approx 0.293 \quad (4.20)$$

$$d_{\cos}(x, z) = 1 - 0 = 1 \quad (4.21)$$

$1 > 0.586$ である。

角度そのものを使えば距離になる。

$$d_{\text{ang}}(x, y) = \arccos \left(\frac{x \cdot y}{\|x\| \|y\|} \right) \quad (4.22)$$

これは球面上の測地線距離 (球面に沿って測った最短距離) なので、三角不等式を満たす。 $d_{\cos}$ は d_{ang} の単調変換なので、**順位だけが問題なら $d_{\cos}$ で構わない**。

ここで章の冒頭の 4 条件と折り合いを付けておく。第 4.6 節で同著者と異著者を最もよく分けたコサイン距離は、距離の公理を満たしていない。それで困らなかったのは、あの実験でやったことが「近い順に並べる」ことだけだったからである。順位しか使わない操作に公理は要らない。一方、三角不等式を前提に候補を枝刈りする近傍探索や、距離の値そのものを足し引きする手法に公理を破る指標を持ち込むと、高速化が誤った答えを返したり、保証が失われたりする。4 条件は守るべき戒律ではなく、その指標でどの操作までが安全かを示す仕様である。

4.8 相関係数は偏差ベクトルのコサインである

表 4.3 でコサイン距離と相関距離の効果量が小数第 2 位まで一致した。偶然ではない。

ピアソンの相関係数 (correlation coefficient) は次のように書ける。

$$r(x, y) = \frac{\sum_j (x_j - \bar{x})(y_j - \bar{y})}{\sqrt{\sum_j (x_j - \bar{x})^2} \sqrt{\sum_j (y_j - \bar{y})^2}} \quad (4.23)$$

式 (4.8) と見比べると、形がまったく同じである。

$$\cos \theta = \frac{\sum_j x_j y_j}{\sqrt{\sum_j x_j^2} \sqrt{\sum_j y_j^2}} \quad (4.8)$$

違いは、相関係数のほうが x_j の代わりに $x_j - \bar{x}$ を使っている点だけである。

そこで**偏差ベクトル**を定義する。

$$\tilde{x} = x - \bar{x}\mathbf{1} = (x_1 - \bar{x}, x_2 - \bar{x}, \dots, x_n - \bar{x}) \quad (4.24)$$

$\mathbf{1}$ はすべての成分が 1 のベクトルである。これを使うと、相関係数は次の一行で書ける。

$$r(x, y) = \cos \theta(\tilde{x}, \tilde{y}) = \frac{\tilde{x} \cdot \tilde{y}}{\|\tilde{x}\| \|\tilde{y}\|} \quad (4.25)$$

相関係数とは、偏差ベクトルどうしのなす角の余弦である。

図 4.10 が示すように、平均を引く操作はベクトルの向きを大きく変える。(a) では 2 つのベクトルが同じような方向を向いているが、(b) では 90 度近くまで開いている。

補足 なぜ平均を引くのか

アンケートで 5 段階評価をつけてもらったとする。A さんは全体に甘く、平均 4.2 をつける。B さんは辛く、平均 2.1 である。

生の点数でコサインを取ると、2 人とも「全部の項目に正の値をつけている」という共通点だけで高い類似度が出てしまう。

平均を引けば、「A さんが自分の基準より高くつけた項目」と「B さんが自分の基準より高くつけた項目」が一致するかを見ることになる。これが相関係数の意味である。

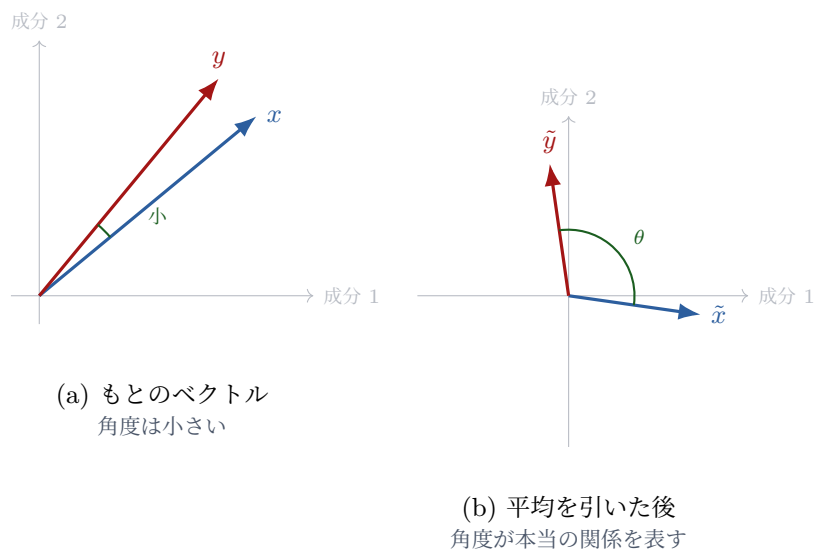


図 4.10 平均を引くと角度が変わる。相関係数が測っているのは (b) の角度である。

使い分けは1点で決まる。各サンプルの**絶対的な大きさ**に意味があるならコサイン、サンプルごとの**基準の違い**を消したいなら相関である。評価者ごとの甘辛を消したいなら相関、文書ごとの長さを消したいならコサイン、という具合になる。

本書のデータは第3章で行方向の正規化を済ませてあるので、消すべき基準の違いがすでになく、2つの指標がほぼ一致した。

4.8.1 散布図の細さは相関の強さではない

相関係数を式 (4.25) のように角度として読むと、1つ注意すべきことが出てくる。**ベクトルの長さを変えても角度は変わらない**。つまり、どちらかの変数を何倍しても相関係数は変わらない。

このことは、散布図の見た目と食い違う。図 4.11 の3枚は、**まったく同じ点の集まり**を、縦軸の縮尺だけ変えて描いたものである。

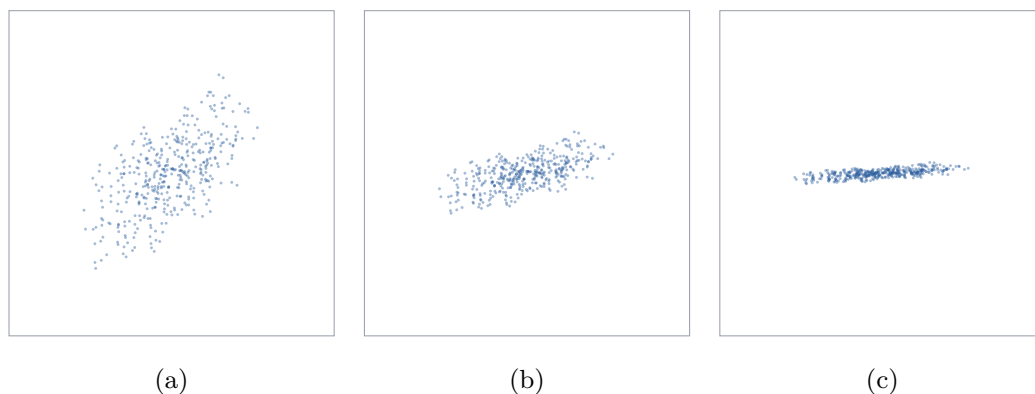


図 4.11 同じデータを、縦軸の縮尺だけ変えて描いた3枚。点の並びは (c) がいちばん細く見えるが、3枚の相関係数はまったく同じ値である。

(c) は細い帯に見えるので、いかにも相関が強そうである。(a) は雲のように広がっていて、弱そう

に見える。だが3枚は同じ点を描いていて、相関係数はどれも同じ値になる。変えたのは縦軸の目盛りだけだからである。

見た目の細さを決めているのは、**縦の広がり**と**横の広がり**の比であって、相関ではない。2つを混同すると、次のような読み違いが起きる。

- 単位を変えただけの図を見て「相関が強くなった」と言う
- 縦軸を切り詰めた図を見て「はっきりした関係がある」と言う
- 逆に、縦に引き伸ばした図を見て「関係がない」と言う

× アンチパターン 4.6 — 散布図の見た目の細さから相関の強さを判断する

帯が細いかどうかは、軸の縮尺しだいでいくらでも変わる。判断したいなら**数値を計算する**。逆方向の間違い(数値だけを見て図を描かない)は、次項のアンスコム の例で確かめる。

補足 軸の縮尺は図を作る側の裁量である

同じことが、相関以外の図でも起きる。

棒グラフの縦軸を0から始めずに途中で切ると、わずかな差が大きな差に見える。支持率45%と48%を、縦軸40%からの目盛りで描けば、棒の高さの比は5対8になる。実際の比は45対48である。棒グラフは面積で量を伝える図なので、0から始めないと約束が崩れる(折れ線グラフは変化の向きを伝える図なので、切ってもよいことが多い。ただし切ったことは明示する)。

2つの系列を左右別の縦軸で重ねる図も同じ性質を持つ。軸を独立に伸縮できるので、無関係な2つの線を「連動している」ように見せることができる。広告費と売上を重ねて因果があるかのように描くのは、その典型である。

図は嘘をつかなくても、**縮尺の選び方で印象を作れる**。自分が図を作るときは、選んだ縮尺が結論を作っていないかを一度疑う。

4.8.2 同じ数値からまるで違う形が作れる

前項とは逆向きの話をする。縮尺を変えても相関係数は変わらなかった。では相関係数が同じなら、データの形も同じかというと、そうではない。

統計学者 Anscombe は1973年、**平均・分散・相関係数・回帰直線がすべてほぼ一致するのに、散布図はまったく違う**4組のデータを作ってみせた[63]。Anscombe の四重奏と呼ばれる。

図4.12の4組は、どれも x の平均9.0、 y の平均7.5、相関係数0.816、回帰直線 $y = 3.00 + 0.500x$ である。それでも中身は次のように違う。

- (a) 素直な直線的関係。相関係数を読んでよい唯一の組
- (b) 実際には曲線。直線を当てはめること自体が間違い
- (c) 1点の外れ値が直線を引っ張っている。その1点を除けば完全な直線
- (d) x が1つの値に固まっており、離れた1点だけが傾きを決めている

(c) と (d) では、**たった1点を消すだけで結論が変わる**。それでも要約統計量には何の変化も現れない。

2017年には、同じ発想をさらに推し進めて、平均・分散・相関を小数第2位まで保ったまま、点

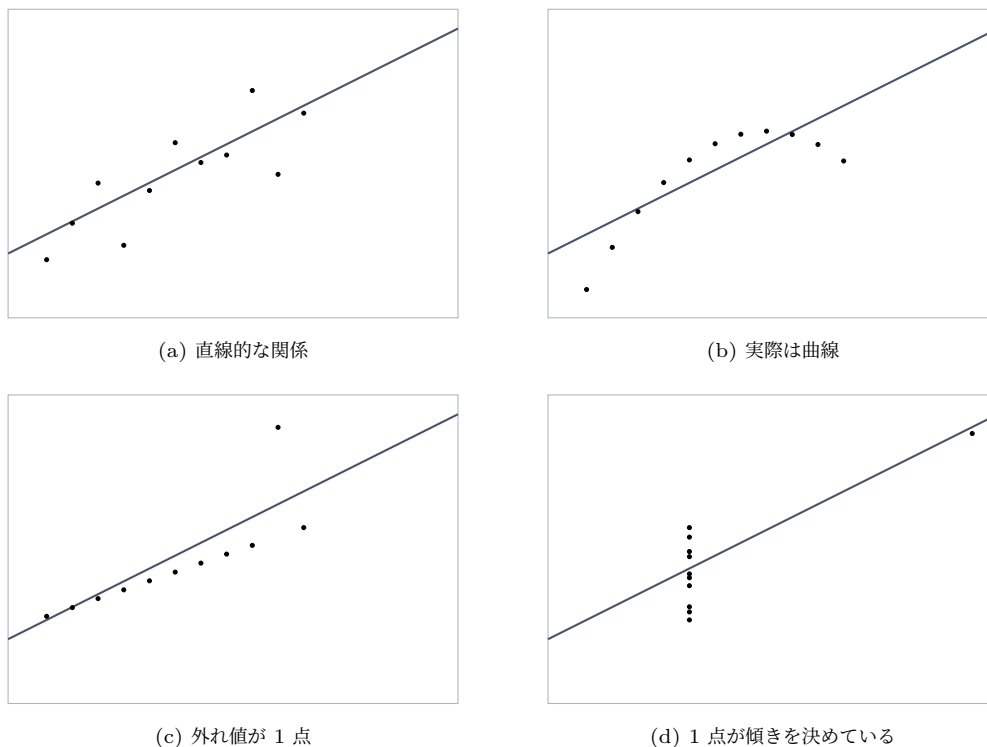


図 4.12 Anscombe の四重奏。4 組とも平均・分散・相関係数 ($r = 0.816$)・回帰直線 (灰) がほぼ一致するが、散布図の形はまるで違う。

の配置を恐竜の絵にまで変形してみせた研究がある [64]。

— ○ 定石 4.3 — 要約した数値を報告する前に必ず散布図を描く —

平均、分散、相関係数、決定係数。どれも分布を数個の数に潰したもののなので、潰す前の形を復元できない。

散布図やヒストグラムは、外れ値、非線形な関係、二峰性、打ち切りといった要約統計量では見えないものを 1 枚で見せてくれる。描くのに数秒しかかからない。

4.9 相関は「伝わらない」

式 (4.25) から、意外な結論が出てくる。

A と B の相関が 0.7、B と C の相関が 0.7 だったとしても、
A と C の相関は負になりうる。

これは直感に反する。「A と B が似ていて、B と C も似ているなら、A と C も似ているはずだ」と考えたくなる。

だが相関係数は推移律を満たさない。

4.9.1 角度で考えると分かる

相関係数が余弦なら、角度に直して考えればよい。

$$r(A, B) = 0.7 \implies \theta_{AB} = \arccos(0.7) = 45.6^\circ \quad (4.26)$$

$$r(B, C) = 0.7 \implies \theta_{BC} = \arccos(0.7) = 45.6^\circ \quad (4.27)$$

角度は三角不等式を満たす (第 4.7.2 節)。したがって θ_{AC} の取りうる範囲は

$$|\theta_{AB} - \theta_{BC}| \leq \theta_{AC} \leq \theta_{AB} + \theta_{BC} \quad (4.28)$$

$0^\circ \leq \theta_{AC} \leq 91.1^\circ$ である。

上限の 91.1° に注目してほしい。 90° を超えている。 $\cos(91.1^\circ) = -0.020$ なので、相関係数は負になりうる。

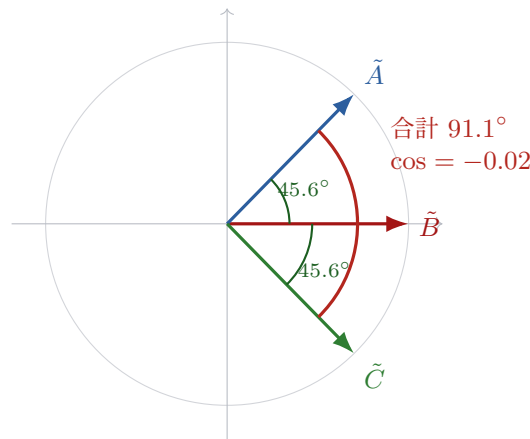


図 4.13 A と C が B の反対側に開くと、両者の角度は 90° を超える。

図 4.13 が状況を示している。A も C も B から 45.6° 離れているが、**反対方向に**離れることができる。そのとき両者は 91.1° 開き、相関は負になる。

4.9.2 3次元で見る — 相関が捉えていない角度*

図 4.13 では話を平面に押し込めたが、偏差ベクトルは本来もっと高い次元にいる。ベクトルが 3 本なら、その 3 本が張る空間は高々 3 次元なので、3 次元の図で全体を正確に描ける。すると、2 次元の図では見えなかったものが見えてくる。

$r(A, B)$ が決めるのは $\tilde{B}$ と $\tilde{A}$ のなす角 θ_{AB} だけである。つまり $\tilde{A}$ は「 $\tilde{B}$ の周りに角度 θ_{AB} で立てた円錐」の上ならどこにいてもよい。 $\tilde{C}$ も同様である。自由に残っているのが、 O, B, A が張る平面と O, B, C が張る平面のなす角 (二面角) φ である。

図 4.14 で、2 枚の面を蝶番のように $\tilde{B}$ の周りで開閉してみてほしい。閉じれば ($\varphi = 0$) $\tilde{A}$ と $\tilde{C}$ は同じ面の同じ側に来て最も近づき、開ききれば ($\varphi = 180^\circ$) 反対側に来て最も離れる。3 つの角の関係は、球面三角法の余弦定理そのものである。

$$\cos \theta_{AC} = \cos \theta_{AB} \cos \theta_{BC} + \sin \theta_{AB} \sin \theta_{BC} \cos \varphi \quad (4.29)$$

$\cos \varphi$ が $+1$ から -1 までの値を取ることが、 $r(A, C)$ に幅を生む。相関係数 $r(A, B)$ と $r(B, C)$

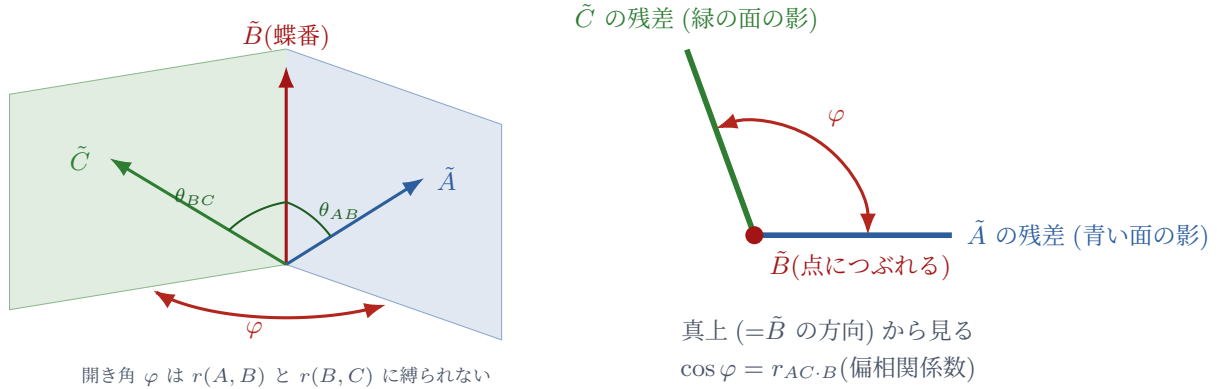


図 4.14 左: 3本の偏差ベクトルを3次元で描く。2枚の面は $\tilde{B}$ を蝶番として屏風のように手前へ折れており、 $r(A, B)$ と $r(B, C)$ が固定するのは2つの角 θ だけ。開き角 (二面角) φ は自由に残る。右: 同じ配置を $\tilde{B}$ の真上から見下ろすと、 $\tilde{B}$ は点につぶれ、2枚の面は2本の線になる。この φ の余弦が偏相関係数である。

は θ を固定するだけで、 φ の情報をまったく持っていない。これが、相関が推移しない理由の幾何的な正体である。

4.9.3 一般の場合*

$r(A, B) = a$ 、 $r(B, C) = b$ のとき、 $r(A, C) = c$ の取りうる範囲は次の式で与えられる。

$$ab - \sqrt{(1-a^2)(1-b^2)} \leq c \leq ab + \sqrt{(1-a^2)(1-b^2)} \quad (4.30)$$

発展 相関の取りうる範囲 (式 (4.30)) の導出

3変数の相関行列

$$R = \begin{pmatrix} 1 & a & c \\ a & 1 & b \\ c & b & 1 \end{pmatrix} \quad (4.31)$$

は半正定値でなければならない。相関行列は $R = \frac{1}{n} \tilde{Z}^T \tilde{Z}$ (正規化した偏差ベクトルのグラム行列。第6章で改めて扱う) の形をしているので、任意のベクトル u に対して $u^T R u = \frac{1}{n} \|\tilde{Z}u\|^2 \geq 0$ である。半正定値の必要条件として $\det R \geq 0$ が要る。

$$\det R = 1 + 2abc - a^2 - b^2 - c^2 \geq 0 \quad (4.32)$$

c についての2次不等式に整理する。

$$c^2 - 2abc - (1 - a^2 - b^2) \leq 0 \quad (4.33)$$

2次方程式 $c^2 - 2abc - (1 - a^2 - b^2) = 0$ の解は

$$c = ab \pm \sqrt{a^2 b^2 + 1 - a^2 - b^2} \quad (4.34)$$

$$= ab \pm \sqrt{(1-a^2)(1-b^2)} \quad (4.35)$$

発展（続き）

平方根の中を因数分解しているところが要点である。 $a^2b^2 - a^2 - b^2 + 1 = (1 - a^2)(1 - b^2)$ が成り立つ。

したがって c は 2 解の間になければならない。これが式 (4.30) である。

角度で書き直すと、加法定理そのものになる。 $a = \cos \alpha$, $b = \cos \beta$ と置けば $\sqrt{(1 - a^2)(1 - b^2)} = \sin \alpha \sin \beta$ なので

$$\cos \alpha \cos \beta - \sin \alpha \sin \beta \leq c \leq \cos \alpha \cos \beta + \sin \alpha \sin \beta \quad (4.36)$$

つまり $\cos(\alpha + \beta) \leq c \leq \cos(\alpha - \beta)$ である。先ほどの三角不等式による議論と完全に一致する。そして式 (4.29) で $\varphi = 180^\circ, 0^\circ$ と置いたものが、ちょうどこの下限と上限になっている。行列式の条件・三角不等式・二面角の開閉という 3 つの見方が、同じ式に合流するわけである。

$a = b = r$ とすると、下限は

$$c_{\min} = r^2 - (1 - r^2) = 2r^2 - 1 \quad (4.37)$$

これがゼロになるのは $r = 1/\sqrt{2} \approx 0.707$ のときである。

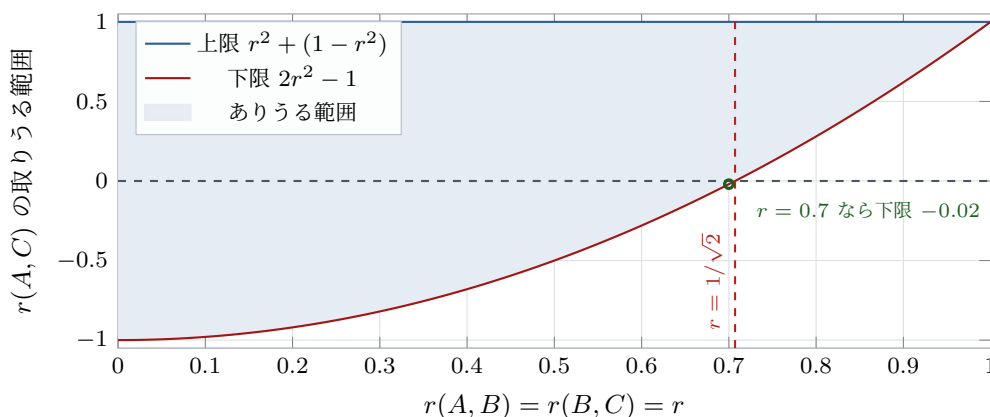


図 4.15 $r(A, C)$ が取りうる範囲。 $r < 1/\sqrt{2}$ では負になりうる。

図 4.15 から読み取れることをまとめる。

$r = 1/\sqrt{2}$ が境界である。これより強い相関なら、間接的な相関も必ず正になる。これより弱ければ、負になる余地が残る。

相関 0.7 は「かなり強い」と言われることが多い。

だがそれでも、推移律を保証するにはわずかに足りない。

4.9.4 本書のデータでの実例

理論だけでは実感が湧かない。実際に本書の 52 個の特徴から 3 列の組をすべて取り出し、2 組の相関係数がどちらも 0.45 を超えていて、残る 1 組の相関係数が負になっているものを探すと、2 組見つかった。そのうち残る 1 組の相関が最も負に振れた組を挙げる。

読点の直前に来る平仮名のうち、「え」「ゃ」「ら」の 3 つである。

表 4.4 $r(A, B) = r(B, C) = r$ のときの $r(A, C)$ の下限

r	角度	$r(A, C)$ の下限	負になりうるか
0.9	25.8°	+0.62	いいえ
0.8	36.9°	+0.28	いいえ
$1/\sqrt{2} \approx 0.707$	45.0°	0.00	境界
0.7	45.6°	-0.02	はい
0.6	53.1°	-0.28	はい
0.5	60.0°	-0.50	はい
0.3	72.5°	-0.82	はい

表 4.5 実データに現れた非推移性

組	相関係数	角度
「え」と「ゃ」	0.474	61.7°
「ゃ」と「ら」	0.548	56.8°
「え」と「ら」	-0.090	95.2°

2つの角度を足すと $61.7 + 56.8 = 118.5^\circ$ になる。これが θ_{AC} の上限である。実際の 95.2° は上限を 23.3° 下回って範囲に収まっており、それでいて 90° は 5.2° 超えている。だから相関が負になった。

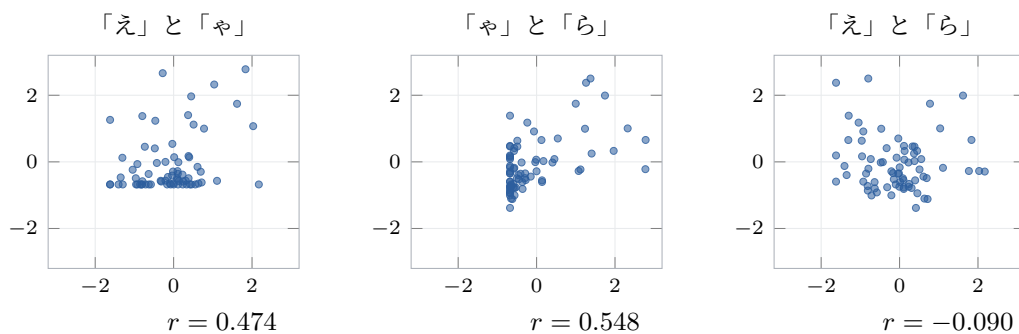


図 4.16 実データの散布図 (標準化済み)。左 2 つはゆるやかな右上がり、右は無相関に近い。

× アンチパターン 4.7 — 相関を連鎖させて推論する —

「A と B に相関がある。B と C にも相関がある。だから A と C にも関係があるはずだ」
推論は成り立たない。相関係数が $1/\sqrt{2}$ を超えていない限り、A と C が無相関どころか負の相関を持つ可能性が残る。

特徴選択で「目的変数と相関の高い特徴を集める」だけでは不十分なのも同じ理由による。

個々に相関が高くても、特徴どうしの関係によって組み合わせたときの寄与はまったく変わる。

4.9.5 どの単位で数えたかで相関は変わる

相関について、もう1つ落とし穴がある。何を1件と数えるかで値が変わることである。

本書のデータで、平仮名の比率と漢字の比率のピアソン相関係数を2通りの数え方で計算してみる。

- **作品を1件と数える** — 74作品それぞれの値で計算する。 $r = -0.915$
- **著者を1件と数える** — 著者ごとに作品の平均を取り、8個の点で計算する。 $r = -0.717$

同じデータから、絶対値で0.198違う2つの相関係数が出た。どちらも計算は正しい。測っているものが違う。

作品単位の-0.915は「**どの作品でも**、平仮名が多ければ漢字が少ない」という関係の強さを表す。第3章で見たとおり、これは合計がほぼ1という制約から来る構造的な負相関である。

著者単位の-0.717は「**どの著者でも**、平仮名をよく使う人は漢字をあまり使わない」という関係の強さである。ただし、著者ごとに平均を取った時点で点は8個しか残らない。8点での相関係数は標本を取り直せば大きくぶれるので、弱まったという向きそのものに意味を読み取ることはできない。

向きが決まっているわけではない。集約すると相関が強くなることもある。個体内のばらつきが消えて、集団どうしの違いだけが残るからである。本書では逆に弱く出た。**どちらに転ぶかは、データの構造しだい**で予測できない。

値の大小より、**集団で計算した相関をそのまま個体にあてはめられないこと**のほうが重い。「著者単位で見ると平仮名と漢字が連動している」からといって、「ある作家が平仮名を多く使った作品では漢字が少ない」とは限らない。前者は著者の間の話、後者は1人の著者の中の話で、別の問題である。

これは**生態学的誤謬** (ecological fallacy) と呼ばれる。都道府県別に集計した相関から個人の傾向を語る、企業別の平均から社員個人の傾向を語る、といった形でよく現れる。

× アンチパターン 4.8 — 集計した単位の相関をその中の個体の話として読む

県ごとの平均所得と平均寿命に相関があっても、「所得の高い人ほど長生きする」とは言えない。測ったのは県の性質であって、人の性質ではない。

報告するときは**何を1件として数えたか**を必ず書く。読むときも、そこを確かめてから解釈する。

補足 偏相関という道具 — 正体は二面角の余弦

「AとCの相関は、Bを介した見かけ上のものではないか」を調べたいときは**偏相関係数** (partial correlation) を使う。Bの影響を取り除いた上での相関である。

説明としては「Bの値を固定したときのAとCの相関」という言い方をよく見かける。条件付きの分布を思い浮かべると身構えてしまうが、幾何の像で見れば、測っているのは1つの角度にすぎない。

補足（続き）

$$r_{AC \cdot B} = \frac{r_{AC} - r_{AB}r_{BC}}{\sqrt{1 - r_{AB}^2}\sqrt{1 - r_{BC}^2}} \quad (4.38)$$

分子と分母は、式 (4.30) とそっくりである。それもそのはずで、球面三角法の式 (4.29) を $\cos \varphi$ について解くと

$$\cos \varphi = \frac{\cos \theta_{AC} - \cos \theta_{AB} \cos \theta_{BC}}{\sin \theta_{AB} \sin \theta_{BC}} = \frac{r_{AC} - r_{AB}r_{BC}}{\sqrt{1 - r_{AB}^2}\sqrt{1 - r_{BC}^2}} \quad (4.39)$$

となり、右辺は式 (4.38) そのものである。つまり

偏相関係数とは、図 4.14 の開き角 φ の余弦である。

偏相関を測るとは、 $\tilde{B}$ の真上に視点を移して、そこから A と C を見下ろすことである (図 4.14 右)。真上からの視点では3つのことが同時に起きる。

- $\tilde{B}$ は1点につぶれて見えなくなる。視線と同じ方向を向いたベクトルは、長さを失うからである。「Bの影響を取り除く」という言い回しの、これが文字どおりの姿である
- $\tilde{A}$ と $\tilde{C}$ は、それぞれ $\tilde{B}$ 方向の成分 (回帰でいえば「Bで説明できる部分」) を視線方向に失い、**残差だけが影として床に残る**。青い面と緑の面は、真上から見れば2本の線である
- その2本の線のなす角こそが、屏風の開き角 φ である。だから「残差どうしの相関」(偏相関の教科書的な定義) と「Bの周りの開き角の余弦」(図の読み方) は同じ量になる

相関係数 $r(A, B)$ と $r(B, C)$ が捉えそこねていた「残りの1つの角」を測る道具が、ちょうど偏相関だったわけである。 $\cos \varphi$ が -1 から 1 に収まることから、偏相関の値域も、式 (4.30) の範囲も同時に出る。

4.10 高次元では距離が壊れる

もう一つ、知っておくべき性質がある。次元が上がると、距離という概念そのものが働かなくなる。 d 次元の単位立方体に点をランダムに撒き、すべてのペアの距離を測って、その分布を描いてみる。比べやすいように、距離は各次元での平均距離で割ってある。

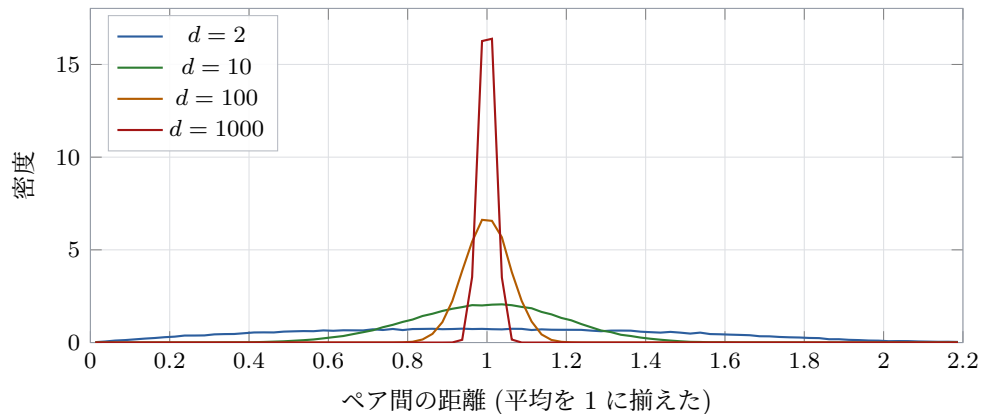


図 4.17 ランダムな点どうしの距離の分布。次元が上がるほど平均のまわりに突き刺さるように集中していく (距離の標準偏差を平均で割った変動係数は $d = 2$ で 0.47、 $d = 1000$ で 0.02)。

図 4.17 を低い次元から順に追う。 $d = 2$ では距離は幅広く散らばっている。近いペアも遠いペア

もある。 $d = 100$ になると分布は細い山になり、 $d = 1000$ ではほとんど 1 本の針である。**どのペアを取っても、距離がほぼ同じになっていく。**これを**距離の集中** (concentration of distances) という。「近い」「遠い」という区別が、次元とともに意味を失っていく。

同じことを別の切り口で測る。次元ごとに単位立方体へ 200 点を撒き、全ペアのユークリッド距離のうち最も小さいものを $d_{\min}$ 、最も大きいものを $d_{\max}$ として、 $(d_{\max} - d_{\min})/d_{\min}$ を見る。

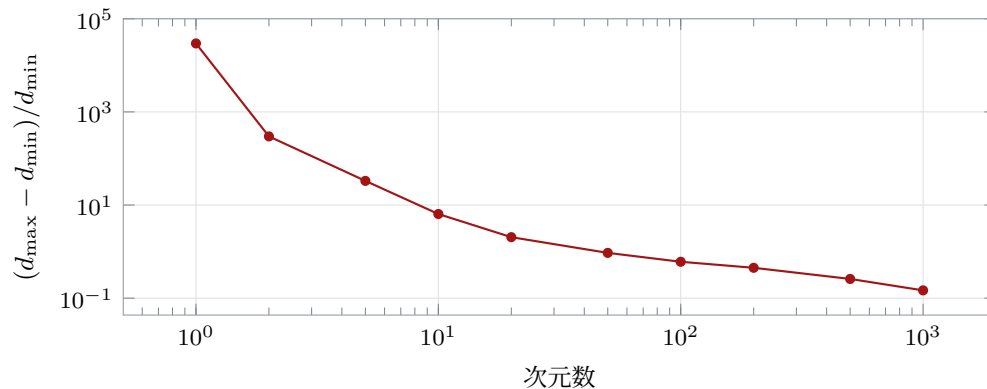


図 4.18 次元と距離の相対的な広がり。次元が上がるほど、最も近い点と最も遠い点の差が消えていく。

1 次元では、最も遠いペアの距離が最も近いペアの 3 万倍ある。1000 次元では、 $d_{\max}$ と $d_{\min}$ の差が $d_{\min}$ の 15% しかない。**すべての点がほぼ等距離になる。**

「近い点を探す」という操作が意味を失う。これが**次元の呪い** (curse of dimensionality) の一つの側面である。第 9 章でこれが実際の分類精度にどう影響するかを測り、付録 B でその数学的な背景を扱う。

4.10.1 p を下げても集中が遅くなるだけで消えない

第 4.6 節では、高次元では p の小さいミンコフスキー距離のほうがよく分ける、と述べた。ならば p を工夫すれば次元の呪いから逃げられるかということ、半分だけ正しい。 $p = 1$ が高次元で相対的に有利なのは、定性的な理由がある。次元が増えるほど、**どこか 1 つの次元で、たまたま大きな差が出る**機会が増える。52 次元で 2 作品を比べるのは、くじを 52 回引くようなもので、外れ値めいた成分が紛れ込む余地は次元とともに広がっていく。大きな p は、その偶然の 1 つの差を増幅して距離全体を支配させてしまう ($p = \infty$ はその極限で、最大の差しか見ない)。 $p = 1$ なら、偶然の差も 1 票としてしか数えないので、多数の次元に薄く広がった本物の差が生き残る。外れ値に対するマンハッタン距離の強さと同じ理屈が、高次元では「外れ値的な成分が確率的に必ず混ざる」という形で常時はたらく。

だがこれは呪いの**緩和**であって、解決ではない。図 4.17 で見た**距離の集中**は、どの p を選んでも起きる。集中の速さが変わるだけで、「全ペアがほぼ等距離」へ向かう流れそのものは止められない。

4.10.2 p を決めているのは次元ではなく、データの狂い方である

p を次元の問題として扱うのをやめると、選び方の見当がつく。

同じものを指しているはずの 2 つのベクトルの値がずれるとき、そのずれ方には型がある。すべて

の座標が少しずつ揺れる型と、少数の座標だけが大きく壊れる型である。後者は身の回りに多い。文字認識の読み違い、センサの値の飛び、写真に散る白黒の点。1 か所か 2 か所だけが大きく狂って、残りは無傷である。

100 次元・800 点の人工データで確かめた。もとの点に狂いを加えたものを問い合わせにして、 L_p の最も近い点がもとの点を返す割合を測る。狂いの大きさ (二乗和の期待値) は両条件でそろえてあり、片方は 100 座標すべてに薄く配り、もう片方は無作為な 4 座標に集めた。

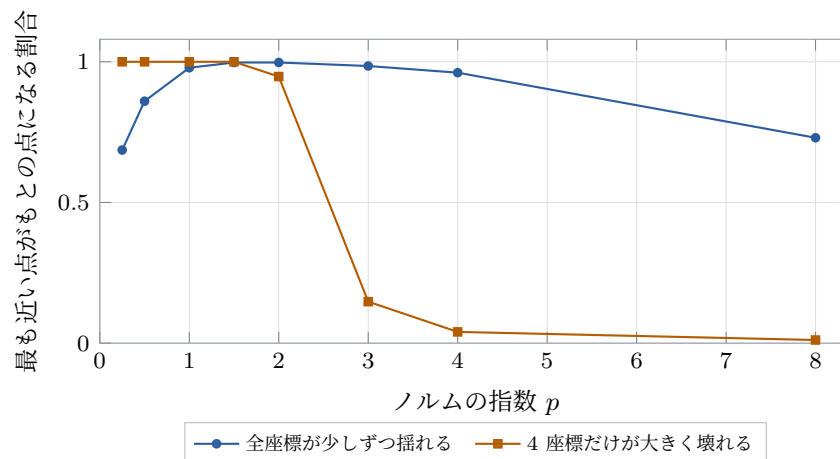


図 4.19 狂い方の型ごとの、最も近い点の当たり方。次元は両条件とも 100 で、狂いの総量もそろえてある。それでも最適な p が 2 付近から 1 以下まで移る。

全座標が少しずつ揺れるときは、もとの点を返す割合が $p = 1.5$ から 2 で最も高く 0.998、 $p = 0.25$ では 0.686、 $p = 8$ では 0.730 と両側で下がる。4 座標だけが壊れるときは、同じ割合が $p \leq 1.5$ ではすべて 1.000、 $p = 2$ で 0.948、 $p = 3$ で 0.147、 $p = 8$ で 0.011 である。 $p = 1.5$ と $p = 3$ のあいだで、割合が 1.000 から 0.147 へ落ちる。

理屈は $p = 2$ が差を二乗することにある。4 つの座標が大きくずれていると、その 4 つだけで距離が決まってしまう、無傷の 96 座標が一致していることが結果に表れない。 p を下げると大きい成分の支配力が下がるので、96 座標の一致が生き残る。逆に、全座標が薄く揺れているときに p を下げすぎると、本物の差まで薄まって当たらなくなる。

次元は両条件とも 100 である。それでも最適な p が移るのだから、 p を決めているのは次元ではない。見るべきは、そのデータがどう狂うかである。

4.10.3 集中を決めるのは列の数ではなく内在次元

本書の表は 74 行 $\times$ 52 列である。列が 52 もあるのだから集中も深刻だろう、と考えたくなる。だが列が 52 個あることと、点が 52 次元いっぱいには散らばっていることは別である。

確かめるには列シャッフルという操作を使う。列ごとに、行の順番をばらばらに入れ替える。こうすると各列のヒストグラムはそのまま、列どうしの連動だけが消える。「平仮名の多い作品は漢字が少ない」のような結び付きが無くなり、点は本当に 52 次元へ散らばる。

表 4.6 の数値を読む。もとのデータは、ばらつきの 9 割を主成分 18 本で説明できる。列シャッフルで連動を壊すと、同じ 9 割に主成分が 31 本要る。相対コントラストは 3.180 から 0.749 へ、4 分

表 4.6 列の連動を壊すと集中が強まる (74 作品、列ごとに標準化)。相対分散は中心からの距離のばらつきを平均で割った量、相対コントラストは、各点から見て最も遠い点と最も近い点のユークリッド距離の差を、最も近い距離で割った量である。どちらも大きいほど近傍を区別しやすい。

データ	相対分散	相対コントラスト	9 割に要る成分数
本書の 52 列	0.317	3.180	18
列シャッフル後	0.183	0.749	31
同じ大きさの独立乱数	0.107	0.540	31

の 1 以下に下がる。連動をまったく持たない独立乱数の 0.540 に近い値である。同じ 74 行 $\times$ 52 列の形をしていても、連動があるほうが近傍をはっきり区別できる。

つまり呪いの深刻さを決めているのは列の数ではなく、データが実際に広がっている自由度の数である。これを**内在次元**という (第 6 章)。François らはこの点を示して、実際の場面での深刻さは数学の予想より軽い、と述べている [7]。

× アンチパターン 4.9 — 列の数だけを見て次元の呪いを心配する

「52 列もあるから距離が壊れているはずだ」とは言えない。列がいくつあっても、それらが連動していれば点は低い次元の面の上に乗っていて、近傍は区別できる。

逆も言える。列を 10 個に減らしても、その 10 個が互いに無関係なら集中は残る。減らすべきなのは列の数ではなく、無関係な方向である。

心配する前に測る。相対コントラストか、主成分分析で累積寄与率 9 割に要る成分数を見れば見当がつく。

4.10.4 みんなの近傍に何度も現れる点がある

集中とは別に、高次元ではもう一つのこと起きる。 k 近傍をすべての点について求めると、どの点も平均すればちょうど k 回だけ誰かの近傍リストに載るはずである。ところが高次元では、この回数が特定の点に偏る。

表 4.7 ユークリッド距離で求めた k 近傍リストに、各点が現れた回数。歪度が大きいほど偏っている。平均は次元によらず k である。

データ	k	出現回数の歪度	最大
独立乱数 $n = 1500, d = 3$	10	-0.49	19
独立乱数 $n = 1500, d = 20$	10	3.25	123
独立乱数 $n = 1500, d = 100$	10	3.84	160
本書の 52 列	5	0.24	13
列シャッフル後	5	1.09	20

独立な乱数 1500 点について $k = 10$ の近傍リストを作り、各点が何回載ったかを数えた。3 次元では最大が 19 回で、平均の 10 回の 2 倍に満たない。100 次元では最大が 160 回で、1 つの点が 1500 点の 1 割以上の近傍リストに載っていた。こういう点を**ハブ** (hub) と呼ぶ [8]。ハブになりやすいの

はデータの中心に近い点で、 k 近傍法はこの点を答えとして返しがちになる。正規化やコサイン距離への切り替えでは消えない。

本書の 74 作品を列ごとに標準化して $k = 5$ で同じ数え方をすると、出現回数の歪度は 0.24、最大は 13 回で、偏りは出ていない。列シャッフルで連動を壊すと歪度が 1.09、最大が 20 回になる。偏りが出ていない理由は前項と同じく、内在次元が低いことにある。

対処としては、距離を近傍の分布に照らして測り直す方法が知られている。各点の k 番目の近傍までの距離で正規化する局所スケーリングや、「互いに近いと思っているか」を対称に測る相互近接度がある。指数 p をいじるより、こちらのほうが偏りは減る。

4.10.5 呪いの弱い低次元の構造を探して、そこで勝負する

呪いを正面から解く方法は知られていない。実務での対処は、どれも「高次元空間をそのまま相手にしない」ことに帰着する。

1. **低次元の構造に賭ける。** データが 52 次元いっぱい広がっていることは実際には少なく、多くは低い内在次元の面の上に乗っている (多様体仮説、第 6 章)。特徴選択と次元削減が働くのは、賭けが当たるときである。
2. **モデルの自由度を絞る。** 正則化、木の深さの制限、単純なモデル。高次元で好き放題に境界を引ける自由こそが過学習の源なので、その自由を削る (第 7・8 章)。
3. **データを増やす。** ただし高次元空間を点で「埋める」には次元に対して指数的な数が要る (付録 B)。10 倍程度に増やしたくらいでは足りないことが多い。
4. **距離のいらぬ情報を足す。** 距離の定義をいじり続けるより、判別に直結する特徴を 1 つ作るほうが早いことが多い (第 3 章)。

4 つとも、呪いを解くのではなく避けている。第 6 章の次元削減と、第 7・8 章の正則化は、そのための道具である。

○ 定石 4.4 — 距離を使う手法にける前に次元を落とす —

k 近傍法、 k -means、コサイン類似度による検索。どれも距離が意味を持つことを前提にしている。次元が高いままこれらにかけると、第 9 章で見るとおり、無関係な列に埋もれて成績が落ちる。

順序を決めておく。次元を落としてから距離を測る。落とし方は特徴選択でも次元削減でもよい。落とさずに済ませたいなら、木ベースの手法のように距離を使わない手法を選ぶ。

4.11 章末の課題

1. 図 4.1 にならって、 $p = 0.5$ のときの単位球を描け。これは距離の 4 条件を満たすか。三角不等式を確かめよ。
2. 標準化する前と後で距離行列がどう変わるか、図 4.7 を作り直して比べよ。
3. コサイン距離と相関距離の効果量がほぼ同じになった。2 つの定義を見比べ、なぜそうなるか説明せよ。
4. 第 3 章の特徴のうち、文長 μ と文長 σ を除いて表 4.3 を作り直せ。順位は変わるか。

5. 表 4.2 で、標準化なしの SVM の正解率 0.284 が「最頻クラスを答え続ける箱」であることを、混同行列 (第 8 章。行を正解、列を予測として数えた表) を描いて確かめよ。(ヒント: 予測列がすべて同じ著者になっているはずである。なぜ吉川英治なのかも考えよ。)

第5章

まとまりを見つける

5.1 正解を見せずに分ける

ここまでで、74 作品それぞれが 52 個の数値になり、作品どうしの距離が測れるようになった。

本章では**著者名を一切使わずに**、作品をグループに分ける。これを**クラスタリング (clustering)** という。

正解を隠すのは、実際の場面では正解がないことが多いからである。著者不明の文書群を整理したい、顧客をタイプ分けしたい、遺伝子の発現パターンをまとめたい。どれも正解ラベルは存在しない。

本書では著者名という正解を持っているが、それは**答え合わせのため**だけに使う。分ける作業そのものには使わない。

5.2 階層クラスタリング

考え方は素朴である。一番近いもの同士をくっつける。それを繰り返す。

アプローチは 2 種類ある。

- **凝集型 (agglomerative)** — 各点を 1 つのクラスタとして始め、近いものから併合していく。1 つになるまで続ける
- **分割型 (divisive)** — 全体を 1 つのクラスタとして始め、分割していく。点の数だけのクラスタになるまで続ける

実際に使われるのはほぼ凝集型である。分割型は「どこで 2 つに割るか」の候補が点の数に対して爆発的に増えるので、計算が現実的にならない。以下では凝集型を扱う。

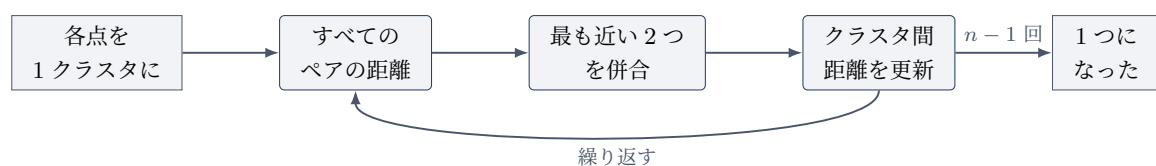


図 5.1 凝集型階層クラスタリングの手順。

5.2.1 5 点で追いかける

5 個の点で実際に動かしてみる。図 5.2 の各段階を追ってほしい。

併合の順番と距離を表にする。

併合の履歴を木の形に描いたものが**デンドログラム**である。横軸 (この図では縦軸) が併合したときの距離になる。

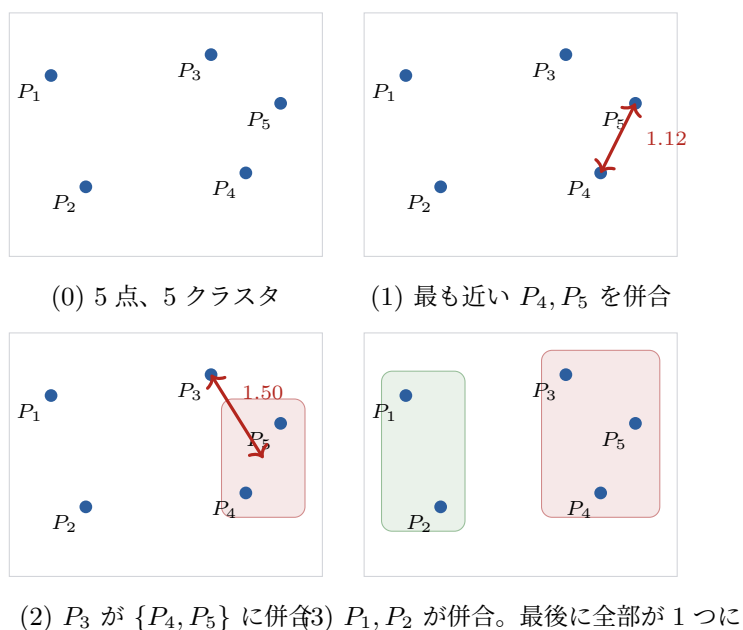


図 5.2 凝集型階層クラスタリングの過程 (群平均法)。数字は併合したときの距離。

表 5.1 併合の履歴

段階	併合したもの	距離	併合後の大きさ
1	P_4, P_5	1.118	2
2	$P_3, \{P_4, P_5\}$	1.496	3
3	P_1, P_2	1.676	2
4	$\{P_1, P_2\}, \{P_3, P_4, P_5\}$	2.791	5

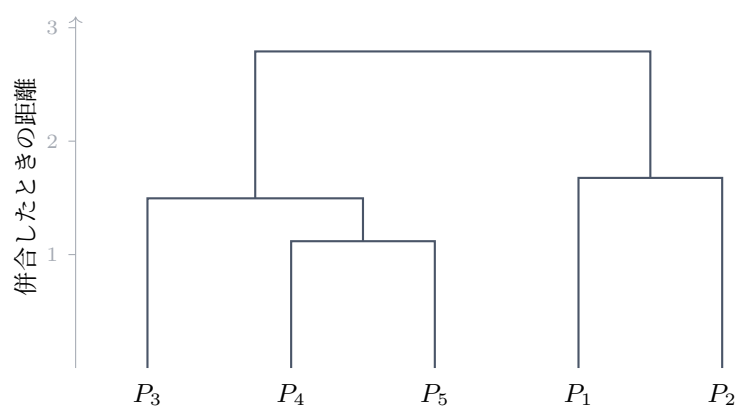


図 5.3 図 5.2 に対応するデンドログラム。

補足 デンドログラムはトーナメント表として読める

デンドログラムはトーナメント表と同じ構造である。下から勝ち上がっていく代わりに、近いもの同士が合流していく。

高さは「どれだけ離れていたか」を表す。低い位置で合流したペアは似ており、高い位置まで別々だったものは似ていない。

5.3 クラスタ間距離の4つの定義

図 5.1 の手順のうち「クラスタ間距離を更新」の段では、**クラスタとクラスタの距離**を決めなければならない。点と点の距離は第4章で決めたが、複数の点を含む集まりどうしの距離は別に定義しなければならない。

主に4つの流儀がある。クラスタを C_i, C_j とし、点間の距離を $d(x, y)$ とする。

はじめの3つは、点と点の距離から**クラスタ間の距離**を作る。

$$\text{単連結 (最短距離法)} \quad D(C_i, C_j) = \min_{x \in C_i, y \in C_j} d(x, y) \quad (5.1)$$

$$\text{完全連結 (最長距離法)} \quad D(C_i, C_j) = \max_{x \in C_i, y \in C_j} d(x, y) \quad (5.2)$$

$$\text{群平均法} \quad D(C_i, C_j) = \frac{1}{|C_i||C_j|} \sum_{x \in C_i} \sum_{y \in C_j} d(x, y) \quad (5.3)$$

4つ目の **Ward 法**だけは、発想が違う。測るのは距離ではなく、**併合するとどれだけ損をするか**である。クラスタ内の平方和 (各点から重心までの距離の2乗の和) を E と書くと、 C_i と C_j をくっつけたときの増分は次の形になる。

$$\Delta E(C_i, C_j) = \frac{|C_i||C_j|}{|C_i| + |C_j|} \|\bar{r}_i - \bar{r}_j\|^2 \quad (5.4)$$

$\bar{r}_i$ は C_i の重心である。毎回、増分が最も小さい組を選んで併合する。

式の形を言葉にしておく。後ろの $\|\bar{r}_i - \bar{r}_j\|^2$ は2つの重心の隔たりで、離れた塊どうしをくっつけるほど大きくなる。前の $\frac{|C_i||C_j|}{|C_i| + |C_j|}$ はクラスタの大きさによる重みで、どちらも大きい塊どうしをくっつけるほど大きくなる。**遠いものをくっつけても、大きいものをくっつけても損をする**、という形をしている。

式 (5.4) を上の3つと見比べてほしい。**量の種類がそろっていない***1。上の3つは距離そのもの、つまり長さの量だが、 ΔE は**ユークリッド距離の2乗**に比例するので、面積にあたる量である。Ward 法だけを「クラスタ間距離」と呼ぶのは、正確には言い過ぎである。

デンドログラムの縦軸に何を置くかも、この違いから来る。多くの実装は、長さの量に戻した $\sqrt{2\Delta E}$ を高さに使う (本書の図もそうである)。2点をくっつける場合、式 (5.4) で $|C_i| = |C_j| = 1$ とすると係数が $\frac{1 \times 1}{1+1} = \frac{1}{2}$ になるので、 $\Delta E = \frac{1}{2}d^2$ である。これを $\sqrt{2\Delta E}$ に入れば d に戻る。つまり高さはちょうど点間の距離になる (実際、(0,0) と (1,0) の2点を scipy に渡すと、デンドログラムの高さは 1.000 になり、2点のユークリッド距離 1 に一致する)。

*1 ここでいう「種類」は、長さ・面積・体積のような量の区別のことで、空間の次元の話ではない。

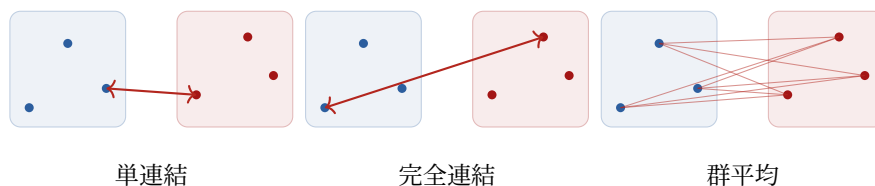


図 5.4 クラスタ間距離の測り方。赤い線が採用される距離。

それぞれに癖がある。

表 5.2 4 つの方法の得失

方法	長所	短所
単連結	細長い形のクラスタも見つけられる	近い点が 1 つあれば次々に併合する。鎖効果 (chaining effect) で細長く伸び、外れ値に弱い
完全連結	直径のそろったクラスタができる	大きなクラスタが小さなクラスタを飲み込む。外れ値に弱い
群平均	外れ値に強い。単連結と完全連結の折衷	球形のクラスタを作る傾向
Ward	外れ値に強く、大きさのそろったクラスタ。k-means と似た結果	球形のクラスタを作る傾向。ユークリッド距離が前提 (第 5.6 節)

発展 Lance-Williams の漸化式

4 つの方法は、実は一つの式で書ける [27]。 C_i と C_j を併合して C_{ij} を作ったとき、別のクラスタ C_k との距離は

$$D(C_{ij}, C_k) = \alpha_i D(C_i, C_k) + \alpha_j D(C_j, C_k) + \beta D(C_i, C_j) + \gamma |D(C_i, C_k) - D(C_j, C_k)| \quad (5.5)$$

で与えられる。 $n_i = |C_i|$ として、係数は次のとおりである。

方法	α_i	α_j	β	γ
単連結	1/2	1/2	0	-1/2
完全連結	1/2	1/2	0	1/2
群平均	$\frac{n_i}{n_i + n_j}$	$\frac{n_j}{n_i + n_j}$	0	0
Ward	$\frac{n_i + n_k}{n_i + n_j + n_k}$	$\frac{n_j + n_k}{n_i + n_j + n_k}$	$\frac{-n_k}{n_i + n_j + n_k}$	0

漸化式の形にしておくと、距離行列を毎回作り直さずに更新できる。実装が一つで済むのも利点である。

なお、係数の表が示すように、 $\gamma \neq 0$ の単連結・完全連結は距離の順序しか使わない。だから任意の距離と組み合わせられる。一方 Ward は重心の差の 2 乗を使うので、ユークリッド距離を前提とする。

Ward の行を使うときは、何を漸化式に入れるかに注意がいる。式 (5.4) が 2 乗の量なので、表の係数は距離の 2 乗に対して書かれている。距離をそのまま入れると、Ward の基準を実装していないことになる。

発展（続き）

歴史的な混乱がここにある。R の `hclust` には `ward.D` と `ward.D2` の 2 つがあり、前者は距離をそのまま、後者は 2 乗してから漸化式に入れて最後に平方根を取る。Ward の基準を実装しているのは後者だけである [35] (`scipy` の `linkage(method="ward")` も後者にあたる)。2011 年より前の R では、前者が単に "ward" という名前で提供されていた。

本書のデータで両方を計算する。74 作品をクラスタリングして $k = 8$ で切り、正解の著者との一致度を ARI で測った (ARI は第 5.5 節で定義する指標で、完全一致で 1 になる)。読点前 47 列を行ごとに合計 1 へ正規化し、ユークリッド距離を使った場合、ARI は `ward.D2` で 0.703、`ward.D` で 0.522 である。52 列を列ごとに標準化した場合も、0.872 対 0.691 になる。どちらの入力でも、同じ「Ward 法」という名前で ARI が 0.18 も違う。論文や報告で「Ward 法を使った」とだけ書かれていても、どちらなのかは分からない。

5.3.1 計算量

n 点に対して、すべてのペアの距離を保持するので空間は $O(n^2)$ 必要である*²。時間は素朴な実装で $O(n^3)$ 、工夫して $O(n^2 \log n)$ になる。

74 点なら一瞬だが、数万点を超えると現実的でなくなる。そこで次節の k-means が使われる。

5.4 切る高さはデンドログラムだけでは決まらない

デンドログラムを、高さのある位置で水平に切る。図 5.3 のように縦軸に距離を取った図なら、横線を 1 本引くことになる。線が枝を横切った本数が、そのときのクラスタ数である。高い位置で切れば少数の大きなクラスタ、低い位置で切れば多数の小さなクラスタになる。

利点はクラスタ数を先に決めなくてよいことである。木を作ってから、後で切る位置を選べばよい。

問題はどこで切るのが正しいのかである。デンドログラムは答えを教えてくれない。切る位置を選ぶには別の物差しが要る。次節で 2 種類用意する。

5.5 答え合わせ — 2 種類の評価

クラスタリングの結果を評価する方法は 2 種類ある。正解を使うものと、使わないものである。

5.5.1 正解を使う — Adjusted Rand Index

評価の前に、1 つ困ったことがある。クラスタリングが出すのは「クラスタ 1」「クラスタ 2」という番号だけで、どの番号が誰にあたるかは教えてくれない。番号の付き方は実行するたびに変わる。だから、著者名とクラスタ番号を直接つぎ合わせるができない。

そこで 2 作品の組を単位にして比べる。「この 2 作品が同じ組に入ったかどうか」なら、番号の付き方に関係なく決まるからである。

Rand Index は、すべての作品ペアについて次を数える。

- 同じ著者で、同じクラスタに入った (a 組)。正解

*² $O(n^2)$ は「点の数が 10 倍になると、必要な量がおおよそ 100 倍になる」という増え方を表す記法である。 $O(n^3)$ なら 1000 倍になる。

- 違う著者で、違うクラスタに入った (b 組)。これも正解
- それ以外 (誤り)

$$RI = \frac{a + b}{\binom{n}{2}} \quad (5.6)$$

分母はペアの総数である。

ただしこれには問題がある。クラスタを細かく割るほど、「違う著者どうしがたまたま違うクラスタに入る」ペアが自動的に増える。 b の項がひとりでに膨らむので、でたらめに分けても k が大きければ高い値が出てしまう。

そこで、**でたらめに分けたときに期待される値**を引いてから、完全一致が 1 になるように割り直す。これが **Adjusted Rand Index (ARI)** である。

$$ARI = \frac{RI - E[RI]}{\max(RI) - E[RI]} \quad (5.7)$$

$E[RI]$ がでたらめな分割での期待値、 $\max(RI)$ が完全一致のときの値である。引き算と割り算で目盛りを付け直しただけで、完全一致で 1、でたらめな分割で 0 前後になる。偶然より悪ければ負にもなる。

5.5.2 正解を使わない — シルエット係数

実務では正解がない。そのときは「クラスタがどれだけよくまとまっているか」を測るしかない。各点 i について 2 つの量を計算する。

$$a(i) = \frac{1}{|C_i| - 1} \sum_{j \in C_i, j \neq i} d(i, j) \quad (\text{同じクラスタ内の平均距離}) \quad (5.8)$$

$$b(i) = \min_{C_k \neq C_i} \frac{1}{|C_k|} \sum_{j \in C_k} d(i, j) \quad (\text{最も近い他クラスタへの平均距離}) \quad (5.9)$$

C_i は点 i が属するクラスタである。 $a(i)$ は同居している仲間との近さ、 $b(i)$ は**いちばん引っ越したくなる隣のクラスタ**との近さを表す。この 2 つを比べる。

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))} \quad (5.10)$$

分子は「隣より仲間のほうがどれだけ近いか」で、分母は値を -1 から 1 に収めるための割り算である。

- 1 に近い — 自分のクラスタによくまとまっている
- 0 付近 — クラスタの境界上にいる
- 負 — 別のクラスタに入れたほうがよい

全点の平均が**シルエット係数** (silhouette coefficient) である。

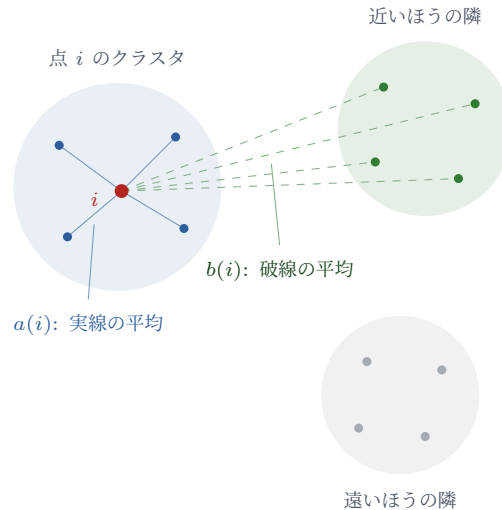


図 5.5 シルエット係数が見ている 2 つの距離。 $a(i)$ は同じクラスターの仲間まで (実線) の平均、 $b(i)$ は他のクラスターのうち最も近いものまで (破線) の平均である。遠いほうの隣は $\min$ を取る時点で捨てられるので、値には現れない。実線が短く破線が長いほど $s(i)$ は 1 に近づく。

5.5.3 2 つの指標は一致しない

52 列を列ごとに標準化した 74 作品を、Ward 法とユークリッド距離で階層クラスタリングする。クラス数 k を 2 から 20 まで変えて切り、そのつど ARI とシルエット係数を計算したのが図 5.6 である。

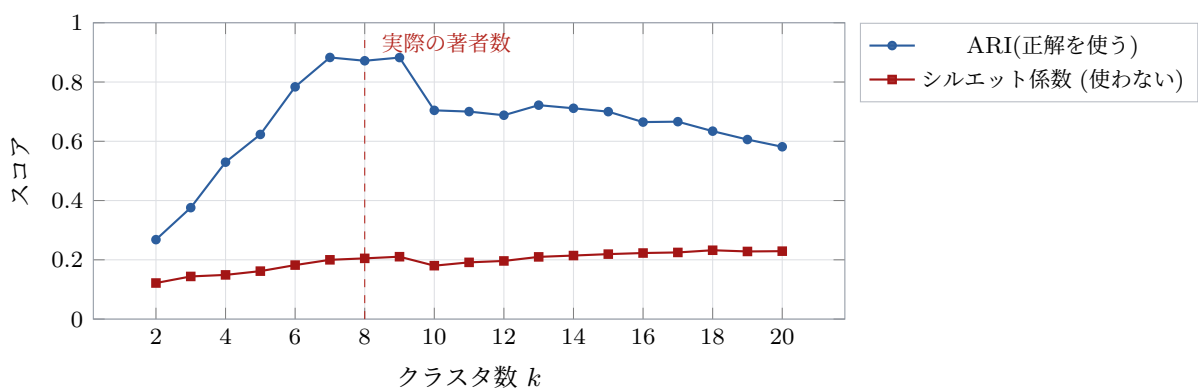


図 5.6 クラス数と 2 つの評価指標 (Ward 法)。

ARI は $k = 7$ から $k = 9$ で頭打ちになり、著者数と同じ $k = 8$ では ARI が 0.87 である。ARI は完全一致で 1、でたらめな分割で 0 前後になる目盛りなので、0.87 は正解の著者分けにかなり近い。文字の使い方だけで、ここまで著者が分かれるのは注目してよい。

一方シルエット係数が最大になるのは $k = 18$ である。著者数の倍以上に細かく割ったところである。

細かく割ると、同居している仲間との距離 $a(i)$ は小さくなるが、最も近い他クラスターとの距離 $b(i)$ も同時に小さくなるので、細かいほど $s(i)$ が上がると一般には言えない。本書のデータで $k = 18$

が最大になるのは、著者の内側に「本当によく似た数点」の密な小集団があり、そこまで割ると $a(i)$ の減少が $b(i)$ の減少に勝つからである。シルエット係数は著者という正解を見ていないので、「割りすぎだ」と判断する材料をそもそも持っていない。

× アンチパターン 5.1 — シルエット係数が最大の k を採用する —

シルエット係数は「球状にまとまったクラスター」を高く評価する。実際の構造が細長かったり、大きさがばらばらだったりすると、正しい k で最大にならない。

内的指標は参考であって決定基準ではない。分野の知識、後段の使い道、結果の解釈しやすさを合わせて決める。

5.6 距離と併合方法の相性

評価の道具がそろったので、ここで距離と併合方法の組み合わせに戻る。最もよく使われる Ward 法は、式 (5.4) のとおりユークリッド距離を前提にしている (他の距離を渡すと何が起きるかは本節の発展で、実務上の扱いは節末の定石で詳しく見る)。

第 4 章で、本書の題材ではコサイン距離のほうがよく分けることを見た。では Ward 法 + コサイン距離にすればよいかというと、そう単純ではない。

図 5.7 に、方法を変えた 2 つのデンドログラムを並べる。

(a) の単連結では、木が一方向に伸びている。表 5.2 で述べた鎖効果である。点が数珠つなぎに併合されていき、クラスターらしいまとまりが見えにくい。

(b) の群平均 + コサイン距離では、まとまりがはっきりしている。

発展 Ward 法にユークリッド距離以外を渡すと何が起きるか

式 (5.4) は重心の差の 2 乗で書かれていて、そこには「距離が 2 乗和で測られている」という前提が入っている。

ところが実務では、 $p = 1$ のマンハッタン距離と Ward を組み合わせたものが最も ARI が高くなるという報告をよく聞く。筋の通らない組み合わせに見える。確かめてみる。

読点前の平仮名 47 列だけを取り出し、行ごとに合計 1 へ正規化したものを入力にする (字種と文長は混ぜない)。つまり各行は **47 文字の上の確率分布**である。これを距離と併合方法のすべての組み合わせでクラスタリングし、正解の著者との一致度を ARI で測った ($k = 8$ で切った)。

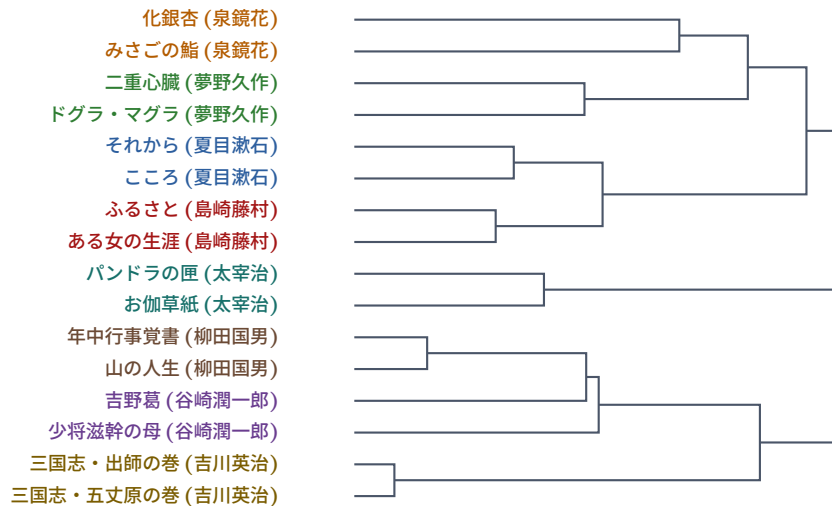
距離	生の比率 (行和 1)			列を標準化		
	完全連結	群平均	Ward	完全連結	群平均	Ward
$p = 0.5$	0.425	0.079	0.600	0.642	0.085	0.633
$p = 1$ (マンハッタン)	0.592	0.310	0.891	0.363	0.029	0.661
$p = 2$ (ユークリッド)	0.338	0.576	0.703	0.109	0.029	0.621
$p = 4$	0.384	0.570	0.434	0.094	0.034	0.692
コサイン	0.330	0.464	0.654	0.612	0.911	0.617

生の比率のままなら、 $p = 1$ と Ward の組み合わせの ARI が **0.891** で最も高い。 $p = 2$ (ユークリッド距離) と Ward の 0.703 を 0.188 上回っている。報告のとおりになった。

$p = 1$ が良い理由は、データが確率分布だからである。2 つの分布 u, v のあいだのマンハッタン距離^a



(a) 単連結・ユークリッド距離



(b) 群平均・コサイン距離

図 5.7 方法を変えると木の形が変わる。上の単連結は鎖状に伸びている。

発展 (続き)

は総変動距離の 2 倍に等しく、「一方の書き手の癖をもう一方に作り変えるのに、どれだけの確率の質量を移せばよいか」という読み方ができる。分布どうしを比べるのに素直な量である。第 4.7.2 節で触れたとおり、自然言語処理で分布どうしの近さを測る尺度を比較した研究でも、コサインのような幾何的な尺度より、分布の差そのものを見る尺度のほうが、類義語の判定で上回っている [19]。

$p = 2$ が劣るのは、2 乗が頻度の高い数文字を強調するからである。47 列の比率を 74 作品で平均すると、「て」が 0.176、「は」が 0.122、「が」が 0.089 で、この 3 文字だけで各行の合計 1 のうち 4 割近くを占める。その 3 文字の差だけで距離がほぼ決まってしまう。マンハッタン距離なら、どの文字も比率の差そのままの重みで距離に加わる。

では p は小さいほどよいかというと、そうではない。Ward 法のまま $p = 0.5$ にすると、ARI は 0.600

発展 (続き)

に下がる。 $p = 1$ の 0.891 を 0.291 下回っている。1 を下回ると、めったに出ない文字のわずかな差が過大に扱われる。比率 0.001 と 0.002 の違いは、たいてい書き手の癖ではなく数え上げの揺らぎである。最良の p は 1 のあたりにある。

Ward が良い理由は距離とは別である。併合の費用に $|C_i||C_j|/(|C_i| + |C_j|)$ という係数が掛かるので、**大きいクラスタどうしの併合が高くつく**。これが単連結の鎖効果と完全連結の飲み込みを両方抑える。距離の選択と併合規則の選択は別々の問題を担当しているので、組み合わせても打ち消し合わない。

計算そのものは、前の発展「Lance-Williams の漸化式」に任意の距離行列を入れれば回る。ただしそのとき得られているのは**平方和の最小化ではなく、大きい併合を罰する併合規則**である。デンドログラムの高さを「クラスタ内平方和の増加量」としては読めない。

上の表の数値は、距離を 2 乗してから漸化式に入れる実装 (scipy の `ward`、R の `ward.D2`) で計算した。距離をそのまま入れる古い実装 (R の `ward.D`) では別の数値になる。その違いは前の発展の末尾で測ったとおりである。

^a 同じ距離が 3 通りに呼ばれるので、ここで整理しておく。手法として呼ぶときは第 4 章で導入した**マンハッタン距離**、ミンコフスキー距離の族の中での位置を示すときは $p = 1$ 、ノルムの性質そのものを指すときは ℓ_1 距離である。とくに分布どうしの比較や、正則化との対応を論じる文脈では ℓ_1 と書く慣習が強い。本書も 3 つを文脈で使い分ける。なお、 $p = 1$ の p はミンコフスキー距離の指数であって、確率を表す p ではない。紛れないう、ここでは分布を u, v と書いた。

5.6.1 前処理は ARI の高さでは選べない*

前の箱の表には、まだ説明していない列がある。右半分、**47 列を列ごとに標準化した場合**である。測り方は左半分と同じで、 $k = 8$ で切ったときの ARI である。そこではコサイン距離と群平均法の組み合わせの ARI が 0.911 で最も高く、生の比率での最良である $p = 1 + \text{Ward}$ の 0.891 を 0.020 上回っている。

結果だけ見れば標準化したほうが良い。ところが第 4.4.2 節の場合分けに従うなら、**47 列はすべて同じ単位の比率であり、標準化する理由がない**。しかも合計が 1 という制約があるので、列ごとに平均を引いて標準偏差で割れば、その制約は壊れる。組成データには `clr` 変換を使えというのが第 3.7 節の話でもあった。結果と原則が食い違っている。どちらを信じるかを決める前に、標準化が実際に何をしているかを見る。

標準化は列の重みを 40 倍まで変える 標準化は各列を $1/s_j$ 倍する。 s_j は列の標準偏差なので、比率の小さい文字ほど大きく引き伸ばされる。

文字	平均比率	実測 SD	数え上げ SD	実測/数え上げ	F 値
て	0.1760	0.0419	0.0055	7.6	17.2
は	0.1217	0.0465	0.0047	9.9	26.5
が	0.0892	0.0322	0.0041	7.8	21.0
ん	0.0073	0.0082	0.0012	6.7	12.4
う	0.0070	0.0054	0.0012	4.5	6.6
っ	0.0012	0.0020	0.0005	4.1	22.4
げ	0.0012	0.0012	0.0005	2.4	5.1
び	0.0011	0.0011	0.0005	2.2	9.9

最も比率の大きい「て」の $1/s_j$ は 24 倍、最も小さい「び」は 945 倍で、両者の倍率の比は 40 倍になる。標準化は列どうしの相対的な重みを最大 40 倍まで書き換える操作である。前処理というより、特徴量の作り直しに近い。

引き伸ばされる列ほど中身が薄い 問題は、引き伸ばされる列に何が入っているかである。比率は数え上げの結果なので、読点 N 個から推定した比率 $\hat{p}$ には $\sqrt{p(1-p)/N}$ 程度の揺らぎが必ず乗る。これを表の「数え上げ SD」に載せた (1 作品あたりの読点の数は、74 作品の中央値で 4809 本である)。

実測 SD を数え上げ SD で割った値が、「観測された散らばりのうち、数え間違いでは説明できない分」の目安になる。この比の中央値は、平均比率の上位 10 列で 6.3、下位 10 列で 2.6 である。まれな文字ほど、見えている散らばりの多くが数え上げの揺らぎで説明できてしまう。

著者を分ける力そのものを F 値^{*3}で測っても同じ向きになる。 F 値の中央値は、平均比率の上位 10 列で 16.6、下位 10 列で 7.4 である。47 列について比率の対数と F 値の対数の相関を取ると +0.43 で、**頻度の高い文字のほうに著者の情報を多く持っている**。標準化は、情報の薄い列を情報の濃い列と同じ重みまで引き上げている。

それでも ARI は上がる にもかかわらず ARI は良くなる。矛盾ではない。生の比率では、平均比率の上位 3 文字だけで 47 列の合計の 39% を占め、距離のほとんどがその 3 文字で決まってしまう。残り 44 列は、1 列ずつなら弱い、誤差の向きがそろっていないので、44 本まとめれば 3 本より強くなりうる。弱い判定器を多数束ねると強くなるのと同じ理屈である。標準化は、その束ね方をたまたま用意している。

「たまたま」と書いたのには理由がある。74 作品の 80% を無作為に抜き出してクラスタリングし、 $k=8$ で切ったときの ARI を測る。これを 300 回繰り返す。300 回の ARI の平均と標準偏差は、生の比率での $p=1+\text{Ward}$ が 0.813 ± 0.091 、標準化でのコサイン + 群平均が 0.869 ± 0.072 である。同じ抜き出しどうして比べて標準化のほうが高いのは 300 回のうち 70% で、3 回に 1 回は負ける。この程度の勝率では、**決着はついていない**。

^{*3} 分散分析で使う統計量で、著者どうしの平均の開きを同じ著者の作品内のばらつきで割った比である。その列の値が著者ごとにまとまって離れているほど大きくなる。scikit-learn の `f_classif` が計算するのも F 値で、第 9 章の特徴選択にも同じものが出てくる。

原則どおりの前処理が勝つとも限らない では組成データの作法どおり clr 変換を使えばよいかというと、そうでもない。clr のあとでコサイン距離と Ward を組み合わせると ARI は 0.740、ユークリッド距離なら 0.655 である。生の比率での最良 0.891 にも、列の標準化での最良 0.911 にも及ばない。clr は比率の比を保つ変換なので、「0.001 が 0.002 になった」という、まれな文字の 2 倍の変化を大きく扱う。ここではその変化の大半が数え上げの揺らぎだった。

○ 定石 5.1 — 前処理は結果ではなく測り方の理屈で選ぶ

本節で並べた 3 つ (生の比率・列の標準化・clr) のうち、ARI が最も高かったのは列の標準化である。しかしクラスタリングを使う本当の場面には正解ラベルがない。ARI を見て前処理を選ぶことは、答えを見て問題の解き方を選ぶことに等しい。

選ぶ根拠にできるのは、ARI ではなく次のような問いである。

- 列の単位はそろっているか (そろっていれば標準化は不要)
- 合計が固定されているか (組成データなら clr 系を検討する)
- 各列の値は、どれくらいの数から推定されたか (少ない数から推定した列を引き伸ばすのは、雑音を引き伸ばすことになる)
- 引き伸ばした列に、引き伸ばすだけの情報があるか (F 値や効果量で先に確かめられる)

比率の表なら、まず生の比率と clr を試し、列ごとの標準化は「理由を言えるとき」に限るのが安全である。

× アンチパターン 5.2 — 合計が 1 に固定された比率の表を列ごとに標準化する

すべての列が同じ単位の比率で、しかも合計が 1 に固定されている表では、標準化は単位をそろえる操作ではなく**列の重みを付け替える操作**になる (上の例では最大 40 倍の付け替えが起き、引き伸ばされたのは主に数え上げの揺らぎだった)。それでも ARI が上がったのはたまたまで、別のコーパス、別の文字集合では逆に転ぶ。選ぶ根拠は、直前の定石に挙げた問いに求める。

○ 定石 5.2 — Ward 法はユークリッド距離と組み合わせる

- Ward 法 — ユークリッド距離とだけ使う
- 単連結・完全連結・群平均 — 任意の距離と使える

ライブラリは警告を出さずに計算してしまうので、自分で気をつけるしかない。

定石が守っているのは**式 (5.4) の意味**である。Ward 法に他の距離を渡しても計算は回るし、本節の発展「Ward 法にユークリッド距離以外を渡すと何が起きるか」で見たように、ARI が上がることもさえる。デンドログラムの入れ子構造も壊れない (Lance-Williams の Ward の係数は、高さが逆転しないための条件を満たしている)。

壊れるのは**解釈**のほうである。ユークリッド距離でなければ、そもそも重心も平方和も定義できないので、得られた高さが何の増分なのかを言えなくなる。残るのは「大きいクラスタどうしの併合を重く見る併合規則」という手続きだけである。

それでも使うなら、次の3つを報告に残す。

1. 渡した距離 (ユークリッドでないなら、その旨を明示する)
2. 使った実装 (ward.D か ward.D2 か。発展「Lance-Williams の漸化式」で見たとおり、読点前 47 列にユークリッド距離を使うと、同じデータで ARI が 0.703 対 0.522 になる)
3. 高さを読んでいないこと (切ったクラスタだけを使ったのか、高さの大小も解釈したのか) 高さを読まないなら支障は小さいが、再現できる形で書き残すところまでが手順である。

5.7 k-means

階層クラスタリングは $O(n^2)$ 以上かかる。数万点では現実的でない。

k-means はクラスタ数 k を先に決めて、次を繰り返す。

(0) クラスタ中心 $\mu_1, \dots, \mu_k$ をランダムに決める

1. 各点を、最も近い中心のクラスタに割り当てる
2. 各クラスタの中心を、所属する点の平均に移動する

1 と 2 を、中心が動かなくなるまで繰り返す。

最小化しているのは**クラスタ内平方和**である。

$$J = \sum_{k=1}^K \sum_{x \in C_k} \|x - \mu_k\|^2 \quad (5.11)$$

手順 1 は中心 μ を固定したまま、各点を最も近い中心に付け替える。より近い中心に移すのだから、その点の $\|x - \mu\|^2$ は減るか同じである。手順 2 は割り当てを固定したまま、中心を各クラスタの平均に置き直す。平均は**そこからの 2 乗距離の和を最小にする点**なので、これも J を減らす。

つまり反復のたびに J は減るか、同じ値にとどまる。ただしそれだけでは、いつまでも少しずつ減り続ける可能性がある。止まることが言えるのは、**点の割り当ての仕方が有限通りしかない**からである。

割り当てが 1 つでも変われば、 J は必ず**狭義**に減る (同じ距離で迷ったときは、いま属しているクラスタに留まると決めておく。こうすると割り当てが変わるのは、いまの中心より真に近い中心が見つかったときだけになる)。 J が減ったのなら、その割り当ては過去に現れたどの割り当てとも違う。つまり同じ割り当てには二度と戻らない。候補は K^n 通りしかないのだから、戻らずにたどっていればいつかは打ち止めになる。打ち止めになるのは、割り当てが 1 つも変わらなくなったときである。

5.7.1 反復を目で追う

仕組みが見えるように、はっきり 3 つの塊がある人工データ (各 40 点) で $k = 3$ として動かす。中心の初期値は乱数で、**3 つとも右下に固まった状態**から出発させる。正解 (3 つの塊の重心) から遠い場所である。

動きを追ってみる。(0) では 3 つの中心がどれも右下にいたので、左の塊も上の塊も、いちばん近い中心はたまたま同じものになり、点の大半が 1 つの中心に割り当てられている。(1) でその中心が

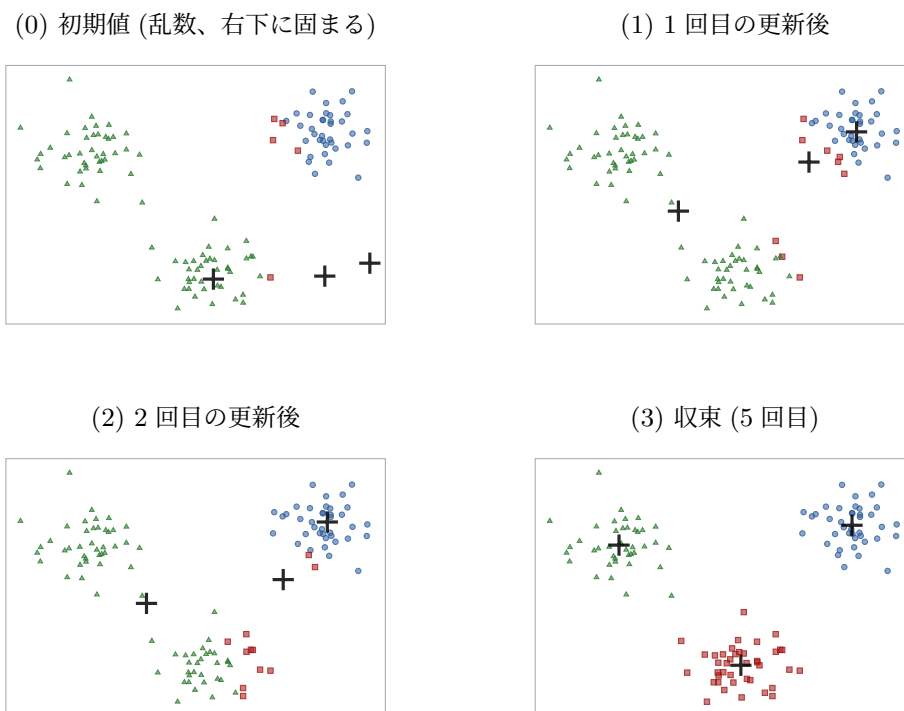


図 5.8 k-means の反復。黒い十字がクラスタ中心。右下に固まって出発した 3 つの中心が、更新のたびに塊の重心へ引き寄せられ、5 回で 1 つずつ収まった。

大量の点の重心へ大きく跳ぶ。1 回の更新で中心が動いた量を、3 つの中心の座標の変化のうち最大のもので測ると、このときは 5.53 である。残りの中心は空いた縄張りを引き継ぐ。以後、(2)(3) と中心どうしが点を取り合いながら移動し、同じ測り方での移動量は 2 回目以降、順に 0.99、2.58、1.84、0.31 になる。5 回目で 3 つの塊に 1 つずつ収まって停止した。**どの中心も、正解の位置を教わっていない。**「割り当て」と「重心への移動」を交互に繰り返すだけで、自然に塊の中心へたどり着くのが k-means の仕組みである。

5.7.2 初期値に依存する

k-means は必ず収束するが、**たどり着く先が最良とは限らない**。式 (5.11) を最小化する問題は NP 困難^{*4}であり、実際に得られるのは局所最適解である。

3 つの明確な塊がある人工データ (各 40 点) で確かめる。 $k = 3$ とし、初期中心を手で置いて `n_init=1` で 1 回だけ実行する。

(b) では初期中心のうち 2 つが同じ塊の中にあったため、その塊を 2 つに割ってしまった。式 (5.11) のクラスタ内平方和は $J = 549.7$ で、正しい解の $J = 49.2$ の 11 倍である。

^{*4} 現実的な時間で厳密な最適解を求める方法が知られていない問題の一群を指す。点の数が増えると、すべての分け方を試す計算量が爆発する。

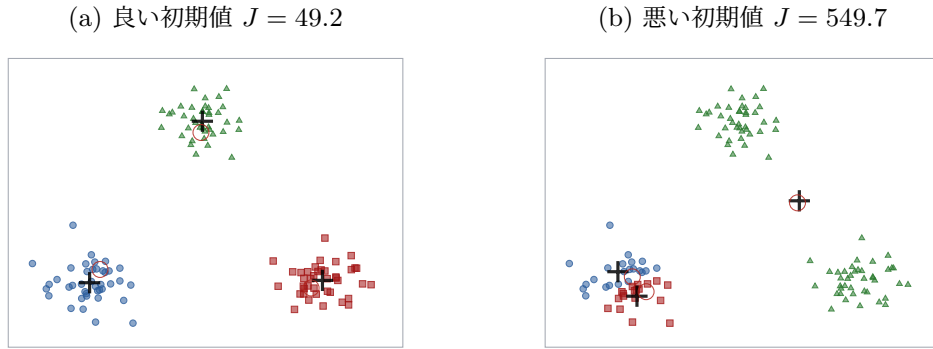


図 5.9 同じデータ、初期値だけが違う。赤い丸が初期中心。(b) では左の塊を 2 つに割り、右の 2 つを 1 つにまとめた。

○ 定石 5.3 — k-means は初期値を変えて複数回試す —

- `n_init` を大きくして複数回試し、 J が最小の結果を採用
- `k-means++` を使う。初期中心が互いに離れるように選ぶ手法で、scikit-learn の既定値になっている

× アンチパターン 5.3 — k-means を 1 回だけ実行する —

自分で実装したり `n_init=1` を指定したりすると、図 5.9(b) のような解を掴んだまま気づかない。

目的関数 J の値は必ず記録しておく。複数回実行して J がばらつくなら、局所解に落ちている。

5.7.3 k-means の制約

もう一つ、押さえておきたい性質がある。k-means ではユークリッド距離が構造的に前提になっていて、他の距離に差し替えることができない。

理由は手順 2 にある。中心を各クラスターの平均に置き直しているが、平均とは「そこからの 2 乗距離の和を最小にする点」のことである。2 乗距離、つまりユークリッド距離を測っていることが、平均を使うと決めた時点で確定してしまう。距離だけをコサイン距離に取り替えても、中心の置き方はユークリッド距離のままなので、辻褄が合わなくなる。

コサイン距離で k-means を回したければ、先にベクトルを ℓ_2 正規化する (第 4 章の式 (4.10))。すべての点が半径 1 の球面に乗り、球面上ではユークリッド距離とコサイン距離が対応するので、実質的に同じことになる。ただし平均を取ると球面から内側へ外れてしまうので、中心を求めるたびに ℓ_2 正規化して球面へ戻す。これを球面 k-means と呼ぶ。

5.7.4 エルボー法とその限界

k を決める目安としてエルボー法がよく紹介される。式 (5.11) の J を k に対してプロットし、「肘」を探す。

なぜ肘を探すのかを先に説明する。 k を 1 つ増やすと、クラスターが 1 つ細かく割れて J が減る。そ

の減り方に注目する。

データに k^* 個の塊が本当にあるなら、 k が k^* に達するまでは、1 つ増やすたびに別の塊どうしが切り離されるので J は大きく減る。 k^* を超えると、今度は 1 つの塊を無理に 2 つに割ることになるので、減り方は鈍くなる。つまり減りの勢いが変わる位置が k^* の目印になる。グラフではそこが折れ曲がって見えるので、肘と呼ばれる。

理屈は通っている。問題は、その肘が実際に見えるかどうかである。

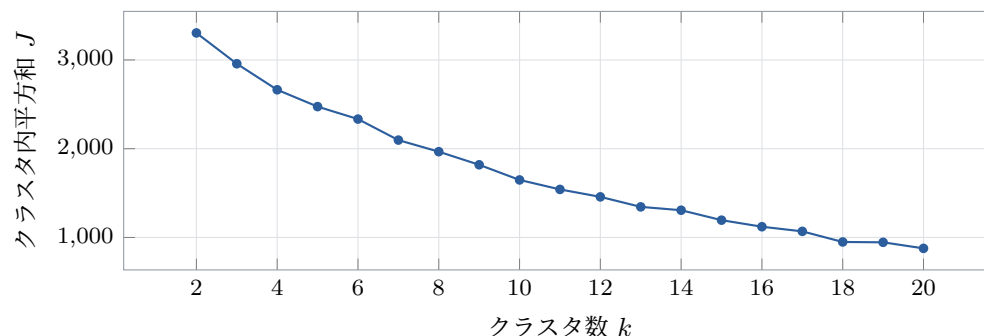


図 5.10 エルボー法。折れ曲がりの位置を探す、はっきりしないことが多い。

図 5.10 に、はっきりした折れ曲がりは見当たらない。 $k = 4$ にも $k = 6$ にも見えるし、そもそもなめらかに減っているだけにも見える。本書のデータには著者という 8 つの塊があるはずなのに、 $k = 8$ に目印は出ていない。

× アンチパターン 5.4 — エルボー法だけでクラスタ数を決める

k を増やせば J の最適値は必ず下がる。 $k = n$ 、つまり全部の点を別々のクラスタにすれば 0 になる。だから「 J が小さいほど良い」では決まらない。

正確には、単調に下がるのは最適値のほうである。k-means が返すのは初期値に依存する局所解なので、 k を増やしたのかえって悪い解に落ち、 J が増えることもありうる (本書は `n_init=20` で 20 通りの初期値から最良を採っている、図 5.10 では起きていない)。どちらにせよ「小さいほど良い」で k は決まらない。

決め手は減り方の変化のはずだが、その変化がどこで起きたかを読む作業に客観的な基準がない。

はっきりした肘が出ないほうが普通だと思ったほうがよい。本書のデータのように、塊どうしの境目がぼやけている場合はとくにそうである。

5.8 階層クラスタリングと k-means の使い分け

本書のデータは 74 点なので階層クラスタリングで十分である。ただし k-means の反復の考え方は、第 7 章で扱う勾配降下法と同じく、目的の値が下がる方向へ少しずつ動かすという形をしている。機械学習の学習の多くはこの形なので、ここで押さえておきたい。

表 5.3 2つの手法の比較

	階層クラスタリング	k-means
クラスタ数	後から決められる	先に決める
計算量	$O(n^2)$ 以上	$O(nkt)$ 、 t は反復回数
結果の安定性	決定的	初期値に依存
距離の自由度	任意 (Ward 除く)	ユークリッドのみ
出力	木構造 (階層)	平坦な分割
向く場面	数千点まで、構造を見たい	大規模、 k が既知

5.9 章末の課題

1. 図 5.2 の 5 点について、単連結と完全連結でも併合の履歴を作れ。群平均法 (表 5.1) と順序は変わるか。
2. Ward 法とコサイン距離を組み合わせてデンドログラムを描き、図 5.7 と比べよ。どこがおかしいか説明できるか。
3. $k = 8$ のクラスタと著者の対応表 (混同行列) を作れ。どの著者が 2 つのクラスタに割れたか。
4. k-means を `n_init=1`、乱数の種を変えて 30 回実行し、 J のヒストグラムを描け。何種類の局所解があるか。
5. Lance-Williams の漸化式 (発展) で、単連結の場合に $\alpha_i = \alpha_j = 1/2$, $\gamma = -1/2$ が $\min$ を与えることを確かめよ。
6. 図 5.8 の人工データで $k = 2$ と $k = 4$ を試し、収束後の J と見た目を $k = 3$ と比べよ。 J の値だけで k を選べない理由を、この実験結果から説明せよ。(ヒント: k を増やすと J は必ず下がる。エルボー法とシルエット係数の項を見直すとよい。)

第 6 章

次元を減らす

6.1 見えないものを見る

52 次元の空間に 74 個の点が散らばっている。散らばりを「見る」ことはできない。人間が直接見られるのはせいぜい 3 次元、紙の上なら 2 次元である。

そこで**次元削減** (dimensionality reduction) を使う。高次元の配置をできるだけ壊さずに、低次元に写し取る技術である。

代表的な考え方が 2 つある。

- **主成分分析 (PCA、principal component analysis)** — データの**分散**が大きい方向を残す
- **多次元尺度構成法 (MDS)** — 点どうしの**距離**を保つ

どちらも「何を捨てて何を残すか」の判断が違うだけである。章の後半では、この 2 つでは扱えない、曲がった構造のための非線形手法 (t-SNE・UMAP など) へ進む。

6.2 主成分分析 — いちばん見晴らしのよい方向から眺める

PCA の発想は単純である。

データを、いちばん全貌を捉えやすい位置から眺める。

写真を撮るときのことを考えてほしい。細長い建物を真横から撮れば形がよく分かるが、真正面から撮ると奥行きが潰れて何も見えない。データにも同じことが言える。

ここから、比喩を数値に置き換える一段が必要になる。「奥行きが潰れて見えない」とは、その方向に点が散らばっていないということである。逆にいえば、**点が最もばらけて見える向きこそ、潰さずに残すべき向き**である。そして「点のばらけ具合」を 1 つの数で表したものが、第 3 章で使った**分散**である。

つまり「いちばん全貌を捉えやすい位置」は、**射影したときの分散が最大になる方向**と言い換えられる。言い換えたとたんに、計算で探せる問題になる。

6.2.1 手順を 3 次元で追う

第 3 章で作った特徴のうち 3 つだけ取り出す。平仮名の比率、漢字の比率、文長 μ の 3 つである。これを標準化して 3 次元空間に置く。

手順を言葉にする。

(0) **センタリング**。データの平均を原点に平行移動する

1. データの中心を通して、もっとも分散の大きい方向に串を通す。これが PC1

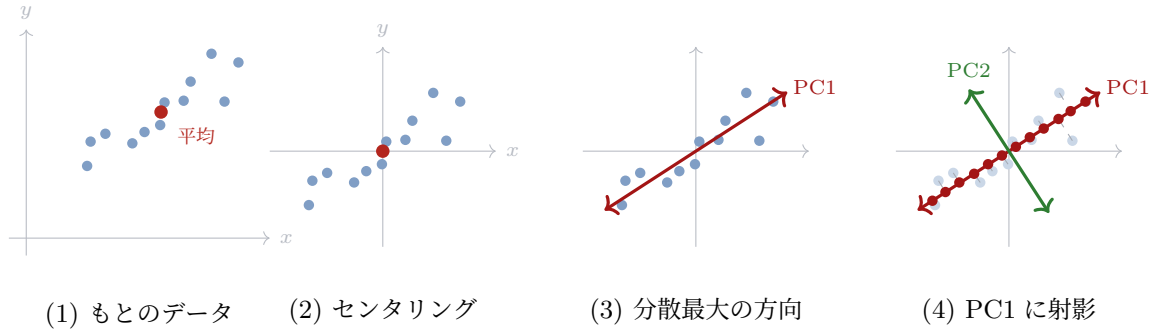


図 6.1 PCA の手順。平均を原点に移し、分散が最大になる方向に軸を通す。

2. データの中心を通り、**PC1 が点になる位置から眺めて**、もっとも分散の大きい方向に串を通す。これが PC2

3. PC1 と PC2 の両方に直交する方向に串を通す。これが PC3

「PC1 が点になる位置から眺めて」というのが要点である。串の根元から串の先を覗き込む姿勢を思い浮かべてほしい。そこから見えるのは、串に垂直な平面である。PC1 方向の広がりには点に潰れて見えない。

PC2 はその平面の中から選ぶことになる。平面は PC1 に垂直なのだから、その中のどの方向も PC1 と直交する。**PC1 と PC2 が必ず直交するのは、探す場所をそう決めたからである。**直交するように後から調整しているのではない。

6.2.2 見る向きを変えてみる

手順が言葉だけでは掴みにくいので、本書のデータを実際に回してみる。平仮名の比率・漢字の比率・文長 μ の 3 列を標準化し、74 作品を 3 次元空間に置いた。

図 6.2 は、**同じ点の集まりを 3 つの向きから見たものである**。点は 1 つも動かしていない。動かしたのは見る位置だけである。

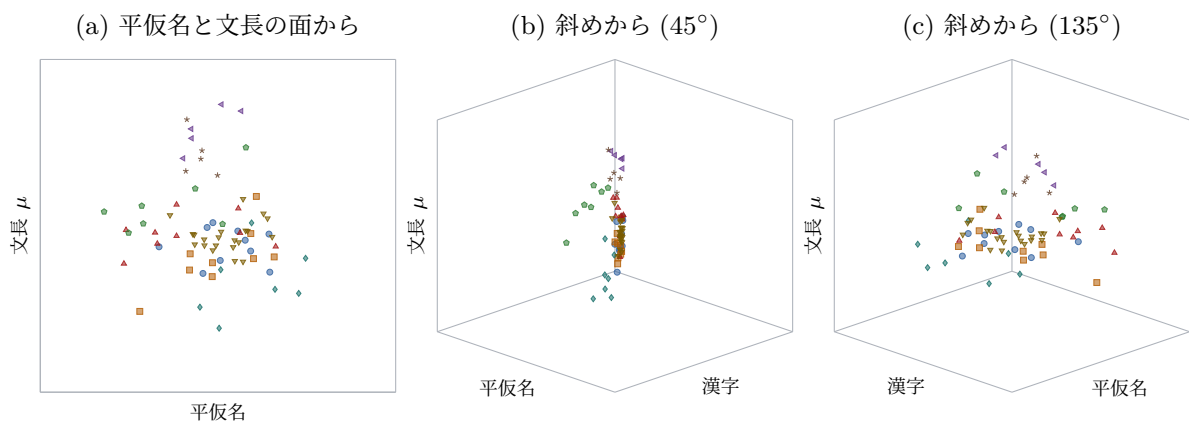


図 6.2 同じ 74 作品を 3 つの向きから見たもの。(b) では点が細い帯に潰れ、(c) では横いっぱい広がる。

(b) の向きから見ると、点は細い帯に潰れる。この向きには、作品どうしの違いがほとんど現れて

いない。逆に (c) からは、点が横いっぱいには散らばって見える。

潰れる理由ははっきりしている。74 作品について平仮名の比率と漢字の比率の相関係数を取ると -0.91 で、一方が増えればもう一方が減る関係にある。(b) はこの 2 つを足す向きから見ているので、増えた分と減った分が打ち消し合う。(c) は逆に、差を取る向きから見ている。図の横軸方向に測った 74 点の標準偏差は、(b) が 0.29、(c) が 1.38 で、(c) が (b) の 4.7 倍である。

写真の比喻に戻れば、(b) が「細長い建物を真正面から撮った」、(c) が「同じ建物を真横から撮った」状態である。PCA がやっているのは、(c) のような向きを計算で見つけることである。

図 6.3 に、見つかった 3 本の軸を重ねた。矢印の長さは、その向きに点がどれだけ広がっているか、つまり標準偏差 $\sqrt{\lambda_k}$ に比例させてある (分散そのものではないので、長さの比と寄与率の比は一致しない)。

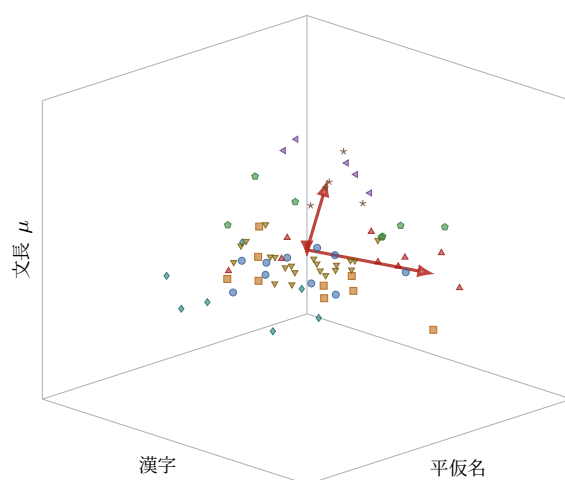


図 6.3 PCA が見つけた 3 本の軸 (赤の矢印)。矢印の長さは、その向きの標準偏差に比例させてある。第 1 主成分の向きにいちばん広く散らばっており、第 3 主成分の向きはその 5 分の 1 ほどしかない。

3 列 (平仮名の比率・漢字の比率・文長 μ) を標準化した 74 作品について、3 本の軸が説明する分散の割合、すなわち寄与率は、順に 65.6%、31.5%、2.8% である。第 3 主成分の矢印は、第 1 主成分の 5 分の 1 ほどの短さである (分散の比 $2.8/65.6$ の平方根が、およそ $1/5$ になる)。この向きの情報を捨てても、失われるのは 2.8% だけである。残りの 97.2% は、はじめの 2 本が張る平面に乗っている。

6.2.3 実際に 3 次元から 2 次元へ

図 6.4 の左が 3 次元の配置 (等角投影で描いたもの)、右が第 1・第 2 主成分の平面に射影した結果である。

3 つの主成分の寄与率は、順に 65.6%、31.5%、2.8% であった。第 3 主成分を捨てて失われるのは、全分散のうちの 2.8% である。ただし、失われるのは分散であって、課題に必要な情報の割合ではない。分散の小さい方向に著者を見分ける手がかりが隠れていることもあるので、著者の手がかりが残っているかは、近傍の変化や分類の成績で別に確かめる (付録 A の教材で、第 3 主成分を軸に加えたときの近傍の変化を見られる)。

第 3 主成分が余る理由は、第 3 章で作ったデータの性質にある。平仮名・片仮名・漢字の比率は合計

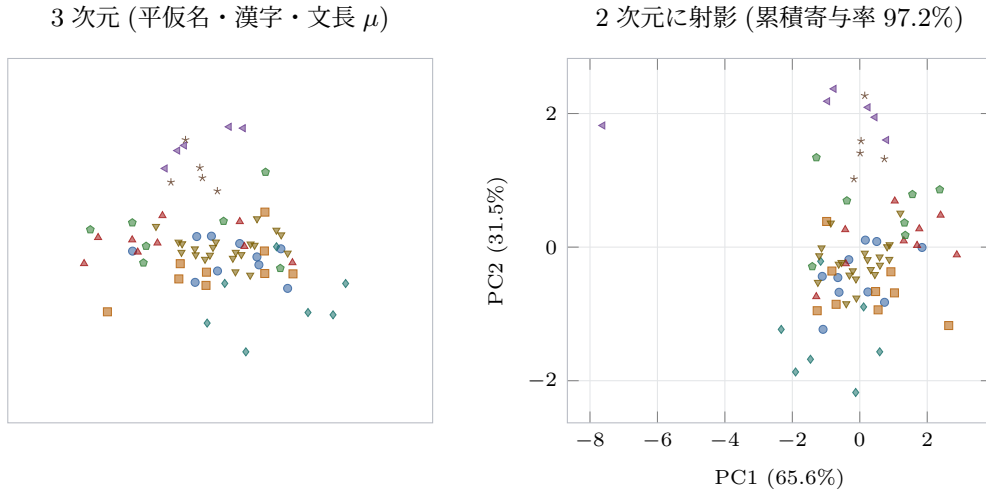


図 6.4 3次元から2次元へ。第3主成分の寄与率は2.8%なので、捨てて失われる分散は全体の2.8%である。

が1になるよう正規化してあった。片仮名の比率は小さいので、平仮名と漢字はほぼ $r_{\text{漢字}} \approx 1 - r_{\text{平仮名}}$ という直線関係にある。

つまりこの3次元データは、もともとほぼ平面上に乗っていた。PCAはそれを見つけ出した。

補足 次元を落としても情報が残る理由 — 多様体仮説

高次元のデータが、実はもっと低い次元の構造の上に乗っていることは多い。これを**多様体仮説** (manifold hypothesis) という。ここでいう**多様体**は、「局所的に見れば平らな面と同じように扱える図形」くらいに読んでおけばよい。地球の表面は球だが、足元の数メートルだけを見れば平面と変わらない。

たとえば手書き数字の画像は $28 \times 28 = 784$ 次元だが、「どの数字か」「傾き」「太さ」「大きさ」といった数十個の要素でほぼ説明できる。784次元の空間の中で、実際のデータはずっと低い次元の面のような領域の上になら存在しない。

いま扱った平仮名・漢字・文長 μ の3列は、ほぼ平面に乗っていた。第3章の字種3列 (平仮名・片仮名・漢字) は別の例で、こちらは合計が1という制約があるので、点は**厳密に**平面の上になら乗らない。このように、データが住んでいる面そのものの次元は、それを取り囲む空間の次元と別の量である (後の節で内在次元として定式化する)。両者がずれているとき、次元削減には意味がある。

ここまでの例では面が平らだったので、PCAのような線形の手法で足りた。面が**曲がっている**とどうなるかは、第6.5.2節のスイスロールで扱う。そこが線形と非線形の分かれ目になる。

6.2.4 数式で書く*

m 個のデータ、 n 次元とする。センタリング済みのデータ行列を $A \in \mathbb{R}^{m \times n}$ と書く (各行が1つのデータ)。

主成分 $v(\|v\| = 1)$ の方向にデータを射影した値は Av である。その標本分散は

$$\text{Var}(Av) = \frac{1}{m} \|Av\|^2 = \frac{1}{m} v^\top A^\top A v = v^\top \Sigma v \quad (6.1)$$

ここで $\Sigma = \frac{1}{m} A^\top A$ は**分散共分散行列** (covariance matrix) である。

第1主成分を求める問題は、次の制約付き最大化になる。

$$\max_v v^\top \Sigma v \quad \text{subject to} \quad \|v\|^2 = 1 \quad (6.2)$$

発展 第1主成分をラグランジュ未定乗数法で求める

制約 $\|v\|^2 = 1$ のもとで $v^\top \Sigma v$ を最大化する。ラグランジュ関数は

$$L(v, \lambda) = v^\top \Sigma v - \lambda(v^\top v - 1) \quad (6.3)$$

v で微分してゼロと置く。

$$\frac{\partial L}{\partial v} = 2\Sigma v - 2\lambda v = 0 \implies \Sigma v = \lambda v \quad (6.4)$$

式 (6.4) は固有値問題そのものである。つまり主成分は分散共分散行列の固有ベクトルであり、そのときの分散は

$$v^\top \Sigma v = v^\top (\lambda v) = \lambda \|v\|^2 = \lambda \quad (6.5)$$

すなわち固有値そのものが、その方向の分散になる。

Σ は対称かつ半正定値なので、固有値はすべて実数で非負であり、固有ベクトルは互いに直交するように取れる。固有値の大きい順に並べれば、それが PC1, PC2, ... である。

発展 $A^\top A$ と $AA^\top$ — 列の類似度と行の類似度

分散共分散行列 $\Sigma = \frac{1}{m} A^\top A$ の中身をよく見ると、 (j, k) 成分は「列 j と列 k の内積を m で割ったもの」である。列は特徴なので、 $A^\top A$ は特徴どうしの類似度の表にほかならない (列を z 化してあれば相関行列そのものになる)。

行列の順序を入れ替えた $AA^\top$ のほうも見ておく。 (i, ℓ) 成分は行 i と行 ℓ の内積で、行は作品なので、こちらは作品どうしの類似度の表である (グラム行列 (Gram matrix) と呼ぶ。行を z 化してあれば、第 4.8 節の相関距離の世界とつながる)。

同じ行列 A を、列の側から見るか行の側から見るかの違いでしかない。実際、両者は固有値を共有する。 $A^\top A v = \lambda v$ の両辺に左から A を掛けると

$$(AA^\top)(Av) = \lambda(Av) \quad (6.6)$$

となり、 $u = Av$ がそのまま $AA^\top$ の固有ベクトルになる (逆向きも同様。サイズが違う分だけ、余った固有値は 0 が埋める)。

対応から、使える手が 3 つ出てくる。

- **小さいほうを解けばよい。** m 作品 $\times n$ 特徴で n が巨大なとき (遺伝子データの検体 $300 \times$ 遺伝子 2 万など)、 $n \times n$ の $A^\top A$ ではなく $m \times m$ の $AA^\top$ を固有分解すれば足りる。顔画像認識の固有顔 (eigenface) で有名になった計算法である。
- **座標がなくても主成分が出る。** $AA^\top$ は内積の表なので、点どうしの距離の表から逆算できる。距離だけから PCA と同じ配置を復元する流儀が古典的 MDS であり、第 6.4 節で扱う MDS の原点にあたる。
- **行と列を同じ図に置ける。** 2 つの固有分解は $u = Av$ で結ばれているので、作品の座標と特徴の座標が同じ軸の上で対応する。第 6.3 節のバイプロットが 1 枚の図に点と矢印を同居させられるのも、同じ対応による。

6.2.5 もう一つの見方 — 射影誤差の最小化

PCA には等価な別の定式化がある。

主成分 v の方向にデータを正射影したとき、もとの点と射影点の間の距離の 2 乗和が最小になる。

図 6.1(4) の点線が、その「誤差」にあたる。

2 つの定式化が一致するのは、三平方の定理による。点 x について、 $\|x\|^2$ は「射影の長さの 2 乗」と「誤差の長さの 2 乗」の和に分解される。

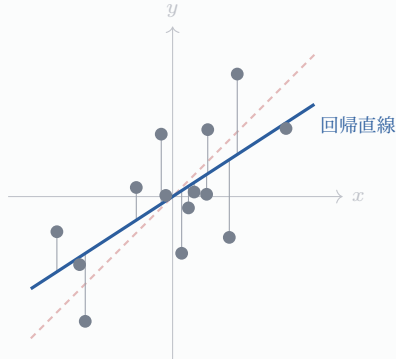
$$\underbrace{\|x\|^2}_{\text{一定}} = \underbrace{(v^\top x)^2}_{\text{射影の分散}} + \underbrace{\|x - (v^\top x)v\|^2}_{\text{射影誤差}} \quad (6.7)$$

左辺はデータで決まる定数だから、右辺の第 1 項を最大にすることと第 2 項を最小にすることは同じである。

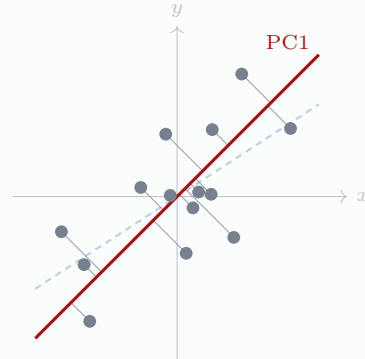
補足 PCA と最小二乗回帰の違い

「誤差を最小にする直線」と聞くと最小二乗法を思い浮かべるかもしれないが、測っている誤差が違う。最小二乗回帰は y 方向のずれを測る。 x は誤差なしと仮定している。PCA は直線への垂直方向のずれを測る。どの変数も対等に扱う。

だから同じデータでも、回帰直線と第 1 主成分は一致しない。同じ 14 点に 2 本の直線を引いてみる。



(a) 縦のずれを最小にする



(b) 垂直のずれを最小にする

細い線が、それぞれ何を最小にしているのかを示している。薄い破線はもう一方の直線で、傾きが違うことが見て取れる。(b) の垂直なずれは x と y を入れ替えても変わらない。だから PC1 は変数の役割を入れ替えても同じ直線になるが、回帰直線は x で y を説明するか y で x を説明するかで別の直線になる。相関が弱いほど回帰直線は水平に寝ていき、PC1 との開きは大きくなる。

6.2.6 寄与率 — 52 列では 2 軸で 3 割ほどしか説明できない

第 k 主成分の寄与率 (explained variance ratio) は

$$\frac{\lambda_k}{\sum_{j=1}^n \lambda_j} \quad (6.8)$$

で定義される。その軸がデータ全体の分散のうち何割を説明しているかを表す。

52 次元すべてを使ったときの寄与率を図 6.5 に示す。

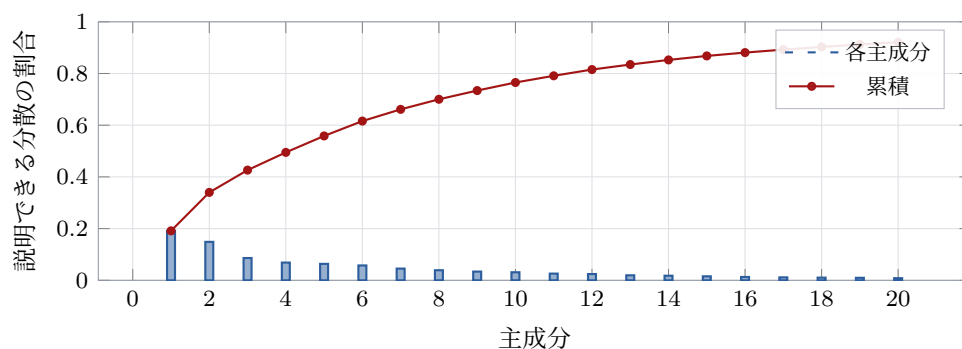


図 6.5 52 列を標準化した 74 作品の寄与率 (スクリープロット)。上位 20 軸のみを示す。累積寄与率は 2 軸で 34%、8 割を超えるのは 12 軸目である。

3 列だけのときは、第 2 主成分までの累積寄与率が 97% だった。52 列を列ごとに標準化した場合の累積寄与率は 2 軸で 34% にしかならず、8 割を超えるのは 12 軸目、9 割を超えるのは 18 軸目である。つまり情報が特定の少数の方向に集中していない。

読点前の平仮名 47 文字は、互いにあまり相関していない。それぞれが独立した情報を持っているとも言えるし、低次元にまとめるのにも難しいとも言える。

6.2.7 2 次元の散布図

それでも 2 次元に落として眺めてみる。

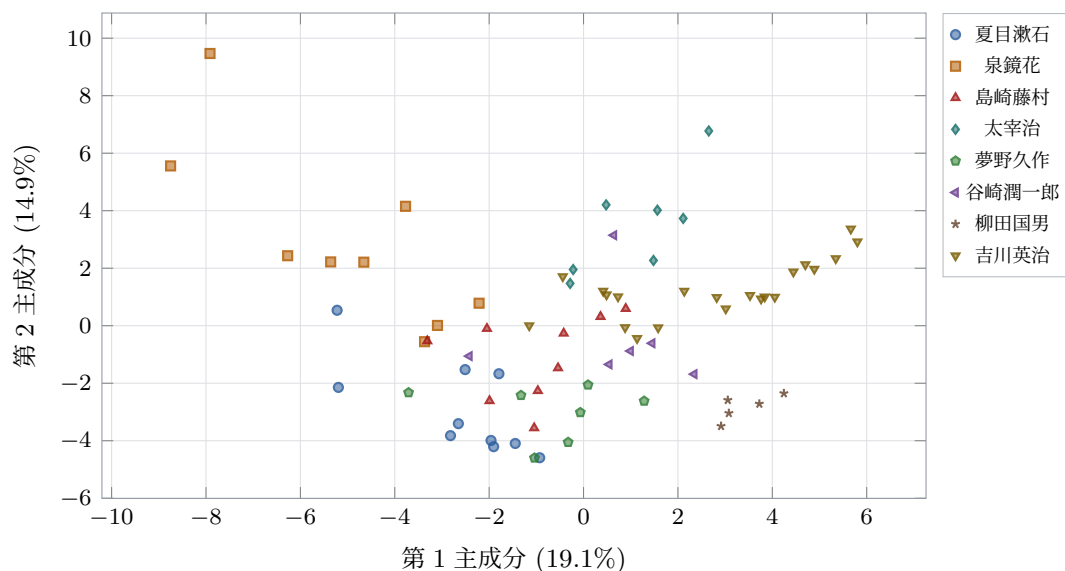


図 6.6 52 次元を PCA で 2 次元にした結果。

著者ごとにある程度まとまって見える。特に吉川英治 (右下) が密集している。

だが軸のラベルを見てほしい。第 1 主成分と第 2 主成分の寄与率を合わせても、52 列全体の分散

の 34.0% しか説明していない。

× アンチパターン 6.1 — 寄与率を見ずに 2 次元の図で「近い」と判断する

図 6.6 で重なって見える 2 作品が、52 次元空間では遠く離れていることは十分あり得る。逆もある。

散布図を出したら**必ず寄与率を確認する**。寄与率が高いほど図の上の距離は当てになるが、それは全体の平均としての話で、個々の 2 点についての保証ではない。図 6.6 のように累積寄与率が 34% なら、図はヒントであって証拠ではない。

補足 寄与率が低くても図を描く意味はある

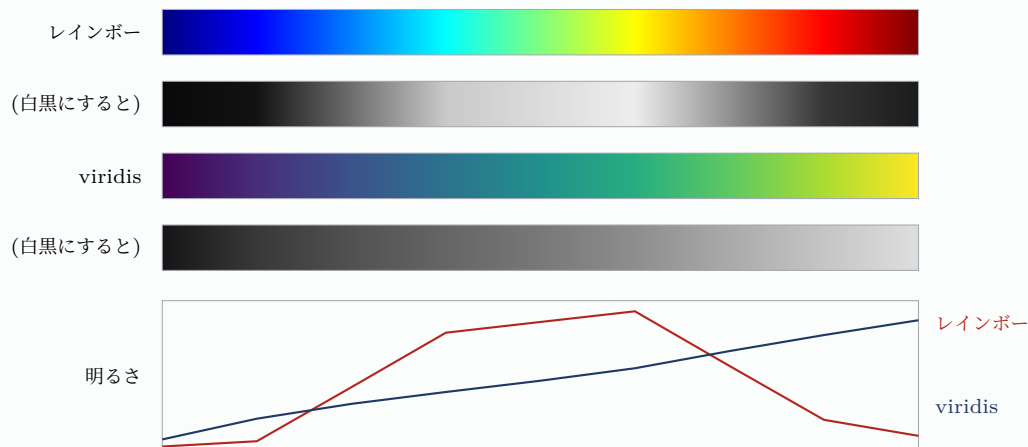
では図 6.6 は無意味かという、そうではない。

「吉川英治だけ極端に密集している」という観察は、図 6.6 から得られた。これは第 9 章で決定的に重要になる手がかりである。可視化は**仮説を思いつくために使い、結論を出すためには使わない**。

補足 色で量を表すときは明るさが順に変わる配色を選ぶ

本書では読点のヒートマップ (第 3.6 節) のように、色の濃さで量を表す図をいくつか使っている。配色の選び方には、意外なほど落とし穴がある。

長らく既定だったのが**レインボー (jet)** と呼ばれる、青から緑・黄を経て赤へ向かう配色である。下に、レインボーと viridis を並べ、それぞれのすぐ下に**同じ配色を白黒に変換したもの**を置いた。



上段のレインボーは、白黒にすると真ん中がいちばん明るく、両端が暗い。左から右へ増える量を表していたはずなのに、明るさだけを見れば**左端と右端がほとんど同じ**になってしまっている。このため、白黒印刷では大小の区別がつかなくなる。

色のままでも問題が残る。緑から黄への変わり目だけ明るさが急に変わるので、データには何もない値域に縞や境界が見えてしまう。気象や医用画像でこの配色を使うと、その値域にだけ偽の構造が現れることが知られている。

下段の viridis は、白黒にしても左から右へ一様に明るくなる。明るさの折れ線を重ねると、レインボーが山型なのに対し、viridis は単調に上がっている。

対処は簡単で、**明るさが端から端まで単調に変わる配色**を選べばよい。matplotlib は 2015 年に viridis(紫から黄へ)を導入し、バージョン 2.0(2017 年)で既定にした。正負の両方向に意味があるデー

補足（続き）

タには、中央が白で両端に向かって濃くなる発散型の配色を使う。本書のヒートマップも、単色の濃淡で描いてある。

明るさが単調な配色を選ぶことは、**色覚多様性**への配慮でもある。日本人男性のおよそ5%、女性のおよそ0.2%は赤と緑の区別が付きにくい型の色覚を持つ[44]。レインボーの赤と緑は、この読者には近い色に見える。一方、明るさで順序がつく配色なら、色の見え方によらず大小が読み取れる。赤と緑で「良い・悪い」を描き分けるのも同じ理由で避け、色に加えて位置・形・濃淡のいずれかで区別をつける。

6.3 バイプロット — 点と変数を同時に描く

PCAには、本章の後半で扱う非線形の手法にはない利点がある。**軸の意味を解釈できる**ことである。

第1主成分は、もとの変数の線形結合で書ける。

$$PC1 = w_1x_1 + w_2x_2 + \cdots + w_nx_n \quad (6.9)$$

係数 w_j (固有ベクトルの成分) が大きい変数ほど、その軸への寄与が大きい。矢印を描くときは、 w_j をそのまま使う流儀と、軸の標準偏差 $\sqrt{\lambda}$ を掛けた**負荷量**を使う流儀がある。本書は後者を使う。標準化したデータでは負荷量がその変数と主成分の相関に一致し、矢印の長さや角度をそのまま統計量として読めるからである。この矢印を点の上に重ね描きしたものが**バイプロット**である。

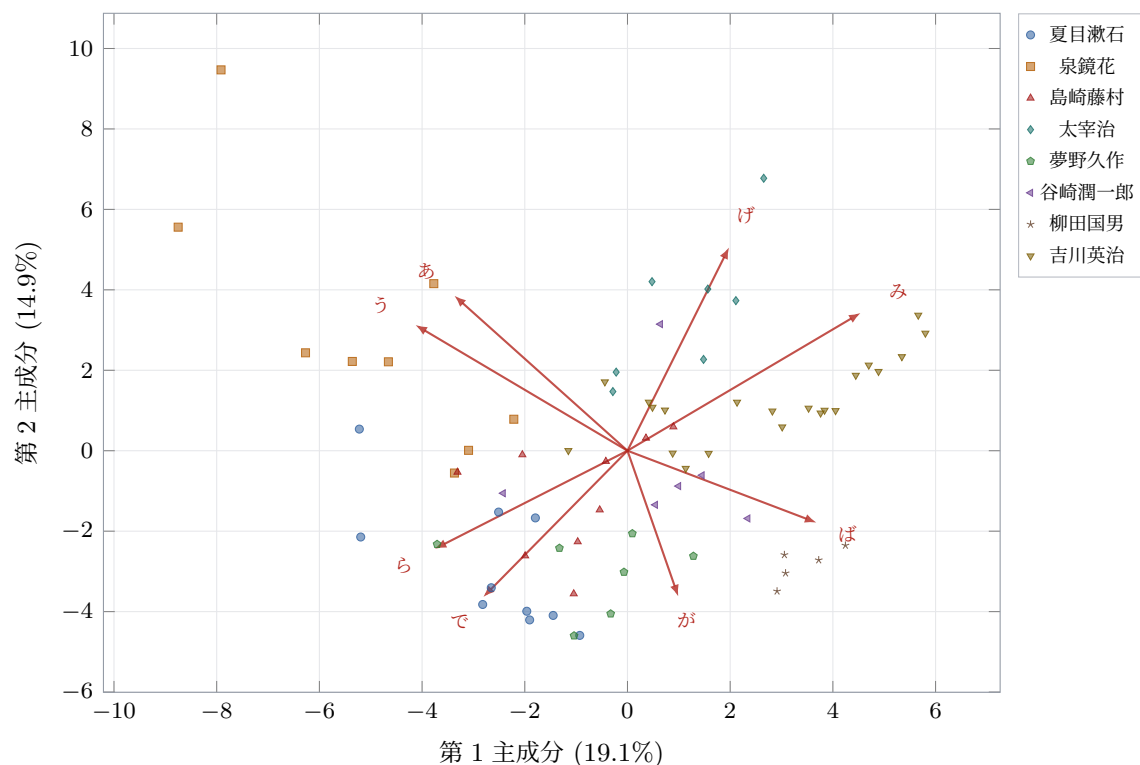


図 6.7 バイプロット。赤い矢印は変数（読点前の平仮名）の寄与を表す。

矢印の読み方は次のとおりである。

- **長さ** — その変数が、この 2 軸の平面にどれだけよく乗っているか。短い矢印は、この平面では表現しきれていない変数なので、向きを読んでも当てにならない
- **向き** — その変数が増える方向
- **矢印どうしの角度** — 変数間の相関 (第 4.8 節)。同じ向きなら正の相関、逆向きなら負の相関、直交なら無相関。ただしこの読みは近似で、2 軸によく乗っている (=長い) 矢印どうしでしか成り立たない
- **点と矢印の関係** — 矢印の方向にある点は、その変数の値が大きい

図 6.7 の矢印は、方角ごとに負荷の大きい変数を 1 本ずつ選んで描いた (すべて重ねると束になって読めないためである)。見どころは矢印どうしの**角度**にある。「み」と「ら」はほぼ逆向き、つまり負の相関で、「み、」で区切る作家は「ら、」では区切らない傾向がある。「ば」と「げ」のように直交に近い組は、ほぼ無相関に使われる。読点の癖が 1 本の軸に沿って強弱するのではなく、互いに独立なくつかの方向を持っていることが、矢印の開き方から読み取れる。

○ 定石 6.1 — 分かれた理由を説明したいときはバイプロットを使う

「なぜこの 2 つのグループが分かれたのか」を説明したいときは、分かれた方向に伸びている矢印を見ればよい。どの変数が違いを作っているかが読み取れる。

t-SNE や UMAP ではこれができない。軸に意味がないからである。

補足 バイプロットの矢印は長さを何倍しても図の意味が変わらない

点と矢印は単位が違うので、そのまま重ねるとどちらかが極端に大きくなる。そこで矢印側に適当な定数を掛けて見やすくするのが慣習である。

矢印の絶対的な長さには意味がない。比較できるのは矢印どうしの相対的な長さと向きだけである。

6.4 多次元尺度構成法 (MDS)

PCA が「分散」を残すのに対し、MDS は「距離」を保とうとする。

n 個の点があり、その間の距離 D_{ij} が与えられているとする。低次元空間に配置 $z_1, \dots, z_n$ を作り、そこでの距離 $\|z_i - z_j\|$ が D_{ij} に近くなるようにする。

最小化する量は**ストレス関数**である。

$$\text{Stress} = \sqrt{\frac{\sum_{i < j} (D_{ij} - \|z_i - z_j\|)^2}{\sum_{i < j} D_{ij}^2}} \quad (6.10)$$

分子は「もとの距離と配置での距離のずれ」を全ペアについて足したものである。分母で割るのは、**全体の大きさの影響を消す**ためである。距離をすべて 2 倍したデータを渡せば、最適な配置も 2 倍に伸びるので、分子と分母がともに 4 倍になり、値は変わらない。そのおかげで、単位の違うデータどうしてもストレスの値を比べられる。

解き方は反復である。点をとってあえてならめな位置に置き、1 点ずつ「ストレスが下がる向き」

へ少し動かす。これを全体が落ち着くまで繰り返す^{*1}。

補足 距離だけあれば配置が作れる

MDS の面白い点は、**もとの座標が分からなくてもよい**ことである。点どうしの距離さえあれば配置を復元できる。

たとえば都市間の距離表だけから地図を復元できる。「東京と大阪は 400km、大阪と福岡は 480km、東京と福岡は 880km」という情報から、3 都市の相対的な位置が決まる。

DNA 配列の類似度、アンケートの回答の似ている度合いなど、座標がそもそも存在しないデータにも使える。

距離がユークリッド距離なら、反復に頼らず一度の固有値計算で配置が求まる (古典的 MDS)。距離の表を内積の表に直すと、第 6.2.4 節の発展で見た AA^T の固有分解に帰着するからである。

MDS は PCA と違って、三角不等式を満たさない「距離」も埋め込むことができる。ストレスを小さくするだけの手続きなので、**距離の公理を一度も使っていない**からである。公理を破った表を渡しても、手続きは止まらずに配置を返す。ひずみは大きくなるが、とにかく絵は出てくる。

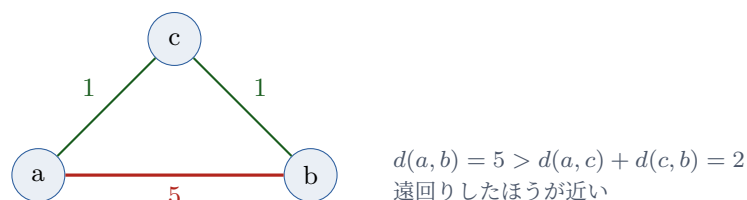


図 6.8 三角不等式が破れている例。ユークリッド空間には正確には埋め込めない。

6.5 非線形の次元削減 — 近傍のつながりを保つ

PCA は**線形**の手法である。データが曲がった面の上に乗っていると、面に沿った距離を直線距離と取り違える。

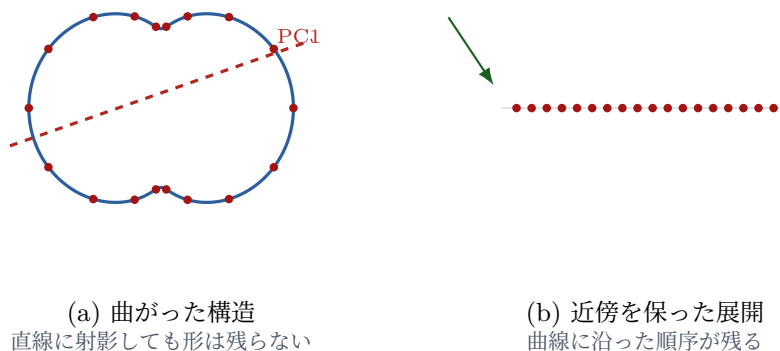


図 6.9 線形手法 (a) と非線形手法 (b)。後者は曲がった構造をほどく。

^{*1} 代表的な手順に SMACOF がある。名前は Scaling by MAjorizing a COmplicated Function の略で、扱いにくい関数を、それより上にある扱いやすい関数で押さえながら最小化していく方法を指す。

UMAP (Uniform Manifold Approximation and Projection, 2018)[29] と t-SNE (t-distributed stochastic neighbor embedding, 2008)[28] は、非線形の次元削減として広く使われている。

6.5.1 UMAP の考え方

原論文の理論的な裏づけは圏論とリーマン幾何に頼っているが [29]、実際にやっていることは 3 段階に分けられる。

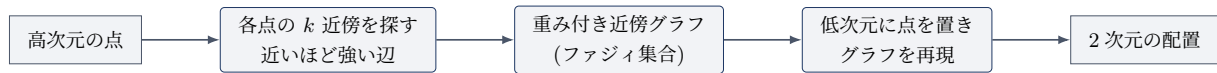


図 6.10 UMAP の手順。距離ではなく「つながり」を保とうとする。

1. **近傍グラフを作る。** 各点について最も近い k 個 (`n_neighbors`) を探し、辺を張る。辺の重みは距離が近いほど大きい
2. **重みを確率に直す。** 各点から見た「つながりの強さ」を $\exp(-(d - \rho)/\sigma)$ の形で表す。 ρ はその点から最も近い点までの距離である
3. **低次元で同じつながりを再現する。** 低次元での距離も同じように確率に直し、高次元での確率にできるだけ近づくよう点を動かす

手順 2 で ρ を引く意味を補っておく。点が密集した場所では ρ が小さく、まばらな場所では大きい。引き算をすると、どの点についても「自分にとっていちばん近い相手」が起点になるので、**密集しているかどうかの違いが消える**。どの点も同じだけの近所を持っているかのように扱えるようになる。

手順 3 で最小化する量は次の形をしている。

$$\mathcal{L} = \sum_{(i,j)} \left[p_{ij} \log \frac{p_{ij}}{q_{ij}} + (1 - p_{ij}) \log \frac{1 - p_{ij}}{1 - q_{ij}} \right] \quad (6.11)$$

p_{ij} が高次元でのつながりの強さ、 q_{ij} が低次元でのそれである。2 つの確率のずれを測る量で、**交差エントロピー**と呼ばれる。

第 1 項が「高次元でつながっていた点は低次元でも近づける」引力、第 2 項が「つながっていない点は離す」斥力にあたる。

補足 UMAP と t-SNE は何が違うのか

t-SNE が最小化するの**は KL ダイバージェンス** $\sum p_{ij} \log(p_{ij}/q_{ij})$ である（これも 2 つの確率分布のずれを測る量で、交差エントロピーと近い親戚にあたる）。

違いは、UMAP の式にある第 2 項の有無である。t-SNE にも点を離す働きはあるが、確率を全体で 1 に正規化することを通じた間接的なものととまる。UMAP は「つながっていない点を離す」項を明示的に持つので、クラスタどうしがより大きく離れた図になりやすい。

第 6.5.7 節で見る「大域構造の保存」の指標での UMAP と t-SNE の違いにも、この差が絡みうる。ただし、大域構造の差の大半は初期値の置き方だけで説明できるという報告もある（第 6.5.6 項の末尾で紹介する）。また、UMAP は離す相手を全部ではなく一部だけ選ぶ工夫（負例サンプリング）で t-SNE よ

補足（続き）

り計算が速い。

6.5.2 スイスロール — 曲がった面をほどく

非線形手法の威力を見るのに、**スイスロール**という人工データがよく使われる。2次元の長方形を、渦巻き状に巻いて3次元に埋め込んだものである。

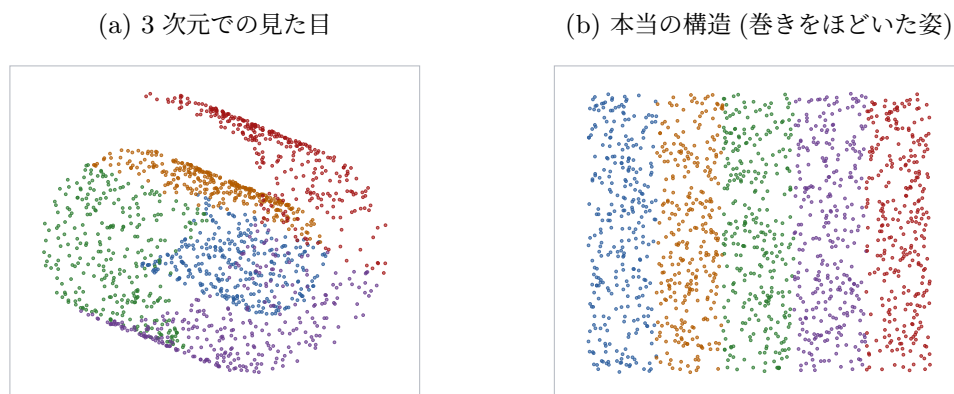


図 6.11 スイスロール。色は巻きに沿った位置。(b) の2次元の面を巻いて、3次元空間に埋め込んだものである。

(a) は3次元の複雑な形に見えるし、実際、点の座標を書き下すには3つの数が要る。だが形そのものは、(b) の長方形を巻いて3次元空間に配置したものである。つまりこのデータは、**本質的には2次元の面が3次元空間に埋め込まれた状態**にある。埋め込み先の空間の次元(3)と、データが実際に住んでいる面の次元(2)は別の量である。面の上の住所は「巻きに沿ってどれだけ進んだか」と「奥行き」の2つで書ける。2という数を**内在次元** (intrinsic dimension) と呼ぶ。

これが第6.2.3節で触れた**多様体仮説**の状況である。字種の3列と違うのは、面が**曲がっている**ことである。平らな面なら線形の射影で開けるが、巻かれた面を平らに戻すには、面に沿った距離を測り直す必要がある。

問題は、**渦巻きの内側と外側**である。3次元空間では近いが、面に沿って歩けば遠い。逆に、面に沿って隣どうしの点は、必ず3次元でも近い。

ここで保ちたいのは、**面に沿った近さ**のほうである。

4枚を見比べると、次のことが分かる。

- **PCA** — 巻きを真上から潰したような形になる。色が重なり、内側と外側が混ざっている。線形変換では巻きをほどけない
- **MDS** — PCA と似た結果。3次元での直線距離を保とうとするので、渦巻きの内外を「近い」と判断してしまう
- **t-SNE** — 色ごとにまとまって並ぶ。局所的な近さを保つので、面に沿った構造が現れる。ただし帯が分断されることもある
- **UMAP** — 同じく色ごとに分かれる。`n_neighbors` を大きくすると、より連続的につながる

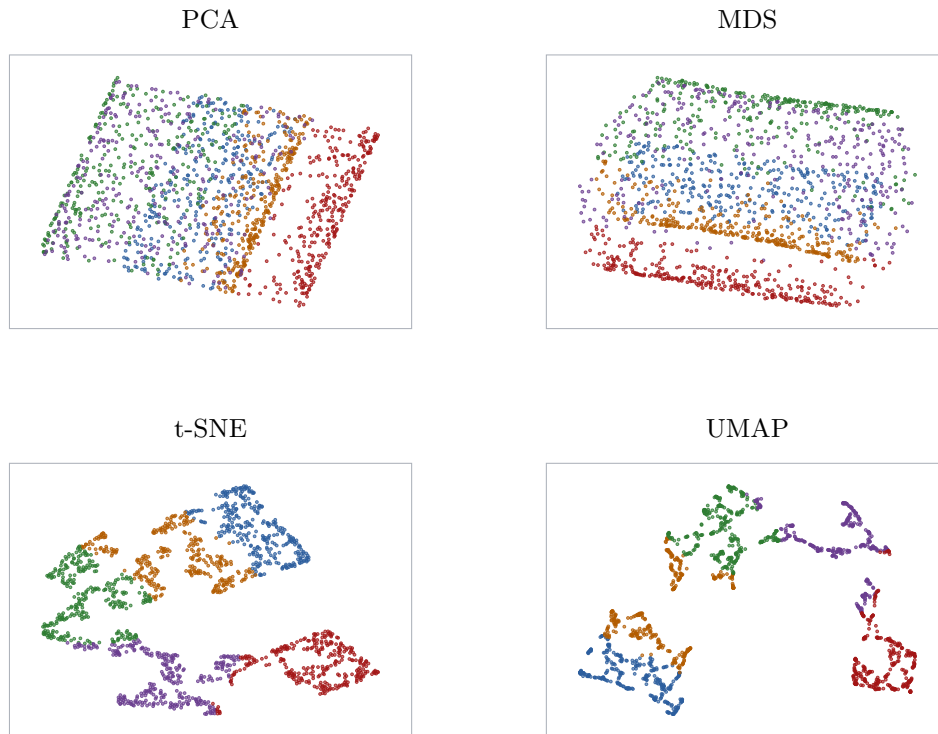


図 6.12 スイスロールを 4 つの手法で 2 次元に。色が図 6.11(b) のように帯状に並べば成功である。

補足 測地距離という考え方

スイスロールを正しくほどく古典的な手法に **Isomap** がある。考え方は明快で、直線距離の代わりに面に沿った距離を使う。

近傍グラフを作り、その上の最短経路長を「距離」として MDS にかける。これを測地距離という。地球上の 2 都市の距離を、地中を貫く直線ではなく地表に沿って測るのと同じ発想である。

UMAP も近傍グラフを作る点で共通している。「近くのつながりだけを信じ、遠くは推測する」というのが、近傍グラフに基づく非線形手法 (Isomap・LLE・t-SNE・UMAP) に共通する戦略である。

6.5.3 超立方体 — 次元を上げながら眺める

もう一つ、次元そのものの効果を見る例を挙げる。

d 次元の超立方体の頂点をすべて取る。座標が 0 か 1 の点なので、 2^d 個ある。辺は「1 ビットだけ違う頂点对」を結んだもので、 $d \cdot 2^{d-1}$ 本になる。

$d = 2$ なら正方形、 $d = 3$ なら立方体。ここまでは絵に描ける。 $d = 4$ 以上は描けないので、次元削減して 2 次元で眺めるしかない。

この図形をわざわざ持ち出すのには理由がある。実際のデータでは、もとの空間でどうつながっていたかが分からない。分からないから次元削減して眺めるのであって、出てきた絵が正しいかどうかを確かめる相手がいない。

超立方体では、この確かめる相手が最初から手元にある。どの頂点とどの頂点が隣り合っているかは定義から決まっており、高次元でありながら、そのつながり方を人が完全に把握できる。だから答えを知ったうえで手法を試せる。出てきた絵ともとの形を突き合わせれば、その手法が何を保ち、何

を作り変えたのかが分かる。本節で超立方体を使うのは、手法の性質を測るための物差しとしてである。

ここで大事なのは、**頂点だけでなく辺の上にも点を打つ**ことである。頂点だけを写しても、どの頂点とどの頂点が隣り合っていたかは失われる。辺を細かく分けた点列として一緒に写せば、低次元でも**辺がつながった線**として残るかどうかが目に見える。

色には各点のハミング重み (座標の和、つまり 1 が立っているビット数) を使う。超立方体は重みでレベル分けされるので、これが構造の目印になる。

まず PCA で試すと、意外なことが起きる。

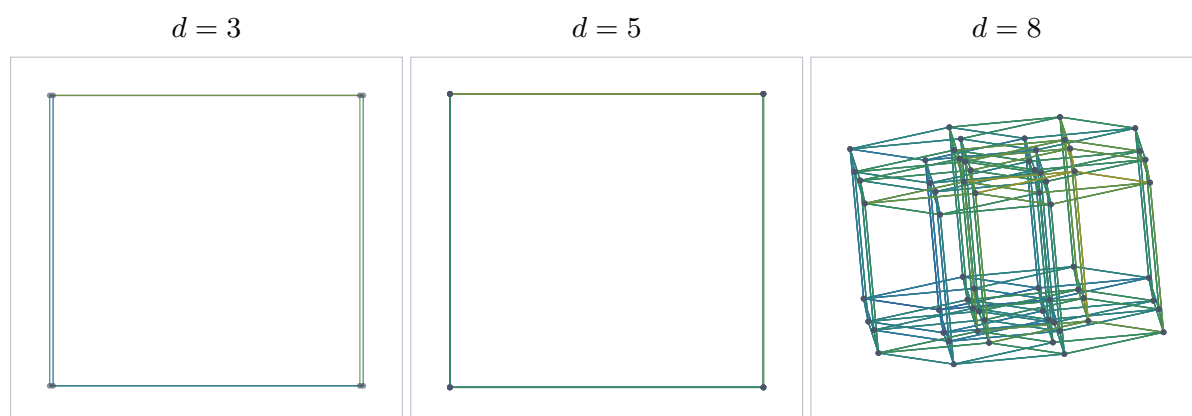


図 6.13 超立方体を PCA で 2 次元に。辺も点列として一緒に写している。色は「1 の個数」(ハミング重み)。 $d=3$ と $d=5$ は正方形に潰れ、 $d=8$ は別の平面に写っている。どちらも PCA の出力として正しい。

$d=3$ と $d=5$ では、 2^d 個の頂点が**正方形の 4 隅**に潰れてしまった。 $d=8$ だけは違う平面に写っているが、どの向きに情報が残ったかを読み取ることはできない。

向きが読み取れないのは、超立方体の頂点が各座標 0 か 1 の点で、**どの軸の分散も等しい**からである。PCA は分散が最大の方を選ぶが、どの方向も同じなので、**選ぶべき方向が決まらない**。 $d=3$ と $d=5$ では座標軸そのものが選ばれ、 $d=8$ では別の 2 方向が選ばれた。実装が返す平面が違うだけで、どれも同じ資格を持つ。いずれにせよ、残りの $d-2$ 次元の情報はそのまま捨てられる。

補足 PCA に向かないデータ

PCA は「分散に偏りがある」ことを前提にしている。すべての方向が対等なデータでは、選ぶべき方向が存在しない。

第 6.2.4 節の式 (6.4) を思い出そう。分散共分散行列の固有値がすべて等しいとき、固有ベクトルは一意に決まらない。超立方体がちょうどその状況にあたる。

距離を保とうとする MDS なら、様子が変わる。

$d=2$ は正方形、 $d=3$ は立方体を斜めから見た形になる。ここまでは直感どおりである。

$d=4$ で少し怪しくなり、 $d=6$ 、 $d=8$ では**何が描いてあるのか分からない**。頂点が円周上に均等に並び、辺が中を埋め尽くしている。

これは投影の失敗ではない。**高次元の超立方体は、2 次元にはどうやっても収まらない**。 $d=8$ な

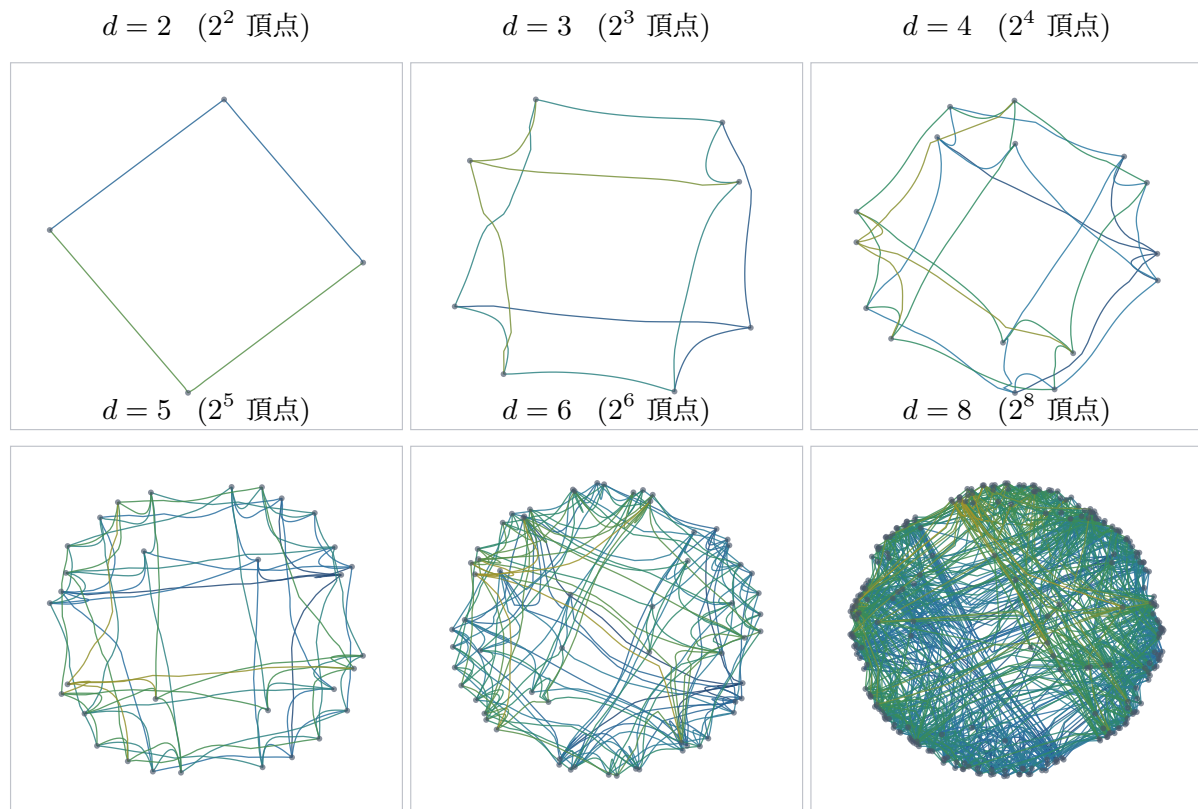


図 6.14 超立方体を MDS で 2 次元に。線が 1 本の辺にあたる。色はハミング重み (濃紺=0 に近い、黄土= d に近い)。次元が上がるほど辺が絡まり合って構造が読めなくなる。

ら 256 頂点と 1024 本の辺を、平面の上で交差させずに配置することはできない。

表 6.1 超立方体の頂点間距離

d	頂点数	辺の数	平均距離	変動係数	$(d_{\max} - d_{\min})/d_{\min}$
2	4	4	1.138	0.172	0.414
3	8	12	1.282	0.207	0.732
4	16	32	1.428	0.216	1.000
5	32	80	1.571	0.214	1.236
6	64	192	1.710	0.206	1.449
8	256	1024	1.970	0.186	1.828

表 6.1 を見ると、頂点間距離の平均は $\sqrt{d/2}$ に従って増えるが、その変動係数 (標準偏差を平均で割った、ばらつきの相対的な大きさ) は $d=4$ の 0.216 を境に頭打ちになり、 $d=5$ の 0.214、 $d=6$ の 0.206 と減り始めている。第 4 章で見た**距離の集中** (付録 B で詳しく扱う) が始まっている。

6.5.4 手法を変えると見え方が変わる

同じ $d=6$ の超立方体を、4 つの方法で描いてみる。

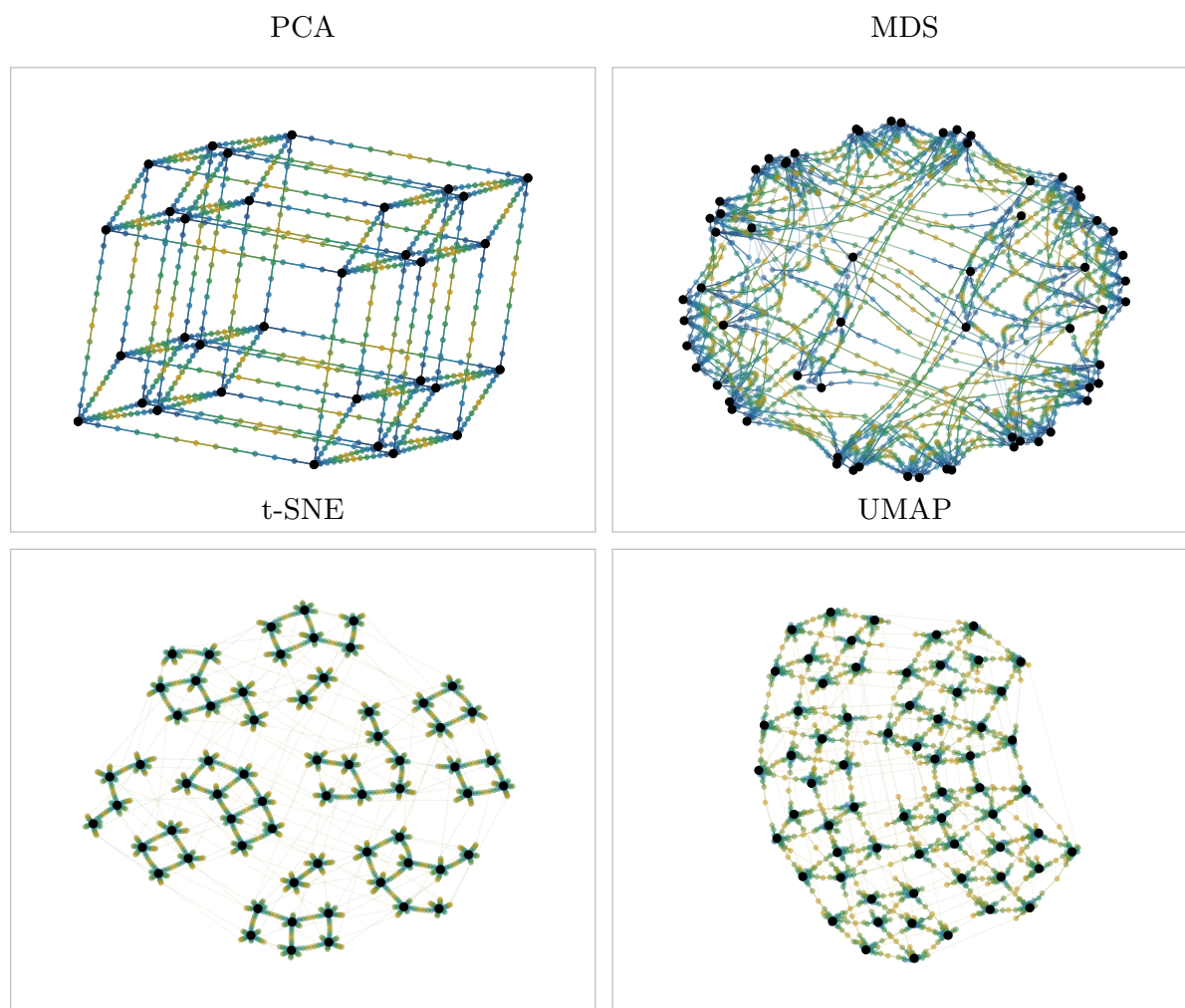


図 6.15 6次元超立方体 (64 頂点、192 辺) を 4 つの方法で。黒い丸が頂点、色のついた丸が辺の途中に等間隔で取った点 (1 本の辺につき 11 個) である。色はもとの辺の上での位置を表す。頂点のとなりが濃紺で、辺の中央へ向かうほど黄土に近づく (図 6.13・図 6.14 のハミング重みとは色の意味が違う)。線の濃さは点の間隔で、まばらな区間ほど薄い。UMAP は `n_neighbors=60`、`min_dist=0.3`。t-SNE では辺の中央の色をした点が頂点に貼りついており、辺が潰れたことが色で分かる。

PCA は辺をすべて直線のまま写す。ただしどの平面に写るかは決まらない。先ほど述べたとおり 6 つの固有値がすべて等しいので、2 本の軸の選び方は無数にあり、実装が返した平面がたまたまこれである。どの平面が返っても、直線が直線に写ることだけは変わらない。

MDS は対称性を保ち、円形に近い配置になる。すべての頂点が対等なので、MDS の振る舞いが正しい。

t-SNE と UMAP は、辺のつながりは残すものの、どの辺がどの辺の隣にあるかを作り替えてしまう。6次元では対等だった 192 本の辺が、平面上では中央に密な部分と外周の疎な部分に分かれる。密なところを見て「ここにクラスタがある」と読むことはできない。超立方体の頂点に密度の差など存在しない。すべて対等である。

6.5.5 辺がまっすぐに見えるのは、点が両端に潰れているから

この図からは、辺の形についてもう一つのことが読み取れる。6 次元での辺はまっすぐな線分であり、その上に 13 個の点を等間隔に並べてある。それが写った先で曲がっていれば、その手法は直線を直線のまま写していない、つまり非線形だということになる。

曲がり具合は、折れ線の長さを両端を結ぶ直線の長さで割った値で測れる。以下ではこれを「曲がり」と呼ぶ。1.00 なら直線のまま、大きいほど曲がっている。192 本の辺すべてについて曲がりを測った中央値は、PCA が 1.00、MDS が 1.15、t-SNE が 1.02、UMAP が 1.08 である。

素直に読めば「t-SNE はほとんど曲げていない」となる。だがこれは読み違いである。

表 6.2 6 次元超立方体の辺 192 本が、写った先でどう並んだか (中央値)。最長区間の割合は、1 本の辺の長さのうち、隣り合う 2 点のあいだのいちばん長い区間が占める割合

手法	曲がり	最長区間の割合	端 4 区間の割合
PCA	1.00	0.08	0.33
MDS	1.15	0.15	0.39
t-SNE	1.02	0.78	0.07
UMAP	1.08	0.23	0.14
等間隔なら	1.00	0.08	0.33

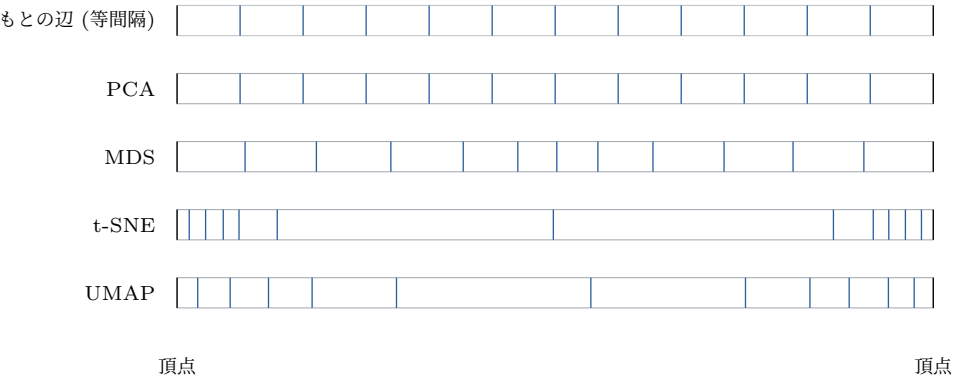


図 6.16 1 本の辺を長さ 1 に伸ばし、13 個の点が来た位置に縦線を引いたもの (192 本の辺の中央値)。両端の太い黒線が頂点。もとは等間隔なので、素直に写っていれば縦線も等間隔に並ぶ。t-SNE では 11 本が両端に張りついて、真ん中が大きく空いている

図 6.16 に、13 個の点が辺のどこに来たかを 1 本の帯で描いた。PCA の縦線は等間隔のままである。t-SNE では 11 本が両端に貼りつき、帯の真ん中は縦線が 1 本もない空白になっている。

表 6.2 の 2 列目「最長区間の割合」は、これを数にしたものである。13 点が等間隔のまま写っていれば、1 区間は辺の長さの $1/12 = 0.08$ を占めるはずである。PCA の最長区間の割合は、192 本の辺の中央値でちょうど 0.08 である。

t-SNE は 0.78 で、等間隔の 0.08 を 0.70 上回る。1 本の辺の長さの 8 割近くを、たった 1 区間が占めている。残る 11 区間は合わせて 2 割しかない。つまり 13 点のほとんどが両端の頂点に吸い寄せら

れ、真ん中は一度の飛躍で結ばれている。3 列目「端 4 区間の割合」も同じことを裏から言っている。両端から 2 区間ずつが辺に占める割合は、等間隔なら 0.33 のところ、t-SNE では 0.07 しかない。

だから「曲がり方が 1.02」は、**直線が直線のまま写ったことを意味しない**。線分が 2 つの点と 1 本の渡り線に潰れた結果、測りようがなくなっているだけである。ここで図 6.15 を見直す。点の色は**もとの辺の上でその点がどこにいたか**を表している。頂点のとなりの点は濃紺、辺のいちばん中央の点は黄土である。

PCA と MDS では、1 本の辺をたどると濃紺から黄土へ変わってまた濃紺に戻る。色が辺に沿って並んでいるので、点も辺に沿って散っている。

t-SNE と UMAP では色の並び方が違う。**黄土の点が黒い頂点に貼りついて**いる。辺の中央にいるはずの点が、頂点と同じ場所にまで引き寄せられている。点の間隔が開いた区間の線を薄くしてあるので、辺の中ほどは線も消え、格子ではなく**点の塊**に見えてくる。すべての区間を同じ濃さで描き、点を打たずにいたときの「きれいにつながった格子」は、描き方が作り出した像だった。

MDS は曲がり方が 1.15、最長区間の割合が 0.15 で、両方とも中くらいである。距離の食い違いを全体で減らそうとするので、点は辺に沿って残るが、線は素直に曲がる。UMAP の最長区間の割合は 0.23 で、等間隔の 0.08 は上回るものの、t-SNE の 0.78 ほど極端ではない。潰れの向きは t-SNE と同じである。

× アンチパターン 6.2 — t-SNE や UMAP が見せた図を、そのまま信じる

局所構造を保つ手法は、**近くの点をさらに近づける**。その結果、もとは連続していたものが**点の塊と隙間**に変わり、**構造がないところにも塊が生まれる**。

図 6.15 の t-SNE は、線だけ描けば「辺がまっすぐ保たれた美しい図」に見える。実際には辺の上の点の大半が頂点に潰れており、線はその隙間を結んだだけである。同じ図から「中央に密な集団がある」と報告することもできない。実際には、密度の差は存在しない。設定を変えれば、はっきり分かれた塊が現れることもある。

対策は次のとおりである。

1. **線ではなく点を描く**。塊と隙間はそれで見える
2. 隣り合う点の間隔を測る。等間隔だったものが等間隔で写っているか。1 つの要約値 (ここでは曲がり 1.02) だけでは「歪んでいない」とは判断できない
3. 複数の手法で描いて比べる (図 6.15 のように)
4. パラメータを変えて安定性を見る (第 6.5.6 項)
5. 見つけた塊が本物か、**もとの高次元空間で確かめる**

補足 「曲がっていないから線形だ」とは言えない

非線形な手法でも、辺の曲がりだけを見ると小さく出ることがある。上で見たとおり、点が潰れていれば曲がりようがないからである。UMAP を既定に近い `min_dist=0.1` で走らせると、潰れがさらに進んで、曲がりの中央値は表 6.2 の `min_dist=0.3` での 1.08 から、t-SNE と同じ 1.02 程度まで下がる。

この向きの推論だけが成り立つ。**曲がっていれば線形ではない**。曲がっていないことは、線形であるこ

補足（続き）

との証拠にならない。

6.5.6 パラメータを振ってみる

前項の測定は、t-SNE と UMAP をそれぞれ 1 つの設定で走らせた結果である。設定を変えれば絵は変わる。どう変わるのかを、同じ 6 次元超立方体で確かめておく。

振るパラメータは 3 つである。t-SNE の `perplexity`、UMAP の `n_neighbors`、そして UMAP の `min_dist` である。前の 2 つは「どれくらいの範囲を近所とみなすか」を、最後の 1 つは「写った先で点をどこまで詰めてよいか」を決める。

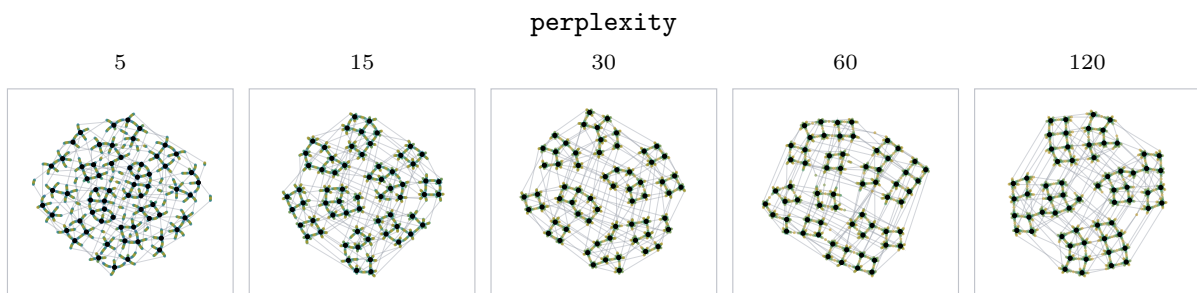


図 6.17 t-SNE の `perplexity` を変えて、同じ 6 次元超立方体を写したもの。黒い丸が頂点、色のついた点が辺の上の点で、色は図 6.15 と同じくもとの辺の上での位置 (頂点=濃紺、辺の中央=黄土)

3 枚の図から読み取れることを順に見る。

t-SNE は、どう設定しても辺を潰す `perplexity` を 5 から 120 まで 24 倍に広げても、192 本の辺の最長区間の割合の中央値は、0.69 から 0.80 のあいだを動くだけである (等間隔に写っていれば 0.08)。変わるのは塊の大きさと配置であって、辺が潰れること自体は変わらない。`perplexity` を選り直しても、この潰れを避けることはできない。

補足 t-SNE に `min_dist` にあたる設定が無いのはなぜか

どちらの手法も、2 つの段階を踏んで平面に写している。第一段階では、もとの高次元空間で「どの点とどの点を近いとみなすか」を決める。第二段階では、写った先の平面で「近い点どうしをどこまで近づけてよいか」を決める。2 つの手法の違いは、この第二段階にある。

第一段階を決めるのが、t-SNE の `perplexity` と UMAP の `n_neighbors` である。名前は違うが役割は同じで、どちらの手法もこの段階の設定を持っている。

第二段階では、平面上の距離 d を「近さ」に直す関数を使う。t-SNE はこの関数を、自由度 1 の t 分布 $q \propto (1 + d^2)^{-1}$ に固定している。この式には調整できる値が入っていないので、使う側が詰まり具合を指定する手段は用意されていない。

UMAP は同じ役割の関数に $(1 + a d^{2b})^{-1}$ という形を使い、 a と b を `min_dist` から当てはめて決めている^a。第二段階の関数そのものが `min_dist` で変わるので、使う側が詰まり具合を指定できる。

前項で見た辺の潰れは、この第二段階が作り出している。UMAP では `min_dist` がその関数を変えるので潰れ方が動き、t-SNE では関数が固定されているので `perplexity` を振っても動かない。2 つの手

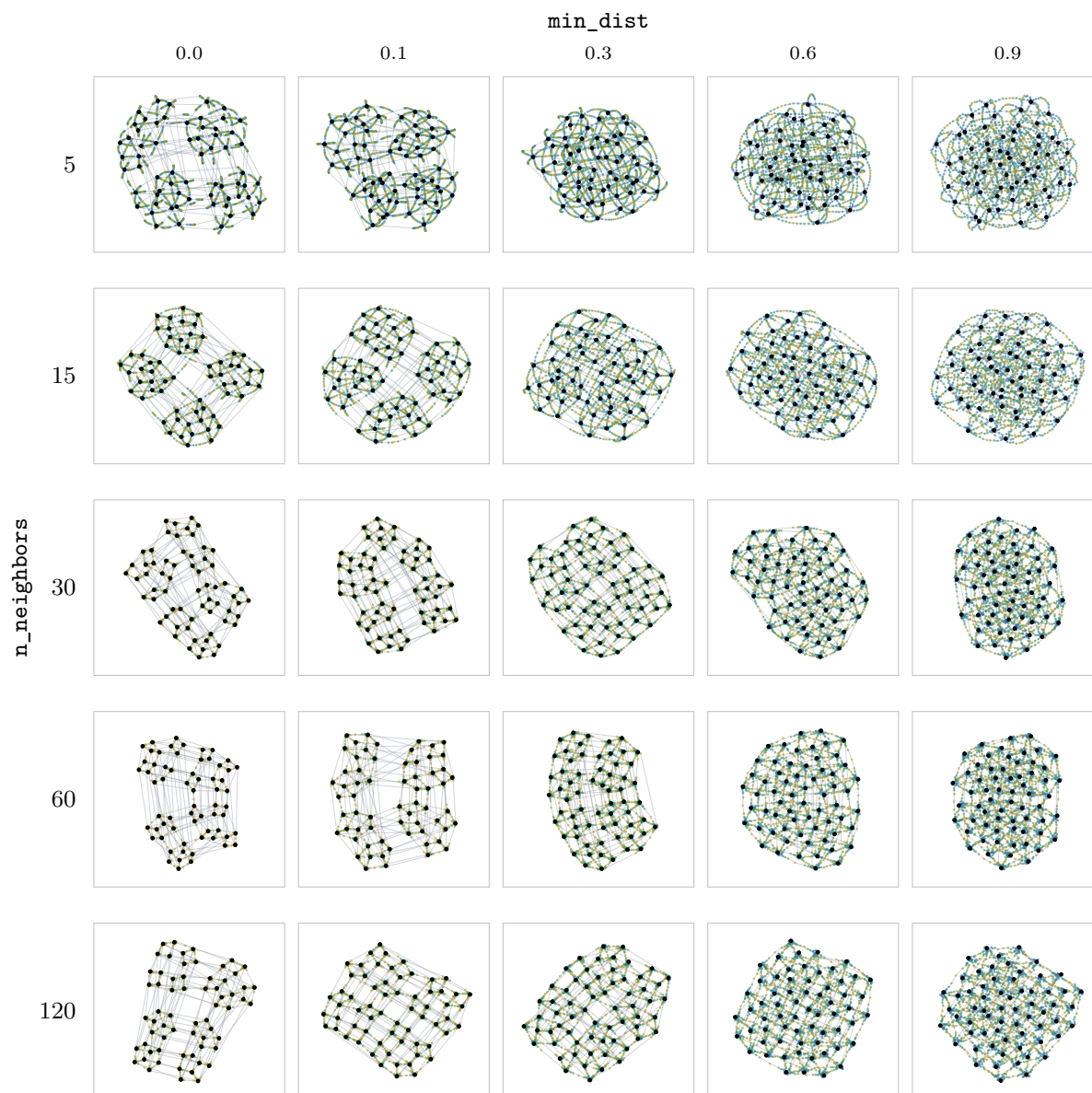


図 6.18 UMAP の 2 つのパラメータを組み合わせた 25 通り。行が `n_neighbors`、列が `min_dist` である。色の意味は図 6.17 と同じ。左端の列 (`min_dist=0`) では点が団子になっており、右へ行くほどほどこけていく

補足（続き）

法で振る舞いが違ったのは、この設計の違いによる。

したがって、t-SNE でも第二段階の関数を差し替えれば潰れ方は変わる。実際、t 分布の自由度 ν を 1 から動かせるようにした研究があり、 ν を小さくして裾を重くするとより細かい塊が見えるようになると報告されている [42]。ただし標準の実装にこの設定は用意されていない。

t-SNE にも `early_exaggeration` や `learning_rate` といった設定はあるが、これらは最適化をどう進めるかを定めるものである。初期値 (`init`) のように結果を大きく変えるものもある (第 6.5.7 節)。ただし、平面上でどこまで詰めてよいかを指定する設定は、この中には含まれていない。

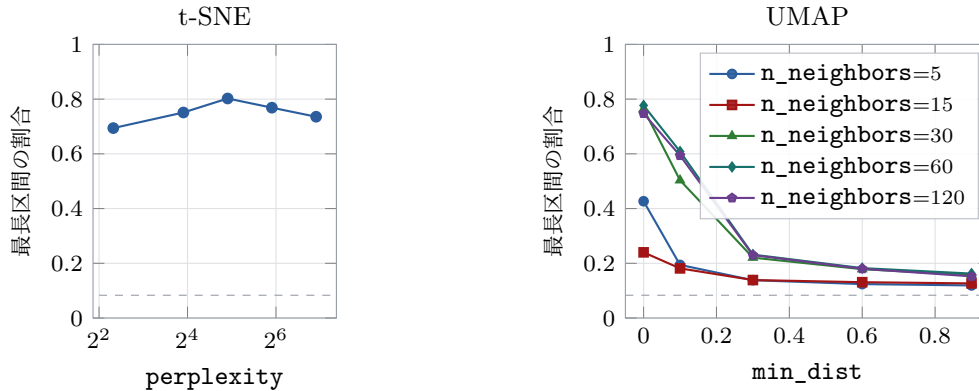


図 6.19 辺の潰れ具合を、パラメータの関数として測ったもの。縦軸は前項の「最長区間の割合」の中央値で、点が等間隔のまま写っていれば 0.08 になる。上に行くほど、辺の上の点が両端に潰れている

補足（続き）

^a 正確には `min_dist` と `spread` の 2 つを与えると、「`min_dist` までは 1、そこから先は指数的に減る」という折れ線に最も近くなるように a と b を数値的に当てはめる。

UMAP で潰れを決めているのは `min_dist` である 図 6.18 を列の方向に見ていくと、左端の `min_dist=0` では点が頂点のまわりで団子になり、右へ行くほどほどけて辺に沿って並ぶ。`n_neighbors` を 5・15・30・60・120 と振ったときの幅で見ると、最長区間の割合の中央値も、左端の `min_dist=0` での 0.24~0.78 から、右端の `min_dist=0.9` では 0.12~0.16 へ下がる。`min_dist` は「写った先で近い点をどこまで詰めてよいか」の下限そのものなので、これが小さいほど点は団子になる。既定値の 0.1 は、まだ潰れが残っている領域にあたる (`n_neighbors=15` での最長区間の割合は 0.18 で、等間隔の 0.08 を上回っている)。

2 つのパラメータは独立に働かない 2 つが独立に働くのであれば、行と列を 1 本ずつ見れば全体が分かるはずである。だが図 6.18 の左端の列は、上 2 行と下 3 行で様子が違う。`n_neighbors` が 5 と 15 のうちは `min_dist=0` でも辺が残っているのに、30 を超えると同じ `min_dist=0` で団子になる。近所を広く取るほど、詰めてよいという指示が強く働くためである。`min_dist` を 0.3 にすると、`n_neighbors` を 5 から 120 まで振ったときの最長区間の割合の幅は、0.09 に縮む。つまり `min_dist` の影響の大きさ自体が `n_neighbors` で変わる。片方を固定したまま他方だけを振った結果から、そのパラメータの働きを結論することはできない。

○ 定石 6.2 — t-SNE や UMAP の図には、必ずパラメータを書き添える

図 6.17 と図 6.18 の 30 枚は、すべて同じデータである。どれか 1 枚だけを見せられたら、そのデータについてまったく違う印象を受ける。

図のキャプションに書くのは次の 3 つである。

- 手法名と実装 (t-SNE は実装によって既定の初期化が違う)

- 主要なパラメータ (t-SNE は `perplexity` と `init`、UMAP は `n_neighbors` と `min_dist`)
- 乱数の種

読むときも同じで、書いていない図から構造を読み取らない。

補足 構造のないデータに塊が現れるのは有名な落とし穴

t-SNE が構造のないデータに塊を作ることは、Wattenberg らの “How to Use t-SNE Effectively” (2016) で詳しく示されている [34]。正規分布からランダムに取った点の集まり、つまり構造がまったくないデータでも、`perplexity` の設定次第で「きれいに分かれたクラスタ」が現れる。

UMAP にも同じ性質があり、同じ趣旨の解説が用意されている (Coenen と Pearce の “Understanding UMAP” [32])。そこでも、塊の大きさと塊どうしの距離には意味がないこと、`n_neighbors` と `min_dist` の設定しだいで絵の印象が変わることが強調されている。

もう一つ、両者に共通する落とし穴がある。「UMAP は t-SNE より大域構造をよく保つ」と言われることがあるが、Kobak と Linderman は、その差が初期値の置き方だけで説明できることを示した [33]。t-SNE の実装は既定で乱数から始め、UMAP はスペクトル埋め込みから始める。t-SNE も同じ初期化にすれば差は消える。本書が第 6.5.7 節で t-SNE を `init="pca"` で走らせているのも、この指摘に沿った設定である。

論文の図を見せられて「クラスタが見つかりました」と言われたら、まずどの手法で、どのパラメータで、どの初期値から描いたかを確認するとよい。

6.5.7 4つの手法を並べる

入力を確認してから比べる。1 点は 1 作品で、その座標は第 3 章で作った 52 次元のベクトルである (字種の比率 3 列、文長分布の μ と σ の 2 列、読点前の平仮名 47 列)。列ごとに単位も桁も違うので、各列を平均 0・標準偏差 1 に標準化してある (第 4.4.2 節)。この 74 行 × 52 列の表を、4 つの方法で 2 次元にした。主なパラメータは t-SNE が `perplexity = 15`、UMAP が `n_neighbors = 15`、`min_dist = 0.35` である (2 つの値をどう変えると結果がどう変わるかは次項で確かめる)。

補足 読点前 47 列を列ごとに z 化してよいか

読点前の平仮名 47 列は、足すと 1 になる組成データである (第 3 章)。列ごとに平均 0・標準偏差 1 へ揃える操作は、合計の制約を壊すし、めったに出ない文字の列を、よく出る文字の列と同じ重みまで引き上げる。筋としては `clr` 変換のほうがよい。第 3 章で測ったとおり、`clr` を通すと著者判別の正解率は 0.918 から 0.960 へ上がった (表 3.4)。

それでもここで標準化を使うのは、52 列に文長の 2 列が混じっているためである。生のままでは文長 μ (対数で 3.4 前後) が他を圧倒し、標準化なしの PCA では第 1 主成分の寄与率が 0.821 になる (第 4 章)。列ごとに標準化した場合の第 1 主成分の寄与率 0.191 (図 6.5) の 4 倍以上である。桁を揃える操作は避けられない。

では図の読みが前処理に依存しているかどうかを確かめる。読点前 47 列と字種 3 列に `clr` をかけ、文長 2 列だけ標準化した表で次項の図 6.21(a) を計算し直すと、数値は変わるが並びは動かない。大域構造の保存 (距離の順位相関) は MDS の 0.847 が最も高く、局所構造の保存 (5 近傍の一致率) は t-SNE の 0.687 が最も高い。LLE とスペクトル埋め込みが左に沈むという読みも変わらない。

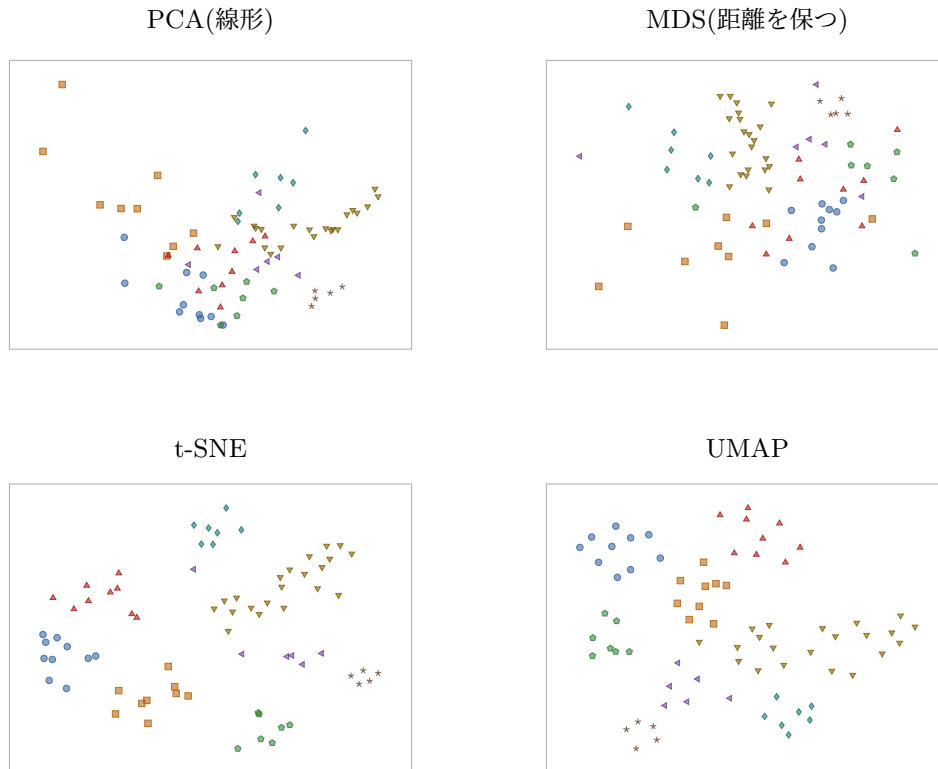


図 6.20 同じ 74 作品を 4 つの方法で 2 次元に。色と記号は著者 (図 6.6 と同じ)。

見た目には t-SNE と UMAP のほうが「きれいに分かれている」。だが、見た目に分かれていることと、元の構造を保っていることは別である。

図 6.20 の 4 枚には目盛りを描いていない。PCA 以外の 3 枚は、軸に読むべき意味がないためである。

× アンチパターン 6.3 — t-SNE や UMAP の軸に意味を読む

PCA の軸は元の変数の線形結合として書けるので、「この方向は漢字の比率が高い側」と言える。第 6.3 節のバイプロットは、それを図にしたものだった。

t-SNE、UMAP、そして SMACOF 法で解く MDS は違う。これらが最小化するのは**点どうしの距離や近さのずれ**だけで、その量は配置全体を回しても裏返しても変わらない。軸の向きを決める力が目的関数の中にない。

52 列を標準化した 74 作品を、同じ設定で乱数の種だけを変えて 2 回 2 次元にする。1 回目と 2 回目で第 1 軸の座標がどれだけ対応するかを、相関係数で測った (比較のため、74 作品それぞれの 5 近傍が 1 回目と 2 回目でどれだけ一致するかも並べる)。

手法	第 1 軸の相関	5 近傍の一致率
PCA	1.000	1.000
MDS(SMACOF 法)	0.467	0.622
t-SNE(<code>init="pca"</code>)	1.000	1.000
t-SNE(<code>init="random"</code>)	0.305	0.892
UMAP	0.543	0.695

t-SNE を乱数から始めると、第 1 軸の相関は 0.305 にとどまる。2 回の座標が完全に対応していれば PCA と同じ 1.000 になるので、同じ図を 2 枚描くと、横軸の値はほとんど別物になっている。それでも 5 近傍の一致率は 0.892 で、9 割近くが保たれている。**誰と誰が近いかは保たれ、軸だけが毎回変わる**。MDS も同じで、第 1 軸の相関 0.467 に対して 5 近傍の一致率は 0.622 である。

`init="pca"` にすると第 1 軸の相関は 1.000 になるが、これは t-SNE が軸を見つけたからではない。その第 1 軸の座標は PC1 の座標と相関 0.741 で、初期値に置いた PCA の軸がそのまま残っているだけである。乱数を変えても結果が変わらないのは、**乱数の入る余地を初期値が奪っているからにすぎない**。**再現することと、意味があることは別である**。

だから、これらの図から読んでよいのは**塊があるか、どの点と同じ塊に入るか**までである。「横軸の右にあるほど文が長い」のような読み方をしたければ、PCA のように軸の意味が定義される手法を使う。塊どうしの間隔についても同じ制限がかかることは、第 6.5.8 節の末尾で改めて扱う。

6.5.8 大域と局所はトレードオフになる

2 つの指標で評価する。

- **大域構造** — 高次元での点对距離と、低次元での点对距離の**順位相関**^{*2}。全体の位置関係をどれだけ保っているか
- **局所構造** — 高次元での 5 近傍と、低次元での 5 近傍がどれだけ一致するか

手法も増やしておく。ここまでの 4 手法に、多様体仮説を前提とする 3 つを加える。いずれもまず各点の近傍グラフを作り、その上で何を保つかが違う。

- **Isomap**[30] — 近傍グラフ上の最短経路長を**測地線距離**の近似とみなし、その距離を MDS で再現する。スイスロールの図 6.11 で「面に沿った距離」と呼んだものである
- **LLE**(局所線形埋め込み、locally linear embedding) [31] — 各点を近傍点の重み付き和で表し、**その重みを保ったまま低次元に置き直す**。距離ではなく「近所での混ぜ方」を保つ
- **スペクトル埋め込み** — 近傍グラフをグラフラプシアンという行列で表し、その固有ベクトルを座標にする。つながりの強い点どうしが近くに置かれる

^{*2} 値そのものではなく、小さい順に並べたときの順位に直してから相関係数を計算したもの。スピアマンの順位相関係数という。直線関係でなくても、一方が増えれば他方も増えるという関係が保たれていれば 1 に近づく。

比べ方には一工夫が要る。t-SNE の perplexity と UMAP の k はどちらも「どれくらいの範囲を近所とみなすか」を決める値で、これを変えると塊の数も塊どうしの間隔も変わる。片方だけ調整して比べては公平にならないので、t-SNE は perplexity を $5 \cdot 15 \cdot 30$ と振る。

UMAP にはもう一つ、写った先で点をどこまで詰めてよいかを決める `min_dist` がある。第 6.5.6 項で見たとおり、この 2 つは独立に働かない。そこで UMAP は k を $5 \cdot 15 \cdot 40$ 、`min_dist` を $0.05 \cdot 0.35 \cdot 0.8$ として 9 通りすべてを走らせた。

ただし 9 点をそのまま図に描くと読めなくなる。そこで他のどの設定にも両軸で負けていない設定だけを残す。どちらか一方でも勝っている設定が他にないものだけが残るので、「この手法はここまでは伸ばせる」という上限が線として見える。9 通りのうち残ったのは両データとも 4 通りだった。

もう一つ、これらの手法は乱数で初期値を決めるので、同じ設定でも実行のたびに結果が変わる。そこで乱数の種を 5 通り変えて実行し、平均を点、標準偏差を誤差棒で示す。PCA と Isomap には乱数が入らないので 1 回だけ実行している。

さらに、傾向が本書のデータ固有でないことを確かめるため、別のデータでも同じ図を描く。ライトノベル 60 区間を、近代文学と同じ作り方で 52 次元のベクトルにしたものである。ただし列の選び方(まれすぎる平仮名を落とす基準)はライトノベルだけで決め直しているの、47 文字の顔ぶれは同じではない。

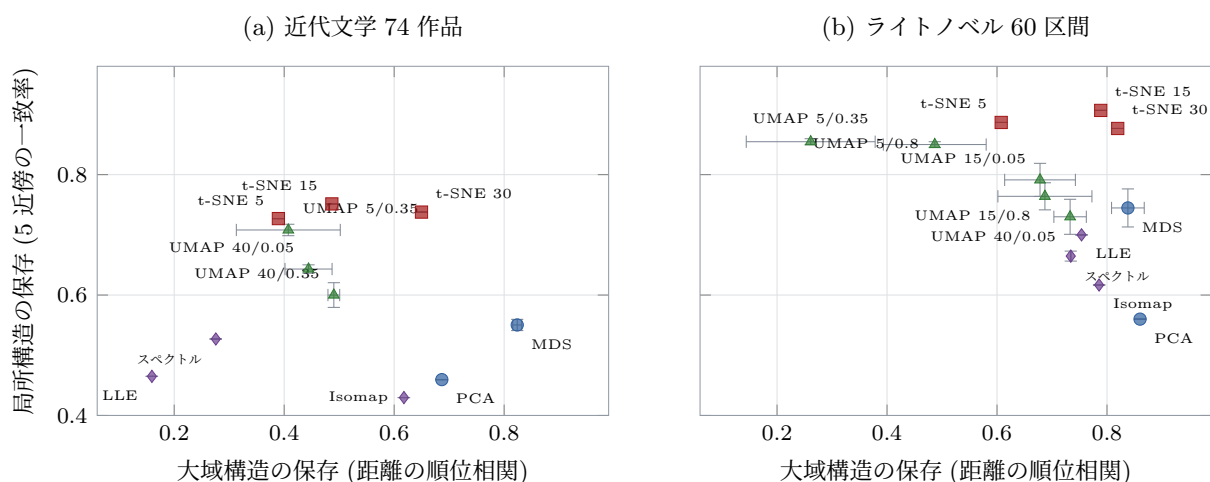


図 6.21 点は乱数の種 5 通りの平均で、誤差棒は縦横それぞれの標準偏差である(横棒が大域構造の指標のばらつき、縦棒が局所構造の指標のばらつき)。縦棒が短いのは、局所の指標が種を変えてもほとんど変わらないためである。右上に行くほど良い。t-SNE に添えた数字は perplexity、UMAP の数字は `n_neighbors/min_dist` である。UMAP は 9 通り試したうち、他のどの設定にも両軸で負けていない設定だけを細い線で結んで載せた。近代文学では右上の角が空くが、まとまりの強いライトノベルでは t-SNE がそこに近づく。トレードオフの強さはデータで変わる。

図 6.21(a) が示すのはトレードオフである。

- MDS は大域構造の保存、すなわち距離の順位相関が 0.824 で最も高い。距離を直接保とうとするので当然
- t-SNE は局所構造の保存、すなわち 5 近傍の一致率が 0.751 で最も高い。perplexity を 5 から 30 に上げると、距離の順位相関が 0.390 から 0.650 へ 0.260 上がる

- UMAP も同じ調整ができる。 k を小さくすれば局所寄り、大きくすれば大域寄りになる。残った 4 通りを結んだ線が、そのまま調整の幅を表している
- PCA・Isomap・LLE は 5 近傍の一致率が下位に並ぶ (それぞれ 0.460、0.430、0.465 で、いずれも t-SNE の 0.751 を 0.28 以上下回る)
- Isomap は PCA の近くに来るが、LLE とスペクトル埋め込みは左に沈む。局所の関係「だけ」を保つ設計の手法は、本書のデータでは大域をほとんど捨てている

誤差棒からも別のことが読み取れる。なおここで描いたのは、乱数の種を 5 通り変えたときの標準偏差である (第 8.3.4 項)。横棒が大域構造の保存 (距離の順位相関) のばらつき、縦棒が局所構造の保存 (5 近傍の一致率) のばらつきを表す。まず横棒と縦棒で長さの桁が違う。図に描いた点でみると、種を 5 通り変えたときの標準偏差は、距離の順位相関では最大 0.117((b) の UMAP $k = 5$) に達するのに、5 近傍の一致率は最大でも 0.032 にとどまる。どの点が誰の近所かは種を変えてもほぼ同じで、その塊を平面のどこに置くかだけが毎回変わるということである。乱数に振り回されているのは全体の配置のほうだと分かる。

横棒の長さは手法によっても大きく違う。t-SNE は誤差棒が点になっている。本書は `init="pca"` で走らせているので、初期値に乱数が入らないためである (前項のアンチパターンで見たとおり、これは安定して見えるだけで、軸に意味が出たわけではない)。一方 UMAP は乱数から始めるので、 $k = 5$ の設定では、距離の順位相関の標準偏差が (a) で ± 0.095 、(b) で ± 0.117 になる。近所とみなす範囲を狭く取るほど、初期値の違いが最後まで残る。手法間の差がこの幅に収まるなら、1 回の実行の値では優劣は区別できない。

(b) は別のデータでの追試である。距離の順位相関も 5 近傍の一致率も、全体に上がる。60 区間はシリーズごとのまとまりが強く、どの手法にも易しいためである。並び方も似ているが、同じではない。t-SNE は perplexity を上げると大域も局所も高く出て、右上の角に近づく。まとまりがはっきりしたデータでは 2 つの指標が衝突しにくくなる。PCA が大域構造の保存で首位に立ち、距離の順位相関が 0.860 になるのも (a) にはなかったことで、シリーズごとの塊がはっきりしていれば線形の射影で足りるという意味である。トレードオフがどれだけきついかはデータで変わる。1 枚のデータで見えた傾向を別のデータで確かめる。一手間かければ、「たまたま」への反論を先回りできる。

× アンチパターン 6.4 — 片方の手法だけ調整して次元削減を比べる

図 6.21 で、t-SNE の perplexity を 1 つしか試さずに描くと、(a) では perplexity = 15 の 1 点、すなわち (距離の順位相関、5 近傍の一致率) = (0.487, 0.751) だけが残る。「局所は強いが大域は弱い手法」という結論がそこから出てしまう。perplexity を 30 まで上げれば (0.650, 0.738) に移り、順位相関が 0.163 上がって一致率は 0.013 下がるだけなので、その結論は手法の性質ではなく設定の選び方から来ていたことになる。

比較の相手にだけ調整の機会を与えれば、自分の推したい手法はいくらでも良く見せられる。論文や記事で「提案手法が既存手法を上回った」と読むときは、既存手法の設定がどこまで振られているかを先に確かめる。自分で比べるときは、各手法の主要なパラメータを同じ本数だけ振ってから並べる。

発展 大域保存と局所保存の 2 指標が測っていないもの

図 6.21 の横軸は「もとの空間での点对距離をどれだけ保つか」である。だが第 6.5.2 節のスイスロールでは、目的は**巻きをほどく**ことだった。ほどくとは、渦の内側と外側で偶然近くにある点对を引き離すことであり、もとの空間の距離を**壊す**ことにほかならない。巻きをほどきたい場面では、横軸の値が高いのはむしろ失敗である。

スイスロールは正解が分かっているので、両方を測って比べられる。「もとの 3 次元空間での距離」との順位相関と、「展開図 (巻き位置と高さ) での距離」との順位相関である。

手法	もとの空間の距離	展開図の距離
PCA	0.816	0.536
MDS	0.840	0.520
t-SNE	0.372	0.594
UMAP	0.309	0.655
Isomap	0.360	0.756
LLE	0.349	0.640
スペクトル	0.348	0.554

どちらの距離を相手にするかで、7 手法の順位が入れ替わる。以下の数値はいずれも距離の順位相関である。MDS はもとの距離で 0.840 の首位なのに、展開図では 0.520 で最下位になる。展開図で首位に立つのは Isomap の 0.756 で、こちらはもとの距離では 0.360 と下位 4 手法の中にいる。UMAP も同じ向きで、もとの距離では 0.309 で最下位、展開図では 0.655 で 2 位になる。そもそも、もとの距離と展開図の距離のあいだの順位相関自体が 0.422 しかない。2 つの「大域構造」は別物である。

Isomap が展開図で首位に来るのは偶然ではない。Isomap は点对距離を**近傍グラフ上の最短経路長**で測る。巻かれた面の上では、その最短経路が面に沿って回り込むので、渦の内側と外側で偶然近くにある 2 点を「遠い」と判定できる。測るものを変えたから、ほどけた。

つまり図 6.21 の 2 軸は、**もとの距離の配置をそのまま 2 次元で眺めたい**という目的に対する指標である。多様体を展開したいなら、測るべきは測地線距離との一致になる。正解の座標が手に入らない実データでは、近傍グラフ上の最短経路長でそれを代用する (Isomap が内部でやっていることである)。

本書の 74 作品で 2 軸を使ったのは、作家の文体に「巻かれた面」のような構造を仮定する理由がなかったからである。仮定を変えれば、測る量も変えなければならない。

同じことは手法の側にも言える。図 6.21 では t-SNE の最良の設定が UMAP のどの設定よりも右上に来ているが、これも「UMAP を使う理由がない」という話にはならない。UMAP が広く使われる理由の一つは**規模**にある。t-SNE の計算量は点の数に対して重く、数十万点になるとつらい。UMAP は大規模データでも速く、メモリも少なく済む。74 点や 60 点の比較では、規模の差は原理的に見えない。もう一つ、UMAP は学習した写像を保持できるので、**あとから来た新しい点を同じ地図に落とせる (transform)**。t-SNE は基本的に全点を作り直しになる。分類の前段に組み込むなら、差は大きい。

まとめると、ベンチマークは、**軸に選んだ性質しか測らない**。軸の選び方が目的と食い違っていることもあれば (スイスロールの例)、軸の外に長所があることもある (UMAP の例)。「この比較は何を測っていて、何を測っていないか」を言えるようにしてから、図の結論を使う。

× アンチパターン 6.5 — t-SNE や UMAP の図で距離を語る

「A と B のクラスタは近いから似ている」とは言えない。これらの手法はクラスタ**内**の近さを保つが、クラスタ**間**の距離は保存しない。

図 6.21(a) のとおり、UMAP の大域構造の保存、すなわち距離の順位相関は、2 つのパラ

メータを9通りに振っても0.37から0.49にとどまる。同じ図でのMDSの0.824には遠く及ばない。クラスタの大きさと塊どうしの間隔も、近傍の広さを決める設定を変えるだけで伸び縮みする。

軸に意味がないこと(前掲のアンチパターン)と合わせると、これらの図で信用してよいのは「どの点とどの点が同じ塊に入るか」だけになる。

6.6 手法は1つに絞らず、並べて描く

表 6.3 次元削減法の比較。「保存が最良」は本書のデータでの結果(図 6.21)による。

手法	長所	短所	向く場面
PCA	軸を解釈できる。パイプロットが描ける。速く決定的。新しいデータも同じ変換で写せる	線形変換のみ。局所構造の解像度が低い。分散が代表値として不適切な分布では弱い	変数の意味を知りたい。前処理として
MDS	距離を直接保つ。座標がなく距離行列だけでも使える。大域構造の保存が最良	$O(n^2)$ 以上で大規模データに弱い。局所構造はt-SNEほど保てない	距離しかないデータ。全体の位置関係が重要
t-SNE	局所構造の保存が最良。クラスタが見やすい	遅い。大域構造・クラスタ間距離が保存されない。新しいデータを写せない。乱数依存	クラスタの存在を確認したい
UMAP	t-SNE より 速い。 <code>n_neighbors</code> で局所と大域のバランスを調整できる。新しいデータも写せる	クラスタ間距離は保存されない。パラメータに敏感。理論が難しい	大規模データの探索的可視化
オートエンコーダ	任意の非線形変換。次元削減と復元を同時に学習	訓練データが大量に必要。設計と学習の手間	画像・音声など大規模データ

○ 定石 6.3 — 次元削減は複数の手法を並べて見る —

どれか一つを選ぶ必要はない。図 6.20 のように並べて描くのが最も安全である。

- すべての手法で同じ構造が見えたら、それは本物らしい
- 一つの手法でしか見えない構造は、その手法の産物かもしれない

そして必ず定量指標も見る。図 6.21 のような評価を一度すれば、どの図をどこまで信じてよいか分かる。

補足 オートエンコーダと PCA の関係

オートエンコーダは、入力を低次元に圧縮してから復元するニューラルネットワークである。

$$x \xrightarrow{\text{エンコーダ}} z \xrightarrow{\text{デコーダ}} \hat{x}, \quad \min \|x - \hat{x}\|^2 \quad (6.12)$$

途中の変換を掛け算と足し算だけで組み、非線形な関数を挟まずに作ると、**PCA と同じ平面を学習**することが知られている。非線形な関数を挟むことで、PCA では表せない曲がった構造まで扱えるようになる。

第 6.2.5 節では「射影誤差の最小化」という定式化を見た。式 (6.12) の $\min \|x - \hat{x}\|^2$ は、「もとに戻したときのずれを最小にする」という同じ形をしている。オートエンコーダは、PCA の当てはめを非線形に拡張したものだと理解できる。

6.7 章末の課題

1. 図 6.4 で使った 3 列に片仮名の比率を加えて 4 次元にし、寄与率がどう変わるか調べよ。
2. 標準化せずに PCA を行くと何が起きるか。第 4 章の図 4.4 を踏まえて予想し、確かめよ。
3. 第 1 主成分の係数 (固有ベクトルの成分) を大きい順に並べ、どの特徴が寄与しているか調べよ。文体の言葉で解釈できるか。
4. MDS で 2 次元に落とし、PCA の結果と比べよ。どちらが著者ごとのまとまりを見せるか。
5. PCA で 2 次元に落としてから距離を測るのと、52 次元のまま距離を測るのでは、第 4 章の効果量 (表 4.3) はどう変わるか。
6. 字種 3 列 (平仮名・片仮名・漢字) だけに、標準化せずセンタリング (平均を引く) のみで PCA をかけ、第 3 主成分の固有値が (浮動小数点の誤差を除いて) 0 になることを確かめよ。固有ベクトルはどの方向を向いているか。(ヒント: 合計 1 の平面の法線方向、つまり $(1, 1, 1)/\sqrt{3}$ に平行になるはずである。第 3 章の共分散の行和ゼロの議論から導ける。)

第7章

著者を当てる

7.1 正解を与えて学ばせる

前章では著者名を隠してグループを作った。今度は逆に、著者名を教えて**未知の作品の著者を当てるモデル**を作る。

これを**教師あり学習** (supervised learning) という。手順を図 7.1 に示す。

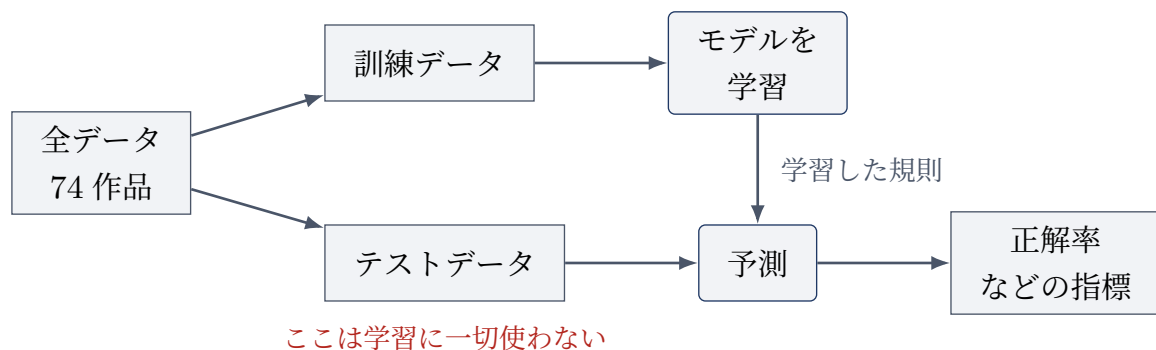


図 7.1 教師あり学習の基本手順。

図の下に注記が肝心である。テストデータは学習に使わない。理由は本章の後半で実際に見る。

7.2 まず当てずっぽうの成績を知る

モデルを作る前にやることがある。**何も学習しない戦略の成績**を計算しておく。

考えられる最も単純な戦略は、「いつも最も作品数の多い著者だと答える」ことである。吉川英治が 21 作品なので、正解率は $21/74 = 0.284$ になる。

表 7.1 基準線

戦略	正解率
8 名から一様ランダムに選ぶ	0.125 ^{*1}
つねに最頻クラス (吉川英治) と答える	0.284

何も考えずに吉川英治と答え続けるだけで、74 作品の正解率は一様ランダムに選んだときの **2 倍以上**になる。

^{*1} 1/8 という理論値である。第 8 章では同じ基準線を実際に 2000 回引いて測っており、そちらは 0.126 になる。

× アンチパターン 7.1 — 基準線を置かずに精度を報告する

「正解率 50% を達成した」と言われても、それが良いのか悪いのか分からない。クラスが偏っていれば、最頻クラスを答えるだけで 50% を超えることもある。

クラス数が K のとき、当てずっぽうの水準を $1/K$ だと思ってしまいがちである。それはすべてのクラスが同じ大きさのときの話で、偏りがあれば**最頻クラスの割合**のほうが高くなる。モデルを超えるべきなのは、高いほうである。ここを取り違えると、無意味なモデルを成功と誤認する。

7.3 k 近傍法 — 最も単純な分類器

k 近傍法 (kNN、 k -nearest neighbors) は、モデルらしいモデルを作らない。新しい作品が来たら、**最も近い k 個の訓練データを探し、その多数決で答える**。手続きはこれだけで終わる。

第4章で距離を定義したので、そのまま使える。

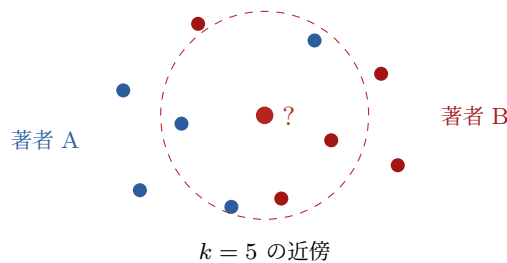


図 7.2 k 近傍法。円内の 5 点の多数決で「?」の著者を決める。この例では著者 A が 3 点、著者 B が 2 点なので、判定は著者 A になる。

k の値で性格が変わる。

- $k = 1$ — 最も近い 1 点をそのまま真似る。訓練データには完璧に合う
- k が大きい — 広い範囲の多数決になり、なめらかな境界になる

補足 kNN は訓練のときに何もしていない — 怠惰学習

kNN は訓練の段階で何もしない。データを覚えるだけである。計算は予測のときに全部行う。訓練を先送りする手法を**怠惰学習 (lazy learning)**と呼ぶ。

利点は、モデルの形を仮定しないので複雑な境界も表現できること。欠点は、予測のたびに全データとの距離を計算するので遅いこと、そして第9章で見るように**次元の呪いに弱い**ことである。

7.4 線形モデル — 1 本の境界で分ける

kNN は境界を持たない。もう一方の極に、**空間を 1 本の境界で仕切る**という考え方がある。

2 つのクラスを分ける境界を、直線 (3 次元なら平面、52 次元なら超平面) で書く。

$$s(x) = w_1x_1 + w_2x_2 + \cdots + w_nx_n + b \quad (7.1)$$

$s(x) > 0$ なら一方のクラス、 $s(x) < 0$ ならもう一方とする。学習とは、係数 w と切片 b を決める

ことである。 w_j は「その特徴が1増えるとスコアがどれだけ変わるか」なので、**どの特徴をどちら向きに使ったかが読める**。決定木と並んで、説明のしやすい手法である。

決め方が2通りある。

7.4.1 ロジスティック回帰 — 確率に直して最尤法で決める

式 (7.1) のスコアは $-\infty$ から $+\infty$ までの値を取るなので、そのままでは確率として読めない。そこで**シグモイド関数**で0から1に押し込める。

$$P(\text{クラス 1}) = \frac{1}{1 + e^{-s(x)}} \quad (7.2)$$

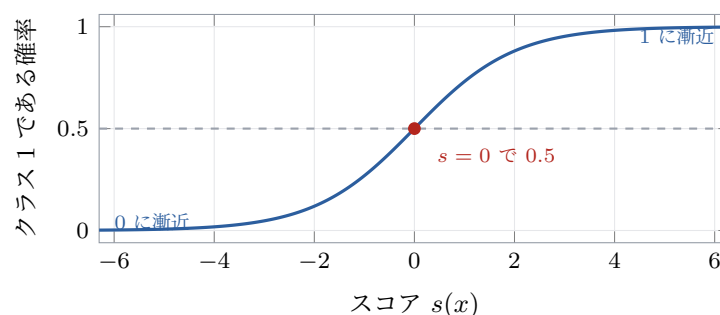


図 7.3 シグモイド関数。 $-\infty$ から $+\infty$ までの値を取るスコアを、0 から 1 の確率に押し込める。原点付近では急に、両端ではゆるやかに変化する。

図 7.3 のとおり、スコアが0のとき確率 0.5、大きいほど1に近づく。境界そのもの ($s = 0$) では「どちらとも言えない」と答え、境界から離れるほど自信のある確率を返す。係数は**最尤法**で決める。手元のデータが最も出やすくなる w と b を選ぶという基準である (第3章の発展「 $n-1$ で割るべきか」でも触れた)。これが**ロジスティック回帰** (logistic regression) である。

本書のように著者が8名いる場合は、著者ごとに線形スコアを1本ずつ、計8本用意し、予測のときは8本のスコアを比べて、最大の著者を答える。学習のさせ方には2つの流儀がある。著者ごとに「その人か、そうでないか」の二値問題を別々に解く流儀 (one-vs-rest) と、8本のスコアをまとめて確率に変換し、全体を1つの尤度として最大化する流儀 (多項ロジスティック回帰) である。scikit-learn の既定は後者で、本書の実験もそれに従う。なお実際に解かれるのは、正則化項 (第 4.4.2 節) を足した罰則付きの最尤推定である。

発展 二値の結果を確率で答えるモデルは交差エントロピーで測る

「太宰治か、そうでないか」のように答えが2つに1つで、モデルの出力が0から1の確率 p である場合を考える。その良し悪しを1つの数にまとめる基準が**交差エントロピー** (cross entropy) である。**対数損失** (log loss) ともいう。正解を $y \in \{0, 1\}$ 、モデルの答えを p と書くと、 N 件の平均は次の量である。

$$L = -\frac{1}{N} \sum_{i=1}^N [y_i \ln p_i + (1 - y_i) \ln(1 - p_i)] \quad (7.3)$$

y_i が1なら $-\ln p_i$ 、0なら $-\ln(1 - p_i)$ だけが残るので、**正解のほうに与えた確率の対数**を集めてい

発展 (続き)

ることになる。正解に 0.5 を与えたときの損失は 0.69、0.1 しか与えなければ 2.30、0.01 なら 4.61 と、言い切って外すほど急に立ち上がる。

クラスが K 個ある場合も同じ形から出る。正解のクラスだけ 1、ほかを 0 にした K 個の数 $y_{i1}, \dots, y_{iK}$ (one-hot 表現) を使うと、次のように書ける。

$$L = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^K y_{ik} \ln p_{ik} \quad (7.4)$$

$K = 2$ とし $y_{i2} = 1 - y_{i1}$ 、 $p_{i2} = 1 - p_{i1}$ を入れれば式 (7.3) に戻る。内側の和は正解のクラス以外がすべて 0 倍で消えるので、実際に残るのは**正解のクラスに与えた確率の対数** 1 項だけで、次のように書き直せる。

$$L = -\frac{1}{N} \sum_{i=1}^N \ln p_i (\text{正解のクラス}) \quad (7.5)$$

この形が選ばれる理由を 2 つ挙げる。

最尤法と同じものである 1 件の観測が確率 p で表が出るコイン投げだとすると、 N 件が観測される確率 (尤度) は $\prod_i p_i^{y_i} (1 - p_i)^{1 - y_i}$ である。対数を取り、符号を変え、 N で割ると式 (7.3) そのものになる。「手元のデータが最も出やすくなる係数を選ぶ」という最尤法の言い方と、「交差エントロピーを最小にする」という言い方は、同じ手続きの別名である。ロジスティック回帰が最尤法で係数を決めるのは、この損失を最小にしていることにほかならない。

勾配が飽和しない シグモイド (式 (7.2)) と式 (7.3) を組み合わせると、係数についての勾配は $(p - y)x$ という簡単な形になる。外している量にそのまま比例するので、大きく外していれば大きく動く。ここで二乗誤差 $(p - y)^2$ を使うと、連鎖律でシグモイドの微分 $p(1 - p)$ が掛かる。 p が 0 や 1 に近い領域ではこれがほぼ 0 になるので、**確信を持って間違えているときほど動けなくなる**。交差エントロピーはこの $p(1 - p)$ をちょうど打ち消す。式の見え目より、この性質のほうが実務では大きい。

なお $-\ln p$ は「その結果を知ったときの意外さ」と読める。確率 0.5 の出来事が起きても意外ではないが、0.001 と言っていた出来事が起きれば大きく驚く。その平均が交差エントロピーである。底を 2 に取れば、結果を符号化するのに要するビット数になる。第 6 章の t-SNE が高次元と低次元の 2 つの分布のずれを測っていたのも、定数の差を除いてこれと同じ量である (あちらは KL ダイバージェンスで、交差エントロピーからエントロピーの項を引いたものにあたる)。

ここでは学習の目的関数として見たが、交差エントロピーは出来上がったモデルの確率を測る側にも使う。Brier スコアとの比較は第 8.6 節で扱う。

関連した列があると係数の符号が入れ替わる 線形モデルの利点は、係数が読めることだとされる。「 w_j が大きい列ほど、判定を強く左右している」という読み方である。

その読み方が壊れる場合がある。**強く関連した列が同時に入っているとき**である。

本書のデータで確かめる。平仮名の比率と漢字の比率は、第 3 章で見たとおり $r = -0.915$ と強く負に相関している。合計が 1 に近い比率どうしなので、片方が増えれば他方は減る。

この 2 列で「太宰治かどうか」を当てるロジスティック回帰を組む。2 列とも標準化してあるので、係数は「その列が 1 標準偏差だけ増えたときにログオッズがどれだけ動くか」を表す。74 作品から重複を許して 74 作品を引き直し、そのたびに係数を求める操作を 300 回繰り返した (ブートストラップ)。ただし引き直した結果、太宰の作品が 1 つ以下しか入らない標本が 3 回出た。片方のクラスが

ほぼ空では係数が意味をなさないで、それらは除いて 297 回とした。比べるのは、平仮名の係数が 2 つの場合でどう変わるかである。

結果は 2 つの形で現れた。

まず**振れ幅が広がった**。297 個の標本にわたる平仮名の係数の標準偏差は、平仮名だけを入れたモデルで 2.4、漢字も入れたモデルで 3.4 である。同じ列の係数なのに、相方がいるだけで 1.5 倍ほど不安定になる。

係数の不安定さより重いのが、2 つ目の変化である。**符号が逆になった**。平仮名だけのモデルでは係数の符号が一方に定まるのに対し、漢字も入れたモデルでは 89% の標本で反対向きになる。「平仮名が多いほど太辛い」が「少ないほど太辛い」に化ける。

ここで気をつけたいのは、89% という数字である。コインを投げるように符号が揺れているだけなら、反対向きになる割合は 50% 前後になるはずである。9 割近いということは、揺らぎではなく**ほぼ毎回そうなる**、つまり系統的な入れ替わりである。

符号が入れ替わるのは、平仮名と漢字がほとんど同じ情報を裏返しに持っているためである。「平仮名に +5、漢字に -5」と置いても「平仮名に -3、漢字に +3」と置いても、足し合わせたスコアはほとんど変わらない。そのうえ漢字の係数が入ることで、平仮名の係数が担う役割そのものが置き換わる。

2 列モデルの平仮名の係数が表しているのは、もはや「平仮名の多さの効果」ではない。**漢字の値を固定したうえでの、平仮名の相対的な位置**である。1 列モデルの係数とは、そもそも別の量を測っている。値が違っても符号が違っても、当然といえば当然である。

損失の形を描くと、何が起きているかが見える。

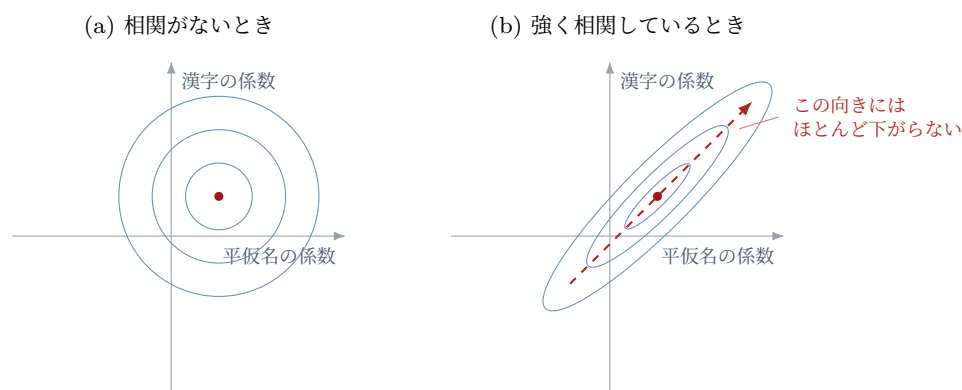


図 7.4 2 つの係数を動かしたときの予測誤差の等高線 (模式図)。中心の点が最も誤差の小さい組合せである。相関がなければ等高線は丸く、中心から外れるとすぐ誤差が増える。強く相関していると等高線は細長くなり、赤い破線の向きにはどこまで動かしてもほとんど誤差が変わらない。標本が少し変われば、中心はこの谷に沿って大きく滑る。

図 7.4 の右が、平仮名と漢字を並べたときの状況である。 $r = -0.915$ の 2 列では、予測誤差の等高線が描く楕円の長軸が、短軸の約 5 倍の長さになる。谷に沿って滑るので、標本を引き直すたびに係数の組は大きく動く。それでも**足し合わせたスコア**は谷の底に沿ってほぼ一定なので、予測の当たり方は変わらない。壊れるのは係数の読みだけである。

この状態を**多重共線性** (multicollinearity) という。強さの目安には **VIF** という量があり、2 列の

場合は $1/(1-r^2)$ で計算できる。本書の平仮名と漢字では 6.1 になる。5 を超えたら係数の解釈に注意、というのが慣用の目安である。

× アンチパターン 7.2 — 相関の強い列を並べたまま係数の大小を比べる

係数は「他の列を固定したときの効果」を表すことになっている。だが平仮名と漢字のように連動する列では、**片方だけを動かす**という状況が現実には存在しない。存在しない状況についての数字なので、大小を比べても意味が薄い。

紛らわしいのは、**予測の性能そのものは落ちない**ことである。足し合わせたスコアは安定しているので、当たり方は変わらない。壊れるのは解釈のほうだけで、成績を見ていると気づけない。

係数を読みたいなら、相関の強い列は片方に絞るか、まとめて 1 つの指標にするか、正則化 (第 4.4.2 節) で係数を安定させる。

ただし L1 正則化で列を絞る場合は、別の注意が要る。関連した列のどれが残るかは標本しだいで入れ替わるので、残った列を「重要な列」と読むことはできない (第 7.11 節のアンチパターン)。

補足 分類なのになぜ「回帰」と呼ぶのか — 一般化線形モデル

式 (7.1) の線形の当てはめをそのまま使っているからである。違いは、出力を確率に直す関数を 1 枚かぶせた点だけにある。

見方を枠組みにしたものが**一般化線形モデル** (generalized linear model、GLM) で、部品は 3 つある [45]。

- **線形予測子** $\eta = w_1x_1 + \dots + w_nx_n + b$
- 当てたい値が従う**分布** (正規分布、二項分布、ポアソン分布など)
- 両者を結ぶ**リンク関数** g 。 g (当てたい値の期待値) $= \eta$ と置く

線形回帰は「正規分布 + 恒等リンク ($g(\mu) = \mu$)」、ロジスティック回帰は「二項分布 + ロジットリンク」である。**ロジット**とは

$$g(p) = \ln \frac{p}{1-p} \quad (7.6)$$

つまり**オッズの対数**で、これを η と等しいと置いて p について解くと、式 (7.2) のシグモイドが出てくる。シグモイドはロジットの逆関数である。回数を当てたいなら「ポアソン分布 + 対数リンク」を選ぶ (ポアソン回帰)。どれも中身は同じ線形の当てはめで、当てたい値の性質に合わせてリンクを取り替えているだけなので、まとめて「回帰」と呼ばれている。

ロジットが**比の対数**であることにも注目してほしい。第 3.7 節で組成データに使った対数比と同じ形である。 p と $1-p$ は合計が 1 の 2 成分の組成データなのだから、同じ道具が出てくるのは自然なことである。

名前が紛らわしいのだが、**一般線形モデル** (general linear model) という別の語もある。そちらは正規分布と恒等リンクに限った枠組み (重回帰や分散分析を含む) を指す。「一般化」の 3 文字があるかどうかで、指すものが違う。

7.4.2 サポートベクトルマシン — 隙間を最大にする

2つのクラスがきれいに離れているなら、境界の引き方は1通りではない。図7.5(a)のように何本でも引ける。

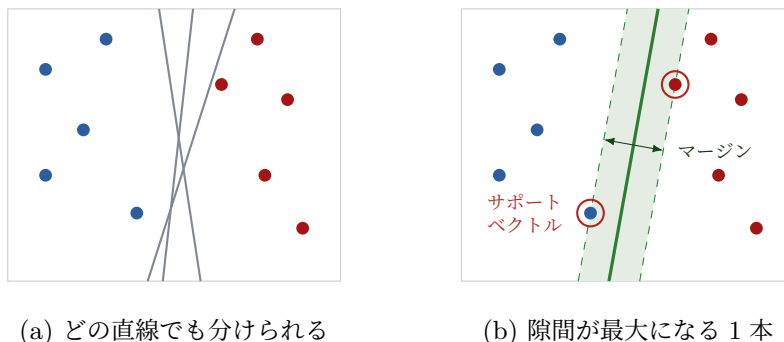


図 7.5 境界の引き方。2枚とも同じ点である。(b)の破線は、境界から左右に等しく離れたマージンの縁で、赤い輪をかけた2点がちょうどその上に乗る。この2点が境界を決めており、これより境界に近い点は存在しない。

サポートベクトルマシン (SVM) は、境界と最も近い点との距離、すなわち**マージン**を最大にする位置に境界を引く(図7.5(b))。境界を決めているのは、境界の近くにある少数の点だけである。境界を支える少数の点を**サポートベクトル**と呼び、名前の由来になっている。遠くの点をいくら動かしても境界は変わらない。

直線で分けられないデータには**カーネル**を使う。データを高次元の空間に写してから直線で分けるという発想である。もとの空間に戻して見れば、境界は曲がって見える。

いちばん小さな例で見る。数直線の上に、原点の近くに青い丸が、その両外側に赤い四角が並んでいるとする(図7.6(a))。1本のしきい値では、この並びを2つに分けられない。赤い四角が、青い丸をはさんで左右に散っているからである。ところが縦軸に x^2 を足して平面に持ち上げると、横一直線で分けられるようになる。

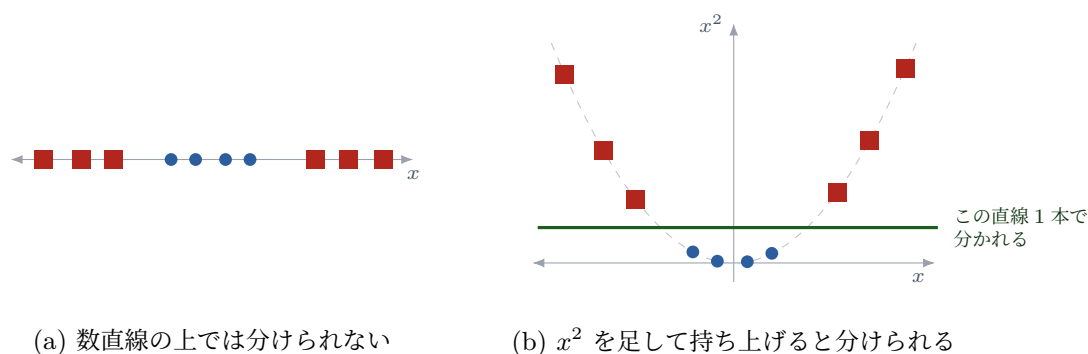


図 7.6 次元を上げると直線で分けられるようになる例。(a)の1次元では赤い四角が両端に分かれているので、どこにしきい値を置いても片方が残る。(b)のように x^2 を2本目の軸として足すと、遠い点ほど高く持ち上がるので、水平な直線1本で分かれる。この直線を(a)に戻すと $|x| = \text{一定}$ 、つまり**2つのしきい値**という曲がった境界になる。

図 7.6 でやったのは、 x という 1 列から (x, x^2) という 2 列を作る操作である。実際のカーネルはこれをずっと高い次元で行う。高次元に写すほど分けやすくなる理屈は、付録 B の Cover の定理にある。

肝心なのは、**高次元に写した座標を実際には計算しない**ことである。SVM の計算に出てくるのは 2 点の内積だけなので、「写した先での内積の値」を別の式で直接求めてしまえば足りる。その式をカーネルと呼ぶ。本書で使う RBF カーネル (radial basis function、動径基底関数) は $\exp(-\gamma\|x - x'\|^2)$ という形で、2 点のユークリッド距離を直接使う。 γ は**どれくらい近い点だけを近いとみなすか**を決める値で、大きくすると境界が個々の点のまわりに細かく巻きつき、小さくすると直線に近づく。

SVM には、もう 1 つ C という設定値がある。これは**訓練データの誤分類をどれだけ許すか**を決める。 C を大きくすると誤りを許さなくなり、境界が訓練データに合わせて無理な形をとる。 γ と C はどちらも複雑さを決めるパラメータで、2 つとも大きくするほど過学習しやすくなる。本書ではどちらも既定値のまま使っている (第 7.10.1 項)。

補足 距離や正則化を使うモデルには標準化が要る

線形モデルはどちらも、複数の列を足し合わせて 1 つのスコアを作る (式 (7.1))。しかも正則化項や距離が入るので、列の単位がそろっていないと特定の列だけで結果が決まってしまう。第 4.4.2 節の表 4.2 は、74 作品の著者を層別 5 分割の交差検証で判別したときの正解率を、標準化の有無で並べたものである。標準化しない SVM(RBF カーネル) の正解率が、最頻クラスを答え続ける基準線 0.284 と同じ 0.284 にとどまったのも、単位のそろわない列を足したからである。

同じことが **k 近傍法**にも当てはまる。こちらは足し算ではなく距離を測るが、単位のそろわない列を混ぜれば距離が 1 列に支配されるという事情は同じである。同じ表の k 近傍法 ($k = 5$) では、正解率が標準化なしの 0.650 から標準化ありの 0.877 へ、0.227 上がる。本書が以降で報告する kNN の数字は、すべて標準化してから測ったものである。

木のモデルは 1 列ずつ大小を見るだけなので、標準化しても結果が変わらない。**列を足すか、距離を測るモデルには標準化が要る**と覚えておけばよい。

7.5 決定木 — 質問を繰り返す

決定木 (decision tree) は、「この特徴がこの値より大きいか」という質問を繰り返してクラスを絞り込む。

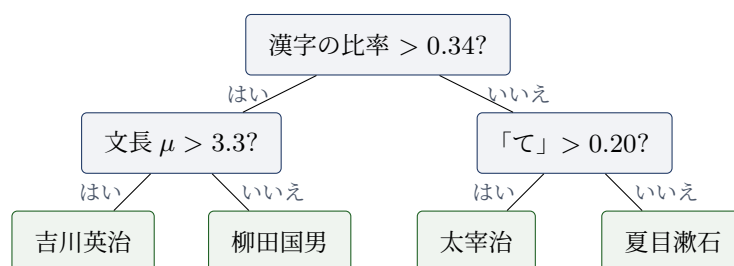


図 7.7 決定木の模式図 (実際の木を単純化したもの)。

木を作るには「どの特徴のどの値で分けるか」を決めなければならない。基準として**情報利得** (information gain) がよく使われる。

まず、集合の乱雑さを**情報エントロピー** (entropy) で測る。クラス i の割合を p_i として

$$H = - \sum_i p_i \log_2 p_i \quad (7.7)$$

すべて同じクラスなら $H = 0$ 、均等に混ざっているほど大きくなる。

特徴 A で分割したときの情報利得は

$$\text{IG}(A) = H(\text{分割前}) - \sum_v \frac{|S_v|}{|S|} H(S_v) \quad (7.8)$$

分割によってどれだけ乱雑さが減ったかを表す。これが最大になる特徴を選んで分割する。それを繰り返す*2。

補足 エントロピーの直感

コインを投げて表裏を伝えるのに必要な情報は1ビットである。 $H = -0.5 \log_2 0.5 - 0.5 \log_2 0.5 = 1$ 。
いかさまコインで表が90%出るなら、 $H = -0.9 \log_2 0.9 - 0.1 \log_2 0.1 \approx 0.47$ ビットで済む。ほとんど表だと分かっているのだから、伝える情報が少なくて済む。
エントロピーは「平均して何ビット必要か」を表している。

決定木の魅力は、**できた木を人が読めること**である。図 7.7 のような木なら、なぜその著者と判定したかを説明できる。これは後で見るランダムフォレストにはない性質である。

7.6 ランダムフォレスト — 木をたくさん作る

決定木は1本だと不安定である。データが少し変わると、まったく違う木ができることがある。

ランダムフォレスト (random forest) はこの弱点を、数を増やすことで解決する [46]。

1. 訓練データから重複を許して同じ件数を引き直す (**ブートストラップ**)。木ごとに見ているデータが違うものになる
2. 各分割では、全特徴のうち一部だけをランダムに候補にする
3. これを数百回繰り返して木を作り、多数決で決める

手順1で、なぜ木ごとにデータが変わるのかを確かめておく。74件から74回引くとき、ある1件が1回の抽選で選ばれない確率は $1 - 1/74$ である。74回とも選ばれない確率はそれを74乗した値で、 $(1 - 1/74)^{74} \approx 0.37$ になる*3。つまり平均して3分の1強の作品が、その木の訓練から漏れる。漏れる顔ぶれは木ごとに違うので、育つ木も違うものになる。

2つのランダム性がここで働く。1本の木が3割間違えるとしても、**どこで間違えるかが木ごとにばらばら**なら、ある1作品について間違える木は全体の一部にとどまる。多数決を取れば、正しく答

*2 この量は、情報理論でいう**相互情報量** (mutual information) にほかならない。第2項は「どの枝に落ちたかを知ったあとに残る乱雑さ」、すなわち条件付きエントロピー $H(Y | A)$ なので、 $\text{IG}(A) = H(Y) - H(Y | A) = I(A; Y)$ が成り立つ。決定木は、分割のたびに「クラスとの相互情報量が最大になる質問」を選んでいくことになる。

*3 一般に $(1 - 1/n)^n$ は n を大きくすると $1/e = 0.368$ に近づく。

えた多数のほうが通る。逆に木どうしが似ていれば、同じ作品でそろって間違えるので、何本集めても多数決の意味がない。だから手順1と2で、わざと木を違うものになっている。

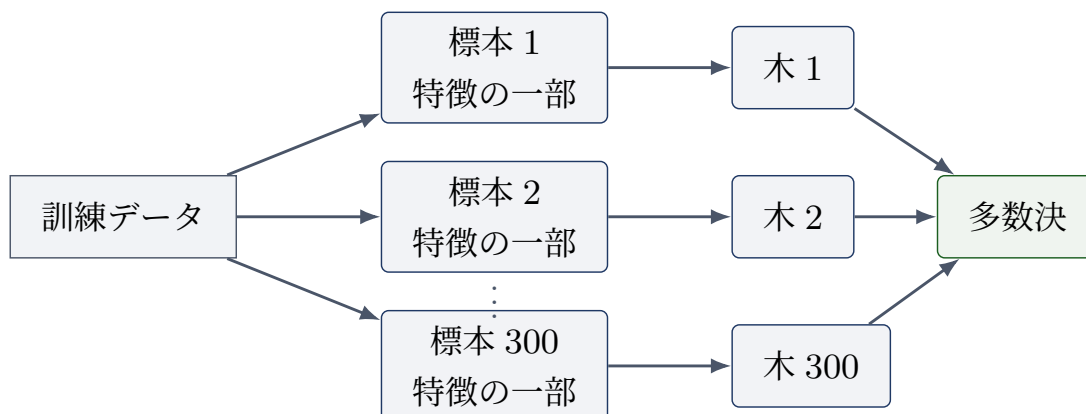


図 7.8 ランダムフォレスト。似ていない木を多数作って多数決する。

代償として、なぜその判定になったかが説明しにくくなる。300本の木の多数決を人間が追うことはできない。精度と説明可能性のトレードオフは、機械学習のあちこちに現れる。

7.7 勾配ブースティング — 間違いを順に埋める

ランダムフォレストは木を並列に作って多数決を取った。これをバギング (bagging) という。

もう一つの方法がある。木を逐次的に作り、前の木が間違えた分だけを次の木に学ばせる。これがブースティング (boosting) である。

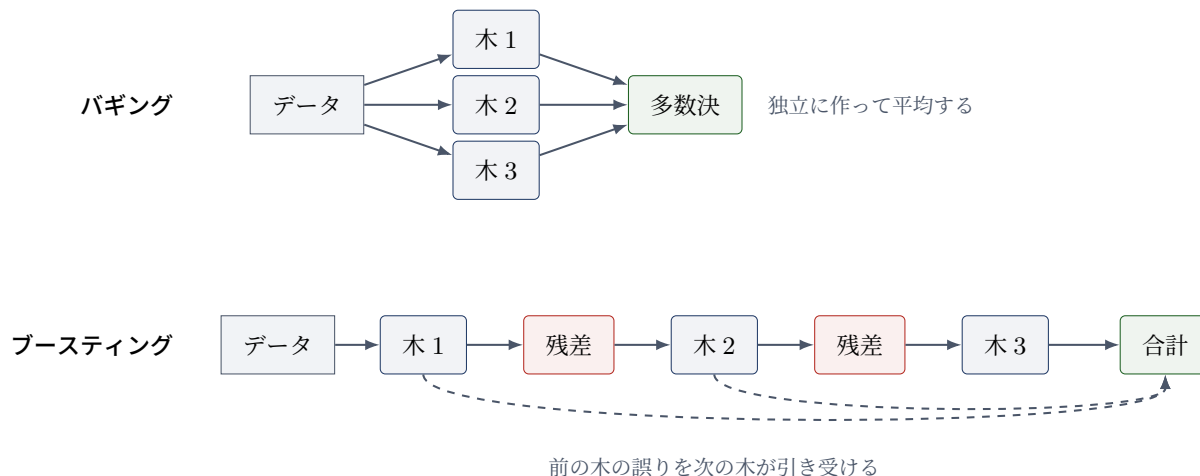


図 7.9 バギングとブースティングの違い。前者は並列、後者は逐次。

7.7.1 残差を埋める過程を見る

分類より回帰のほうが直感的なので、1次元のデータでブースティングの動きを見る。

真の関数は $f(x) = \sin x + 0.3x$ で、そこにノイズを加えたものが観測値である。弱い学習器とし

て深さ 1 の決定木 (決定株) だけを使う。1 本では「ある値を境に 2 つの水準に分ける」ことしかできない。

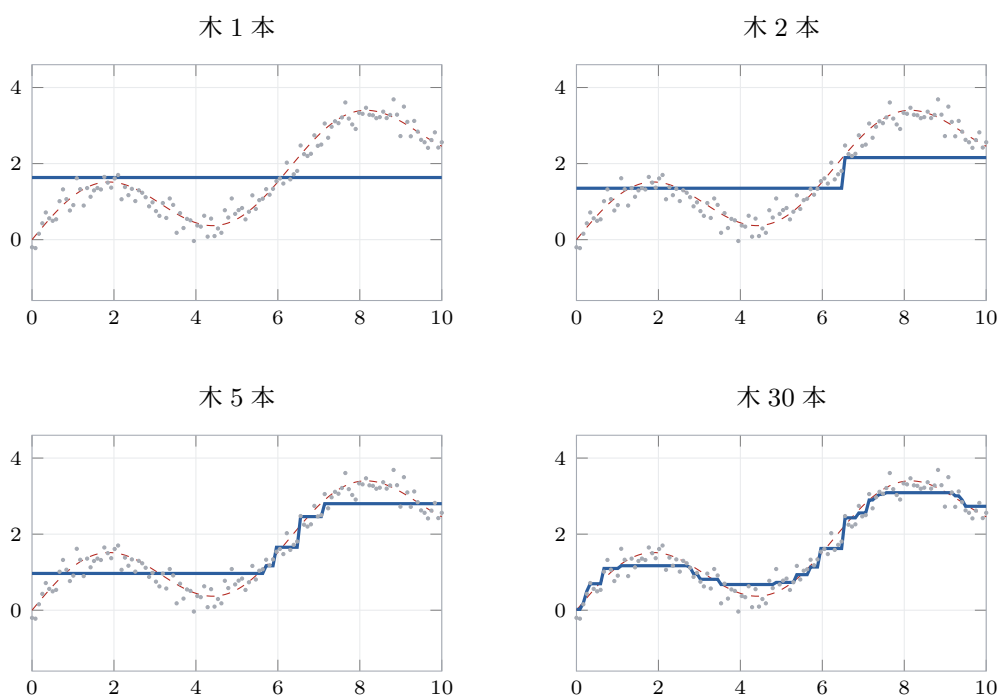


図 7.10 勾配ブースティングの過程。灰色は観測値、赤破線が真の関数、青が予測。決定株を足していくと階段が細くなり、曲線に近づく。

木 1 本のときは、ほとんど平らな直線である。2 本、5 本と足していくと段差が増え、30 本では曲線をよく追っている。

7.7.2 各木が学ぶのは直前までの残差

m 本目の木が学ぶのは、それまでの予測の残差である。

$$r_i^{(m)} = y_i - F_{m-1}(x_i) \quad (7.9)$$

新しい木 h_m を残差に当てはめ、学習率 η を掛けて足し込む。

$$F_m(x) = F_{m-1}(x) + \eta h_m(x) \quad (7.10)$$

図 7.11 に、残差そのものと、それに当てはめた木を示す。

2 本目のときはまだ大きな構造が残っている。30 本目では残差がほぼ平らになり、木が拾えるものが少なくなっている。

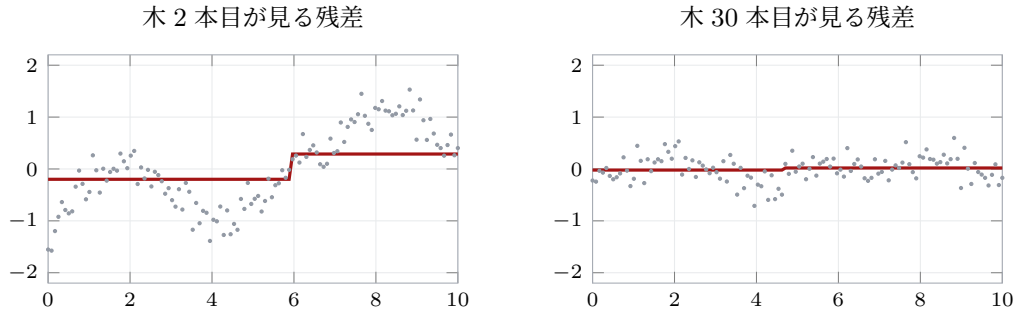


図 7.11 残差 (灰色の点) と、それに当てはめた決定株 (赤)。回を追うごとに残差が小さくなり、木の刻み幅も細くなる。

補足 残差は損失関数の負の勾配である — 「勾配」の由来

2 乗誤差 $L = \frac{1}{2} \sum_i (y_i - F(x_i))^2$ を考える。 $F(x_i)$ で微分すると

$$-\frac{\partial L}{\partial F(x_i)} = y_i - F(x_i) = r_i \quad (7.11)$$

残差はちょうど損失関数の負の勾配である。式 (7.10) は、関数の空間での勾配降下法になっている。

損失関数を変えれば、残差の代わりに別の量を当てはめることになる。分類問題ではロジスティック損失を使い、「確率のずれ」に相当する量を学ぶ。これが勾配ブースティング (gradient boosting) という名前の由来である。

7.7.3 学習率と木の本数

式 (7.10) の η は学習率 (learning rate) である。1 に近いと一気に残差を消し、小さいとゆっくり進む。

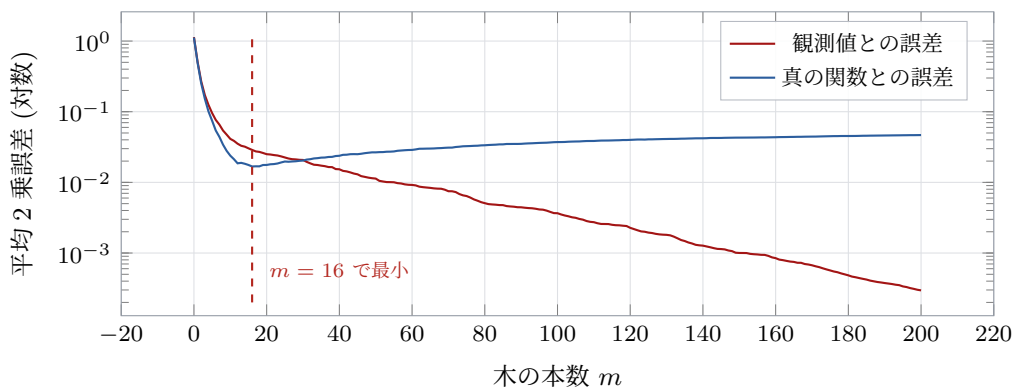


図 7.12 木を足すほど観測値との誤差は下がり続けるが、真の関数との誤差は $m = 16$ で最小になり、その先はむしろ悪化していく。

図 7.12 が重要である。縦軸は、この 1 次元の人工データに決定株を足していったときの平均 2 乗誤差である。観測値との平均 2 乗誤差 (赤) は木を足すほど下がり続け、200 本ではほぼゼロになる。だが真の関数との平均 2 乗誤差 (青) は $m = 16$ のときの 0.017 が最小で、その先は木を足すほど悪化

し、200 本では 0.047 になる。最小のときより 0.030 大きい。

底を過ぎてから起きているのは**ノイズの学習**、つまり過学習である。観測値にどこまでも近づくということは、観測値に混ざったノイズにもどこまでも近づくということだからである。

— × アンチパターン 7.3 — ブースティングの木をランダムフォレストと同じつもりで増やす —

ランダムフォレストは木を独立に作って多数決するので、本数を増やしても害はなく、増えるのは計算時間だけである。ブースティングは違う。前の木の誤りを次の木が引き受ける仕組みなので、足せば訓練誤差をいくらでも下げられ、**本数を増やすほど過学習する**。同じ「木をたくさん作る手法」でも、本数の意味が正反対である。

η と本数はトレードオフの関係にある。 η を小さくすると 1 本あたりの寄与が減るので、同じ性能に達するのに多くの木が要る。一般に「 η を小さく、本数を多く」のほうがテスト誤差は小さくなるが、計算時間は増える。

本数を止める位置をどう決めるかも問題になる。図 7.12 のように真の関数が分かっている底が見えるが、実際のデータでは見えない。そこで訓練データの一部を取り分けて成績を測り、**それが悪化し始めたところで打ち切る**。これを早期打ち切り (early stopping) といい、LightGBM にも XGBoost にも仕組みが用意されている。第 10.2.2 項で見る深層モデルの止め方と同じ考え方である。

本書は本数を 300 本に固定して使った。打ち切りのために訓練データをさらに削ると、74 点では学習側が痩せすぎるためである。

7.7.4 LightGBM

勾配ブースティングの実装として広く使われているのが **LightGBM**(Microsoft, 2017)[48] と **XGBoost**(2016)[47] である。Kaggle^{*4} などの競技では、表形式データにおける定番になっている。LightGBM の工夫はいくつかある。

- **ヒストグラム化** — 連続値を 256 程度のビンに離散化し、分割点の探索を高速にする
- **Leaf-wise 成長** — 木を深さごとに揃えて成長させるのではなく、損失が最も減る葉から伸ばす。同じ葉数でより深い木になる
- **GOSS**(gradient-based one-side sampling) — 勾配の大きい行 (まだ誤差の大きいもの) を優先的に使い、小さい行は間引いて学習を速くする

7.7.5 本書のデータでは、調整が要る設定は 2 つだった

勾配ブースティングは調整できる値が多い。LightGBM の一覧には 100 を超える項目が並ぶ。だが役割は 3 つに分けられる。

- **どれだけの刻みで、何歩進むか** — 学習率と木の本数。この 2 つは対で決まる (第 7.7.3 項)。

^{*4} **Kaggle** は機械学習の競技会を運営するプラットフォーム (2010 年開始、2017 年に Google が買収)。主催者がデータと評価指標を公開し、参加者が予測を提出して順位を競う。表形式データの部門では上位解法の多くが勾配ブースティングを使っており、「表形式ならまず LightGBM か XGBoost」という通念はここから広まった。ただし競技のデータは数万行から数百万行あることが多く、本書の 74 点とは条件が違う。

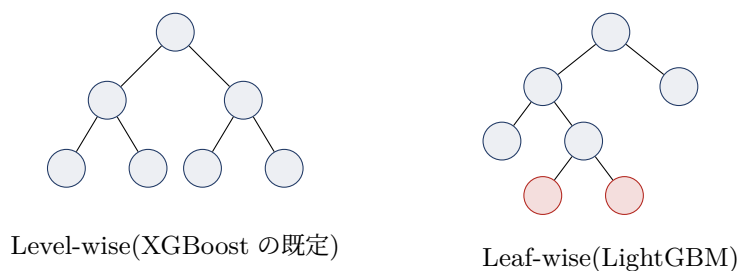


図 7.13 木の成長のさせ方。Leaf-wise は損失が最も減る葉だけを伸ばす。

- **1 本の木をどこまで細かくするか** — 葉の数 (または深さ) と、葉に残す最小データ数。細かくするほど 1 本が強くなり、過学習しやすくなる。
- **どれだけ手加減させるか** — 正則化の強さ、1 本の木に使う行や列の割合。わざと情報を減らして過学習を抑える。

どれをどれだけ触ると成績が変わるのかを、本書のデータで測った。既定値から 1 つのパラメータだけを変え、他は既定値のままにしてある。

表 7.2 既定値から 1 つだけ設定を変えたとき、74 作品の著者を層別 5 分割の交差検証で判別した正解率が動いた幅 (試した値の中での最大と最小の差)。正解率そのものではない。右の 2 列は、正解率が最も高くなったパラメータの値と、そのときの正解率である。いずれも分割の乱数 3 通りで平均した。すべて既定値のままの正解率は、乱数 5 通りの平均で 0.943 である。

パラメータ	正解率の幅	最良の値	そのときの正解率
葉の最小データ数	0.134	10	0.950
学習率	0.117	0.3	0.937
ℓ_2 正則化の強さ	0.032	0.0	0.933
木の本数	0.009	200	0.937
1 本に使う列の割合	0.004	0.5	0.937
1 本の木の本数	0.000	4	0.933

上の 2 つと下の 4 つで、幅の桁が違う。**葉の最小データ数**を 1 から 20 まで動かしたときの正解率の幅は 0.134、**学習率**を 0.01 から 0.3 まで動かしたときの幅は 0.117 ある。残りの 4 つは、 ℓ_2 正則化の強さが 0.032、木の本数が 0.009、1 本に使う列の割合が 0.004、1 本の本数の葉の数が 0.000 で、ほとんど変わらない。葉の数の幅が 0.000 というのは、4 枚に制限しても 31 枚許しても正解率が同じ値だったということである。74 点しかないのだから、31 枚まで許しても実際にはそこまで分かれないうためである。

葉の最小データ数が大きく響くのも同じ理由による。既定値の 20 を 1 まで下げると、1 点だけの葉が作れるようになる。74 点に対してそれは細かすぎて、木がデータを覚えてしまう。**標本が少ないほど、1 本の本を粗くしておくほうがよい。**

学習率と木の本数は、片方を下げたらもう片方を上げる関係にある。

正解率は、学習率 0.02・木 25 本では 0.680 しかない。同じ学習率のまま木を 400 本に増やすと 0.937 になる。0.257 の差である。学習率 0.3 なら、木 25 本の時点ですでに 0.937 である。行き着く

表 7.3 学習率と木の本数の組み合わせごとの正解率 (74 作品、層別 5 分割の交差検証、分割の乱数 3 通りで平均)。学習率を下げたぶんだけ本数を増やせば同じところに行き着く。

学習率	25 本	50 本	100 本	200 本	400 本	800 本
0.3	0.937	0.937	0.937	0.937	0.937	0.937
0.1	0.923	0.928	0.933	0.937	0.937	0.937
0.05	0.855	0.914	0.932	0.933	0.937	0.937
0.02	0.680	0.820	0.901	0.919	0.937	0.937

先は同じで、そこに何本で着くかが違う。一般には学習率を小さく、本数を多くしたほうがわずかに良くなるとされるが、その差は計算時間の増加に見合わないことも多い。

○ 定石 7.1 — ブースティングのパラメータは 3 手に分けて触る

やみくもに全部を同時に振らない。順序を決めておくと探索が短くて済む。

1. 学習率を 0.05 から 0.1 に固定し、木の本数は早期打ち切りで決める。ここで大枠の成績が決まる。
2. 1 本の木の粗さを決める。葉の最小データ数から先に触る。標本が少ないなら大きめにする。
3. 過学習が残っていれば正則化と列のサンプリングを足す。時間が許せば最後に学習率を下げ、本数を増やす。

木の深さや葉の数は、標本が多いときにだけ意味を持つ。74 点のような規模では、そこを刻んでも結果は変わらない。

探索して出た数字を成績として報告しない 74 作品の層別 5 分割の交差検証で、15 通りの設定をランダムに試して最も正解率の高いものを選ぶ。選ばれた設定の正解率は 0.973 で、すべて既定値のままの 0.943 との差は +0.030 である。だがこの 0.973 は、設定を選ぶのに使ったのと同じ分割で測った値である。探索そのものを内側に入れた入れ子交差検証で測り直すと正解率は 0.959 になり、既定値との差は +0.016 まで縮む。見えていた上積みの半分は、選び方が分割に合わせただけの分であった。この測り方の詳細は第 9.7 節で扱う。

7.7.6 本書のデータでは

3 つを同じ枠組みで比べる。成績として測るのは、74 作品の著者を当てた正解率である。74 作品を 5 つに分け、4 つで学習して残り 1 つで試すことを 5 回繰り返して平均した (交差検証。詳しくは第 8 章で扱う)。以下、表の中では交差検証 (cross-validation) を CV と略す^{*5}。

5 つに分けるととき、どの塊にも各著者が元の比率どおり入るようにしている (層別。理由は第 7.9 節で述べる)。以下、単に「層別 CV」と書いたものはこの分け方である。

右側の「シリーズを分けた CV」は、『三国志』のように同じ長編から作った作品を訓練とテストに分けずにまとめて片側へ置いた場合の成績である (分け方が要る理由は本章の最後で述べる)。

^{*5} 第 2 章で変動係数を CV と書いたが、これとは無関係の同じ略語である。

表 7.4 木ベースの手法の比較。74 作品の著者を判別したときの正解率で、左は層別、中央は『三国志』のようなシリーズを 1 つの群として訓練とテストに分けない、いずれも 5 分割の交差検証である。右はその落差 (層別 - シリーズ) である。いずれも分割の乱数は 1 通りである。

モデル	通常の層別 CV	シリーズを分けた CV	落差
決定木 (1 本)	0.688	0.711	-0.023
ランダムフォレスト	0.987	0.975	+0.011
勾配ブースティング (sklearn)	0.812	0.772	+0.041
LightGBM	0.839	0.913	-0.074

ランダムフォレストが最も強い。層別 CV の正解率は 0.987 で、LightGBM の 0.839 を 0.148、勾配ブースティング (sklearn) の 0.812 を 0.175 上回る。ブースティングは Kaggle の定番だが、ここでは負けている。

ただし 0.148 の差のすべてを手法の差と読むことはできない。表の 0.839 は葉 7 枚・学習率 0.05 という控えめな設定での値で、表 7.3 のとおり、学習率と本数を変えるだけで 0.937 までは上がる。それでも 0.987 には及ばない。残る差は標本数の少なさで説明がつく。手元には 74 点しかなく、ブースティングは逐次的に誤差を詰めていく手法なので、データが少ないとノイズを追いかけやすい。少数データでは、ランダムフォレストのほうが安定しやすい。

最右列は、層別 CV の正解率からシリーズを分けた CV の正解率を引いた落差である。52 列では、落差が正になった 2 つのうち大きいほうが勾配ブースティング (sklearn) の +0.041 で、ランダムフォレストは +0.011 にとどまる。決定木の -0.023 と LightGBM の -0.074 は負で、分け方を厳しくしたほうがかえって正解率が高い。落差そのものが小さいので、この列で優劣までは論じられない。なぜ落差が小さいのか、そして特徴を変えると何が起きるのかは、第 9 章で扱う。

○ 定石 7.2 — 分類手法はデータの規模と目的で選ぶ —

- 標本数が数千以下 — ランダムフォレストから試す
- 標本数が数万以上、表形式 — LightGBM や XGBoost
- 説明が必要 — 決定木 1 本、線形モデル
- とにかく速く試したい — ロジスティック回帰

「最新の手法が常に最良」ということはない。データの規模と性質で決まる。

7.8 訓練データで測った正解率は当てにならない

ここまでの数字はすべて交差検証で測ってきた。訓練データでの成績を使うとどれだけ話が変わるかを、本章の最も重要な実験として確かめる。

決定木を全データで学習し、**同じデータで正解率を測ってみる**。

```
tree = DecisionTreeClassifier(random_state=0).fit(X, y)
print(tree.score(X, y))    # 1.000
```

正解率 1.000。完璧である。

だがこれは何も意味しない。決定木は、必要なだけ深くすれば訓練データを丸暗記できる。答えを見ながら答え合わせをしているのと同じである。

データを 7:3 に分けてやり直す。

表 7.5 同じ決定木の正解率を、測り方を変えて出したもの。上は 74 作品すべてで学習して同じ 74 作品で測った値、下は 7:3 に分けて訓練側で学習しテスト側で測った値

測り方	正解率
学習に使ったデータで測る	1.000
学習に使っていないデータで測る	0.783

同じモデル、同じデータなのに、測り方を変えただけで正解率に 0.217 の差がついた。この差が**過学習** (overfitting) である。

差の中身は、モデルが覚え込んだ**その 74 作品に固有のこと**である。「太宰治は『て、』が多い」なら未知の作品にも通じるが、「3 番目の作品は『や、』がちょうど 0.012 である」は、その作品にしか通じない。訓練データで測れば後者も正解を生むので、成績が 1.000 まで上がる。別のデータに当てれば、後者は何の役にも立たない。

分類ではなく、点に線を当てはめる問題で描くと形が見える。

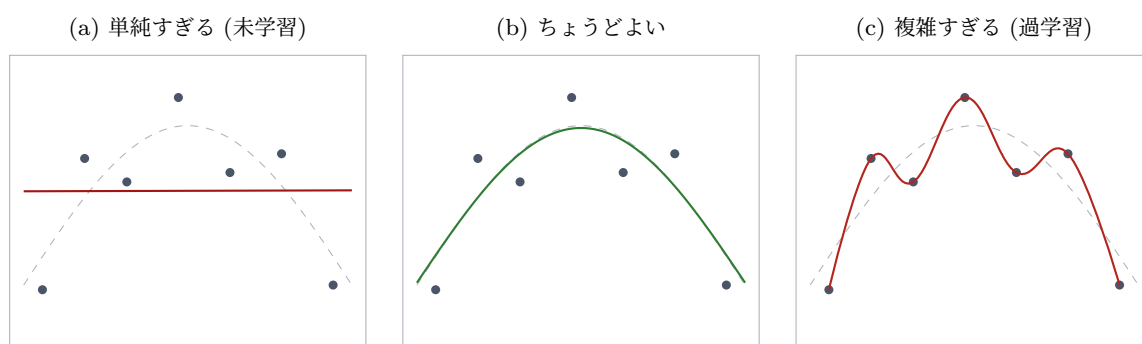


図 7.14 同じ 7 点に 3 通りの当てはめ方をした。灰色の破線が真の関係、黒い点が観測されたデータである。左は直線しか引けないので、どう頑張っても点の傾向を追えない。右は 7 点すべてをぴったり通るが、点と点のあいだで大きく暴れており、真の関係から離れている。訓練データでの誤差は右がいちばん小さい。それでも別のデータで測れば右がいちばん外す。

図 7.14 の右が、いま決定木で起きていることである。「訓練データでの誤差が最も小さいモデル」を選ぶと、このいちばん暴れた曲線が選ばれてしまう。

補足 「過学習」という訳語は誤解を招きやすい

overfit の over は「学びすぎ」ではなく「**訓練データに合わせすぎ**」を指す。過剰適合という訳のほうが原語に忠実だ、という指摘がある。実際、起きているのは適合のしすぎであって、学習の回数や時間が多すぎたわけではない。

補足（続き）

本書は慣用に従って過学習と呼ぶが、言葉に引きずられて「学習を減らせばよい」と読むと的を外す。減らすのはモデルが持つ自由さのほうである。

× アンチパターン 7.4 — 訓練データで性能を報告する

気づきにくく、発表まで残りやすい間違いである。

1.000 に近い正解率を見たら、まず評価に使ったデータが学習と分かれているかを確認する。課題が易しければ高い正解率は正当に出るので、問題は数字の高さそのものではなく、どのデータで測ったかにある。

7.9 分割のときの注意 — 層別化

データを分けるとき、単純にランダムに切ると壊れる場合がある。

柳田国男の作品は5点しかない。ランダムに7:3で分けると、テスト側に1点も入らないことがある。あるいは訓練側に1点しか入らないこともある。

層別化 (stratification) は、分割後もクラスの比率が元と同じになるように分ける。

```
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.3, stratify=y, random_state=0
)
```

stratify=y を書くかどうかだけの違いだが、クラスが偏っているデータでは必須である。

7.10 モデルを比べる

代表的な分類器を同じ条件で比べたのが表 7.6 である。測り方は次章で説明する交差検証を使っている。

表 7.6 分類器の比較。74 作品の著者を 5 分割の層別交差検証で判別したときの正解率で、平均と標準偏差は 5 つのフォールドにわたるもの

モデル	平均正解率	標準偏差
最頻クラス (基準線)	0.284	0.026
k 近傍法 ($k = 1$)	0.973	0.033
k 近傍法 ($k = 5$)	0.877	0.055
決定木	0.688	0.106
ロジスティック回帰	0.986	0.029
ランダムフォレスト	0.987	0.027

ランダムフォレストの平均正解率が 0.987 である。最頻クラスを答え続ける基準線 0.284 を 0.703 上回る。文字の使い方だけで著者がほぼ完全に判別できていることになる。

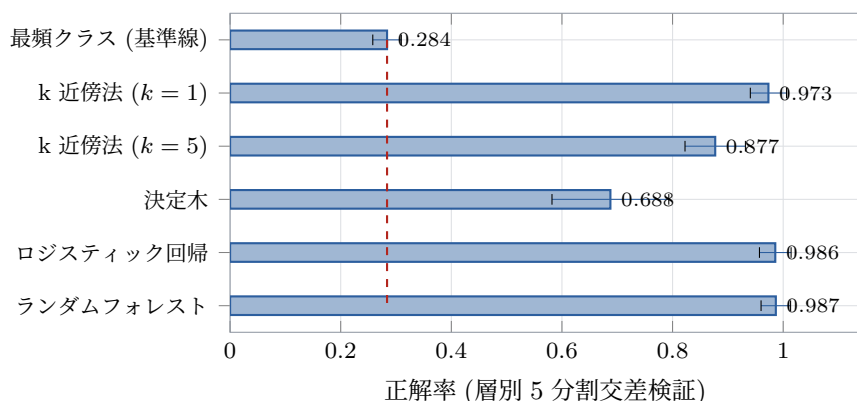


図 7.15 表 7.6 を横棒にしたもの。横向きの細い線は標準偏差、破線は基準線 (最頻クラス) の高さ。上位 3 つは棒の長さも標準偏差もほぼ同じで、決定木だけ標準偏差が 3 倍以上ある

図 7.15 を見ると、上位 3 つ (k 近傍法 $k = 1$ 、ロジスティック回帰、ランダムフォレスト) は棒の長さがほとんど変わらない。太字にした 0.987 を「1 位」として読みたくなるが、標準偏差の細い線が互いに深く重なっている。この重なるの読み方は次章で扱う。

補足 交差検証は標準偏差も見る

平均だけでは比べたことにならない。5 つのフォールドにわたる正解率の標準偏差を見ると、決定木の 0.106 は、上位 3 つのモデル (0.027 から 0.033) の 3 倍以上あり、2 番目に不安定な k 近傍法 ($k = 5$) の 0.055 と比べても 2 倍近い。

これは「分割の仕方によって成績が大きく変わる」という意味で、モデルが不安定であることの兆候になる。ただし、5 つの値から出した標準偏差はそれ自体ぶれの大きい推定値である (第 8 章)。順位を決める根拠にはせず、不安定さの兆候として読む。

7.10.1 設定値はほとんど探索していない

表 7.6 には、書かれていない選択がいくつも隠れている。 k NN の k をいくつにするか、木をどこまで深く育てるか、森を何本にするか。これらは学習で決まらず、人が外から与える値である (第 8 章でハイパーパラメータとして扱う)。

本書がどうしたかを表 7.7 に示す。

つまりほとんど探索していない。これは手抜きでもあり、意図でもある。

意図の側から言うと、探索を入れると比較の意味が変わる。「よく調整された手法 A」と「既定値の手法 B」を比べても、分かるのは調整の手間の差でしかない。本書が見たいのは手法の性格の違いなので、どれも同じだけ手をかけない、という揃え方を選んだ。

手抜きの側も認めておく。表 7.6 の順位は、探索すれば入れ替わりうる。とくに SVM は C と γ で境界の複雑さが決まる手法なので、既定値のままの成績を「SVM の実力」と読むわけにはいかない。

表 7.7 本書が使った主な設定値。特記のないものはライブラリの既定値

手法	設定	備考
k 近傍法	$k = 1$ 、 $k = 5$ 、 $k = 15$	k を変えたときの挙動そのものを見たいので、複数を併記する
決定木	深さ制限なし、図では深さ 4	制限なしだと必ず訓練データを丸暗記する (第 7.8 節)
ランダムフォレスト	木 300 本、他は既定値	本数は多いほどよく、増やしても害がない (第 7.7.3 項)
ロジスティック回帰	既定の正則化のまま	標準化してから使う
SVM	RBF カーネル、既定値	C と γ は探索していない
LightGBM	木 300 本、学習率 0.05、葉 7 枚	小さいデータ向けに葉を絞った
MLP(第 10.2 節)	幅 を 4 通 り、 <code>max_iter=3000</code> 、他 は既定値	止め方は既定 (訓練損失の停滞。検証データは使わない)

○ 定石 7.3 — 設定値を探索したかどうかをはっきり書く —

探索していないなら「既定値のまま」と書く。探索したなら、何を、どの範囲で、どう選んだかを書く。

書いていない報告を読むときは、探索したものとして扱うほうが安全である。そして探索したなら、その探索も評価の内側に入れる。選ぶために使ったデータで成績も測ると、数字が甘くなる。正しい手順は入れ子の交差検証で、第 9 章で実際に測る。

7.10.2 境界の形を見る

表 7.6 の数字は、どの手法も似た成績に見せてしまう。だが同じ成績でも、引いている境界の形は違う。

52 次元の境界は描けないので、2 列に絞る。吉川英治・夏目漱石・泉鏡花の 40 作品を、読点前の「は」と「ら」の 2 列だけで分ける (全 52 列から 2 列を選べば、3 著者はほぼ分かれる)。両方とも標準化してあるので、目盛りは標準偏差を単位とする。

図 7.16 を見比べてほしい。

- $k = 1$ の **k 近傍法**は、点のまわりを囲むように境界が入り組む。1 点でも飛び地があれば、そこに小さな島ができる
- $k = 15$ にすると島が消え、なめらかな 3 分割になる。 k は境界の細かさを決めるパラメータである
- **決定木**の境界は必ず軸に平行である。「は > 0.3 か」という質問しかできないので、斜めの境界を階段で近似するしかない

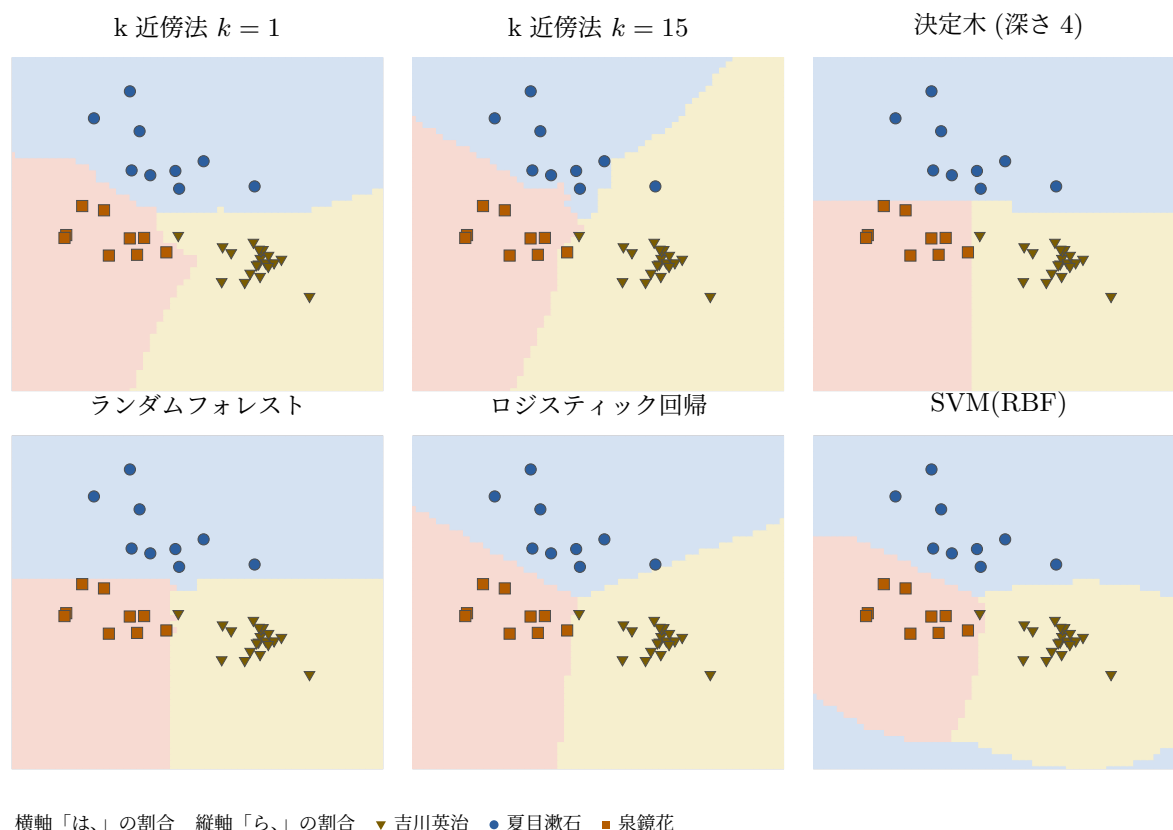


図 7.16 同じ 40 点に 6 つの手法で境界を引いた。背景の色が、その位置に来た作品をどの著者と答えるかを表す (黄が吉川英治、青が夏目漱石、赤が泉鏡花の領域)。この 2 列だけを使った層別 5 分割 CV の正解率はどれも 0.900 から 0.975 の間で大差ないが、境界の形はまるで違う。

- ランダムフォレストも階段だが、300 本の本の多数決なので刻みが細くなり、角が丸く見える
- ロジスティック回帰は直線だけで区切る。3 クラスなので、3 本の直線が中央で交わる形になる
- SVM(RBF) はなめらかな曲線で囲む。カーネルが曲がった境界を許すためである

この 40 作品を 2 列だけで判別したときの層別 5 分割 CV の正解率は、 $k=1$ の k 近傍法で 0.950、決定木で 0.925、ランダムフォレストと SVM で 0.975 である。数字は近いのに、モデルが世界をどう切っているかは違う。未知の点がどこに来るかによって、どの手法が当てるかも変わる。

○ 定石 7.4 — 成績が並んだときは境界の形が誤りの型で選ぶ —

交差検証の平均が 0.01 しか変わらない 2 つの手法は、その差では選べない (第 8 章)。決め手になるのは次のような性質である。

- 境界の形が、その領域で期待される形に近い (軸に平行な境界が自然な問題もあれば、なめらかな境界が自然な問題もある)
- 誤りの型が許せるか (混同行列を見る。第 8 章)
- 説明できるか、速いか、新しいデータに追従できるか

図 7.16 のような図は、2 列に絞らないと描けない。それでも、手法の癖を一度目で見しておく

と、数字だけを見ていたときより判断が速くなる。

7.11 判別を支えている特徴を突き止める

ここまでで正解率は出た。次に問うべきは、何を根拠に当てているのかである。確かめ方を2つ見る。特徴を群ごと抜き差しする方法と、1列ずつ壊してみる方法である。

7.11.1 特徴群の抜き差し (アブレーション)

第3章で作った3種類の特徴を、単独あるいは組み合わせでランダムフォレストに与え、74作品の著者を層別5分割の交差検証で判別したときの正解率を測る。

表 7.8 特徴群ごとの正解率 (74 作品、ランダムフォレスト、層別 5 分割の交差検証、分割の乱数は 1 通り)

使った列	列数	正解率
字種の比率だけ	3	0.419
文長 μ, σ だけ	2	0.569
字種 + 文長	5	0.635
読点前平仮名だけ	47	0.917
すべて	52	0.987

基準線 (最頻クラスを答え続ける) は 0.284 である。表 7.8 を順に読み取っていく。

- 読点前平仮名の 47 列だけで正解率 0.917 に達する。第3章で「本書で最も判別力の高い特徴」と予告したことの実証である。読点をどこで打つかという癖だけで、8人の著者はほぼ判別できてしまう
- 字種 3 列だけでも正解率 0.419 ある。基準線 0.284 を 0.135 上回るなので、無力ではない。図 3.7 で見た著者ごとのまとまりが、そのまま成績になっている
- 52 列を全部合わせた正解率 0.987 は、読点単独の 0.917 を 0.070 上回る。弱い特徴群も、強い特徴群が間違える作品を拾う形で寄与している。第8章の混同行列 (図 8.4) で見る「漱石と藤村を分けるのは読点の癖」という逆算と、表裏の関係にある

7.11.2 置換重要度 — 1列ずつ壊す

群ではなく個々の列の寄与を見るには、置換重要度 (permutation importance) を使う。テストデータの 1 列だけをシャッフルして正解率がどれだけ落ちるかを測る。その列が使われていなければ、壊しても成績は落ちないはずである。

図 7.17 で目を引くのは、上位に入っていない文字のほうである。頻度最大の「て、」は全読点の 18% を占めるのに、置換重要度の順位は 52 列中 26 位である。一方、2 位の「っ、」は第3章のゼロの話で出てきた、74 作品中 42 作品では一度も現れない列である。

理由を考えると腑に落ちる。「て、」は誰もが打つので、差がつきにくい。「っ、」は「あっ、と叫んだ」のような書き方を**する作家としない作家に割れる**ので、出現の有無そのものが署名になる。よく

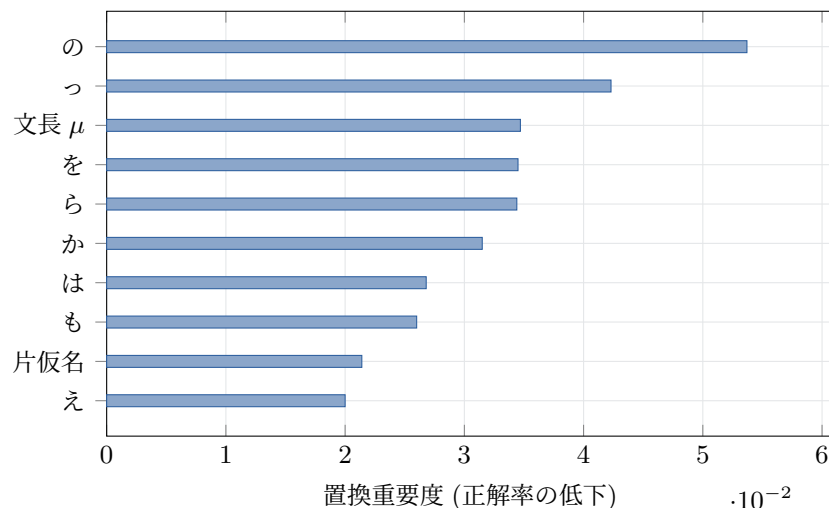


図 7.17 置換重要度の上位 10 列。値は、その列をシャッフルしたときにテスト側の正解率が下がった量である。ランダムフォレストを 74 作品の層別 5 分割の交差検証で学習し、各フォールドのテスト側で 20 回置換した平均を、さらに 5 つのフォールドで平均した。1 位は「の、」の 0.0537、2 位は「っ、」の 0.0423 である。

出るものより、人によって使い方が割れるものが判別の決め手になる。第 2 章で「ポアソン分布に従う機能語を選べ」と言ったのは題材への依存を避けるためだったが、その中でどれが寄与するかは、著者間のばらつきで決まる。

1 位の「の、」は、その中間にあたる。「て、」ほど誰もが打つわけではなく、「っ、」ほど使う人と使わない人に割れるわけでもない。2 つの中間で、そこそこ出るうえに作家ごとに割合が違う。判別に最も寄与するのは、**頻度と個人差が両方そろった列**である。

× アンチパターン 7.5 — 不純度重要度だけを見る —

ランダムフォレストが標準で出す重要度 (`feature_importances_`) は、訓練中の分割でどれだけ不純度を下げたかの集計であり、**訓練データで測った量**である。実際、本書のデータで比べると、不純度重要度の上位には「ら」や「は」が来て、置換重要度の順位とは食い違う。過学習した木が訓練データの中でだけ役立てた特徴も、高く出てしまう。重要度は**テストデータ側**で測る。置換重要度をテスト側で計算すれば避けられる。

× アンチパターン 7.6 — 特徴重要度を因果や本質と読む —

置換重要度が示すのは「**このモデルが、このデータでその列に頼っている度合い**」である。「著者性の本質は特定の 1 文字の前の読点にある」という主張ではない。

相関の強い特徴どうしても、重要度が**分け合われる**ことにも注意がいる。片方をシャッフルしても、もう片方が肩代わりするので、どちらの重要度も実際の寄与より低く出る。似た列をまとめてから測る (本節のアブレーションはその一形態である) か、相関構造と合わせて読む。

同じ現象が、本章で見た 3 つの道具すべてに現れる。線形モデルの係数では符号ごとに入れ替わり (第 7.4 節)、L1 正則化では片方だけが残る (すぐ後で見る)、置換重要度では両方が低く出る。道具を変えても、**関連した列のどちらが寄与しているかは分からない**。分からないものを

分けようとせず、グループとして扱うほうが筋が通っている。

L1 が 0 を作るのは係数の動ける範囲が角を持つからである。罰の大きさを λ で調整するかわりに、**罰の値をある大きさ以下に抑える**という言い方に置き換えると見通しがよい。両者は同じことを別の側から書いたものである。すると係数の動ける範囲が、L1 ではひし形、L2 では円になる。

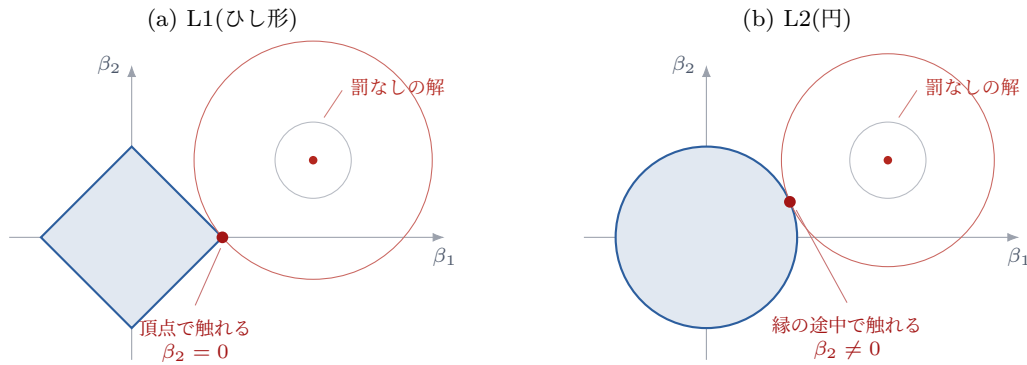


図 7.18 L1 と L2 で解の形が変わる理由。青い領域は罰を受けずに済む範囲、灰色の同心円は予測誤差の等高線で、中心が罰を課さないときの解である。解は等高線が青い領域に最初に触れる点になる。ひし形には尖った頂点があり、そこは軸の上なので係数の 1 つが厳密に 0 になる。円には尖った点がないので、触れる場所は軸から外れる。

図 7.18 のとおり、違いは**角があるかどうか**だけである。ひし形の頂点は軸の上にあるので、そこで触れれば片方の係数が 0 になる。列が増えれば頂点も辺も増えるが、尖っている場所ほど触れやすいという性質は変わらない。L1 がいくつもの係数を同時に 0 にするのは、そのためである。

× アンチパターン 7.7 — L1 が残した列を「重要な列」と読む

変数選択の道具として使えることと、残った列だけが判別に寄与しているということは別である。

強く関連した列が複数あるとき、L1 はそのうち 1 本だけを残して他を 0 にする性質を持つ。どれが残るかは、標本のわずかな違いで入れ替わる。平仮名と漢字のように連動する 2 列があれば、ある分割では平仮名が残り、別の分割では漢字が残る、ということが起きる。

残らなかったほうが無関係なのではない。残ったほうと同じ情報を持っていたから消された。それを「漢字は判別の役に立たない」と読むと、逆の結論になりかねない。

関連した列をまとめて扱いたいときは、罰し方を変える。どれを使うかは、**グループの境目を人が知っているかどうか**で決まる。

- **Elastic Net** — L1 と L2 を混ぜる。グループの指定は要らない。L2 の項が関連した列の係数を一緒に縮めるので、1 本だけ残して他を切る挙動がやわらぐ
- **Group Lasso** — 列をあらかじめグループに分けておき、**グループ単位で丸ごと残すか落とすか**を選ぶ。本書の 52 列なら、字種 3 列・文長 2 列・読点前 47 列という 3 つのグループが自然に決まる。カテゴリ変数をダミー展開した列も、1 つの変数から出たものなのでま

とめて扱いたい

- **Sparse Group Lasso** — グループを選んだうえで、グループの中でも列を絞る

グループの境目が分からないなら Elastic Net、分かっているなら Group Lasso、と考えればよい。

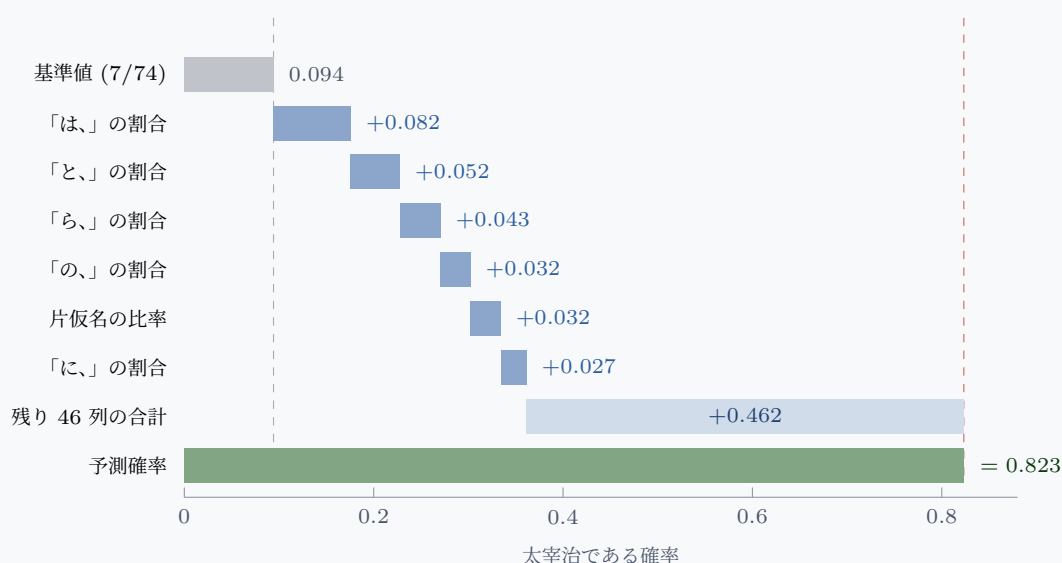
同じ理由で、**L1 が選んだ列の数**も鵜呑みにできない。第 4.4.2 節の正則化パスで最後まで残った 3 列という数字は、「判別に必要な情報が 3 列ぶんある」という意味ではなく、「互いに独立な向きが 3 本ぶん残った」という意味に近い。読点前の 47 列は互いに相関しているので、実際に判別を支えている手がかりの数はもっと多い。

発展 SHAP — 貢献度を公平に配る

置換重要度は「モデルが全体としてどの列に頼っているか」を教えるが、「この作品が太宰治と判定されたのは、どの列のおかげか」には答えない。1 件ごとの判定を特徴に分解する道具が **SHAP** (SHapley Additive exPlanations) である。

原理は協力ゲーム理論の **Shapley 値** (1953 年)[49] である。素朴に言えば次の考え方になる。特徴たちを、共同で予測値を作るプレイヤーとみなす。ある特徴の貢献は「その特徴が加わった瞬間に予測がどれだけ動いたか」。ただしその量は先に誰が入っていたかに依存するので、**参加順のすべての並べ方について平均**する。これが Shapley 値である。Shapley 値の配り方は、「全員の取り分を足すと、予測値と基準値 (平均的な予測) の差に一致する」「同じ働きをする 2 人の取り分は等しい」「何もしない人の取り分は 0」という公平性の要請を (技術的な条件とあわせて) 満たす**唯一**の配り方であることが証明されている。

実例を 1 つ。74 作品すべてで学習したランダムフォレストが『斜陽』を太宰治と判定する確率は 0.823 である。この確率を TreeSHAP で分解すると、基準値 0.094 (モデルの平均予測。ここではほぼ太宰治の事前の割合 7/74 に一致する) の上に、「は、」の SHAP 値 +0.082、「と、」の +0.052、「ら、」の +0.043、……と 52 列分の寄与が積み上がり、**足すとちょうど 0.823 に一致する**。足し算が必ず合うこと (加法性) が SHAP の持ち味で、「どの癖がどれだけ寄与したか」を 1 作品単位で示せる。審査や診断のように、個々の判定に説明責任がある場面で使われる理由である。



発展（続き）

上の図が『斜陽』の分解である。基準値から出発して、各列の取り分を左から積んでいくと、最後の 1 列まで積むと予測確率にぴったり一致する。

問題は計算量で、「参加順のすべての並べ方」は $52! \approx 8 \times 10^{67}$ 通り、連合 (特徴の部分集合) の数でも $2^{52} \approx 4.5 \times 10^{15}$ 通りあって、定義どおりには計算できない。実用になっているのは近似と特化のおかげである。モデルを問わない **KernelSHAP** は、連合をサンプリングして重み付き回帰で Shapley 値を推定する。木のモデルに限れば、木構造を利用して厳密値を多項式時間で計算する **TreeSHAP** がある [50, 51]。上の数字も TreeSHAP による。

注意は置換重要度のときと同じである。SHAP が配っているのは「このモデルがこの予測を作った内訳」であって、著者性の因果ではない。関連した特徴の間でどう配るかにも流儀があり、実装の設定しただけで値は変わる。数字の細部より、「基準値からの差を、公平の公理に従って特徴に配る」という枠組みごと覚えておくのがよい。

なお、ここまでの数字はすべて交差検証で測っている (SHAP の例だけは、判定の説明が目的なので学習済みモデルをそのまま使った)。

手で組み立てた 52 列の代わりに、大規模言語モデルの埋め込みや深層学習のモデルを使うとどうなるか。その比較は、評価の作法と漏れの避け方がそろってからのほうが読みやすいので、第 10 章に置いた。

7.12 同じ長編の巻が訓練とテストに分かれている

74 作品を層別 5 分割の交差検証で判別したときの、ランダムフォレストの正解率が 0.987 である。8 名の著者を、52 個の数値でほぼ完璧に当てている。

だが、素直に喜ぶ前に確かめたいことがある。

第 6 章の図 6.6 で気づいたことを思い出してほしい。吉川英治の作品だけが異様に密集していた。そして第 1 章の表 1.2 によれば、その 21 作品のうち 18 は『三国志』と『宮本武蔵』を巻ごとに分けたものである。

交差検証は作品をランダムに分ける。つまり『三国志』の第 3 巻で訓練し、第 4 巻でテストすることが起きている。

同じ長編の別の巻。登場人物も舞台も語彙もほぼ同じである。モデルは「吉川英治の文体」を学ばなくても、「この長編に固有の癖」を覚えれば当てられてしまう。

これはデータ漏洩 (data leakage) の一種である。第 9 章で、正しく測り直すと数字がどれだけ変わるかを見る。

その前に、次章で「正しく測る」とはどういうことかを整理しておく。

7.13 章末の課題

1. 決定木の深さを 1 に制限して学習し、木を描け。モデルは何を根拠に判定しているか。
2. $k = 1$ の k 近傍法が訓練データでほぼ必ず正解率 1.000 になる理由と、1.000 にならない例外 (どんなデータのときか) を説明せよ。
3. 表 7.6 でロジスティック回帰とランダムフォレストがほぼ同じ成績である。どちらを選ぶべき

か、判断材料を 3 つ挙げよ。

4. `stratify=y` を外して 7:3 分割を 20 回繰り返し、テストデータに含まれる柳田国男の作品数を記録せよ。0 になったのは何回か。
5. 置換重要度を k 近傍法 ($k = 5$ 、標準化つき) で計算し直し、図 7.17(ランダムフォレスト) と上位の順位を比べよ。(ヒント: 重要度は「そのモデルが」使った度合いであり、モデルを替えれば順位も変わりうる。大きく変わる列と変わらない列のどちらが信頼できるかも考えよ。)

正しく測る

8.1 1回の分割では足りない

前章で、74 作品を 7:3 に分けて決定木を学習させ、テストデータでの正解率 0.783 を得た。データの 7 割で学習し、残りの 3 割で成績を測る、というやり方である。このように一度だけ分けて測ることを **ホールドアウト** (hold-out) という。

分け方は乱数で決めた。乱数の引き方が違えば、この 0.783 という正解率も変わりうる。どれだけ変わるのかを確かめる方法がある。**分割の乱数だけを変えて**、同じことを 50 回繰り返す。データもモデルも設定も変えない。変えるのは分け方だけである。

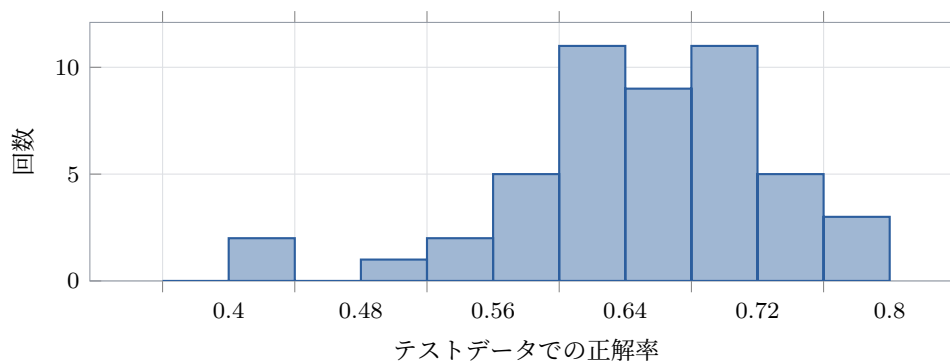


図 8.1 同じ 74 作品・同じ決定木で、7:3 の分割の乱数だけを変えて 50 回測った、テストデータでの正解率の分布。

50 回のテストデータでの正解率は、最小 0.478、最大 0.870、平均 0.690、標準偏差 0.084 であった。

最大と最小の差が 0.39 ある。最悪の引きでは正解率が 0.478 で半分も当たらず、最良の引きでは 0.870 で 9 割近く当たる。どちらも同じモデルを同じデータで測った結果である。

前章の正解率 0.783 が、この 50 回の中でどのあたりに位置するかも確かめておく。50 回の平均 0.690 を 0.093、つまり標準偏差 0.084 の 1 つぶん以上、上回っている。あの数字は、平均よりかなり高めの引きだったことになる。

論文に書かれた「正解率 0.870」が、実は何回も引いた中の当たりくじだったということは起こりうる。そして書いた本人にその自覚がないことも多い。乱数の種を変えて何度か走らせ、いちばん良い数字を残す。悪気なくそうしてしまう。データが小さいほど振れ幅は大きく、本書のデータでは 1 作品の当たり外れで正解率が 0.045 も上下する。「乱数の種を変えたら数字が変わった」という経験がないうちは、1 回の分割を信じてしまいがちである。一度自分で確かめておくとよい。

8.2 交差検証

振れ幅の原因は、テストに回る作品が 22 点しかないことにある。たまたま易しい 22 点を引けば高く、難しい 22 点を引けば低く出る。

かといってテストを増やせば、訓練に回る作品が減る。どちらを増やしても、もう一方が減る。

k 分割交差検証 (*k*-fold cross-validation) は、この板挟みを**全部の作品に 1 回ずつテストを担当させる**ことで抜ける。データを *k* 個の塊に分け、1 つをテスト、残りを訓練にして測る。これをテスト役を入れ替えながら *k* 回繰り返す。*k* 回終われば、どの作品もちょうど 1 回テストに使われたことになる。

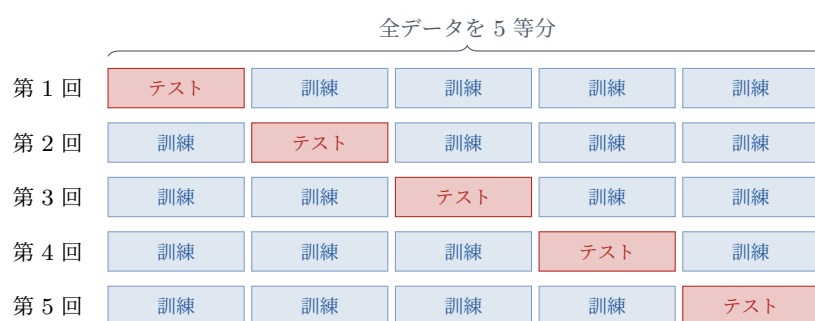


図 8.2 5 分割交差検証。全データが 1 度ずつテストに使われる。

5 回の正解率の平均を報告する。5 回のあいだで数字がどれだけ振れたかも意味を持つので、標準偏差も一緒に出す。

○ 定石 8.1 — 分割数は 5 か 10 から始めてデータが少ないときだけ増やす

$k = 5$ か $k = 10$ が標準的な選び方である。

データが少ないときは k を大きくする。 k を大きくするほど 1 回あたりの訓練データが増えるからである。その代わり学習を k 回やり直すので、計算時間も k に比例して伸びる。

どちらを選んでも、報告するのは平均だけではない。ばらつきの添え方は第 8.3.4 項で扱う。

k を大きくする方向の極端は、データ件数に等しく取ることである。1 件だけをテストにして残り全部で訓練する形になり、これを **1 つ抜き交差検証** (leave-one-out cross-validation、LOOCV) という。訓練データを最大限に使えるので偏りは小さくなるが、学習を件数ぶん繰り返すので計算が重い。それに、 k 個の結果は訓練データをほとんど共有しているので互いに強く関連し、そこから計算したばらつきは信用しにくい。本書は 5 分割を使っている。

8.2.1 層別交差検証

分けるときには、もう一つ気をつけることがある。柳田国男は 5 作品しかないなので、何も考えずに 5 分割すると、柳田の作品が 1 つも入らない回や、逆に 2 つ入る回ができる。前者の回では柳田を当てる機会がなく、後者では 1 件の外れが大きく響く。

そこで分割のあとも各著者の比率が元の表と同じになるように分ける。これを層別化という。StratifiedKFold を使えば自動でそうなる。

74 作品をランダムフォレストで判別し、5 分割交差検証の正解率を層別化ありとなしで比べたのが表 8.1 である。

表 8.1 層別化の有無で 5 分割交差検証の正解率がどう変わるか (74 作品、52 列、ランダムフォレスト)

分割	正解率の平均	最小の回	最大の回
層別なし KFold	0.933	0.867	1.000
層別あり StratifiedKFold	0.987	0.933	1.000

平均の差より、**最小の回**に注目してほしい。層別なしでは 5 回のうち最も悪い回の正解率が 0.867 で、層別ありの最も悪い回の 0.933 を 0.066 下回っている。たまたま少数クラスが偏った分割を引いたためである。

8.3 正解率だけでは足りない

正解率だけを見ることには限界がある。

吉川英治は全体の 28% を占めるのだった。正解率が高くても、**柳田国男の 5 作品を 1 つも当てられていない**かもしれない。5 作品を全部落としても、正解率は $5/74 = 0.07$ しか下がらないからである。全体の数字 1 つでは、少数の著者が丸ごと捨てられていても気づけない。

そこで、著者ごとに成績を出す。

ここで見方を 1 つ変える。「8 名の誰か」を当てる問題として見るのをやめ、「**太宰治か、そうでないか**」という **2 択の問題を、著者ごとに 8 つ立てる**。太宰治についての 2 択、夏目漱石についての 2 択、というふうに 8 回考える。1 つの 2 択に注目すれば、予測と正解の関係は 4 通りしかない。

	その著者と予測	違うと予測
実際にその著者	真陽性 TP	偽陰性 FN (見逃し)
実際は違う著者	偽陽性 FP (誤検出)	真陰性 TN

図 8.3 ある著者 1 名に注目したときの、予測と正解の 4 通り。著者ごとにこの表を 1 枚ずつ作る。

太宰治を例にとる。太宰治の作品を太宰治と当てたものが真陽性 (TP)、太宰治なのに他の著者と答えたものが偽陰性 (FN)、他の著者なのに太宰治と答えたものが偽陽性 (FP)、他の著者を他の著者と答えたものが真陰性 (TN) である。「陽性」は**いま注目している著者だと判定した**という意味で、病気の検査の言い方をそのまま借りている。

この 4 つの数から、3 つの指標を作る。

$$\text{適合率 (precision)} = \frac{TP}{TP + FP} \quad (8.1)$$

$$\text{再現率 (recall)} = \frac{TP}{TP + FN} \quad (8.2)$$

$$F_1 = \frac{2 \cdot \text{適合率} \cdot \text{再現率}}{\text{適合率} + \text{再現率}} \quad (8.3)$$

言葉にすると、

- **適合率 (precision)** — そう予測したもののうち、実際に当たっていた割合。誤検出の代償が大きいときに重視する
- **再現率 (recall)** — 実際にそうだったもののうち、当てられた割合。見逃しの代償が大きいときに重視する
- F_1 — 両者の調和平均。バランスを見る^{*1}

8.3.1 マクロ平均とマイクロ平均

著者ごとに F_1 が 8 個出たので、これを 1 つの数にまとめたい。まとめ方が 2 通りある。

- **マクロ平均 (macro-average)** — 著者ごとの F_1 を 8 個そのまま平均する。21 作品の吉川英治も 5 作品の柳田国男も **同じ 1 票**になる
- **マイクロ平均 (micro-average)** — 著者ごとに分けて、8 名分の TP・FP・FN を全部足してから 1 回だけ計算する。作品数の多い著者の影響が大きい

多クラスでは、マイクロ平均は必ず正解率と一致する。1 件間違えると、間違えた先の著者では偽陽性が 1 つ、本当の著者では偽陰性が 1 つ増える。8 名分を足し合わせると分子は正解した件数、分母は全件数になり、正解率の式と同じものになるからである。

偏りのあるデータでは、2 つの差そのものが情報になる。マクロ平均だけが低ければ、少数の著者を取りこぼしている。

74 作品を 52 列すべてでランダムフォレストに判別させ、層別 5 分割交差検証で著者ごとの適合率・再現率・ F_1 を測った。結果を表 8.2 に示す。

誤りは 74 作品中 1 件だけである。谷崎潤一郎の作品を 1 つ、柳田国男と誤判定している。その 1 件が、谷崎の再現率を $5/6 = 0.833$ に、柳田の適合率を $5/6 = 0.833$ に下げている^{*2}。

^{*1} 算術平均でなく**調和平均**を使う理由は 2 つある。一つは、調和平均が**小さいほうに強く引きずられる**こと。適合率 1.0 でも再現率 0.1 なら $F_1 = 0.18$ にしかならず、片方を犠牲にした「見かけの高得点」を許さない (算術平均なら 0.55 になってしまう)。もう一つは、適合率と再現率が**同じ分子 TP を持つ割合**であること。分子が同じ割合どうしは、調和平均を取ると分母のほうに平均されて $F_1 = 2TP / (2TP + FP + FN)$ というきれいな形にまとまる。誤りの総量 $FP + FN$ を 1 つの分母に束ねた指標、と読める。名前は一般形から来ている。 $F_\beta = (1 + \beta^2) PR / (\beta^2 P + R)$ の β は再現率を適合率の何倍重視するかを表す。添字の 1 は $\beta = 1$ 、つまり両者を等しく重視する場合の意味である。文字 F 自体に意味はない。van Rijsbergen の教科書 *Information Retrieval*(1979) の有効性指標 E に由来する量が、評価会議 MUC-4(1992) で採用された際に、同書の別の関数の記号 F で呼ばれてしまい、そのまま定着したとされる [52, 53]。

^{*2} $73/74 = 0.986$ である。表 8.1 など他の箇所では 0.987 と書いているのは、5 回の分割それぞれで正解率を出してから平均した値である。全体をまとめて数えるか、分割ごとに出してから平均するかで、小数第 3 位が食い違う。

表 8.2 著者ごとの適合率・再現率・ F_1 (74 作品、52 列、ランダムフォレスト、層別 5 分割交差検証)

著者	適合率	再現率	F_1	作品数
吉川英治	1.000	1.000	1.000	21
夏目漱石	1.000	1.000	1.000	10
島崎藤村	1.000	1.000	1.000	9
泉鏡花	1.000	1.000	1.000	9
太宰治	1.000	1.000	1.000	7
夢野久作	1.000	1.000	1.000	7
谷崎潤一郎	1.000	0.833	0.909	6
柳田国男	0.833	1.000	0.909	5
マクロ平均			0.977	
正解率 (マイクロ)			0.986	

作品数の多い 6 名の F_1 が 1.000 なので正解率は 0.986 と高いが、マクロ平均 F_1 は 0.977 にとどまる。少数の著者での取りこぼしは、正解率にはほとんど現れず、マクロ平均 F_1 には現れる。

8.3.2 混同行列 — 誤りの型から足りない特徴が分かる

平均された数値だけでは、どの著者とどの著者を取り違えたかが分からない。混同行列 (confusion matrix) は行が正解、列が予測である。

ただし、そのまま描いても読むものが出てこない。8 名を当てる問題を 52 列すべてでランダムフォレストに解かせると、層別 5 分割交差検証の正解率が 0.9 を超え、行列はほぼ対角線だけになるからである。そこでわざと難しくした設定を右に並べる。特徴を字種 3 列と文長 2 列の計 5 列に間引き、読点の癖という強力な手がかりを封じた状態である。

(b) には模様がある。夏目漱石と島崎藤村が相互に混ざる (漱石 10 作品中 2 作品が藤村に、藤村 9 作品中 3 作品が漱石に化ける)。泉鏡花は 9 作品中 5 作品が吉川英治と誤判定される。字種の比率と文の長さだけで見ると、鏡花の文語調と吉川の講談調は似た位置に来てしまう。全列に戻すとこれらの誤りは消える。つまり漱石と藤村、鏡花と吉川を分けているのは読点の癖だと、誤りの型から逆算できる。

数値の要約より先に混同行列を見る習慣をつけるとよい。「A を B と間違える誤りが多い」というパターンが見えれば、その 2 クラスを分ける特徴が足りないと分かる。これは平均された F_1 からは読み取れない。逆に (a) のような対角行列しか出ないなら、その課題はもう評価の役に立たないほど簡単だということである。

8.3.3 小数第 3 位は偶然の範囲に収まる

ここまで正解率を小数第 3 位まで書いてきた。だがその桁に意味があるかどうかは、別に考えなければならない。

正解率は「74 作品のうち何作品を当てたか」という割合である。割合には、標本の大きさから来る

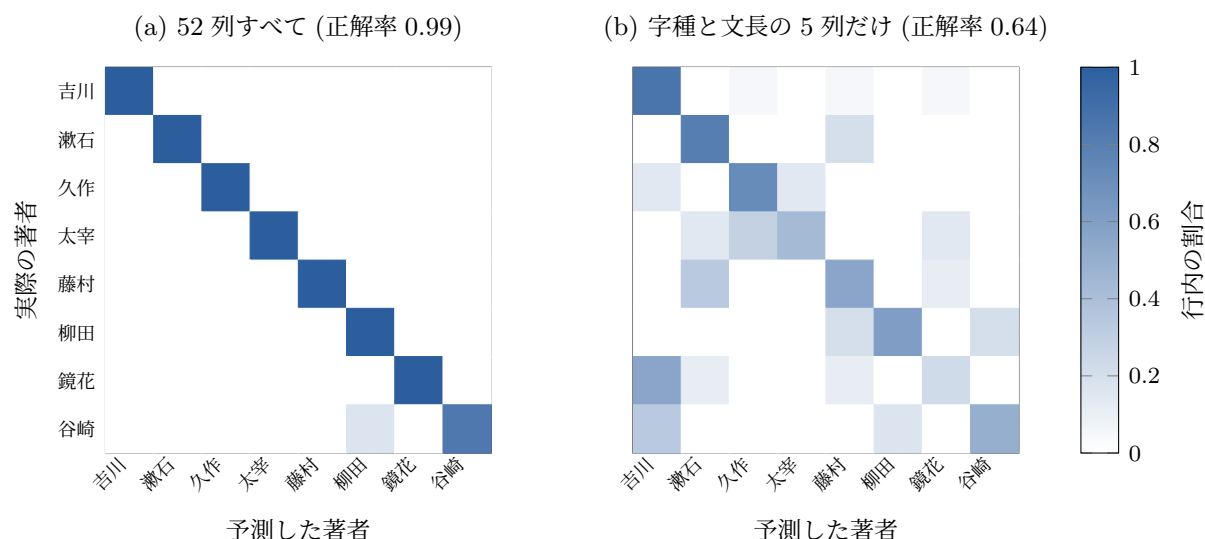


図 8.4 混同行列。行が実際の著者、列が予測した著者で、色は行内の割合。対角線が正解、それ以外が取り違いである。どちらも 74 作品をランダムフォレストで判別した層別 5 分割交差検証の結果で、括弧内は 74 作品全体をまとめて数えた正解率。(a) はほぼ対角線だけで読むものがない。(b) のように課題を難しくすると、誤りの型が現れる。

不確かさがつきまとう。コインを 10 回投げて 7 回表が出ても、そのコインの表の出やすさが 0.7 だとは言えないのと同じである。

74 件中 73 件を当てたときの 95% 信頼区間を計算すると*3、

正解率 0.986 (95% 信頼区間 [0.927, 1.000])

となる。下端は 0.927 である。つまりこのデータから言えるのは「真の正解率が 0.927 を下回るとは考えにくい」までであって、0.986 という数字そのものではない。

補足 「95%」が何にかかっているか

信頼区間の 95% は、**確率**である。ただし、**何の確率か**に注意が要る。

データを取る前の段階を考える。これから取るデータに同じ手続きを当てはめて区間を作ったとき、その区間が真の値を含む確率が 0.95 になるように、区間の作り方が決めている。Neyman はこれを**被覆確率** (coverage probability) と呼んだ。確率が付いているのは**区間を作る手続きのほう**である。真の値は決まっています。動くのは、データとそこから作られる区間である。

同じことは、繰り返しの割合としても書ける。同じ手続きでデータを取り直して区間を作る操作を何度も繰り返せば、そのうち 95% が真の値を含む。「100 回やれば 95 回」という説明はこれを指しており、被覆確率と別のことを言っているわけではない。

一方、いま手元にある**この 1 本**の区間は、すでに 2 つの数値として確定している。真の値を含んでいるか、いないかのどちらかである。「この区間に真の値がある確率は 95%」と言いたければ、真の値の

*3 二項比率の信頼区間で、Clopper-Pearson の方法による。scipy.stats.beta.ppf で計算できる。なお二項比率の区間は各件の当たり外れが独立であることを仮定するが、交差検証の予測は訓練データを共有していて独立ではない。ここでの区間は目安として読む。

補足（続き）

大きさについての事前分布をもう一つ仮定する必要がある、そうして得られる区間は信用区間 (credible interval) と呼んで区別する [43]。どちらの立場を取るべきかは長く議論されてきたところで、本書はその判定に立ち入らない。本書が区間を使う目的、つまり「小数第3位を真に受けない」「差を見るときは差の区間を計算する」は、どちらの立場でも変わらないからである。

そして、どちらの読み方をするにせよ、95% という数字は計算に使った仮定がすべて正しいときの話である。正規分布を仮定したなら、その仮定が現実と合っているかどうかで実際の被覆確率は変わる。実務では、この仮定のほうが解釈の議論よりはるかに大きく結果を左右する。

日常的には「だいたいこのへんらしい」で構わない。気をつけたいのは、区間の幅を、細かく読んでよい桁数の目安として使うことだけである。幅が ± 0.07 なら、小数第2位を比べる意味はない。

もう1件間違えて74件中72件だったとすると、正解率は $72/74 = 0.973$ 、その95% 信頼区間は $[0.906, 0.997]$ になる。73件のときの区間 $[0.927, 1.000]$ と大きく重なる。1件の当たり外れで小数第3位は変わるが、その動きは偶然の範囲に収まっている。

だから小さなテストデータの成績を小数第3位まで比べても意味がない。「手法Aは正解率0.987、手法Bは0.973。Aの勝ち」と書きたくなるが、74件で測った正解率なら、0.987と0.973の差0.014は、たった1件の当たり外れでしかない。件数が少ないほど、1件の重みが大きくなる。差が「1件か2件ぶん」に収まっているなら、どちらが良いとは言えないと書くのが正しい。第9.8節で見るように、たくさんの手法を試して最良を報告すれば、この程度の差は簡単に作り出せる。

— ○ 定石 8.2 — 成績を報告するときはばらつきの幅を添える —

添え方は2通りある。

- 交差検証の標準偏差 — 分割の引き当てかたでどれだけ振れるかを表す (第8.2節)
- 二項比率の信頼区間 — 件数の少なさから来る不確かさを表す

2つは別のものを測っている。前者は「分け方を変えたら」、後者は「もう一度データを集めたら」の話である。どちらか一方でも添えてあれば、読み手は小数第3位を真に受けずに済む。

8.3.4 エラーバーは3通りある

「±」を添えると書いたが、何を添えたのかを書かないと、読み手には伝わらない。グラフに立てるエラーバーにも、よく使われるものが3種類ある。どれも見た目は同じ縦棒なので、図だけを見て区別することはできない。

74作品を52列すべてでランダムフォレストに判別させ、層別5分割交差検証で正解率を測った結果 (5回の平均は0.987) で、3つを並べてみる。

- 標準偏差 (SD) = 0.030 — 5回の正解率が、互いにどれだけばらついたか。個々の測定の散らばりである
- 標準誤差 (SEM) = 0.013 — SD を $\sqrt{5}$ で割った値。「同じ実験をもう一度やったら、平均がどれだけばらつくか」を表す

- **95% 信頼区間** = ± 0.037 — SEM に t 分布の係数 2.78 を掛けたもの^{*4}

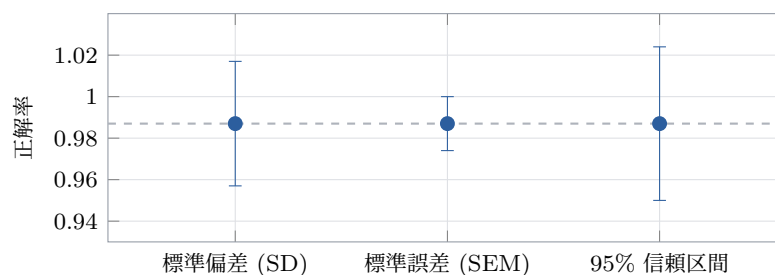


図 8.5 同じ結果から作った 3 種類のエラーバー。中心はどれも 5 回の平均正解率 0.987 で、長さだけが違う。図の見た目では区別できない。

図 8.5 のとおり、同じ結果から**長さの違う 3 本**が作れてしまう。SEM は SD の $1/\sqrt{5}$ 、つまり 2.24 分の 1 である。分割数を増やせばいくらでも短くできる。 $n = 100$ なら 10 分の 1 になる。**短いエラーバーは、精度が高いことを意味しない。**

なお、交差検証の SEM には追加の注意が要る。 $1/\sqrt{n}$ という式は n 個の値が**互いに独立**であることを前提にしているが、 k 分割の各回は訓練データの大部分を共有しているので独立ではない。そのため $SD/\sqrt{k}$ は標準誤差の過小評価になりやすい。交差検証の分散には不偏推定量が存在しないことが知られており [73]、ここでの SEM はあくまで目安として読む。

× アンチパターン 8.1 — エラーバーが何かを書かずに図を出す

「±」の中身を書かなければ、読み手は判断できない。SEM を SD だと思って読めば、散らばりを $\sqrt{n}$ 分の 1 に見誤る。

図のキャプションか凡例に、必ず次の 2 つを書く。

- エラーバーが SD か SEM か信頼区間か
- n がいくつか (平均を何個から計算したか)

読むときも同じで、**書いていない図の棒の長さは信じない。**

使い分けは、何を伝えたいかで決まる。

- 個体がどれだけばらついているかを見せたい → SD。「作品ごとに成績が違う」ことを伝えたいならこちら
- 平均をどれだけ正確に測れたかを見せたい → SEM か信頼区間
- 2 つの条件の差を論じたい → 信頼区間。ただし次の項の注意が要る

8.3.5 区間が重なっていても差はあるかもしれない

2 つのモデルの成績を並べ、それぞれの 95% 信頼区間が少し重なっているのを見て「差はない」と結論するのをよく見かける。**これは正しくない。**信頼区間について広く知られた誤読の一つで、統計の誤用を整理した文献にも項目として挙がっている [43]。

^{*4} 5 回しか測っていないので、正規分布の 1.96 ではなく、自由度 $5 - 1$ の t 分布の値を使う。

作例で確かめる。2つのモデルを同じ手続きで何度も測り、次の結果を得たとする。

	平均正解率	SEM	95% 信頼区間
モデル A	0.900	0.020	[0.861, 0.939]
モデル B	0.835	0.020	[0.796, 0.874]

0.861 から 0.874 のあいだで、2つの区間は重なっている。ここで手が止まる人が多い。

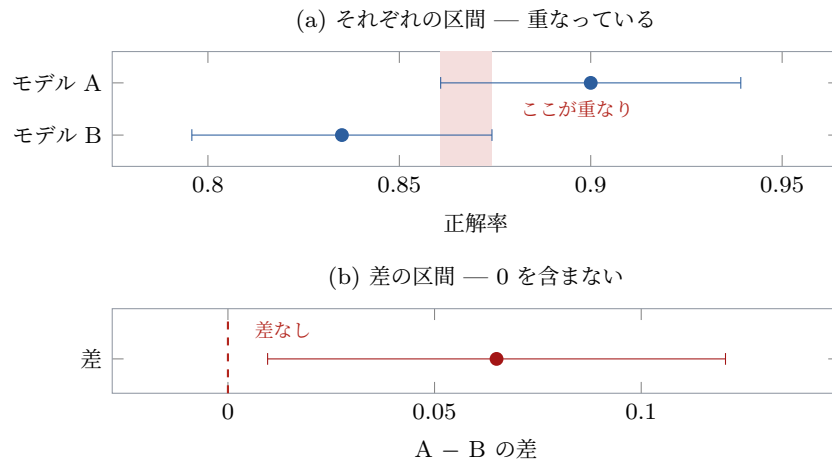


図 8.6 同じ結果を 2 通りに描いた。(a) ではモデル A とモデル B の区間が赤い帯の部分で重なっており、「差はない」と読みたくなる。(b) は差そのものの区間で、こちらは 0(差なし) を含まない。判定に使うのは (b) であって (a) ではない。

実際にすることは 1 つで、**差そのものの信頼区間**を計算する。差の標準誤差は、2つの標準誤差から次のように求める。

$$SE_{\text{差}} = \sqrt{SEM_A^2 + SEM_B^2} \quad (8.4)$$

これは 2つの測定が互いに独立な場合の式である。同じ分割で 2つのモデルを測ったのなら、フォールドごとに差を取る対応ありの方式 (第 8.3.7 項) を使う。

足し算ではなく**二乗和の平方根**なので、2つを単純に足すよりも短くなる。この差が、次に見る「区間は重なるのに差はある」領域を生む。

作例に当てはめる。差は $0.900 - 0.835 = 0.065$ 、その標準誤差は $\sqrt{0.020^2 + 0.020^2} = 0.028$ である。測った回数が十分多いとして、 t 分布の係数を 1.96 で近似すれば、95% 信頼区間は $0.065 \pm 1.96 \times 0.028 = [0.010, 0.120]$ となり、**0 を含まない**。区間が重なっていたにもかかわらず、差はあると言ってよい。

図 8.6 が同じことを 2 通りに描いたものである。(a) だけを見て判断すると結論を取り違える。

理由は 2つの幅を比べれば分かる。2群の SEM がどちらも s のとき、2つの区間の端がちょうど接するのは平均の差が $2 \times 1.96s = 3.92s$ のときである。一方、差の区間が 0 を含まなくなるのは $1.96 \times \sqrt{2}s = 2.77s$ を超えたときである。2.77s から 3.92s のあいだが、**区間は重なっているのに差はある**という領域になる。作例の $s = 0.020$ なら 0.055 から 0.078 で、差の 0.065 はここに入っ

ている。

片側の結論を出すだけなら、目で見て使える判定もある。

- 2つの区間が**まったく重ならない** → 差はあると言ってよい。 $s_A + s_B \geq \sqrt{s_A^2 + s_B^2}$ が常に成り立つので、重ならない時点で必要な幅を超えている
- 一方の区間が**もう一方の平均を含んでいる** → 差があるとは言えない。重なりがここまで深ければ、差の区間は必ず0を含む
- それ以外、つまり**重なってはいるが相手の平均は含まない** → **何も分からない**。差があるかもしれないし、ないかもしれない。作例がちょうどこの場合に当たる

要するに、2つの区間を並べて重なりを目で測るのはやめ、式(8.4)で差の標準誤差を出して差の区間を作る。判断することは「区間が0を含むか」の1つだけになる。3つ以上を並べるときは、図の描き方まで変える必要がある。第8.3.7項で扱う。

図の描き方といえば、平均を棒の高さで表し、その上にエラーバーを1本立てた図もよく見る。簡潔だが、**隠すものが多すぎる**。データが何件あるのか、分布が1つの山か2つの山か、外れ値が1点あるだけなのか全体がばらけているのか。どれもこの図からは読み取れず、まったく違う形のデータが、同じ棒と同じ長さの棒になりうる。第4.8.2項の Anscombe の四重奏と同じ話である。件数が数十以下なら、**生のデータ点をそのまま重ねて描く**のがよい。本書の5分割交差検証なら、点は5個しかないで、平均と棒を描くより、5個をそのまま並べたほうが情報量が多い。件数が多いなら、箱ひげ図やバイオリンプロットで分布の形を見せる。棒とエラーバーだけで済ませるのは、分布の形が明らかに問題にならない場合に限る。

8.3.6 たくさん並べて比べる — フォレストプロット

手法が2つなら差の区間を1本計算すればよい。だが実際には、5つも10つも並べて比べることが多い。そのとき定番になっているのが、1行に1手法、点で平均、横棒で区間を描いた図である。医学の分野で複数の研究をまとめるときに使われてきた形で、**フォレストプロット** (forest plot、森林図) という。横棒がずらりと並ぶ様子が林に見えることから、この名前がついた。

第7章の表7.6にある6手法の正解率(74作品、52列、層別5分割交差検証)を、この形で描く。読み方の順序を決めておくと迷わない。

1. **基準線がどこにあるか**を最初に見る。最頻クラスを答え続ける戦略の正解率の95%信頼区間は[0.25, 0.32]で、他のどれとも重なっていない。並べた手法はすべて基準線を超えている
2. **棒の長さを見る**。決定木の棒がいちばん長い。平均正解率が0.688と低いだけでなく、**測れていない**のである
3. **最後に順位を見る**。上位3つは棒が大きく重なっている。順位はついてはいるが、読み取れる差ではない

棒を2種類描く理由 — 並べた本数だけ外れやすくなる 区間を1本だけ描くときには起きず、たくさん並べたときにだけ起きる問題がある。

1本ずつは95%で正しくても、6本並べれば**少なくとも1本が外れる確率はおおよそ26%**になる。第9.8節の多重テストと同じ構造で、計算もそちらで扱う。なお6本の区間は同じデータから作られ

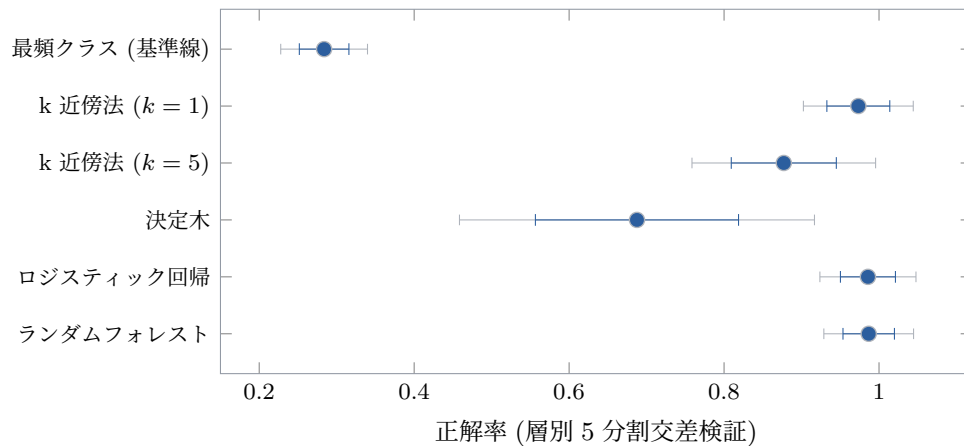


図 8.7 6 手法のフォレストプロット。点が層別 5 分割交差検証の平均正解率、濃い細い棒がその手法だけを見たときの 95% 信頼区間、薄い太い棒が 6 手法をまとめて 95% にした区間である。表を眺めるより、どの差が読み取れてどの差が読み取れないかが早く分かる。区間はいずれもフォールド間の $SD/\sqrt{5}$ を標準誤差とした t 区間で、フォールドが独立でないため (第 8.3.4 項)、正式に 95% の被覆を保証する区間ではない。描き方の説明用と考えてほしい。

ていて独立ではないので、この 26% という値は独立を仮定した近似である。

図全体を「95% の信頼度で読みたい」なら、1 本あたりの信頼度を上げて棒を伸ばす必要がある。最も素朴なやり方はボンフェローニ補正 (Bonferroni correction) で、5% を手法の数で割り、1 本あたり $5/6 = 0.83\%$ の水準にする。本書の場合、掛ける t 分布の係数が 2.78 から 4.85 へ、つまり棒が 1.75 倍に伸びる。これが薄い太いほうの棒である。

伸ばすと読み方が変わる。決定木の正解率の同時区間は $[0.459, 0.917]$ まで広がり、 k 近傍法 ($k = 5$) の同時区間の下端 0.759 と重なってしまう。個別の区間では分かれて見えた 2 つが、6 手法をまとめて主張しようとするのと分けられなくなる。

ただし、ボンフェローニ補正が直すのは本数の問題だけである。もとの 1 本 1 本の区間が、独立でないフォールドから $SD/\sqrt{5}$ で作った目安の区間であることは変わらない。棒を伸ばしても、その被覆が 95% になるわけではない。

× アンチパターン 8.2 — 並べた区間の重なりを見て手法に順位をつける

フォレストプロットは順位表ではない。

第 8.3.5 項で見たとおり、2 本の区間が重なっているかどうかから差の有無は判定できない。本数が増えれば、その判定をたくさん繰り返すことになる。6 手法なら組み合わせは $\binom{6}{2} = 15$ 通りある。15 回も目で判定すれば、そのうちいくつかは間違える。

フォレストプロットに向いているのは次の使い方である。

- どれが基準線を超えたかを見る (1 本ずつの判定で足りる)
- どの手法が測れていないかを見る (棒が異様に長いもの)
- 全体としてどのくらいの幅に収まっているかをつかむ

「A と B のどちらが強いかな」を決めたいなら、図を眺めるのではなく、A と B の差の区間を

計算する。比べたい相手が最初から決まっているなら、そもそも 6 本並べる必要がない。

8.3.7 差だけを描く

前項の図には根本的な弱点がある。**見せたいものが描かれていない**。知りたいのは差なのに、図に描いてあるのは各手法の区間である。だから読み手は重なりを目で測るしかなくなり、第 8.3.5 項で見た誤りに落ちる。

直し方は単純である。**差そのものを横軸にする**。

手順は 3 段である。

1. 基準にする手法を 1 つ決める。本書では平均正解率が最も高かったランダムフォレストにした。ただし**同じ結果を見てから基準を選んでいる**ので、基準を事前に決めた場合 (例えば「いま使っている手法」を基準にする) より差が大きくなりやすい。本来は基準を先に決めておく
2. 残りの 5 手法それぞれについて、基準との正解率の差を計算する
3. 差の点推定と信頼区間を並べ、0 のところに縦線を引く

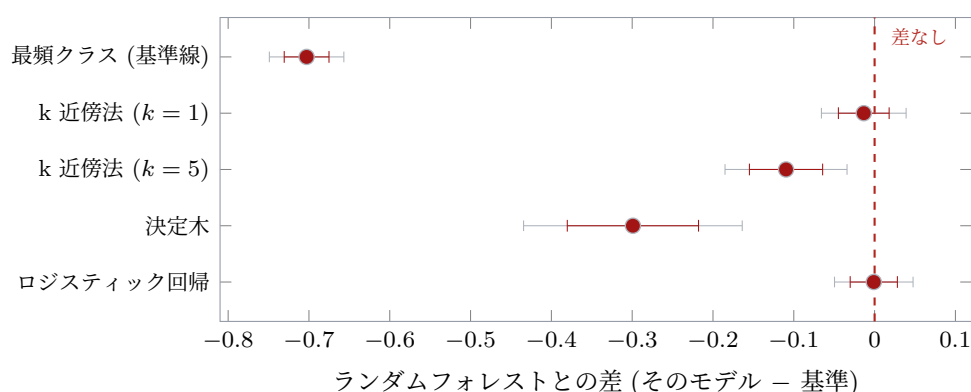


図 8.8 ランダムフォレストを基準にした、層別 5 分割交差検証の平均正解率の差の区間。薄い太い棒は**対応を取らずに**計算した差の区間、濃い細い棒は**同じ分割のフォールドごとに差を取って**計算した区間である。赤い縦線が「差なし」で、区間がこの線をまたいでいれば差があるとは言えない。重なりを目で測る余地がない。区間は図 8.7 と同じくフォールド間の $SD/\sqrt{5}$ から作った目安で、正式な 95% の被覆は保証しない。

読み方は 1 つに減る。**区間が 0 の線をまたいでいるか**だけである。

- またいでいない → 基準との差を示唆する
- またいでいる → 差があるとは言えない

図 8.8 では、5 手法のうち 3 つで、正解率の差の区間が 0 を含まない。残る 2 つは 0 をまたいでおり、ランダムフォレストとの正解率の差があるとは言えない。前項のフォレストプロットで「上位 3 つは棒が重なっている」と読んだのと同じ結論だが、こちらは**目で測っていない**。

「0 を含まない」を「差がある」と断定しないのは、区間の作り方による。フォールドごとに差を取ると、モデル間で共通する上下は消えるが、**フォールドどうしが訓練データを共有している**ことは消えない (第 8.3.4 項)。 $SD/\sqrt{5}$ はやはり標準誤差の過小評価になりやすく、区間は短めに出る。性能差を正式に推論したいなら、目的と分割の単位を先に決め、反復交差検証に対する補正付きの t 検定

[73]などで測り直す。本項の図は、**差を軸にすると読み方がどう変わるか**を示すためのものである。

棒を2種類描く理由 — **差を取ると共通の上下が消える** 薄いほうと濃いほうで長さが違う。濃いほうはフォールドごとに差を取ってから区間を作っている。 k 分割交差検証の1回ぶん、つまり「この塊をテストに回した回」をフォールド (fold) と呼ぶ (第10.1節)。

交差検証では、どのモデルも**同じ分割**で測っている。だから「たまたま難しい作品ばかりがテストに集まったフォールド」では、どのモデルも一緒に成績が下がる。この共通の上下は、モデルどうしの差を取れば引き算で消える。消えたぶんだけ区間が短くなる。本書のデータでは、対応を取らずに作った区間と比べて、長さが平均0.60倍になった。

比べたいのが差なら、最初から差を測る設計にする。 同じ分割で全部のモデルを回しておけば、あとからフォールドごとの差が取れる。モデルごとに別々の分割で測ってしまうと、この短縮は使えない。

ただし、差を軸にしても**本数の問題は消えない**。5本の95%区間を並べれば、少なくとも1本が外れる確率はやはり23%ある(これも独立を仮定した近似の値である)。基準を決めて15通りから5通りに減らしたぶん、前項より軽くはなっているが、ゼロにはならない。図全体を95%で読みたいなら、前項と同じくボンフェローニ補正で棒を伸ばす。本数を減らすことと、補正することは、別々の対処である。

○ 定石 8.3 — 比べたいものが差なら差を軸にした図を描く

区間を並べた図は、読み手に重なりを測らせてしまう。差を軸にすれば、読むのは「0をまたぐか」だけになる。

- **基準を1つ決める。** 全ペアを描くと15通りになるが、基準を決めれば5通りで済む
- **0の線を必ず引く。** これがないと差の図の意味がない
- **同じ分割で全モデルを回す。** フォールドごとの差が取れて区間が短くなる

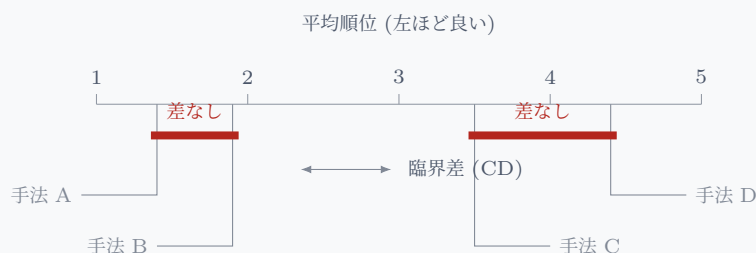
生のデータ点も一緒に見せたいなら、**推定プロット** (estimation plot) という描き方がある。左側に各群の生の点を並べ、右側に**別の軸**を立てて差の分布と区間を描く。提案者の名前からGardner–Altmanプロットとも呼ばれ、医学や心理学の分野で「 p 値の代わりにこれを載せよ」という提案とともに広まった[38]。DABESTなどの実装がある。

発展 複数のデータセットで手法を比べる — CD ダイアグラム

機械学習の論文では、手法を複数のデータセットで走らせて比べることが多い。このとき正解率をそのまま平均するのは筋が悪い。データセットによって正解率の水準も幅も違うからである。どの手法でも正解率が0.99を超えるデータセットと、どれも0.6前後にとどまるデータセットとを平均すれば、手法間の差が大きく出るのは後者のほうである。平均は、差の出やすいデータセットの結果に引きずられる。

そこで**データセットごとに順位をつけ、順位の平均で比べる方法**がある。これを図にしたのが**CD ダイアグラム** (critical difference diagram) で、Demšar が提案した比較の手順[39]とともに広まった。日本語では**臨界差図**と訳されることもあるが、論文や実装ではCD diagram、CD plot と書かれることが多い。本書はCD ダイアグラムと呼ぶ。

発展（続き）



横軸が平均順位で、左ほど成績が良い。赤い横棒でつないだ手法どうしは**差があるとは言えないこと**を表す。つながりかどうかの境目になる順位差が**臨界差** (critical difference、CD) で、手法の数とデータセットの数から決まる (検定には Nemenyi 検定などを使う)。図の名前の CD はこの臨界差のことである。図の A と B は順位差が臨界差より小さいのでつながれ、B と C は臨界差より大きいのでつながれていない。

読み方は1つだけ覚えればよい。同じ棒につながれていれば、順位が違っていても**差はない**。順位の数字そのものを比べることはできない。

本書はデータセットが1つしかないので、CD ダイアグラムは描けない。描くには「手法 × データセット」の表が要る。論文でこの図を見たら、**いくつかのデータセットで測ったか**を先に確かめるとよい。数が少なければ臨界差は大きくなり、ほとんどの手法が1本の棒でつながってしまう。

8.4 指標は「どの誤りが高くつくか」で選ぶ

指標が増えてきたので、選び方を整理しておく。出発点は**どの誤りが高くつくか**であって、指標の名前ではない。がん検診なら再現率を重視する。見逃せば命に関わるが、誤検出は追加検査で解消できる。迷惑メールのフィルタなら適合率を重視する。迷惑メールが1通届くより、大事なメールが1通消えるほうが困る。「 F_1 を最大化する」は思考停止になりうる。自分の問題ではどちらの誤りが高くつくかを先に決める。

表 8.3 の1行目と2行目は、本書のデータでは食い違わなかった。表 8.2 のとおり、正解率 0.986、マクロ F_1 0.977 で、どちらで見ても同じ結論になる。食い違うのは、多数クラスだけを当てて少数クラスを捨てているときである。その場合、正解率の高いままマクロ F_1 だけが落ちる。

補足 指標を目標にすると指標だけが上がる

指標を選ぶことには、もう1つの意味がある。**選んだ指標が、そのまま最適化の目標になること**である。そして目標にされた指標は、たいてい抜け道を持っている。本書の言葉でいえば、「正解率を上げよ」と命じられたモデルが最初に見つけるのは、最頻クラスを答え続ける戦略である。文体を学ばずに正解率 0.284 を得る道が、そこにあるからである。

同じことが人にも起きる。コールセンターの評価を「平均処理時間」で測れば、時間は短くなる。用件が済んでいなくても電話を切ることで達成できるからである。強化学習でボートレースのゲームを走らせた実験では、エージェントがゴールせずに、ボーナスアイテムの湧く場所を延々と周回してスコアを稼いだ [72]。

測るために選んだ量が、目標そのものにすり替わる。これはグッドハートの法則と呼ばれている。指標は目的の代理でしかないので、代理が取りこぼしている部分を別の指標で見張っておく必要がある。正解

表 8.3 指標の選び方

状況	見るもの
クラスの大きさがそろっていて、どの誤りも同じ重さ	正解率。ただし最頻クラスの割合と必ず並べる
クラスが偏っていて、少数クラスも同じくらい大事	マクロ F_1 。正解率は多数クラスに引きずられる
誤検出と見逃しで代償が違う	適合率と再現率を別々に見る。 F_1 にまとめると差が消える
しきい値をまだ決めていない、順位付けの良し悪しを見たい	ROC 曲線と AUC。陽性がごく少ないときは PR 曲線
多クラスで「どこを取り違えたか」を知りたい 数値を当てる (回帰)	混同行列。数字 1 つに要約する前に必ず一度見る 決定係数 R^2 か平均絶対誤差。 $R^2 = 0$ が「つねに平均を答える」

補足 (続き)

率を報告するときにマクロ F_1 を並べるのは、その一例である。

8.4.1 基準線は指標ごとに作る

第 7.2 節で「基準線を置く」と書いたが、**基準線は指標ごとに違う**。当てずっぽうにも何通りかあり、どれが強いかは指標で入れ替わる。

74 作品・8 名のデータで、3 つの当てずっぽうの正解率とマクロ F_1 を実際に測った (ランダムなものは 2000 回の平均)。

当てずっぽうの戦略	正解率	マクロ F_1
一様ランダム (8 名から等確率)	0.126	0.117
クラス比に比例して選ぶ	0.157	0.122
つねに最頻クラス (吉川英治)	0.284	0.055

表の太字は、その指標でいちばん強かった戦略である。2 つの列では、別の戦略が太字になっている。

正解率でいちばん強い「つねに最頻クラス」(正解率 0.284) が、マクロ F_1 では 0.055 でいちばん弱い。吉川英治だけを当てて残り 7 名を全部捨てるので、7 名分の F_1 がゼロになり、8 個の平均が押し下げられるからである。逆に一様ランダムは、正解率では 3 つのうち最低の 0.126 だが、どのクラスも同じように当てにいくぶん、マクロ F_1 では 0.117 と、最頻クラス戦略の 0.055 の 2 倍になる。

つまり「当てずっぽうの成績」は 1 つに決まらない。どの当てずっぽうを相手にするかで、自分のモデルがどれだけ上回ったかの見え方が変わる。

○ 定石 8.4 — 報告する指標と同じ指標で基準線を測る —

「正解率 0.90 を達成」と書くなら、最頻クラス戦略の**正解率**を並べる。「マクロ F_1 0.62 を達成」と書くなら、同じ当てずっぽうの**マクロ F_1** を並べる。指標を混ぜると、基準線が不当に低く見えたり高く見えたりする。

当てずっぽうの候補は次の 3 つで足りることが多い。

- **一様ランダム** — クラス数の逆数。理論値がすぐ出る
- **クラス比に比例** — 正解率の期待値は $\sum_i p_i^2$
- **つねに最頻クラス** — 正解率の期待値は $\max_i p_i$

2 値分類で AUC を報告するなら、基準線は 0.5 である (当てずっぽうの順位付け)。回帰なら $R^2 = 0$ 、すなわち「つねに平均を答える」が基準線になる。

8.4.2 決定係数は誤差の大きさではなく基準線からの改善

表 8.3 の最終行に決定係数を挙げたが、 R^2 については「使うべきでない」という批判が長く続いている。何が問題で、代わりに何を見ればよいかを整理しておく。

定義から始める。 n 個の正解 y_i と予測 $\hat{y}_i$ に対し、

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2} \quad (8.5)$$

分子は予測の二乗誤差、分母は「つねに平均 $\bar{y}$ を答える」ときの二乗誤差である。つまり R^2 は誤差そのものではなく、**基準線に対して誤差が何割減ったか**を表す。第 7.2 節の考え方が式に埋め込まれていると見ればよい。

分母は評価データの散らばりで決まる 問題は、その分母が測る相手によって変わることにある。

第 9 章「時系列の漏れ」で使う連載小説のデータで確かめる。Web 上で 677 話にわたって連載された作品について、各話の文体から**その話がいつ公開されたか**を当てる問題である。正解も予測も「連載開始から何日目か」という日数で、誤差の単位は日になる。

677 話すべてをリッジ回帰で予測し、5 分割交差検証で測ると、 $R^2 = 0.77$ 、平均絶対誤差は 133 日、つまり平均して 4 か月ほどずれる。

ここで、**予測はそのままにして測る範囲だけを変える**。連載中盤の 226 話を取り出し、その範囲だけで測り直す。すると平均絶対誤差は 133 日から 118 日へ 15 日**下がる**のに、 R^2 のほうは 0.77 から -0.94 になった。誤差が小さくなったのに、 R^2 は 0 を割り込み、「つねに平均を答える」という基準線に負けた。

R^2 が下がったのは、比較の相手が変わったためである。全期間では正解 (公開日) の標準偏差が 362 日あるが、中盤だけなら 107 日しかない。狭い範囲では「つねに平均」が強い相手になるので、同じ精度の予測でも R^2 は落ちる。 R^2 が測っているのは予測の質だけでなく、**評価データの散らばり**でもある。

× アンチパターン 8.3 — 別々のデータで測った決定係数を並べて優劣を論じる —

「先行研究は $R^2 = 0.6$ 、本手法は $R^2 = 0.8$ 」は、両者が同じ評価データを使っていない限り比較になっていない。均質な集団に当てれば R^2 は下がり、散らばりの大きい集団に当てれば上がる。比べてよいのは、**同じ正解の並び**に対する R^2 どうしだけである。

列を足せば増える 第二の批判は、説明変数を増やしても訓練データの R^2 が決して減らないことである。役に立たない列を足しても、モデルはその列の係数を 0 にすればよいので、少なくとも元と同じ値は保たれる。実際には 0 でない係数がわずかに当てはまるので、値は上がる。同じデータに、情報をまったく含まない乱数列を足していった (図 8.9)。

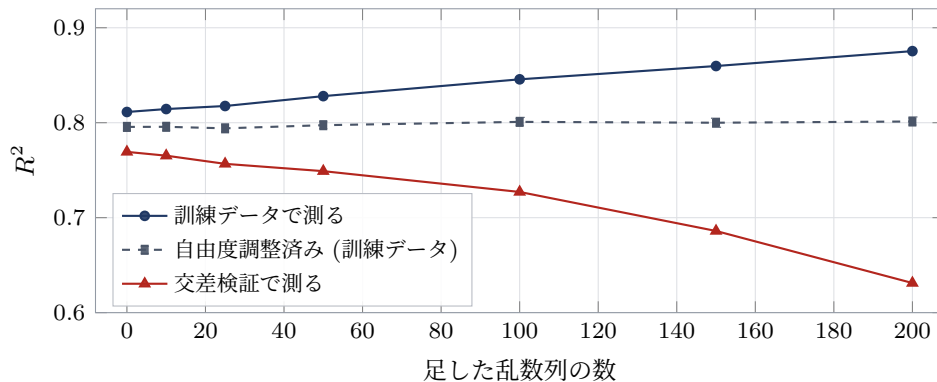


図 8.9 677 話の公開日を線形回帰で当てる問題に、情報ゼロの乱数列を足したときの R^2 。訓練データで測れば増え続ける

同じ 677 話を線形回帰で予測するとき、乱数を 200 列足すと、訓練データで測った R^2 は 0.81 から 0.88 に上がる。同じモデルを 5 分割交差検証で測れば R^2 は 0.63 まで落ちるので、予測が悪化したことは正しく分かる。自由度調整済み R^2 *5 は 0.80 でほとんど変わらない。無情報の列がもたらす見かけの増分を罰で打ち消して横ばいに保つのが調整の役目なので、これは設計どおりの挙動である。ただし、交差検証で見た予測の悪化までは検出できない。

これは R^2 固有の欠点というより、本章で繰り返してきた「訓練データで測るな」の一例である。ただし R^2 は増えることが数学的に保証されているぶん、「 R^2 が上がったから良くなった」という誤読を招きやすい。

発展 R^2 への批判とそれへの反論

R^2 への批判は古い。Kvålseth は 1985 年の論文で、 R^2 と呼ばれる式が少なくとも 8 通り流通していること、それらが一致するのは切片つきの線形回帰という限られた場合だけであることを指摘した [65]。切片を置かない回帰では $\sum (y_i - \bar{y})^2$ を残差と回帰に分解する等式が成り立たなくなり、定義によっては 1 を超える値さえ出る。

非線形回帰ではさらに悪い。Spiess と Neumeyer は薬理学の用量反応曲線を題材に、数千回のシミュレーションで R^2 が良いモデルにも悪いモデルにも一様に高い値を返すこと、 R^2 を基準にモデルを選ぶ

*5 列数 p による罰を入れた $1 - (1 - R^2) \frac{n-1}{n-p-1}$ 。「調整済み」と呼ばれる。

発展 (続き)

と正しいモデルを選べる割合が 28~43% にとどまることを示した [66]。彼らの結論は、非線形では R^2 を捨てて AIC などの情報量規準を使い、というものである。

いっぽう擁護もある。Chicco らは、MSE・RMSE・MAE・MAPE には上限がないため値を 1 つ示されてもよし悪しが判断できない、と論じた [67]。「平均絶対誤差 133」と言われても、正解の分布を知らなければ優れた予測なのか無意味な予測なのか分からない。 R^2 には 1 という上限と、0 という意味のある基準点がある。彼らはこれを理由に、回帰の標準指標として R^2 を推奨している。

ただし彼ら自身が認めるとおり、 $R^2 = 1 - \text{MSE}/\text{MST}$ で分母は評価データごとに固定されるから、**同じデータの上でモデルを順位づけるかぎり**、 R^2 による順序と RMSE による順序は完全に一致する。 R^2 の利点は順位付けの精度ではなく、1 つの値の読みやすさにある。また彼らは、 R^2 が負のときは下限が $-\infty$ なので -0.5 と -50 の差を解釈できない、という限界も挙げている。

批判が当てはまる場面と、当てはまらない場面 批判をまとめると、 R^2 が壊れるのは (1) 別のデータで測った値と比べたとき、(2) 訓練データで測ったとき、(3) 切片なしや非線形の回帰に当てはめたとき、である。逆にいえば、同じ評価データの上で、交差検証で測り、切片つきの回帰に使うかぎり、 R^2 は「基準線からどれだけ改善したか」を素直に表す。

○ 定石 8.5 — 回帰は誤差と基準線を組にして報告する

数値を当てる問題では、次の 3 つを並べる。

- **単位の残る誤差** — 平均絶対誤差や RMSE。「何日ずれるか」が読み手に伝わる
- **基準線** — 「つねに平均を答える」ときの同じ誤差。 R^2 はこの 2 つの比を 1 つの数にしたものである
- **残差の図** — 予測値を横軸、 $y_i - \hat{y}_i$ を縦軸に取る。誤差が特定の範囲に偏っていないかは、数値 1 つでは分からない

R^2 を報告してもよいが、そのときは評価データが何かを必ず書く。非線形回帰や切片なしの回帰では R^2 を使わず、予測誤差そのものか情報量規準で比べる。

第 9 章の時系列の漏れでは、同じ連載小説の公開日を時間順に分けて予測したときの R^2 が -4.44 になる。負の値が出ることも自体が、 R^2 を「説明できた分散の割合」とは読めない証拠である。割合なら負にならない。 R^2 は割合ではなく、基準線との比較である。

8.5 しきい値を動かす — ROC 曲線と AUC

ここまでは「モデルが出した予測クラス」だけを見てきた。だが多くのモデルは、実際には**確率**を出している。

```
proba = model.predict_proba(X)    # 各クラスの確率
pred = proba.argmax(axis=1)       # 最大のものを選ぶ
```

2 値分類なら「陽性の確率が 0.5 を超えたら陽性」と判定するのが既定である。だが 0.5 というしきい値は、単なる慣習にすぎない。

しきい値を下げれば、陽性と判定する数が増える。見逃し (偽陰性) は減り、誤検出 (偽陽性) は増

える。上げれば逆になる。

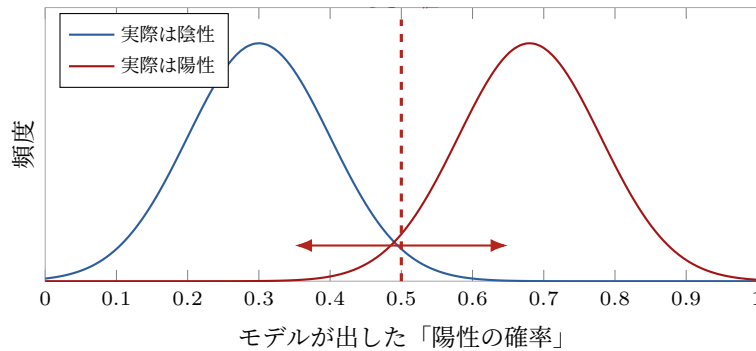


図 8.10 しきい値を左右に動かすと、見逃しと誤検出のバランスが変わる。

しきい値を 0 から 1 まで動かしながら、次の 2 つを記録する。

$$\text{真陽性率 (TPR)} = \frac{TP}{TP + FN} \quad (= \text{再現率}) \quad (8.6)$$

$$\text{偽陽性率 (FPR)} = \frac{FP}{FP + TN} \quad (8.7)$$

横軸を FPR、縦軸を TPR として描いた曲線が **ROC 曲線**である。Receiver Operating Characteristic の略で、第二次大戦中のレーダー信号の判別研究に由来する。

一つ工夫をする。ここから測るのは「太宰治か、そうでないか」という 2 択で、74 作品を層別 5 分割交差検証にかけ、そこで出た確率から AUC を計算する。52 列すべてを使うと、この 2 択は簡単すぎて 3 つの手法の AUC がどれも 0.98 を超え、曲線が左上の角で重なってしまう (ランダムフォレストが 0.983、k 近傍法 ($k = 5$) が 0.997、ロジスティック回帰が 0.996)。差が見えないので、混同行列 (第 8.3.2 項) と同じ工夫をする。特徴を字種 3 列と文長 2 列の計 5 列に間引いた、読点の癖を封じた状態で描く。

こうすると 3 本の曲線がはっきり分かれる。5 列に間引いた設定での AUC は、ロジスティック回帰が 0.968 で最も高く、ランダムフォレストが 0.934、k 近傍法が 0.883 と続く。52 列すべてを使った場合の順位はこれと違い、k 近傍法が 0.997 で最上位、ロジスティック回帰が 0.996、ランダムフォレストが 0.983 で最下位になる。**手法の優劣は特徴集合しだいで入れ替わる。**

× アンチパターン 8.4 — 全員が満点近くになる課題で手法の優劣を論じる —

どの手法でも成績がほぼ満点に張りつくことを**天井効果** (ceiling effect) という。天井に張りついた数値どうしの差は、ほとんど乱数である。「AUC0.997 の手法が 0.996 の手法に勝った」と書いても、乱数の種を変えれば順位は入れ替わる。

比べたいものがあるなら、課題のほうを難しくする。特徴を間引く、クラスを増やす、訓練データを減らす。本節で 52 列から 5 列に絞ったのはそのためである。逆に、難しくしても差が出ないなら、**その比較には差がない**と報告すればよい。無理に順位を付ける必要はない。

曲線上の目印をいくつか押さえる。

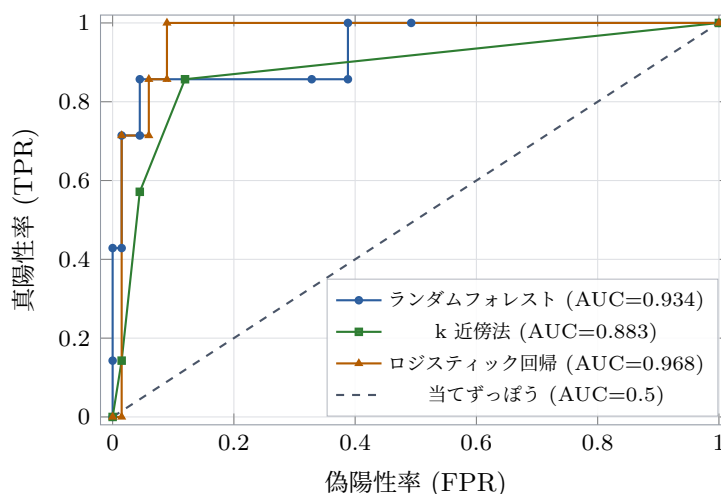


図 8.11 「太宰治かどうか」を判定する ROC 曲線 (字種 3 列と文長 2 列の計 5 列に間引いた設定、74 作品、層別 5 分割交差検証)。左上に近いほど良い。

- 左上の角 (0, 1) — 完璧。誤検出ゼロで全部当てる
- 対角線 — 当てずっぽうと同じ
- 対角線より下 — 予測を反転させたほうがまし

曲線の下面積が **AUC (Area Under the Curve)** である。1 に近いほど良く、0.5 が当てずっぽうに相当する。

この面積には、分かりやすい解釈がある。

陽性のサンプルを 1 つ、陰性のサンプルを 1 つランダムに選んだとき、モデルが陽性のほうに高いスコアを与える確率。

図 8.11 のランダムフォレストの $AUC = 0.934$ なら、無作為に選んだ太宰治の作品と他の作家の作品を 1 つずつ比べて、93% の確率で太宰治のほうに高いスコアがつくという意味である。しきい値をどこに置くかとは無関係に、**順位付けの良さ**だけを測っている。

8.5.1 AUC の落とし穴

AUC は便利だが、万能ではない。

× アンチパターン 8.5 — 不均衡データで AUC だけを見る

陽性が 1% しかないデータを考える。陰性が 99% あるので、偽陽性率 $FP/(FP + TN)$ の分母が非常に大きい。偽陽性が多少増えても FPR はほとんど変わらない。

結果として、実用に耐えないモデルでも AUC が 0.9 を超えることがある。「陽性と判定したものうち 9 割が誤り」でも、AUC は高く出る。

不均衡なデータでは **PR 曲線** (適合率-再現率曲線) を使う。横軸に再現率、縦軸に適合率を取る。不均衡があるほど、AUC と PR-AUC の差が開く。

○ 定石 8.6 — しきい値は誤りのコストで決める

ROC 曲線は「どのしきい値がありうるか」を見せてくれるが、どれを選ぶべきかは教えてくれない。決めるのは、第 8.4 節で指標を選んだときと同じ**誤りのコスト**である。見逃しと誤検出のコストの比が決まれば、最適なしきい値も決まる。

8.6 較正 — 出力された確率が当たっているかを測る

AUC は順位だけを見ていた。モデルが出した確率の 0.9 と 0.6 のどちらが大きいかが分かればよく、その 0.9 という数値自体には踏み込まない。

だが実務では、確率の値そのものを使う場面がある。「この文書が太宰治である確率は 0.9」と表示する、確率で期待損失を計算して判断を決める、複数のモデルの確率を平均する。どれも **0.9 と言われたら、そう判定された文書の 9 割が本当に太宰治であることを前提**にしている。

これが成り立っているかどうかを**較正** (calibration) という。測り方は素朴で、モデルが 0.9 前後と答えた文書だけを集め、その中で実際に正解だった割合を数えればよい。確率の帯ごとにこれを繰り返して、横軸に予測確率、縦軸に実際の割合を取った図が**信頼度曲線** (reliability diagram) である。対角線の上に乗っていれば、確率は額面どおりということになる。2 値化の対象は、ROC 曲線の太宰治から替えて、今度は吉川英治にする。

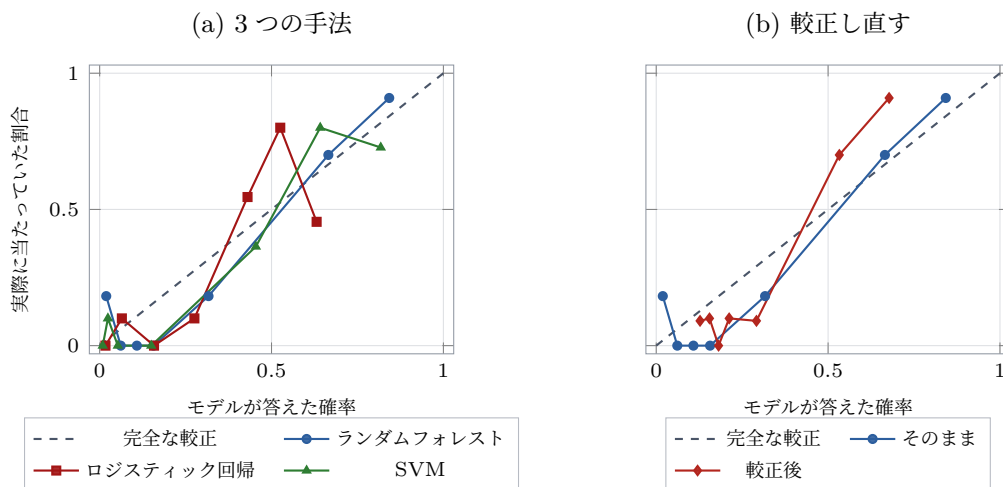


図 8.12 信頼度曲線。「吉川英治かどうか」を字種 3 列と文長 2 列で当てた場合 (図 8.11 と同じ難しくした設定、74 作品、層別 5 分割交差検証)。横軸はモデルが答えた確率、縦軸はその帯で実際に当たっていた割合である。点が対角線より下であれば自信過剰、上であれば自信不足である。

8.6.1 ずれを 1 つの数で測る — Brier スコア

図を見るだけでなく、数値でも比べたい。確率と実際 (当たりなら 1、外れなら 0) の **2 乗誤差の平均**を **Brier スコア**という。

$$\text{Brier} = \frac{1}{n} \sum_{i=1}^n (p_i - y_i)^2 \quad (8.8)$$

小さいほどよい。確率と正解の 2 乗誤差なので、較正のずれだけでなく、陽性と陰性をどれだけ分離できているかも一緒に反映する。**確率予測の総合的な誤差**であって、較正専用の指標ではない。較正そのものは、信頼度曲線の対角線からのずれで見る。「吉川英治か、そうでないか」を字種 3 列と文長 2 列の計 5 列で判定し、層別 5 分割交差検証で出した確率の Brier スコアを測ると、ランダムフォレストが 0.0958、SVM が 0.1084、ロジスティック回帰が 0.1495 である。参考までに、**全員に同じ確率 (陽性の割合) を答える**という何も学ばないモデルでは 0.2033 になり、3 つともこの基準線を下回っている。基準線を置くのは、正解率のときと同じである (第 7.2 節)。

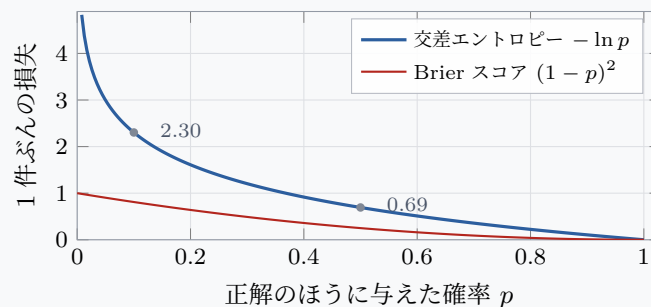
この順位は通説と食い違う。木のアンサンブルは多数決の平均なので確率が 0 や 1 に振り切りにくく、較正が悪くなりやすい、とよく言われる。本節の実測では、確率予測の総合的な誤差が最も小さいのはランダムフォレストで、図 8.12(a) の信頼度曲線を見ても、対角線からのずれが他の 2 つより目立って大きいわけではない。較正の良し悪しは、手法だけでなく**データと問題**で決まる。測らずに手法名から決めつけないほうがよい。

発展 Brier スコアと交差エントロピー — 罰の重さが違う

確率を測る基準は Brier スコアだけではない。第 7 章の発展で見た**交差エントロピー** (1 件あたり $-\ln p$ 。 p は正解のほうに与えた確率) も同じ用途に使う。

2 つは大事な性質を共有している。真の確率が q である事象に対してモデルが p と答えたときの期待値は、どちらも $p = q$ でちょうど最小になり、そこからずらせば必ず増える。この性質を持つ基準を**厳密に固有なスコアリングルール** (strictly proper scoring rule) という。自信を水増ししても、逆に安全側へ丸めても、期待値では損をする。この性質を持たない量 (たとえば正解率) だけを見ていると、正しい値の周りで確率を歪めたほうが得になる余地が残る。

違うのは**言い切ったときの罰の重さ**である。1 件ぶんの罰を、正解のほうに与えた確率 p の関数として並べる。



Brier スコア $(p - y)^2$ の罰は最大でも 1 で、正解に 0.5、0.1、0.01 しか与えなかったときの値は 0.25、0.81、0.98 にとどまる。交差エントロピーは同じ場面で 0.69、2.30、4.61 と立ち上がり、 $p \rightarrow 0$ で正解が 1 だったときには $-\ln p \rightarrow \infty$ となる。「ありえない」と言い切った側が起きたときの罰に上限がない。

どちらを使うかは、外れの重さをどう見るかで決まる。確率そのものが判断に使われる場面 (検査の陽性確率、与信の債務不履行確率) なら、罰に上限のない交差エントロピーが向いている。半面、ラベルが 1 件間違っているだけで値が跳ね上がるので、実装では p を $[\varepsilon, 1 - \varepsilon]$ に丸めてから計算する。本節で Brier スコアを使うのは、値が暴れず、較正のずれと分離の良し悪しをまとめて 1 つの数で見られるからである。

8.6.2 ずれていたら直せる

較正がずれていても、順位付けさえ良ければ後から直せる。やり方は、**予測確率を入力として、正解を当てる 1 変数のモデルを重ねること**である。

- **Platt scaling** — シグモイド関数を当てはめる。パラメータが 2 つだけなので、データが少なくても安定する
- **等調回帰 (isotonic regression)** — 単調増加という条件だけを課して、自由な形の対応表を作る。柔軟だがデータが要る

図 8.12(b) がランダムフォレストに Platt scaling をかけた結果である。Brier スコアは較正前の 0.0958 から較正後の 0.1110 へ上がり、**かえって悪くなった**。何も学ばないモデルの 0.2033 よりは小さいものの、較正しないほうがよかったことになる。較正のための当てはめ自体が、74 点しかないデータでは過学習するからである。

同じ理由で、**小さいデータでは較正曲線を読み込みすぎない**。本節の図は、74 点を 7 つの帯に分けて描いている。1 つの帯に 10 点あるかないかで、そのうち何点が当たったかを割合にしているので、1 点ずれば割合は 0.1 変わる。較正曲線がぎざぎざしていても、それがモデルの性質なのか、点が少ないだけなのかは区別できない。較正を論じるには**帯ごとに数十点は必要**だと考えておくとよく、足りないなら、曲線の形ではなく Brier スコアのような 1 つの数だけを見るほうが安全である。

○ 定石 8.7 — 確率の値を使うなら較正まで確かめる

順位だけを使う場面 (上位 10 件を人が確認する、AUC で比べる) では、較正がずれていても困らない。

確率の値そのものを使う場面では確かめる。

- 確率を表示して、人がそれを見て判断する
- 期待損失 (確率 $\times$ コスト) を計算してしきい値を決める
- 複数モデルの確率を平均する、確率を別のモデルの入力にする

正解率や AUC が高いことと、確率が信用できることは別の性質である。

× アンチパターン 8.6 — 不均衡を「補正」して確率を壊す

クラスの偏りが気になると、少数クラスを水増ししたり (オーバーサンプリング、SMOTE)、多数クラスを間引いたり (アンダーサンプリング) したくなる。

補正すると、モデルが学ぶ**事前の割合**が変わる。本来 100 件に 1 件しか起きないことを、半々のデータで学ばせれば、出てくる確率は系統的に大きくなる。較正が壊れる。

医療の予測モデルで検証した研究では、3 つの補正手法のいずれもが少数クラスの確率を強く過大推定し、それでいて AUC は補正なしのモデルと変わらなかった [61]。**順位付けの力は変わらず、確率だけが狂う**という結果である。

感度と特異度のバランスを取りたいだけなら、データをいじらずに**しきい値を動かせば**同じことができる (第 8.5 節)。そのほうが確率を壊さずに済む。

補正が必要になるのは、少数クラスの件数が絶対的に足りず、モデルがその存在をほとんど

学べないような場合に限られる。

8.7 過学習を目で見る

決定木の最大の深さを 1 から 20 まで変えながら、74 作品を 52 列で判別したときの、訓練データでの正解率と層別 5 分割交差検証での正解率を並べる。

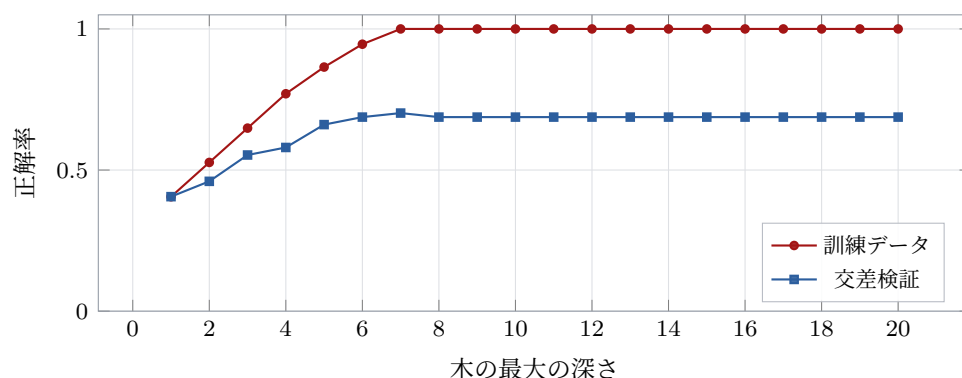


図 8.13 決定木の最大の深さと正解率。2 本の線が開いていく部分が過学習。

訓練データでの正解率は深くするほど上がり、1.000 に張り付く。交差検証での正解率は途中で頭打ちになる。**2 本の差**が過学習である。訓練データの正解率が上がり続けるのは良いことではなく、モデルが暗記を始めた合図である。見るべきは差であって、上の線ではない。

同じことが k 近傍法でも起きるが、向きが逆になる。横軸を近傍数 k に取り替えて、同じ 2 本の線を描く。

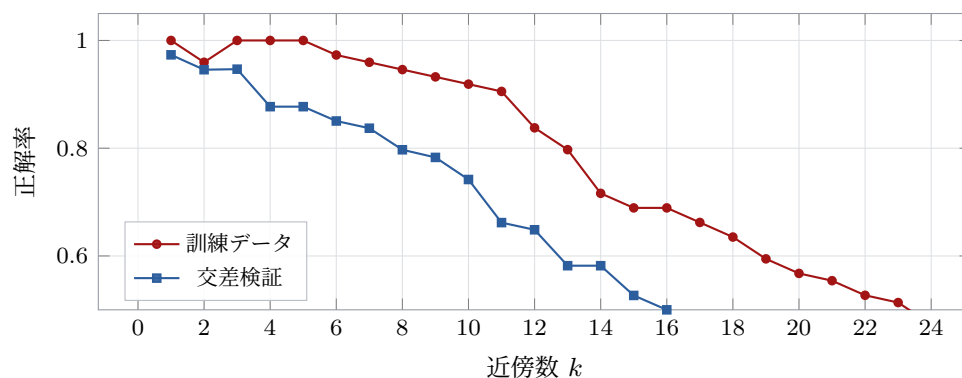


図 8.14 k 近傍法の近傍数 k と正解率。 k が小さいほど複雑なモデルになる。

$k = 1$ が最も複雑で、 k を大きくするほど単純になる。決定木の深さとは向きが逆である。

木の深さや近傍数 k のように、モデルが学習で決めるのではなく人が外から与える設定値をハイパーパラメータ (hyperparameter) という。学習率、正則化の強さ、クラスタ数 k 、t-SNE の perplexity も同類である。モデルの複雑さは、たいていハイパーパラメータで決まる。

× アンチパターン 8.7 — 「パラメータを大きくすると複雑になる」と覚える

そう覚えていると、 k 近傍法で向きを取り違える。

決定木では分岐の数、 k 近傍法では**近傍の狭さ**、正則化つき回帰では**正則化の弱さ**が複雑さを増やす。ハイパーパラメータの値と複雑さの向きは、手法ごとに確かめるしかない。

害は、**過学習を直そうとしてかえって複雑にしまうこと**である。訓練データだけ成績が高いのを見て「単純にしよう」と k を小さくすれば、境界はさらに細くなり、過学習は強まる。正しくは k を大きくする。値を変える前に、それを増やすと複雑さが増すのか減るのかを確かめる。

8.8 学習曲線 — 作品を増やせばまだ伸びる

「精度が足りない。データを増やすべきか、特徴を見直すべきか」という判断に使うのが**学習曲線**(learning curve)である。

訓練に使う作品数を変えながら、ランダムフォレストの正解率の推移を見る。

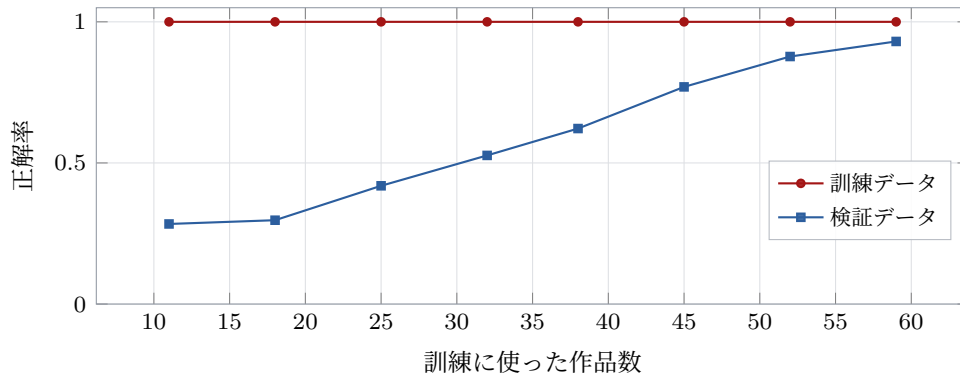


図 8.15 学習曲線 (74 作品、52 列、ランダムフォレスト、層別 5 分割交差検証)。検証データでの正解率の線がまだ上を向いている。

見るのは検証の線の向きである。

- 検証の線が**まだ上を向いている** — データを増やす価値がある
- 検証の線が**水平になった** — 増やしても頭打ち。特徴やモデルを見直すべき
- 2本の線が**大きく開いたまま** — 過学習。正則化を強める
- 2本とも**低い位置で重なる** — 未学習。モデルが単純すぎる

図 8.15 では、訓練に使った作品数が最大の 59 作品になるまで、検証データでの正解率が上がり続けている。作品数を増やせばまだ正解率は伸びる、と読める。

○ 定石 8.8 — モデルをいじる前に学習曲線を描く

モデルを取り替えたりパラメータを調整したりする前に、学習曲線を描く。描けば「データが足りないのか、モデルが合っていないのか」が分かる。

データ不足が原因なのにモデルをいじり続けて時間を失う、というのはよくある失敗である。

逆に、線が頭打ちになっているのにデータを集めに行くのも同じ失敗である。
2本の線をどう読むかは、第9.12節で偏りとばらつきの言葉に置き直す。

8.9 評価の手順をまとめる

図8.16に、ここまでの手順をまとめる。

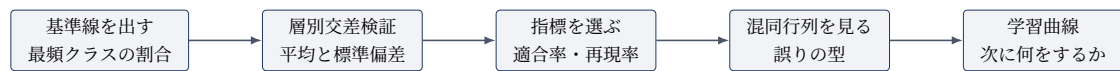


図8.16 評価の手順。どれも省略しない。

残っているのは、次章で扱う問いである。層別5分割交差検証で測ったランダムフォレストの正解率0.987が正しく測られたものかどうかを、そこで確かめる。

8.10 章末の課題

1. 交差検証の分割数を3、5、10、74(1つ抜き)と変え、平均と標準偏差がどう変わるか調べよ。分割数を増やすと、推定の偏りと分割間の標準偏差はそれぞれどちらへ動くか。
2. 表8.2で、谷崎潤一郎の再現率だけが0.833である。混同行列と照らして、何がどう間違えられたか特定せよ。
3. 適合率を重視すべき場面、再現率を重視すべき場面を、著者判別の文脈でそれぞれ考えよ。
4. 図8.15の検証曲線を延長すると、どこで頭打ちになると予想するか。その予想を確かめる方法はあるか。
5. クラスごとの F_1 を $F_2(\beta = 2, \text{再現率重視})$ で計算し直し、順位の変わる著者を探せ。(ヒント: 適合率と再現率の開きが大きい著者ほど大きく変わる。 F_β の式は F_1 の脚注にある。)

第9章

罣を避ける

9.1 出来すぎた結果を疑う

前章までで、ランダムフォレストが74作品の著者を層別5分割交差検証で0.987の正解率で当てるところまで来た。8名の著者を、たった52個の数値ではほぼ完璧に言い当てている。

こんなときは、まず疑う。

機械学習の失敗の多くは、モデルが弱すぎるのではなく**評価が甘すぎる**ことから来る。本番に出して初めて、交差検証で見た成績が出ないと気づく。そのときには手遅れである。

とてもいい性能が出たときは、
過学習かデータ漏洩を疑う。

本章では、甘い評価を生む漏れを一つずつ実際に測る。そのあとで、漏洩とは限らない落とし穴に移る。訓練で見た範囲の外に出られないモデルがあること(第9.10節)、漏洩と交絡と評価設計の不備が1つの事例の中で重なった例(第9.11節)、予測誤差が3つの部分に分かれること(第9.12節)、次元が上がると距離が使いものにならなくなること(第9.13節)、そして生成AIに分析を任せるときに固有の罣があること(第9.14節)である。

9.2 データ漏洩 — 本番では手に入らない情報が訓練に混ざる

訓練の時点で使えるはずのない情報が、モデルに入り込むことを**データ漏洩 (data leakage)**という。

漏れているかどうかの判断基準は一つである。

本番でこのモデルに新しいデータが来たとき、訓練時に使ったのと同じ情報が手に入るか。

答えが「いいえ」なら漏れている。テストデータの平均値も、テストデータの正解ラベルも、本番の時点では存在しない。

9.2.1 漏洩の分類

Kapoor と Narayanan は漏洩を8つの型に整理した [56]。表9.1は、その整理のうち本書で扱うものを土台に、彼らの分類にはない**多重テストの漏れ**を本書が加えたものである。

表 9.1 データ漏洩の型

型	内容
前処理の漏れ	標準化・欠損補完などを分割前に行う
特徴選択の漏れ	全データを見て特徴を選んでから分割する
グループの漏れ	同じ元から来たデータが訓練とテストに分かれる
多重テストの漏れ	同じデータで何度も試し、最良の結果を報告する
ターゲットの漏れ	目的変数から派生した変数、予測時には手に入らない変数を使う
時系列の漏れ	未来のデータで訓練し、過去を予測する

補足 漏洩は珍しい事故ではない

Kapoor と Narayanan の調査では、**17 分野・294 本の論文**に何らかの漏洩が見つかった。医療、政治学、分子生物学など、分野を問わず起きている。

多くは悪意ではなく、手順の順番を一つ間違えただけである。

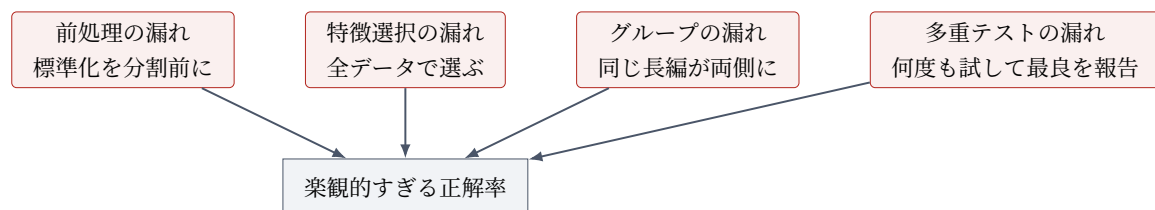


図 9.1 本章で測る漏れのうち代表的な 4 つ。どれも数字を良い方向にだけ歪める。

9.3 前処理の漏れ

標準化は平均と標準偏差を使う。2 つの値を**全データ**から計算すると、テストデータの分布が訓練に混ざる。

```

# 悪い例: 全データで fit してから交差検証
X_scaled = StandardScaler().fit_transform(X)
score = cross_val_score(KNeighborsClassifier(5), X_scaled, y, cv=cv)

# 正しい: Pipeline に入れると fold ごとに訓練データだけで fit される
pipe = make_pipeline(StandardScaler(), KNeighborsClassifier(5))
score = cross_val_score(pipe, X, y, cv=cv)

```

どれだけ漏れるのかを測る。k 近傍法 ($k = 5$) で 74 作品の著者を判別し、分割の乱数を 50 通り変えて、5 分割交差検証の正解率を求めた。漏れた手順の正解率から正しい手順の正解率を引いた差を、以下**水増し**と呼ぶ。正なら漏れたほうが良く見えたことになる。

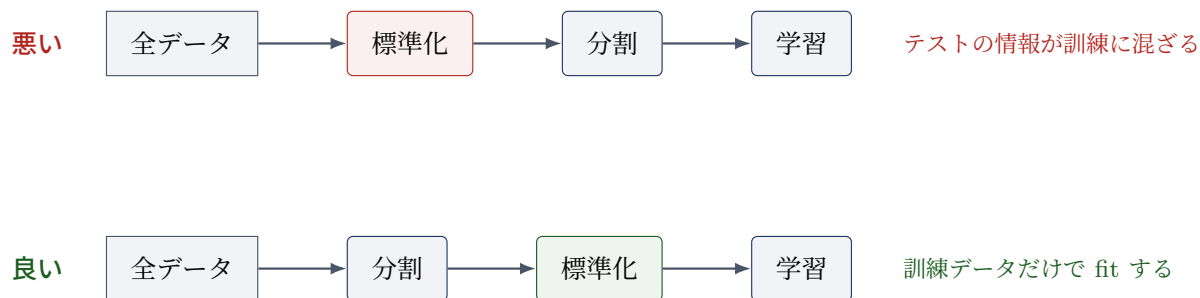


図 9.2 標準化と分割の順序。一つ入れ替えるだけで漏洩になる。

表 9.2 前処理の漏れ (k 近傍法 $k = 5$ 、分割の乱数 50 通りの平均)

手順	正解率の平均	標準偏差
漏れている (分割の外で標準化)	0.923	0.014
正しい (分割の内側で標準化)	0.921	0.022
見かけ上の水増し	+0.002	0.021

水増しの平均は正解率にして +0.002 である。50 通りの水増しの標準偏差は 0.021 で、平均の 10 倍を超える。50 通りのうち、漏れた側の正解率が高かったのは 50%、両者が一致したのが 18%、そして正しく測ったほうが高かったのが 32% である。水増しは -0.053 から $+0.056$ までの幅で散らばる。

つまりこの規模のデータでは、標準化だけの漏れは分割の揺らぎに埋もれる。乱数の種を 1 つだけ選べば水増し +0.029 という値も出るが、それは 50 通りの分布の中の 1 点にすぎない。この数字は、本章の主題である漏洩の代表値にはならない。

補足 今回の作品データで標準化の漏れが小さい理由

標準化が使うのは列の平均と標準偏差である。これはラベルをまったく見ていない。テストデータが訓練に持ち込むのは「その列の値がどのあたりに散っているか」という情報だけで、答えそのものではない。

しかも、テストの 1 行が平均に口を出せる割合は $1/n$ しかない。この実験では、標本を減らしたときの差も測った。

標本数	分割の外で	内側で	差
74	0.989	0.989	-0.000
40	0.967	0.959	+0.008
24	0.954	0.935	+0.020

74 作品では正解率に差が出ないが、24 作品まで減らすと水増しは正解率にして +0.020 になる。小さなデータでこそ、順序を守る意味がある。

9.3.1 標準化だけでも大差が出る場合

作品データでは、標準化にテストデータを含めても正解率の差は小さかった。次に、訓練データとテストデータで値の分布が異なる場合を調べる。

2 クラスの人工データを、訓練用とテスト用に 40 点ずつ作る。各クラスは 20 点ずつである。横軸 x_1 は、どちらの集合でも青が -1 、橙が $+1$ の付近にある。縦軸 x_2 は、訓練側では青が $+1$ 、橙が -1 の付近にあるが、テスト側では関係を逆転させ、青を -10 、橙を $+10$ の付近に置く。測定環境の変化によって、 x_2 の値が大きくなり、二つのクラスの大小関係も逆転した状況を想定している。各点には小さな乱数のずれを加えた。

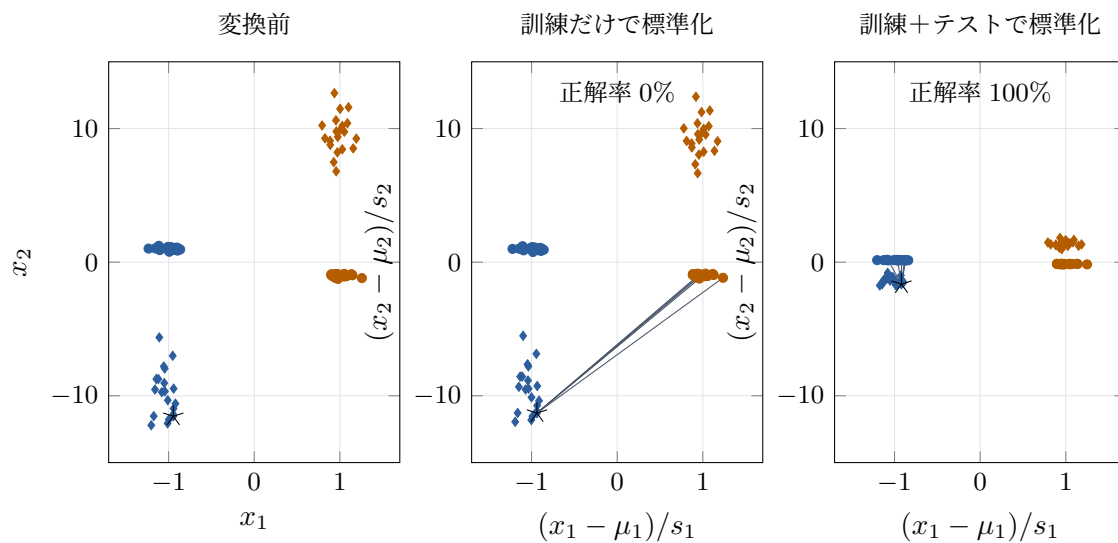


図 9.3 標準化の漏れを調べる人工データの散布図。左は変換前、中央と右はそれぞれの手順で標準化した後の値を描いた。3 図の軸範囲はそろえてある。右では縦方向のばらつきが縮む。青と橙は正解クラス、丸は訓練点、菱形はテスト点。中央と右では、星で示した同じテスト点から近い訓練 5 点へ線を引いた。縦横の 1 単位の長さは異なるため、紙面上の線の長さが計算上の距離を表すわけではない。

二つの手順で、同じ訓練 40 点を使って k 近傍法 ($k = 5$) を学習させる。標準化に使う平均と標準偏差を訓練 40 点から計算した場合、テスト正解率は 0% だった。訓練とテストを合わせた 80 点から計算した場合は 100% になった。テストの正解ラベルはどちらの学習にも渡していない。違いは、標準化にテストの値を含めたかどうかだけである。

近い点が入れ替わるのは、距離の測り方そのものが変わるからである。標準化後のユークリッド距離の 2 乗は、各列の標準偏差を s_1, s_2 とすれば

$$d^2(\mathbf{x}, \mathbf{z}) = \frac{(x_1 - z_1)^2}{s_1^2} + \frac{(x_2 - z_2)^2}{s_2^2}$$

となる。同じ平均を引いた 2 点の差を取るので、平均は消える。訓練側だけでは $(s_1, s_2) = (1.016, 1.022)$ だが、テストを含めると $(1.016, 6.979)$ になる。横軸の標準偏差はほぼ変わらず、縦軸の差の 2 乗に掛かる重みが約 0.021 倍に小さくなる。

図の星から伸びる線を追う。中央では、縦方向で近い橙の訓練点を選んでしまう。右では、縦方向の差が距離に及ぼす影響が小さくなり、横方向で近い青の点が選ばれる。テストデータを使って標準

偏差を計算したために近傍が変わり、正解率が上がった。

この人工例では、標準化の手順による違いを示すため、訓練データとテストデータの分布を意図的に変えた。同じ分布から無作為に分けたデータでも、常にこれほどの差が出るとは限らない。

学習に使っていないデータをどれだけ正しく分類できるかを評価するには、標準化に使う平均と標準偏差を訓練データだけから計算し、テストデータにもその値を使う。テストデータを含めて計算すると、評価対象のデータに合わせて距離の測り方を変えることになる。この例で得られた 100% という正解率は、訓練データだけから学習した分類器の性能を表していない。

授業用デモでは、テストデータの x_2 の値を何倍にするかを変え、標準偏差と近傍の変化を確認できる。

9.3.2 分割の前に済ませると漏れる処理

標準化だけではない。「データから何かを学ぶ」処理はすべて危ない。

表 9.3 分割の内側に入れるべき処理 (scikit-learn の例)

クラス	何を学ぶか
StandardScaler	平均と標準偏差
MinMaxScaler	最小値と最大値
SimpleImputer	欠損を埋める値 (平均・中央値)
SelectKBest	どの特徴を残すか
PCA	主成分の方向
SMOTE	少数クラスの合成サンプル
TargetEncoder	カテゴリごとの目的変数の平均

○ 定石 9.1 — 前処理は Pipeline に入れて分割の内側で fit する

Pipeline は書き方の好みではない。事故を構造的に防ぐための道具である。

前処理をパイプラインに入れておけば、交差検証は各分割で自動的に訓練データだけを使って fit してくれる。自分で順序を管理する必要がなくなる。

使う理由は、漏れが大きいからではない。作品データでは二つの手順による正解率の差は小さかったが、人工例では標準化に使うデータの範囲を変えるだけで 0% から 100% に変わった。それでも全部を内側に入れておくのは、**どれが危ないかを毎回判断しなくて済むから**である。上の表の StandardScaler を SelectKBest に書き換えただけで、正解率の水増しは +0.002 から +0.4 以上変わる (次節で測る)。その判断を書くたびに繰り返すより、順序のほうを固定しておくほうが安全である。

9.4 特徴選択の漏れ

次は、クラスの情報を持たない乱数だけで高い正解率が出る例を見る。

中身が**すべて乱数の特徴量**を 2000 列用意する。著者とは何の関係もない。ここから「著者との関

連が強い 30 列」を選び、その 30 列で交差検証する。

```
X_noise = rng.normal(size=(74, 2000))    # 情報量ゼロ

# 悪い例: 全データを見て30列選んでから交差検証
selected = SelectKBest(f_classif, k=30).fit(X_noise, y).transform(X_noise)
knn = make_pipeline(StandardScaler(), KNeighborsClassifier(5))
score = cross_val_score(knn, selected, y, cv=cv)

# 正しい: 選択もパイプラインに入れ、fold ごとにやり直す
pipe = make_pipeline(SelectKBest(f_classif, k=30),
                     StandardScaler(), KNeighborsClassifier(5))
score = cross_val_score(pipe, X_noise, y, cv=cv)
```

標準化はどちらも訓練側だけで行っている。違うのは特徴選択を分割の外でやるか中でやるかの 1 点だけである。

表 9.4 情報がゼロの乱数から得た正解率

手順	正解率
全データで特徴選択 → 交差検証 (漏れている)	0.570
交差検証の中で特徴選択 (正しい)	0.161
参考: 最頻クラスを答え続ける	0.284

情報が一切ない乱数から、5 分割交差検証の正解率が **0.570** になった。最頻クラスの吉川英治と答え続けるだけで得られる 0.284 の 2 倍である。

正しい手順に直すと、同じ乱数から得られる正解率は 0.161 まで下がる。8 名から等確率に選んだときの 0.125 をわずかに上回るだけで、最頻クラスを答え続ける 0.284 にも及ばない。情報がないのだから、これが正しい姿である。

9.4.1 漏れた側の正解率が 0.570 になる理由

2000 列も乱数を用意すれば、その中にはたまたま著者と相関する列が必ず紛れ込む。

全データを見て選ぶと、テストデータでもたまたま合う列を選んでしまう。そしてその列でテストすれば、当然よく当たる。

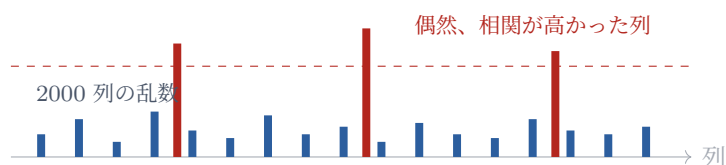


図 9.4 乱数を大量に用意すれば、偶然に相関の高い列が必ず出る。

正しい手順の正解率が 0.161 にとどまるのにも理由がある。選ばれた 30 列は、訓練データの中で

だけ著者と相関していた列である。テストデータに当てれば、その相関は消えて無関係な乱数に戻る。手がかりのない 30 列で距離を測ることになるので、当てずっぽうと変わらない成績になる。

× アンチパターン 9.1 — 特徴選択を前処理として分割の前で済ませる

そう考えると罣にはまる。基準は「前処理かどうか」ではない。

正解ラベル y を見る操作は、すべて分割の内側でなければならない。特徴選択、外れ値除去、しきい値の決定、クラスの再定義。どれも y を見た瞬間に漏洩の候補になる。

9.5 グループの漏れ

漏洩の中で最も見つけにくいのがこれである。

第1章の表 1.2 で見たとおり、吉川英治の 21 作品のうち 11 は『三国志』の各巻、7 は『宮本武蔵』の各巻だった。

74 作品と言っているが、独立した作品は 58 点しかない。『三国志』の 11 巻を 1 点、『宮本武蔵』の 7 巻を 1 点と数え直すと、吉川英治の 21 作品は 5 点になる。 $74 - 21 + 5 = 58$ という計算である。

交差検証は作品をランダムに分けるので、『三国志』の第 3 巻が訓練データに、第 4 巻がテストデータに入る。

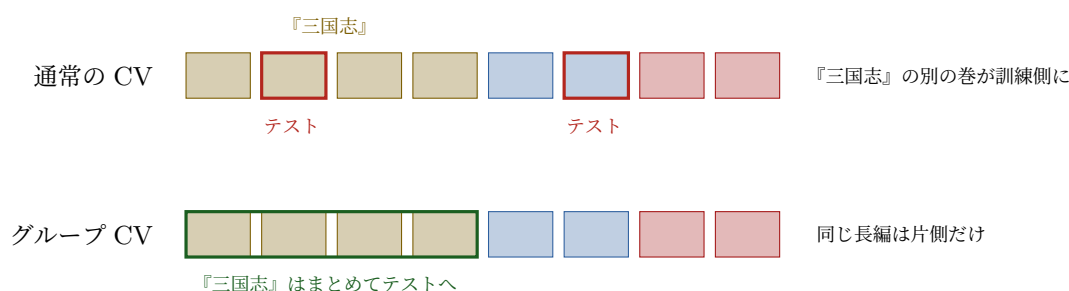


図 9.5 シリーズをまたぐ分割と、またがない分割。

第7章の末尾で予告したとおり、これでは「吉川英治の文体」ではなく「この長編に固有の癖」を覚えるだけで当たってしまう。ここでは、それが数字にしてどれだけの水増しになるかを測る。結果は予想と違う方向に出るので、先に測ってから読み方を考える。

StratifiedGroupKFold は、同じグループのデータが訓練とテストにまたがらないように分割する。表 9.5 の落差は、通常の層別 CV の正解率からシリーズを分けた CV の正解率を引いた差である。正なら、シリーズをまたいで分割したぶんだけ成績が良く見えていたことになる。

表 9.5 グループの漏れ (52 列を使った場合)

モデル	通常の層別 CV	シリーズを分けた CV	落差
k 近傍法 ($k = 5$)	0.877	0.900	-0.023
ランダムフォレスト	0.987	0.975	+0.011

正解率の落差は拍子抜けするほど小さい。ランダムフォレストは通常の層別 CV の 0.987 からシ

リーズを分けた CV の 0.975 へ、0.011 下がただけである^{*1}。k 近傍法にいたっては落差が負で、シリーズを分けたほうの正解率が 0.877 から 0.900 へむしろ高い。

これは失敗ではない。第 3 章の特徴設計が狙いどおりに働いた結果である。52 列は読点の前の文字と字種の比率と文長しか見ていない。そこには『三国志』という物語も、登場人物の名前も入っていない。覚え込むべき「その長編に固有のこと」が、そもそも特徴に含まれていない。

k 近傍法の落差 -0.023 のほうは、意味を読まないほうがよい。74 作品を判別しているので、1 作品の当たり外れが正解率 0.014 に相当する。落差の 0.023 は 2 作品ぶんに満たない。落差が小さいときは符号が反転することもあるという、測り方のばらつきの範囲である。

× アンチパターン 9.2 — 落差が小さいことを「漏れない」の証明として使う

表 9.5 を見て「本書のデータにグループの漏れはない」と結論するのは飛躍である。言えるのはこの特徴の組み合わせでは目立たなかったことだけである。

同じデータでも、特徴を変えれば落差は大きくなる。次節で文字 n -gram に取り替えると、4-gram では落差が正解率にして 0.036 と 3 倍以上に開く。第 11 章でライトノベルを足すと、落差は 0.43 に達する。

漏れの有無を決めるのはデータの集められ方と特徴の作り方であって、モデルでも手法でもない。一度測って小さかったからといって、特徴を変えたあとも小さいままだとは限らない。

9.5.1 文字認識の教科書的な例

グループの漏れを最も明快に示すのが、次の設定である。

100 人に、数字を 0 から 9 まで各 100 回ずつ書いてもらう。データ数は $100 \times 10 \times 100 = 100,000$ 枚。これを訓練用とテスト用にランダムに分割する。

一見なんの問題もない。だが同じ人が書いた文字が訓練とテストの両方に入る。

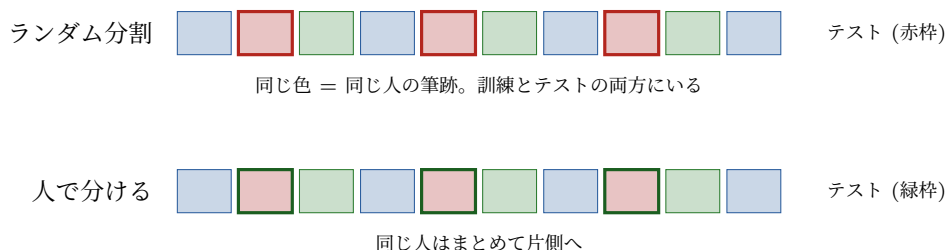


図 9.6 文字認識でのグループ漏れ。人の筆跡の癖を覚えれば当たってしまう。

モデルは「数字の形」を学ばなくても、「この人の筆跡の癖」を覚えれば当てられる。だが本番で使うときには、訓練に参加していない人が書く。

分割は要求仕様に沿って設計する。「未知の人が書いた文字を読みたい」なら、人で分けなければならない。

^{*1} 落差は丸める前の正解率どうしの差である。表示した小数第 3 位の値で引き算すると 0.012 になるが、丸めの影響にすぎない。

9.5.2 Kaggle での実例 — 岩塩層の識別

TGS Salt Identification Challenge[55](2018) は、音波探査の画像から岩塩層を識別する競技だった。訓練画像 4,000 枚、検証画像 18,000 枚、それぞれ 101 × 101 ピクセル。
参加者が気づいたのは、これらの画像がもっと大きな画像を切り分けたタイルだということだった。

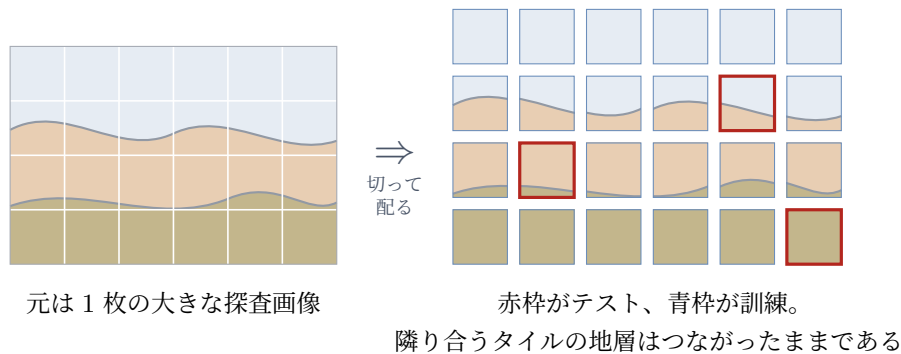


図 9.7 1 枚の画像をタイルに切り分けて訓練とテストに配ると、テストのタイルの地層が、隣の訓練タイルの続きになってしまう。隣を見れば答えが推測できる。

隣り合うタイルは地質構造が連続しているので、片方を見ればもう片方もほぼ分かる。上位陣はタイルをパズルのように組み直し、隣接の情報を利用した。
競技としては巧妙な戦略だが、実務なら評価設計の失敗である。

9.5.3 グループはどこにでもある

表 9.6 グループにすべき単位の例

場面	グループの単位
同じ患者から複数回の検査データ	患者 ID
同じ被験者の複数試行	被験者 ID
同じ人が書いた文字	筆記者 ID
同じ工場・同じロットの製品	ロット番号
同じ長編の各巻、同じ雑誌の各号	作品・媒体
大きな画像を切ったタイル	元画像
同じ都市の複数年のデータ	都市

共通しているのは、行が独立していないという点である。交差検証は行が独立に取られたことを前提にしている。その前提が崩れれば、結果も崩れる。

× アンチパターン 9.3 — 行数をデータ量だと思う —

74 行あるからサンプル数 74、とは限らない。巻の重複を避けるためにまとめると 58 群しかなかった。別の長編どうしが独立かどうかまでは確かめていないので、「独立サンプルが 58 点」

とも言い切れない。群の数は、独立な件数の上限の目安である。

データを受け取ったら、まず行がどうやって作られたかを確認し、同じ元から複数行が生まれていないかを見る。

補足 行を増やしても偏りは薄まらない

「データが少ないから偏る」という言い方をよく聞く。半分は正しいが、半分は違う。

行を増やすと、**偶然のばらつき**は小さくなる。信頼区間は縮み、推定は安定する。だが**集め方の偏り**は、行をいくら増やしても縮まない。偏った集め方で 100 万件集めれば、偏った答えを狭い信頼区間で得ることになる。自信を持って間違えるので、かえって始末が悪い。

2021 年に報告された例がある。週に 25 万件を超える回答を集めた大規模なオンライン調査が、米国のワクチン接種率を公式統計より 17 ポイントも高く見積もった。同じ時期、回答者 1,000 人ほどの伝統的な世論調査は、誤差 5 ポイント以内に収まっていた [60]。250 倍の回答数が、精度では負けた。

差を生んだのは、**誰が回答したか**である。オンラインで自ら回答する人と、無作為に選ばれて依頼される人とは、集団の性質が違う。その違いは件数では埋まらない。

本書のコーパスも同じ性質を持つ。青空文庫から作品をもう 100 点足しても、第 1 章で見た 2 つの選抜 (著作権と電子化) は残ったままである。

9.6 特徴の作り方が漏れを招く — n-gram の罠

ここまでは分割の仕方の問題だった。だが**特徴の設計そのもの**が漏れを引き起こすこともある。

第 3 章で、内容語を避けて文字ベースの特徴を作った。もし手軽に精度を上げようとして、第 2.5.1 項の文字 n-gram を使ったらどうなるか。

```
pipe = make_pipeline(
    CountVectorizer(analyzer="char", ngram_range=(3, 3), max_features=500),
    Normalizer(norm="l1"),
    RandomForestClassifier(n_estimators=300, random_state=0))
score = cross_val_score(pipe, texts, y, cv=cv)
```

頻度上位の 3 文字の並びを 500 種類選び、作品ごとの頻度を行和 1 に正規化して森に渡す。**500 種類を選ぶ操作もパイプラインの中**に入れてある。どの並びを語彙にするかは訓練側の頻度で決まるので、検証側の作品を見ずに語彙を作ることになる (この理由は本節の終わりで扱う)。

表 9.7 n-gram 特徴の成績 (ランダムフォレスト)。語彙の選択は交差検証の各分割で訓練側だけから行った

特徴	通常の層別 CV	シリーズを分けた CV	落差
文字 2-gram (上位 500)	0.973	0.983	−0.010
文字 3-gram (上位 500)	0.987	0.969	+0.018
文字 4-gram (上位 500)	0.960	0.924	+0.036
本書の 52 列	0.987	0.975	+0.011

3-gram の正解率は通常の層別 CV で 0.987 と、本書の 52 列と並んでいる。シリーズを分けた CV では 0.969 で、落差 0.018 は 52 列の 0.011 より大きい、1 作品ぶん (0.014) と変わらない程度である。4-gram になると落差は 0.036 に開く。並びが長いほど、その作品にしか出ない文字列を拾いやすくなる。

正解率だけ見れば、n-gram は本書の 52 列と同程度に使いそうに見える。そこで、モデルが何を見ているかを調べる。ランダムフォレストの重要度が高い 3-gram を並べると、「また」、「が改行、「れども」といった文法的なものに混じって、**漢字 3 文字の並び**が現れる。

表 9.8 ランダムフォレストの重要度が高い 3-gram のうち、漢字 3 文字が連続するもの

3-gram	正体	出典
三四郎	主人公の名前	夏目漱石『三四郎』
小次郎	佐々木小次郎	吉川英治『宮本武蔵』
城太郎	登場人物	吉川英治『宮本武蔵』
左衛門	人名の一部	複数作品
三千代	登場人物	夏目漱石『それから』
万太郎	登場人物	吉川英治『江戸三国志』

登場人物の名前である。

モデルは文体を学んでいない。「『小次郎』が出てくる文書は吉川英治」という規則を覚えている。これは第 2 章で警告した「内容語による分類」そのものである。

× アンチパターン 9.4 — 何を当てたいかを考えずに n-gram を入れる

n-gram は強力で、実装も簡単である。だから安易に使われる。

問題は n-gram そのものではなく、**当てたいものと、n-gram が拾うものが食い違う**ことにある。著者を当てたいのに、文字 n-gram が拾ってくるのは固有名詞だった。「小次郎」を覚えたモデルは、その作品を覚えてだけで、著者の文体については何も学んでいない。未知の作品には別の登場人物が出てくる。

当てたいものが変われば、判定も変わる。

- 話題やジャンルを当てるなら、固有名詞は邪魔者ではなく主戦力である (第 11 章)。表 9.8 の「小次郎」は、時代物という区分の手がかりとしては正しく働く
- 言語判定、文字コードの推定、スパム判定でも n-gram は素直に使える。拾ってほしいものと拾ってくるものが一致しているからである
- 著者判別で n-gram を使うなら、機能語だけの n-gram、品詞の n-gram、漢字列を除いた文字 n-gram のように、内容語が入らない**制限付きの形**にする

どの場合でも、特徴を作ったら**モデルが何を見ているか**を確認する。特徴重要度を上位 20 個ほど眺めるだけで、多くのことが分かる。表 9.8 も、それをやったから出てきたものである。

補足 語彙を全作品で選ぶのも漏れである

表 9.7 では、上位 500 種類の選択を交差検証の内側に入れた。`CountVectorizer(max_features=500)` を全作品に `fit` してから交差検証に渡すと、どの並びを語彙にするかに検証側の作品の頻度が入る。特徴選択を分割の前で済ませると同じ形の漏れである (第 9.4 節)。

本書の初期の版では、この語彙選択を全作品で行っていた。そのときの 3-gram のシリーズを分けた CV は 0.938 で、訓練側だけで語彙を選ぶ現在の手順では 0.969 である。**漏れを直したら正解率が上がった**。検証側の作品に多い並びが語彙に混じると、その作品に固有の文字列が列を占め、訓練側の作品には役立たない列になる。漏れの向きは常に楽観側とは限らない (第 9.3 節の標準化と同じである)。だから、正解率が上がったか下がったかでは漏れの有無を判定できない。判定の根拠は数字ではなく**手順**にある。

9.6.1 ライトノベルではもっと極端になる

同じことをライトノベル 60 区間 (6 シリーズ) で試すと、事態はさらに悪化する。

表 9.9 3-gram 特徴、ライトノベルでの著者判別

対象	通常の CV	シリーズを分けた CV
ライトノベル 6 名	1.000	0.000
近代文学 + ラノベ 14 名	0.985	0.560

通常の CV で 1.000 だった正解率が、**シリーズを分けると 0.000 になった**。

理由は 2 つある。一つは、ライトノベルは長期シリーズが中心で、6 名がそれぞれ 1 つのシリーズしか書いていないこと。つまり**著者とシリーズが 1 対 1 に対応している**。シリーズで分割すると、テストに現れる著者の作品が訓練データに 1 つも存在しない。当たるはずがない。

もう一つは、重要度の高い 3-gram に「猫猫」が入っていたこと。『薬屋のひとりごと』の主人公の名前である。

補足 著者ごとにシリーズが 1 つしかないと正解率は 0.000 になる

「正解率ゼロ」は失敗した実験ではない。**このデータでは著者判別を学習できない**という事実の発見である。

著者ごとに 1 シリーズしかないなら、「著者の文体」と「シリーズの世界観」を切り分ける手立てがない。必要なのはより良いモデルではなく、より良いデータである。

第 11 章では、ライトノベルで何なら学習できるかを考える。

9.7 ハイパーパラメータ選択の漏れ

もう一つ、見落とされやすい漏れがある。

第 8 章で、 k 近傍法のハイパーパラメータ k を 1 から 25 まで変えて交差検証した (図 8.14)。最も良かったのは $k = 1$ で、そのときの正解率は 0.973 だった。

0.973 を「このモデルの性能」として報告することはできない。 k を選ぶのに、評価と同じデータを使っているからである。

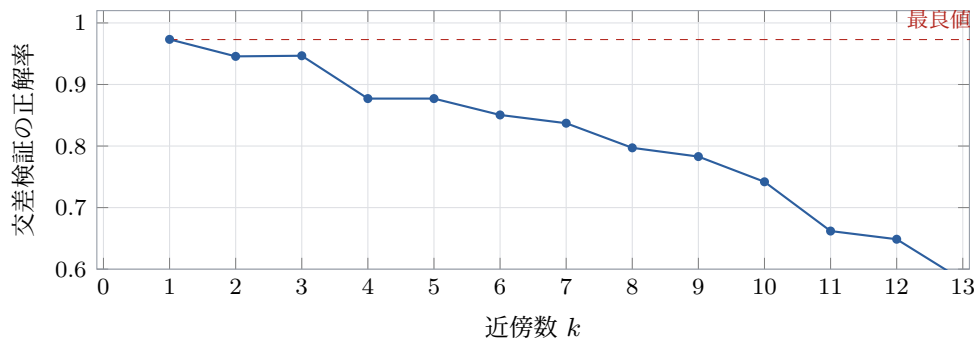


図 9.8 k を変えたときの交差検証スコア。最大値を性能として報告するのは漏洩である。

25 通り試して最も良いものを選んだのだから、その値は偶然の上振れを含んでいる。

正しい手順は入れ子交差検証 (nested cross-validation) である。

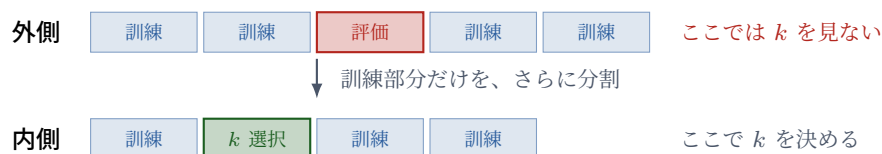


図 9.9 入れ子交差検証。外側で性能を測り、内側でハイパーパラメータを選ぶ。

```
# 内側: k を選ぶ
grid = GridSearchCV(
    make_pipeline(StandardScaler(), KNeighborsClassifier()),
    {"kneighborsclassifier__n_neighbors": range(1, 26)},
    cv=StratifiedKFold(index{StratifiedKFold}(5, shuffle=True, random_state=1),
)
# 外側: 性能を測る。grid 自体をモデルとして渡す
score = cross_val_score(grid, X, y, cv=StratifiedKFold(5, shuffle=True, random_state=0))
```

表 9.10 ハイパーパラメータ選択と評価の分離

手順	正解率
同じ CV で k を選び、最良値を報告 (漏れている)	0.973
入れ子交差検証 (正しい)	0.973
入れ子 + シリーズを分ける (さらに正しい)	0.911

本書のデータでは、入れ子にしても正解率がほとんど変わらなかった。どの分割でも $k = 1$ が最良だったので、内側で選び直しても同じモデルに行き着くためである。差が出ないこともあるというのも大事な観察である。

ただし、差が出なかったのは本書のデータが素直だからで、手順が問題ないことの証明ではない。

差が出る状況は人工的に作れる。特徴を**情報ゼロの乱数 60 列**に差し替えて (次節と同じ道具立てである)、 k ・距離・重み付けの組を同じ手順で選んでみる。

表 9.11 情報ゼロの乱数 60 列での比較。漏れた手順だけが基準線を超える

手順	正解率
同じ CV で設定を選び、最良値を報告 (漏れている)	0.310
入れ子交差検証 (正しい)	0.243
参考: 基準線 (最頻クラス)	0.284

正解率の差は 0.310 対 0.243 で、0.067 しかない。だがこの 0.067 が**基準線 0.284**をまたいでいる。漏れた手順は「乱数に予測力がある」と報告し、入れ子は「情報はない」と正しく報告する。結論の向きが逆になる。

だが差が出るかどうかを事前には知ることができない。入れ子にして初めて分かる。

9.8 多重テストの漏れ

前節で試したのは k の 25 通りだった。もっと試す回数を増やすと、事態は変わる。

k だけでなく、距離の種類 (ユークリッド・マンハッタン・チェビシェフ)、重み付け (一様・距離の逆数) も変えてみる。 k を 8 通り取れば $8 \times 3 \times 2 = 48$ 通りである。ただし特徴量は**情報ゼロの乱数 60 列**にする。

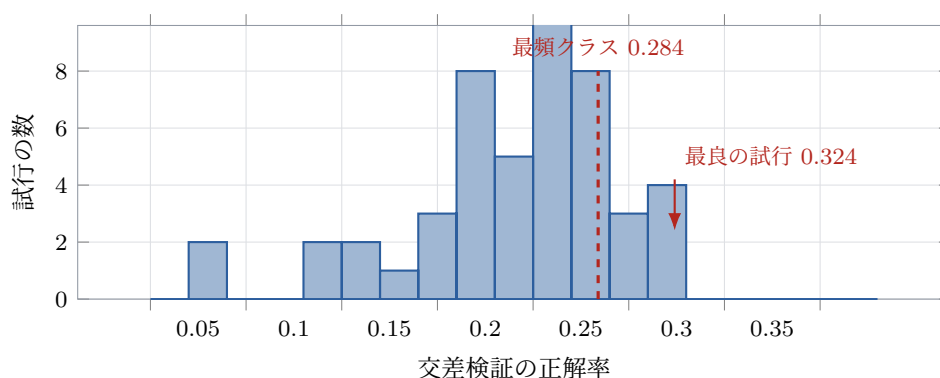


図 9.10 情報ゼロの乱数 60 列で 48 通りの設定を試したときの、5 分割交差検証の正解率の分布。

48 通りの設定それぞれについて、74 作品の著者を 5 分割交差検証で判別したときの正解率を測った。48 通りの平均は 0.242 で、最頻クラスを答え続ける基準線 0.284 を下回っている。乱数の列には著者を見分ける情報がないのだから、これは当然の結果である。

ところが 48 通りのうち最も良かった設定では、正解率が 0.324 になった。基準線を 0.040 上回っている。

「マンハッタン距離を使い、 $k = 21$ 、一様な重みという設定で正解率 0.324 が得られました」と報告すれば、それらしく聞こえてしまう。48 通り試したうちの 1 つだと書かなければ、読み手には見分けようがない。

どれくらい起きやすいかは計算できる。1回の試行が偶然に「当たり」になる確率を5%とすると、 n 回試して1回も当たらない確率は 0.95^n である。少なくとも1回当たってしまう確率は $1 - 0.95^n$ になる。なお48通りの試行は同じデータを使い回して独立ではないので、この値は厳密には成り立たないが、試すほど当たりやすくなるという方向は変わらない。

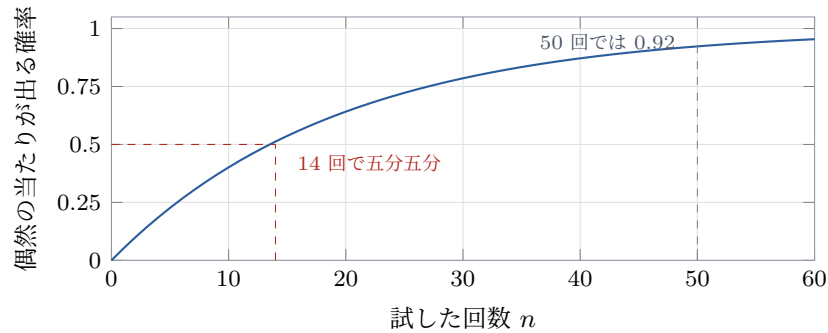


図 9.11 1回あたり5%の確率で偶然に当たる試行を、 n 回繰り返したとき、少なくとも1回当たってしまう確率。14回でもう五分五分になり、50回では0.92に達する。「いろいろ試したら1つうまくいった」は、試した回数を知らなければ評価のしようがない。

上で試した48通りは、この図の右端に近い。最良の設定が最頻クラスを超えたのは、たまたまではなくそうなるまで試したからである。

× アンチパターン 9.5 — 試行回数を報告しない —

何通り試したかを書かずに最良の結果だけを報告するのは、統計でいう**多重比較の問題**である。

同じデータに何度も問いを投げれば、そのうち偶然に良い答えが返ってくる。
対策は次のとおりである。

- 入れ子交差検証で、選択と評価を分ける
- 試行回数を明記する
- 最終確認用のデータを完全に取り分けておき、一度だけ使う

補足 多重テストの漏れは AI との対話でも起きる

「もう少し精度を上げて」と AI に繰り返し依頼して調整を続けると、テストデータが実質的に選択基準になっていく。

これも多重テストの漏れである。人間が手で調整しても、AI に任せても、同じデータを何度も見た事実は変わらない。

9.9 そのほかの漏れ — ターゲットと時系列

9.9.1 ターゲットの漏れ

目的変数から派生した変数、あるいは**予測の時点ではまだ存在しない変数**を使ってしまう型である。

- クレジットカードの不正検出で「返金の有無」を特徴にする。返金は不正と判定された**後**で発生する
- 病気の予測に「その病気の治療薬の処方歴」を使う
- 退職予測に「引き継ぎ資料の作成有無」を使う

補足 KDD Cup 2008 の患者 ID

マンモグラフィ画像から乳がんを予測する競技で、予測に最も寄与した特徴は**患者 ID** だった。ID が検査機関ごとに連番で振られており、機関によって患者の症状の重さが違ったためである。ID という、本来なんの意味も持たないはずの数字に、答えが埋め込まれていた [54]。

9.9.2 時系列の漏れ

時間の順序があるデータをランダムに分割すると、**未来のデータで訓練して過去を予測すること**になる。

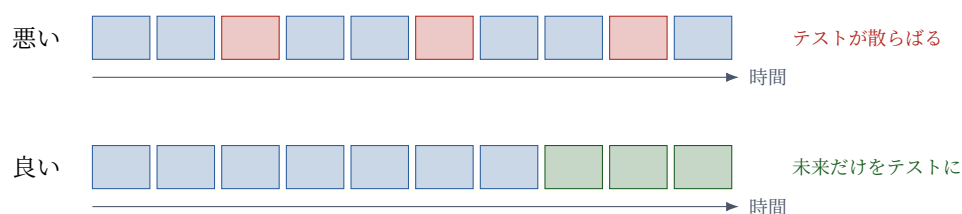


図 9.12 時系列データの分割。過去で訓練し、未来を予測する。

本書のデータで実演できる。『本好きの下剋上』は 677 話が 3.5 年かけて投稿された。各話の文体特徴 (字種・文長・読点) から、**その話がいつ投稿されたか**を Ridge 回帰 (第 4.4.2 節で触れた、正則化つきの線形回帰) で当ててみる。目的変数は連載開始からの経過日数である。

普通に 5 分割交差検証 (ランダム分割) をすると、決定係数 (第 8.4.2 項) は $R^2 = 0.77$ 。「文体から執筆時期が当てられる」ように見える。ところが**先の 451 話で訓練し、残りの 226 話を予測する** (時間順の分割) と、 $R^2 = -4.44$ 。平均を答え続けるより悪い。モデルの予測は訓練期間の終わり付近に張りついたままで、その先の伸びをまったく追えていない (図 9.13)。

からくりは**ドリフト** (drift) である。シリーズの文体は連載が進むにつれて少しずつ変わる。実際、列ごとに標準化した特徴ベクトルどうしのユークリッド距離は、隣り合う話では平均 9.0 なのに、投稿日が連載期間の 3 分の 1 以上離れた話の組では平均 10.7 になる。ランダム分割では、テストの話の**直前と直後の話**が訓練に入るので、モデルは両側からの内挿で答えを写せる。どのフォールドでも、テストの話より未来の話が訓練側に入っている。時間順に分けたとたん、写しができなくなって

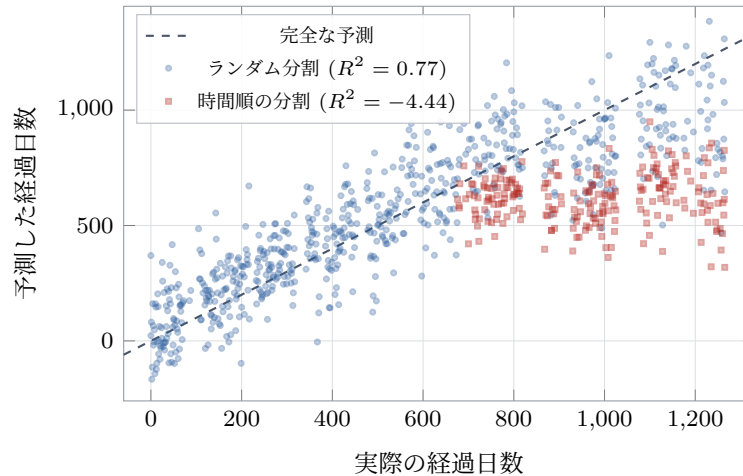


図 9.13 『本好きの下剋上』各話の投稿時期を文体から予測する。ランダム分割では対角線に沿うが、未来を知らずに予測した 226 話 (赤) は訓練期間の終わり付近に張りつく。

化けの皮が剥がれる。

× アンチパターン 9.6 — 未来で訓練して過去を予測する

時間の順序があるデータ (売上、センサ、経過観察、連載作品) をシャッフルして交差検証すると、検証成績は「未来を知っている場合」の成績になる。運用時のモデルは未来を知らないのだから、数字は再現されない。

質の悪いことに、ドリフトが強いデータほどランダム分割の成績は良く出る。隣接時点からの内挿で補えてしまうからである。「時系列なのに交差検証の成績が妙に良い」は、喜ぶところではなく疑うところである。

TimeSeriesSplit を使えば、常に過去で訓練し未来で検証できる。時間の順序を無視した分割は、Kapoor と Narayanan が整理した 8 つの型のうちの 1 つに数えられている [56]。売上予測、災害予測、疫学のモデルなど、時間とともに性質が変わるデータを扱う分野では繰り返し起きている。

9.10 訓練で見た範囲の外に出られないモデルがある

ここからは漏洩を離れる。分割を正しく設計しても残る落とし穴である。前節で時間順に分けたついでに、もう 1 つ確かめておきたいことがある。モデルによっては、そもそも未来の値を出せない。

同じ連載データで、先の 451 話で訓練して残りを予測する。使うモデルを Ridge 回帰・ランダムフォレスト・k 近傍法の 3 つにして比べた (図 9.14)。

訓練に使ったのは 675 日目までで、テストは最大 1265 日目までである。3 つの予測の最大値を並べる。

ランダムフォレスト 605 日 k 近傍法 630 日 Ridge 回帰 949 日

前の 2 つは訓練で見た範囲を 1 日も超えていない。

これは実装の都合ではなく、予測値の作り方から来る性質である。2 つとも、予測値を訓練データ

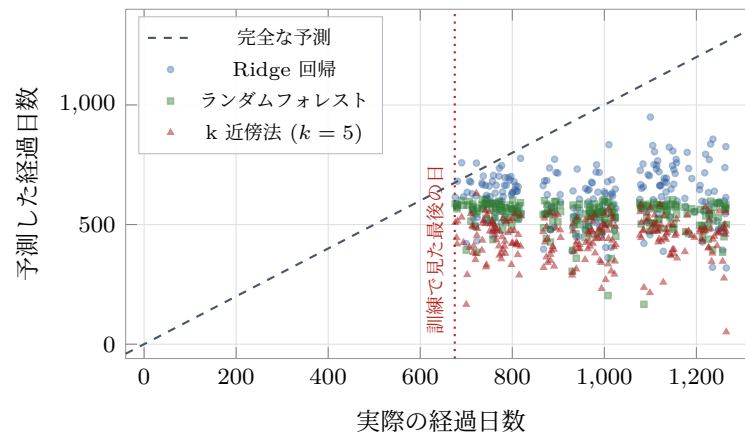


図 9.14 訓練期間より先の話を予測させた結果。ランダムフォレスト (緑) と k 近傍法 (赤) の予測は、訓練で見た最大値のあたりで頭打ちになる。

のラベルの平均として作っている。

- 決定木の予測値は、その葉に落ちた訓練データのラベルの平均である。森はその平均をさらに平均する
- k 近傍法の予測値は、最も近い k 件のラベルの平均である

平均は、元の値の最大値を超えない。どんな入力を与えても、訓練ラベルの最大値より大きな値は出てこない。入力が訓練データからどれだけ離れていても、返ってくるのは「いちばん端の訓練データの値」で頭打ちになる。

Ridge 回帰のほうは直線を延ばすので、範囲の外にも出られる。図でも訓練期間より先まで予測が伸びている。ただし 949 日どまりで、テストの最大 1265 日に及ばない。外に出られることと、外で当たることは別である。

× アンチパターン 9.7 — 訓練データの範囲の外を平均を返すモデルに予測させる

売上、株価、利用者数、累積の何か。上がり続けるデータを、木ベースのモデルや k 近傍法で予測すると、未来の予測は必ず頭打ちになる。モデルが慎重なのではなく、その値を出す能力がない。

見分け方は「予測値が訓練データのラベルの平均で作られているか」である。木、森、k 近傍法、カーネル回帰はこの側にある^a。

対処を挙げる。

- トレンドを先に取り除く (差分を取る、時間で割る)。残った変動だけを学ばせる
- 線形の項を持つモデルと組み合わせる
- そもそも外挿しない。訓練で見た範囲の中だけで使う

^a 勾配ブースティングは、初期値に木の出力を足し込んでいく形なので、原理上は訓練ラベルの範囲を超えられる。ただし範囲外の入力に対しては各木が端の葉を返すので、実際には同じように頭打ちになりやすい。

補足 予測が当たることと効果が測れることは違う

本書が扱ってきたのは、一貫して**予測**である。文体から著者を当てる、文体から時期を当てる。どれも「入力から出力を言い当てる」問題だった。

実務では、似て非なる問いが出てくる。**介入の効果を知りたい**という問いである。「この施策を打ったから売上が伸びたのか」「この機能を足したから利用者が増えたのか」。

2つは別の問題である。右肩上がりのデータで施策の前後を比べて「伸びた」と言っても、何もしなくても伸びていた分が混ざっている。本書の言葉でいえば、**時間そのものが交絡**している。

取り出すには、同じ時期に施策を受けなかった集団を用意して、**両者の変化の差**を取る。共通して起きた伸びが打ち消されて、施策を受けた側にだけ起きた分が残る。差の差 (difference-in-differences) と呼ばれる考え方である。

本書はここから先に踏み込まないが、**予測の精度をいくら上げても、効果の推定にはならない**。必要な道具立てがまったく違う。

9.11 COVID-19 診断モデルの教訓

ここまで個別に見てきた落とし穴は、実際の研究では1つの事例の中で重なる。漏洩・交絡・評価設計の不備が同時に起きた例を見ておく。

2020年から2021年にかけて、胸部X線やCTからCOVID-19を診断するモデルの論文が大量に発表された。

Roberts ら [57] は2,212本の論文から候補を絞り込み、最後に残った62本を精査したうえで、**臨床利用に耐えるものは一つもなかった**と結論している。

原因は漏洩に限らない。型の違う3つに整理できる。

- **Frankenstein データセット** (グループの漏れ) — 公開データが統合・再配布される過程で、同じ患者の画像が訓練と評価の両方に入った
- **子供の画像の混入** (交絡) — 陰性サンプルに小児科のデータが多く含まれていた。子供は大人よりCOVID-19にかかりにくいので、モデルが学んだのは「子供と大人の見分け方」だった。第11章の時代とジャンルの交絡と同じ構造である
- **外部検証の欠如** (評価設計の不備) — 別の病院のデータで確かめた研究はごく少数だった。1つの病院のデータだけで訓練と評価を済ませると、撮影装置や患者層の癖まで込みで学んでしまう

3つとも、モデルや手法の問題ではない。**データがどう集められたかを確認しなかった**ことから来ている。

補足 手元のデータで測れることには限りがある

外部検証がなぜ要するのかは、本書のデータで考えると分かりやすい。

本書は青空文庫の74作品で訓練し、同じ74作品を分けて評価している。どれだけ分割を厳しくしても、測れるのは「青空文庫のこのコーパスの中での成績」までである。

たとえば底本の入力者による表記の癖が全作品に共通していれば、モデルはそれも込みで学ぶが、同じコーパスで評価するかぎり、それは誤りとして現れない。別の出典から集めたテキストで試して、はじめて分かる。

補足（続き）

第11章でライトノベルを足したのは、話を広げるためだけでなく、別の出所のデータに当てるとどうなるかを見るためでもある。

9.12 誤差を3つに分ける — 偏りとばらつき

ここから2節も、漏洩とは別の落とし穴を扱う。まず、これまで別々に出てきた「深さ」「 k 」「正規化の強さ」「木の本数」を、1つの軸の上に並べ直す。

モデルの予測誤差は、3つの部分に分けられる。

- **偏り (bias)** — モデルの形が単純すぎて、真の関係を表せないぶん。データを取り直しても同じ向きに外れる
- **ばらつき (variance)** — 訓練データが変わると答えが変わってしまうぶん。平均すれば合っているが、1回ごとには外れる
- **雑音** — データそのものに乗っている、どうやっても説明できないぶん

言葉だけでは掴みにくいので、的当てにたとえる。的の中心が**真の値**、1発の矢が**1回ぶんの訓練データで作ったモデルの予測**である。訓練データを取り直すたびに矢を1本射ると思えばよい。

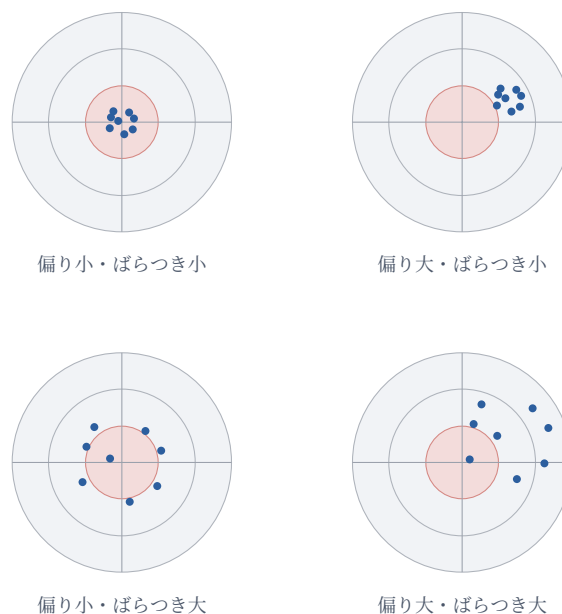


図 9.15 偏りとばらつきを的当てで見る。中心が真の値、点1つが1回ぶんの訓練データから得た予測である。偏りは点の集まりが中心からどれだけずれているか、ばらつきは点どうしがどれだけ散らばっているかにあたる。右上は毎回同じ答えを出すのが的を外し、左下は平均すれば中心に来るが1回ごとには大きく外れる。

図 9.15 の右上と左下に注目してほしい。どちらも当たっていないが、外し方が違う。右上は**何度やっても同じ場所に外れる**ので、データを増やしても直らない。モデルの形を変えるしかない。左下は**平均を取れば中心**なので、データを増やすか、複数のモデルを平均すれば近づく。ランダムフォレストが多数の木を平均するのは、深い木を左下から左上へ寄せる操作にあたる。

2乗誤差で測るとき、3つはちょうど足し算になる。

$$\text{予測誤差} = (\text{偏り})^2 + \text{ばらつき} + \text{雑音} \quad (9.1)$$

式(9.1)の3つが実際にどう振る舞うかを、人工のデータで測ってみる。本物のデータでは真の関数が分からないため、偏りとばらつきを別々に取り出せない。そこで真の関数をこちらで決めてしまう。 $f(x) = \sin x + 0.3x$ と定め、そこに標準偏差0.35の雑音を乗せた点を60個作る。これを訓練データとして決定木で当てはめ、**データを作り直して400回繰り返す**。各点における予測の平均が真の値からどれだけずれているかが偏り、400回の予測の**散らばり**がばらつきである。

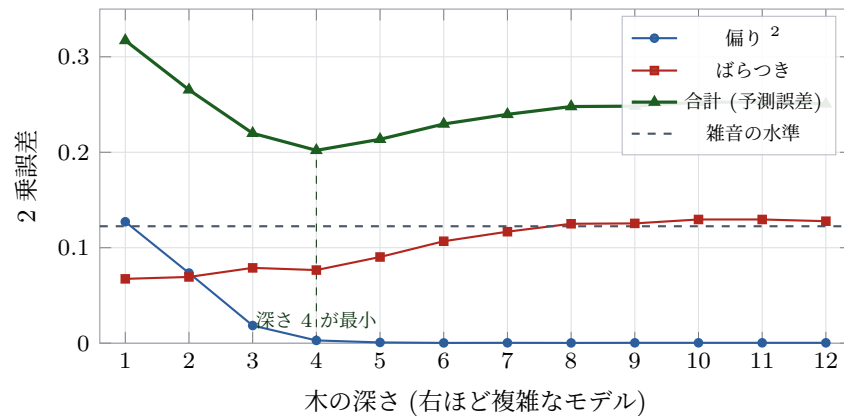


図 9.16 木の深さを変えたときの誤差の分解 (60 点のデータを 400 回作り直した)。偏りは深くするほど減り、ばらつきは増える。合計が最小になるのは、その途中である。

図 9.16 が式 (9.1) の 3 項である。

以下の値はすべて式 (9.1) の各項、つまり 2 乗誤差の単位である。

深さ 1 の木は、 x をある値で 2 つに割って、それぞれの平均を答えるだけである。 $\sin$ の曲線を 2 段の階段で近似することになるので、偏りの 2 乗が 0.1272 と大きい。一方で、データを取り直しても答えはあまり変わらないので、ばらつきは 0.0674 にとどまる。これが**未学習 (underfitting)** の状態である。

深さ 12 まで伸ばすと、偏りの 2 乗は 0.0004 とほぼ消える。真の曲線を細かく追えるようになったからである。ところがばらつきが 0.1278 まで増える。訓練データに乗った雑音まで木が拾うので、データを取り直すたびに違う形になる。これが**過学習**である。

合計が最小になるのは深さ 4、つまり両端ではなく途中である。そして雑音そのものが持つ 2 乗誤差 0.122 は、どこまで行っても引けない。**誤差をゼロにはできない**という下限がここにある。

9.12.1 これまで調整してきた値はすべて同じ軸の上にある

本書でこれまで調整してきた値を並べると、どれも図 9.16 の横軸にあたる。

第 3 章で「 $n - 1$ で割ると偏りは消えるが二乗誤差は最も大きい」と書いたのも、同じ綱引きの一例である (偏りをゼロにする代償として、ばらつきを引き受けている)。

アンサンブルの 2 つの流儀も、同じ軸で説明がつく。

表 9.12 偏りとばらつきの綱引き。左に行くほど偏りが大きく、右に行くほどばらつきが大きい

調整する値	偏りが大きくなる側	ばらつきが大きくなる側
木の深さ (第 8 章 図 8.13)	浅い	深い
k 近傍法の k (第 8 章 図 8.14)	k が大きい	$k = 1$
正則化の強さ (第 4 章 図 4.6)	α が大きい	α が小さい
特徴の数 (第 9.13 節)	少なすぎる	多すぎる
$n - 1$ か n か (第 3 章)	n で割る	$n - 1$ で割る

- ランダムフォレストは、似ていない木を並列に作って平均する。平均を取るとばらつきが減る。偏りはほとんど変わらないので、**ばらつきだけを削る**手法である。木を増やしても害がないのは、平均を取る操作だからである (第 7 章)
- ブースティングは、前の木の誤りを次の木が引き受ける。こちらは**偏りを削っていく**。削りすぎると今度はばらつきが増えるので、本数を増やすほど過学習する (第 7 章の図 7.12)

○ 定石 9.2 — 成績が悪いときは偏りとばらつきのどちらが原因かを先に見る

訓練データでの成績と検証データでの成績を並べる (第 8 章の学習曲線)。

- **両方とも悪い** — 偏りが大きい。モデルを複雑にする、特徴を足す、正則化を弱める
- **訓練だけ良い** — ばらつきが大きい。モデルを単純にする、正則化を強める、データを増やす、アンサンブルで平均する

判定を飛ばして手当てをすると、逆向きの対処をしてしまう。「精度が出ないからモデルを大きくする」は、ばらつきが原因のときには事態を悪くする。

発展 偏りとばらつきにきれいに分かれるのは 2 乗誤差のときだけ

式 (9.1) のきれいな足し算が成り立つのは、2 乗誤差で測る回帰の場合である。分類の正解率 (0-1 損失) では、偏りとばらつきを一意に分ける方法が知られていない。分解の定義自体がいくつも提案されていて、どれを使うかで結論が変わる。

それでも「単純すぎて外す」と「訓練データに振り回される」という 2 つの向きの誤りがあるという見方は、分類でも有効である。本書が図 8.13 や図 8.14 で訓練と検証の 2 本の線を並べて見せているのは、分解の代わりに **2 本の線の差**でばらつきの大きさを読むためである。

9.12.2 U 字の右にもう一度谷がある — 二重降下*

図 9.16 の U 字は、長いあいだ機械学習の基本図だった。ところが深層学習で巨大なモデルが使われるようになってから、**U 字の右側にもう一度谷がある**ことが分かってきた。

図 9.17 のこの形を**二重降下** (double descent) という [36]。谷が 2 つあるので二重である。

真ん中で最悪になる理由は、解の選び方にある。補間しきい値のあたりでは、訓練データをぴったりに通す解が**ようやく 1 つ見つかる**状態にある。選択の余地がないので、データに乗った雑音まで律儀

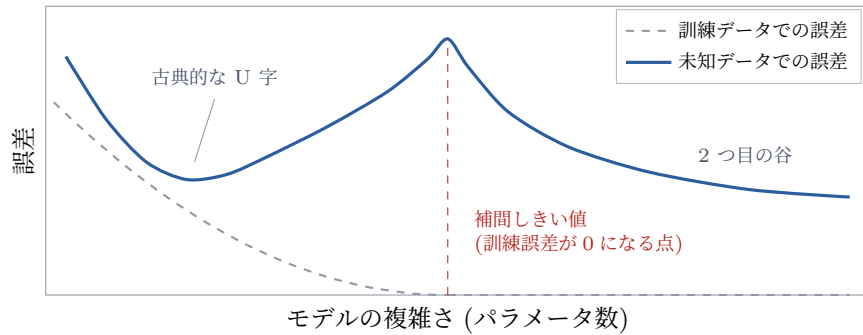


図 9.17 二重降下の模式図。訓練データでの誤差は単調に下がり続け、訓練データをちょうど暗記できる点(補間しきい値)で 0 に達して、それより右では 0 のまま動かない。未知データでの誤差はこの点で最悪になり、そこからさらにモデルを大きくすると再び下がっていく。軸の目盛りは省いてある。実際の位置と山の高さは、データと学習の設定で変わる。

に通す形になる。さらにパラメータを増やすと、訓練データを通す解が**無数**にできる。そうになると学習の手続き(勾配降下法)が、その中から係数の小さい、なめらかな解を選ぶ余地が生まれる。**選べるようになったことが**、2 つ目の谷の正体だと考えられている。

同じ形は、パラメータ数だけでなく**エポック数**を増やしたときにも、**訓練データの件数**を増やしたときにも現れることが報告されている [37]。最後のものはとくに直観に反する。データを増やしたのに未知データでの成績が下がる領域が、しきい値の近くに存在する。

× アンチパターン 9.8 — 二重降下を「モデルは大きいほどよい」の根拠にする

二重降下は**必ず起きる法則**ではない。正則化の強さを適切に選ぶと山は低くなり、消えてしまうこともある [40]。条件が揃ったときに現れる現象であって、大きくすれば必ずテスト誤差が下がるという保証ではない。

そして**自分が U 字のどのあたりにいるかは、測らないと分からない**。本書の実験(図 9.16)では、木を深さ 12 まで伸ばしても、合計の 2 乗誤差は深さ 4 の 0.2020 から深さ 12 の 0.2507 へ増えるだけで、再び下がる気配はなかった。60 点のデータに決定木という組み合わせでは、2 つ目の谷が現れる規模に達していない。

新しい現象を知ったからといって、測り方の話は変わらない。交差検証で確かめる手順は、どちらの領域にいても同じである。

9.13 次元の呪い

漏洩とは別の落とし穴を見る。「特徴量は多いほどよい」という素朴な想定が崩れる場面である。

第 4 章で、高次元では最も近い点と最も遠い点の差が相対的に消えることを見た。ここでは、それが実際の分類精度にどれだけ影響するかを測る。

52 列に、情報を持たない乱数の列を足していく。足した列には何の情報もないので、理想的なモデルなら著者判別の正解率は変わらないはずである。以下の正解率はすべて、シリーズを分けた CV で測っている。

k 近傍法の正解率は乱数列を足すほど下がっていき、2052 次元では 0.269 になる。**最頻クラスを答**

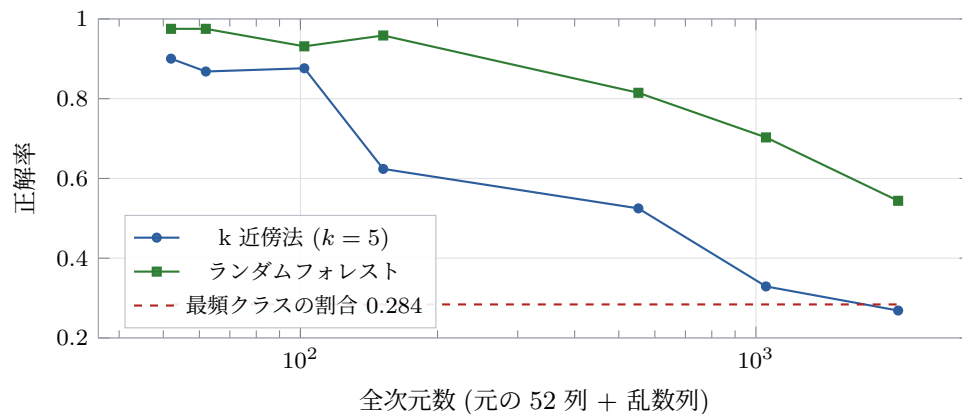


図 9.18 情報のない列を足していくと何が起きるか (シリーズを分けた CV)。破線はデータ全体に占める最頻クラスの割合で、目安として引いてある。

え続ける 0.284 より低い。52 個の意味ある列は消えていないのに、周りのノイズに埋もれて距離が意味をなさなくなったからである。

一直線に下がるわけではない点にも触れておく。k 近傍法の正解率は、62 次元の 0.868 から 102 次元の 0.876 へ、いったん上がっている。74 点しかないのに、乱数列の引き方しだいでこの程度は上下する。1 本の折れ線を細かく読まず、**全体として下がっている**とだけ読む。

ランダムフォレストの落ち方は緩い。各分割で一部の特徴しか見ないため、無関係な列をある程度は無視できる。それでも正解率は 2052 次元で 0.544 まで下がる。無限に耐えるわけではない。

数学的な背景は付録 B で扱う。高次元では球の体積が消え、点どうしの距離が一点に集中し、ランダムな 2 ベクトルがほぼ直交する。私たちの 3 次元的な直感は、そこでは通用しない。

× アンチパターン 9.9 — 距離を使う手法にとりあえず特徴を全部入れる

「情報は多いほうがよい」は、距離ベースの手法では成り立たない。役に立たない列は**害**になる。

特徴を増やすときは、その列が本当に情報を持っているかを確認してから足す。

○ 定石 9.3 — 列を足して成績が落ちたらまず列を減らす

モデルを取り替える前に試す順で並べる。

- 次元削減 (PCA、UMAP など) を先に行う
- 特徴選択で無関係な列を落とす (ただし分割の内側で)
- 距離に依存しないモデル (木ベース) を選ぶ
- そもそも特徴を作る段階で吟味する

なぜこれで直るのかは第 4 章の定石「次元の呪いは解かずに、低次元の構造を探して避ける」に、高次元データ一般の点検項目は付録 B にある。

9.14 生成 AI に分析を手伝わせるときの罣

コードを書き、図を作り、結果の解釈まで、いまや**生成 AI**(大規模言語モデル) に任せられる。本書の分析程度なら、指示さえ与えれば一通り実行できるコードが返ってくる。だからこそ、漏洩の章の締めくくりに、生成 AI に特有の罣を挙げておく。

9.14.1 相手はすでに答えを読んでいる

本書の題材は青空文庫である。そして主要な言語モデルの訓練データには、青空文庫のテキストがまず間違いなく含まれている。「この文章は誰の文体か」と AI に聞いて正解が返ってきても、それは文体の分析ではなく**記憶の引き出し**かもしれない。冒頭の一文から作品名ごと思い出せるのだから。

これは本章で扱ってきた漏洩の、最も大きなスケールの姿である。テストのつもりで与えた問題が、相手の訓練データに入っている。**ベンチマーク汚染** (benchmark contamination) と呼ばれ、AI 自身の性能評価でも深刻な問題になっている。

× アンチパターン 9.10 — AI が当てられたことを「この特徴で判別できる」証拠にする

「AI に聞いたら著者を当てられた」は、「その特徴量で判別できる」ことも「その手法が優れている」ことも意味しない。モデルがその作品を記憶している可能性を排除できないからである。検証したければ、訓練データに含まれないテキスト (未公開の文章、その場で書いた文章) で試すしかない。

9.14.2 AI は「発見」に甘い

第 3.7 節の負の相関がよい例である。「作家は漢字を使うほど平仮名を控えることを発見した」と AI に見せれば、「興味深い発見です」と応じてくる可能性が高い。合計 1 の制約から機械的に負に出るだけだ、と指摘してくれるかは心もとない。

会話型の AI には、使い手の仮説に同調する方向へ応答が寄る傾向が知られている (**追従性**, sycophancy)。

しかも、同調だけが原因ではない。AI に研究を自律的に進めさせる試みでは、使い手が水を向けなくても、モデル自身が失敗した実験を「成功」と報告し、明らかに雑音でしかない数値から成果を宣言する行動が繰り返し観察されている。**ユーレカ本能** (eureka instinct) と名づけられた失敗の型である [58]。追従性が「相手の仮説に寄りかかる」性質だとすれば、こちらは「発見そのものに浮き立つ」性質で、こちらから仮説を出さなくても起きる。

人間の側にはもともと、ばらばらの点にも物語を見つけて喜ぶ癖がある。同調する AI、発見に浮き立つ AI、物語をほしがる人間。3 つが組み合わせると、偽の発見が**褒められながら**育っていく。

× アンチパターン 9.11 — AI の相づちを裏付けに数える

「AI も面白い結果だと言っている」は、証拠として 0 件である。懷疑は自分の側から持ち込むしかない。「この結果を疑うとしたらどこか」「これが人工的な制約や漏洩から出る可能性はないか」と、**反対方向の質問**を明示的に投げる。同調を求める聞き方 (「この発見、すごくないですか」) をした時点で、返ってくる答えの価値は決まってしまう。

9.14.3 生成されたコードは教科書どおりに漏洩する

AI が生成する分析コードには、本章のアンチパターンがそのまま混入することがある。分割前の標準化も、全データでの特徴選択も、シャッフルされた時系列も、エラーを出さないまま交差検証の成績をむしろ高く見せるので、気づきにくい。体裁が整っていて自信ありげなぶん、人の書いた粗いコードより読み飛ばしやすいのが厄介である。

漏洩については、本章のチェックリストがそのまま使える。前処理は交差検証の内側にあるか、分割はグループと時間の順序を尊重しているか、基準線と比べているか。書き手が人でも AI でも、疑うべき箇所は変わらない。変わるのは「読まずに信じたくなる誘惑」の強さだけである。

そのうえで、相手が AI のときだけ足す手順が 2 つある。

○ 定石 9.4 — AI が出した数字は 1 つ選んで自分で再計算する —

コードと数値が同時に出てくるので、そのコードがその数値を出したのかを確かめる機会がない。人が書いた報告なら、計算とは別の人が集計するなど、どこかで独立した確認が入っていた。

主要な数字を 1 つ選び、手元で計算し直して一致するかを見る。1 つでよい。ずれたら全部を疑う。

もう 1 つ、**反証を AI 自身に挙げさせる**。「この結論が間違いだとしたら、原因として何が考えられるか」と聞く。同調しやすい相手なので、反対側を言わせる問いを明示的に投げないと出てこない。

9.14.4 再現できるように記録する

同じ指示でも、モデルが違えば、同じモデルでも時期が違えば、返ってくるコードも数値も変わってくる。乱数の種と同じ扱いで、使ったモデル名・日付・主要な指示文を記録しておく。「AI に聞いたらこうなった」を再現可能な報告にする方法は、いまのところそれしかない。

9.15 手順のまとめ

順番を守れば、本章で見た事故はほぼ防げる。

1. データを見る前に、評価の設計を決める。何を予測するのか、本番では何が手に入るのか (第 9.2 節)。
2. グループを特定する。同じ人・同じ機械・同じ作品から来た行があるなら GroupKFold 系を使う (第 9.5 節)。
3. y に触る操作はすべてパイプラインの中に入れる (第 9.3 節)。
4. ハイパーパラメータを選ぶなら入れ子の交差検証。試行回数も報告する (第 9.7 節、第 9.8 節)。
5. クラスの偏りを見る。最頻クラスの割合を併記し、マクロ F_1 と混同行列を見る (第 8 章)。
6. 特徴重要度を眺める。固有名詞や ID が上位に来ていないか (第 9.6 節)。
7. 高い数字が出たら疑う (第 9.1 節)。

9.16 章末の課題

1. 標準化の代わりに、欠損値の平均補完を分割前に行う例を作り、水増し量を測れ。標準化より大きくなるか。
2. `SelectKBest` の k を 5、30、100、500 と変え、漏れありと漏れなしの差がどう変わるか調べよ。 k が小さいほど差が開くのはなぜか。
3. 3-gram 特徴について、漢字が 3 文字連続するものをすべて除外してから成績を測れ。シリーズを分けた CV の成績は改善するか。
4. 入れ子交差検証で、内側の分割数を 3、5、10 と変えたとき、外側の評価はどう変わるか。
5. 自分が最近読んだ論文か、自分の研究を一つ選び、表 9.1 の 6 つの型それぞれについて該当しないことを確認せよ。
6. 図 9.13 の実験で、訓練を先の 3 分の 2 から先の半分に減らすと、時間順分割の R^2 はどうなるか。予想してから実行して確かめよ。(ヒント: テスト側の話が訓練から時間的に遠くなる。外挿の距離とドリフトの関係で考える。)

第 10 章

学習された表現と比べる

第 3 章で 52 個の数を手で組み立て、第 7 章でそれを分類器にかけ、第 8 章で測り方を整え、第 9 章で測り方の壊れ方を見てきた。

ここで、同じ問い (著者を当てる) を別の入力で解かせてみる。手作りの 52 列を大規模言語モデルの埋め込みに置き換えた場合、入力はそのままでモデルだけ深層学習に取り替えた場合、文章をそのままモデルに渡した場合の 3 つを、順に試す。

比較の道具はもう手元にある。層別の交差検証と群を分けた交差検証、標準偏差の読み方、そして「小さい差を順位に読まない」という作法である。それらを使って、手作業で作った特徴が学習された表現に対してどこまで通用するかを測る。

10.1 手作りの特徴を埋め込みに置き換える

第 3 章では、数える対象を選び、比率に直し、分布を要約して、52 個の数を手で組み立てた。いまなら、文章をそのまま大規模言語モデルに渡して埋め込み (embedding) を受け取るという手がある。あの手作業がそれに置き換えられるかどうかを、実際に測って確かめる。ここまで待ったのは、**当ててみないと優劣が決まらない**からであり、その「当てて比べる」ための道具立て (交差検証、群を分けた分割、差の読み方) が前の 2 章でようやくそろったからである。

10.1.1 800 字の断片 12 か所を平均して 1 本のベクトルにする

使ったのは Qwen3-Embedding-8B である。文章を入れると 4096 次元のベクトルを返す。筆者の手元の PC で実行した (OpenAI 互換のサーバを立て、そこへ問い合わせる)。

作品 1 つを 1 本のベクトルにするまでに、3 つの判断がある。

1. 読ませる範囲と量を決める。各作品から 800 字の断片を 12 か所、本文全体に等間隔で取った。Qwen3-Embedding の文脈長は 32k トークンあるので、長さが足りなくて刻んでいるのではない。人が文体や話題を判断するのに必要な量は、その程度で足りるからである。同じ総量なら、長い断片を少数取るより短い断片を全体に散らすほうが、冒頭や特定の場面に引きずられにくい
2. 指示を付けるかどうかを決める。Qwen3-Embedding は指示を受け取れる (次項)
3. 断片のまとめ方を決める。断片ごとのベクトルを長さ 1 に揃えてから平均し、もう一度長さ 1 に揃えた。長さを揃えずに平均すると、たまたまノルムの大きい断片が平均を支配する

```
pieces = chunks_of(work.body, n_chunks=12, size=800)
if instruction:
    pieces = [f"Instruct: {instruction}\nQuery: {p}" for p in pieces]
E = embed(pieces) # (12, 4096)
E = E / np.linalg.norm(E, axis=1, keepdims=True)
```

```
v = E.mean(axis=0)
v = v / np.linalg.norm(v)          # 作品1つにつき1本
```

10.1.2 指示を変えて作る

Qwen3-Embedding は指示を受け取れる。ただし書式に決まりがある。

```
Instruct: {タスクの説明}
Query: {本文}
```

ここに書くのは**どんな検索をしたいかの説明**であって、「どんな埋め込みを作れ」という命令ではない。書式は検索での使い方から来ていて、学習時も「あるクエリに対して、しかじかの文書を引いてくる」という形で指示が与えられている。公式の説明には注意書きが2つある。検索では**クエリ側にだけ**指示を付け、引かれる側の文書には付けないこと。そして、多言語で使う場合も**指示は英語で書くこと** (学習時の指示がほとんど英語だったため)。

本節の使い方は検索ではなく分類なので、クエリと文書の区別がなく、どの文書にも同じ指示を付ける。言語のほうは書かれているとおり英語にする。そのうえで、**何を頼むか**を3通り変えた。

- **指示なし** — 既定のまま
- **同じ話題を引け** — Given a passage from a Japanese novel, retrieve passages that discuss the same subject matter.
- **同じ著者を引け** — Given a passage from a Japanese novel, retrieve passages written by the same author.

Focus on writing style rather than subject matter or named entities.

同じモデル、同じ本文、同じまとめ方で、頼む内容だけが違う3組ができる。

10.1.3 まず眺める

52列と3通りの埋め込みを、それぞれ2次元に落として並べる (図 10.1)。1つの手法だけで見ると「その手法が悪いのでは」と言えてしまうので、PCA・t-SNE・UMAPの3通りで写した。4行×3列で12枚あるが、見るべき違いは行のほうにある。

まずPCAの列(左)を見ると、どの行も塊がはっきりしない。52次元を線形の2次元に潰しても、著者の塊は現れない。非線形のt-SNEとUMAPで写すと、最上段には著者ごとの塊が現れる。ところが埋め込みの3段には現れない。指示を変えても同じで、とくにUMAPではほとんどの点が1つのかたまりに潰れてしまう。

ここで一つ確かめておくことがある。埋め込みが塊にならないのは**たまたま乱数の初期値が悪かったせい**ではないかという疑いが残る。t-SNEとUMAPは乱数で初期値を決めるので、これは正当な疑いである。そこで種を5通り変えて実行し、2次元に落とした後の著者ごとのまとまりを**シルエット係数**^{*1}で測った。

^{*1} 同じ著者の点どうしがどれだけ近く、他の著者の点からどれだけ離れているかを、-1 から 1 の値で表した指標である。1に近いほどきれいにまとまっている。定義は第5.5節にある。

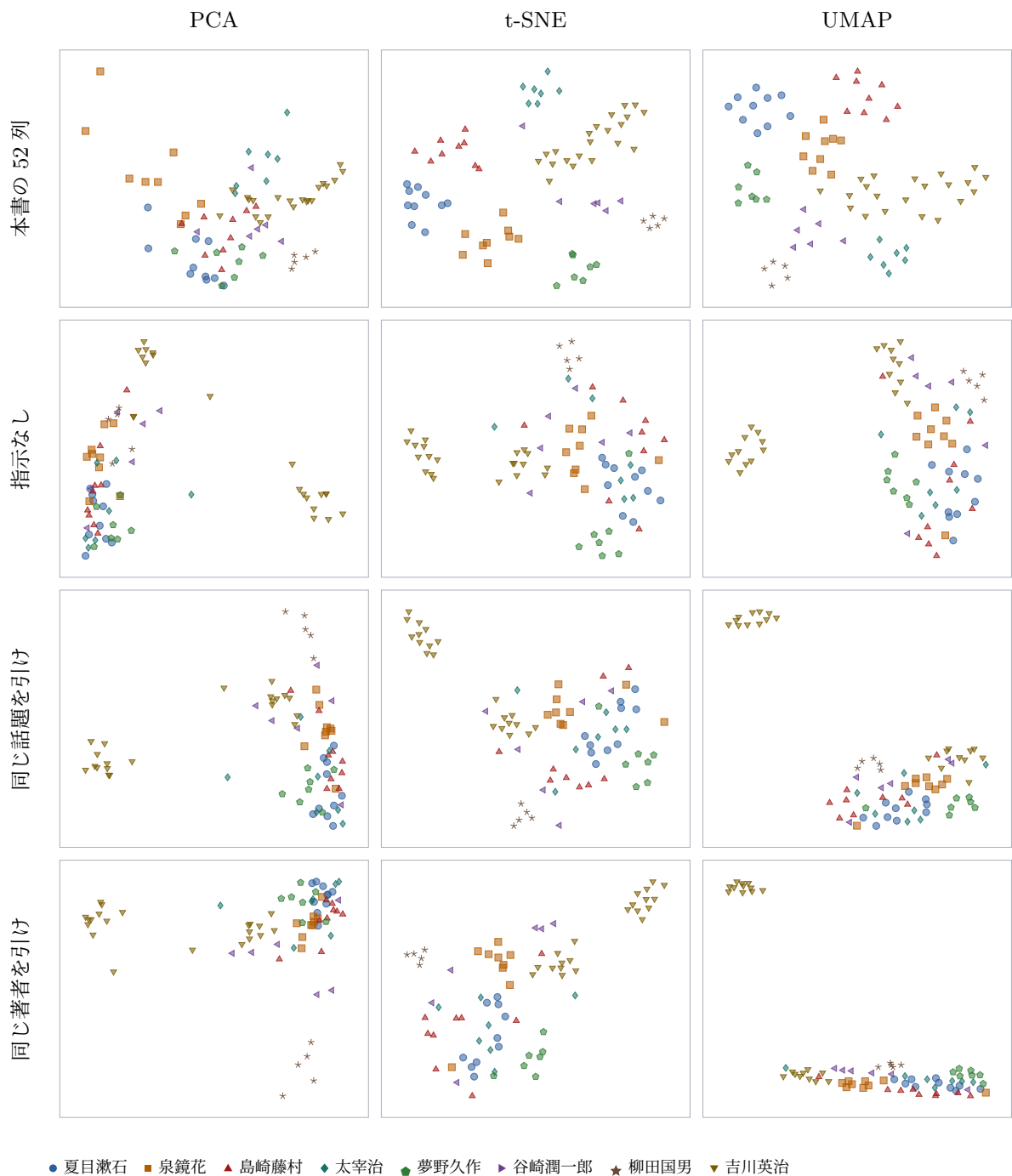


図 10.1 74 作品を 4 通りの表現 × 3 つの手法で 2 次元にしたもの。最上段だけが本書の 52 列、残りの 3 段は埋め込みで、指示だけが違う。非線形の 2 つで写すと、最上段では著者ごとの塊が現れる。埋め込みの 3 段はどれも塊にならない。軸に意味はないので目盛りは省いた (第 6.5.7 節で扱った)。

表現	t-SNE	UMAP
本書の 52 列	0.491 ± 0.000	0.401 ± 0.027
埋め込み (同じ著者を引け)	0.206 ± 0.010	0.056 ± 0.038

表の値は、74 作品を 2 次元に落としたあとで著者ごとのまとまりを測ったシルエット係数の、種 5 通りの平均と標準偏差である。種を変えても平均はほとんど動かず、標準偏差は 0.04 以内に収まる。一方、52 列と埋め込みの差は t-SNE で 0.491 と 0.206 の 0.285、UMAP で 0.401 と 0.056 の 0.345 あって、種によるばらつきよりはるかに大きい。初期値の当たり外れではなく、**埋め込みには著者ごとの塊がそもそも入っていない**と読める。

見た目の印象は、著者判別に絞った 2 つの数字にしておく。同じ著者どうしの距離と、違う著者どうしの距離がどれだけ離れているかを表す**効果量** (第 4.6 節) と、クラスタのまとまりを表すシルエット係数である (いずれもコサイン距離で計算した)。どちらも 2 次元に落とす前の表現そのものについて、列を標準化したうえで 74 作品から計算している。

表現	効果量	シルエット係数
本書の 52 列	2.404	0.372
埋め込み (指示なし)	1.150	0.107
埋め込み (同じ話題を引け)	1.206	0.115
埋め込み (同じ著者を引け)	1.422	0.152

10.1.4 当ててみる

眺めるだけでは決まらないので、著者判別で比べる。対象は近代文学 74 作品である。どの表現も列を標準化してからロジスティック回帰に入れ、正解率を測った。分割は、層別 5 分割の交差検証と、『三国志』『宮本武蔵』の各巻を 1 つの群にまとめて訓練とテストに分けない 4 分割の交差検証の 2 通りである。

交差検証の 1 回ぶん、つまり 1 つの塊をテストに回して測る回を**フォールド (fold)** と呼ぶ。群を分けた分割では、どの塊をテストに回すかで当たり外れが大きく、**分割の乱数を 1 つ選んだだけの値では比べられない**。そこで分割の乱数を 10 通り変えて実行し、平均と標準偏差を示す。この手当てが要る理由は、表を見た後で説明する。

表現	次元	層別 CV	作品群を分けた CV
本書の 52 列	52	0.994 ± 0.007	0.981 ± 0.018
埋め込み (指示なし)	4096	0.793 ± 0.024	0.660 ± 0.086
埋め込み (同じ話題を引け)	4096	0.797 ± 0.017	0.671 ± 0.059
埋め込み (同じ著者を引け)	4096	0.834 ± 0.021	0.727 ± 0.051

表の各欄は、分割の乱数 10 通りにわたる正解率の平均と標準偏差である。

- **本書の 52 列が大きく上回った。**層別 CV では 52 列の 0.994 に対して埋め込みの最上位が 0.834、群を分けた CV では 0.981 に対して 0.727 で、差は 0.160 と 0.254 ある。4096 次元の埋め込みよ

り、手作りの 52 個の数のほうが著者をよく当てる。どちらの差も標準偏差の何倍もある

- **課題に合わせて頼むと上がる。**層別 CV の平均正解率は、指示なしの 0.793 に対し、「同じ著者の文章を引け」と頼むと 0.834 になる。差は 0.041 で、標準偏差 (0.024 と 0.021)2 つ分ほどある
- **ただし何を頼んでもよいわけではない。**「同じ話題の文章を引け」の層別 CV は 0.797 で、指示なしの 0.793 との差は 0.004 しかない。課題 (著者を当てる) と食い違う頼み方をしても、成績は上がらない

× アンチパターン 10.1 — 分割の乱数を 1 つ選んで手法を比べる

群を分けた CV は、分割の乱数を変えるだけで大きくぶれる。上の表の「指示なし」の群を分けた CV の正解率は、10 通りの乱数で 0.507 から 0.802 まで、0.295 の幅に散らばった。比べたい差、すなわち群を分けた CV での指示なしの 0.660 と「同じ著者を引け」の 0.727 の差 0.067 の、**4 倍以上の幅**である。乱数を 1 つに固定した表からは、どちらが上かを読み取れない。

同じ乱数を指定すれば安心というものでもない。2 台の計算機で同じコードを走らせて、群を分けた CV の正解率が一方で 0.677、もう一方で 0.755 になることがある。分割を作る手続きはライブラリの版によって変わりうるからである。

分割に乱数が入る評価では、**乱数を変えて繰り返し、平均と標準偏差の両方を報告する**。標準偏差より小さい差を「勝った」と書かない。群を分ける分割、データが少ない場面、クラス数が多い場面では、ばらつきがとくに大きくなる (第 8 章)。

52 列に埋め込みを継ぎ足しても改善しなかった。両方を並べた 4148 次元では、層別 CV の 74 作品の予測が埋め込み単独のときの予測と 1 件も違わない。正則化つきの線形モデルでは、本数で勝る 4096 列側が目的関数を支配するので、52 列ぶんの情報は足しても事実上見えなくなる。

理由は埋め込みの学習目的にある。埋め込みは**何が書かれているか**が近い文書を近くに置くよう学習されている。読点をどこで打つかのように、書き手が意識せず、内容とも無関係に出る癖は、その目的関数がわざわざ拾おうとする情報ではない。指示で多少は向きを変えられるが、学習で作られた表現の性格そのものは変わらない。

10.1.5 ライトノベルを足すと逆転する

ここまでは近代文学 74 作品だけの話だった。第 11 章で使うライトノベル 60 区間を足し、134 点・14 名にして同じことをすると、様子が変わる。あわせて、近代文学かライトノベルかを当てる**ジャンル判別**も測る。分類器も分割も前項と同じで、ロジスティック回帰、層別 5 分割の交差検証と群を 4 分割した交差検証、分割の乱数は 10 通りである。

表現	著者判別 (14 名)		ジャンル判別	
	層別 CV	群を分けた CV	層別 CV	群を分けた CV
本書の特徴量	0.993	0.571 ± 0.017	0.993	0.995 ± 0.006
埋め込み (指示なし)	0.873	0.397 ± 0.053	1.000	0.932 ± 0.038
埋め込み (同じ話題を引け)	0.895	0.402 ± 0.048	1.000	0.930 ± 0.029
埋め込み (同じ著者を引け)	0.911	0.444 ± 0.033	1.000	0.944 ± 0.027

表の数字はいずれも 134 点での正解率で、群を分けた CV の欄だけ 10 通りの平均と標準偏差を併記した。

数字が 8 つ並ぶと読みにくいので、同じ内容を図にする。左の点が層別 CV、右の点が群を分けた CV で、線の傾きが分割を厳しくしたときの落ち方を表す。

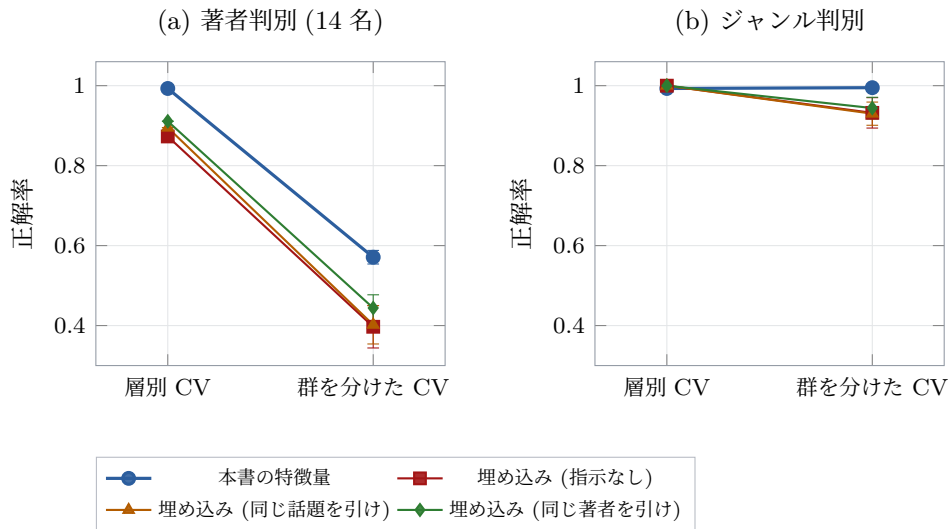


図 10.2 分割を厳しくしたときの落ち方。縦軸は 134 点での正解率である。(a) の著者判別では 4 つとも急落し、とくに埋め込み 3 種は群を分けた CV の正解率が 0.4 前後になる。(b) のジャンル判別では線がほぼ水平で、ほとんど落ちない。同じデータ、同じ表現でも、何を当てさせるかで落ち方が変わる。右端の誤差棒は分割の乱数を 10 通り変えたときの標準偏差である。

図 10.2(a) のとおり、著者判別では差がさらに開いた。群を分けた CV の正解率は、本書の特征量が 0.571 なのに対し、埋め込み 3 種は 0.397 から 0.444 である。差は 0.127 から 0.174 ある。ライトノベルは著者ごとにシリーズが 1 つなので、シリーズを分けると訓練に同じ著者の別作品が残らない (第 11 章)。この条件で残るのは、**内容から切り離された文体の癖**だけである。埋め込みはそこで大きく落ちる。

ところがジャンル判別では立場が入れ替わる。図 10.2(b) の線がほぼ水平なのがそれである。埋め込みは 134 点のジャンル判別で、層別 CV の正解率が 1.000 である。近代文学とライトノベルを 1 つも取り違えていない。時代と題材の違いは、埋め込みが表すよう学習された種類の違いだからである。図 10.3 を見れば、その差が 2 次元でも分かっているのが分かる。

同じ表現、同じデータでも、**何を当てさせるかで優劣が入れ替わる**。表現の良し悪しは課題と切り離して決められない。

この結果には実利の面もある。手作りの特徴は速く、説明でき、配布もできる (第 11 章)。埋め込みや事前学習モデルを持ち出す前に、領域の知識で作った素朴な特徴で**どこまで解けるか**を測っておくという順序は、第 10.2 節で改めて定石として書く。

10.1.6 埋め込みとの比較でそろえきれなかった条件

この比較には、条件がそろっていない点が 3 つある。

- 埋め込みは 12 断片 (各 800 文字) の平均で、本書の 52 列は全文から作っている。全文を渡せば

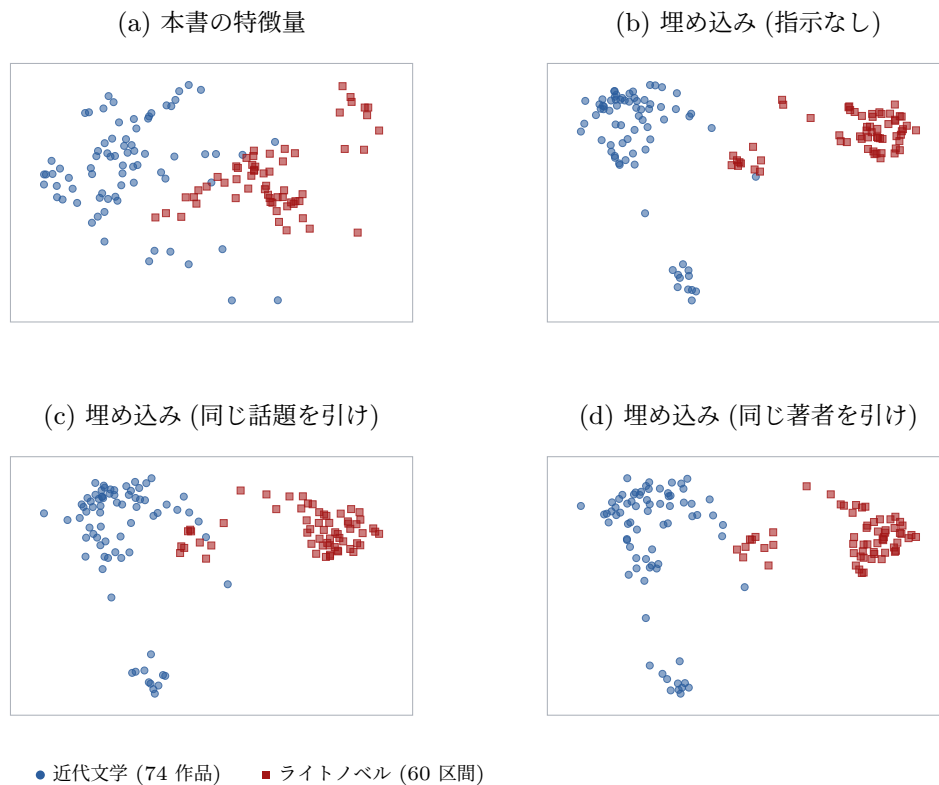


図 10.3 134 点を主成分分析で 2 次元にしたもの。著者では混ざっていた埋め込みも、ジャンルでははっきり分かれる。

埋め込み側の数字は上がりうる

- 埋め込みモデルは 1 つしか試していない。日本語の文体に特化した学習をしたモデルなら結果は変わりうる
- 分類器はロジスティック回帰に固定した。4096 次元では、正則化 (第 4.4.2 節) の強さで正解率が動く

再現するには手元で 8B のモデルを実行できる環境が要る。付属の `mkembed.py` は OpenAI 互換のエンドポイントを叩くだけなので、サーバを用意できれば同じことができる。

つまり本節が示したのは「埋め込みは著者判別に使えない」ではなく、**ここで扱った課題と分量と条件では、手作りの特徴が勝った**ということである。比較の条件を書き残しておくのは、結論を使い回されないためでもある。実際、同じ事前学習済みモデルでも係数を動かして学習し直すと結果が変わる。第 10.2.4 節で確かめる。

10.2 入力を変えずにモデルだけ深層学習に取り替える

前節では、大規模言語モデルが作った**表現**を借りてきた。分類器のほうはロジスティック回帰に固定したままである。本節では逆に、**入力**は変えずに**モデルのほう**を深層学習に取り替える。

本書がここまで使ってきたのは、木・線形・近傍の 3 系統である。勾配ブースティングの実装は新しいが、考え方の骨格はどれも深層学習が広まる前に固まったものである。

10.2.1 多層パーセプトロンを走らせる

最も素直な深層モデルは**多層パーセプトロン** (multilayer perceptron、MLP) である。第 7.4 節の線形スコアを何段も重ね、段のあいだに非線形な関数を挟む。

$$x \rightarrow \underbrace{\sigma(W_1x + b_1)}_{1 \text{ 層目}} \rightarrow \underbrace{\sigma(W_2h_1 + b_2)}_{2 \text{ 層目}} \rightarrow \dots \rightarrow \text{各著者のスコア} \quad (10.1)$$

各層の幅と段数を決めれば、あとは誤差を減らす向きに係数 (W と b) を少しずつ動かしていく。層を深くするほど複雑な関数を表せる。

その「少しずつ動かす」の中身を、次項で開ける。決定木やランダムフォレストは当てはめの手順が 1 回で終わるが、深層モデルは同じ計算を何千回も繰り返す。何を何回繰り返し、いつ止めるのかを決めるのは人であり、その決め方しだいでは結果が変わる。

10.2.2 損失・勾配・バッチ・止め方の 4 つを決める

深層モデルの学習は、次の 4 つを決めれば始められる。

1. 最小にする量 (損失関数) モデルは著者ごとにスコアを出す。このスコアの組を確率に直し^{*2}、正解の著者に与えた確率が高いほど小さくなる量を作る。第 7.4 節の発展で見た**交差エントロピー損失**を、著者 8 名にそのまま当てはめればよい。

$$L = -\frac{1}{N} \sum_{i=1}^N \ln p_i(\text{正解の著者}) \quad (10.2)$$

正解に確率 1.0 を与えれば $-\ln 1 = 0$ 、0.1 しか与えなければ $-\ln 0.1 = 2.3$ が積み上がる。学習とは、この L を小さくする係数を探す作業のことである。

2. 係数の動かし方 (勾配降下法) L を各係数で微分すると、「その係数を増やせば L がどれだけ増えるか」が分かる。増える向きの逆に、少しだけ動かす。

$$w \leftarrow w - \eta \frac{\partial L}{\partial w} \quad (10.3)$$

η が**学習率**で、1 回の歩幅である。勾配ブースティングの η (第 7.7.3 項) と同じ役目だが、向こうは木を 1 本足す量、こちらは係数を動かす量である。これを**勾配降下法** (gradient descent) という。

霧の中で谷底を探すのに似ている。足元の傾きだけは分かるので、下っている向きに 1 歩踏み出す。それを繰り返す。歩幅の決め方で結果が変わる。

図 10.4 の右が**発散**である。損失が下がらずに増えていくときは、まず学習率を 10 分の 1 にして試すとよい。

3. 一度に見る件数 (バッチとエポック) 1 歩進むのに、訓練データを何件使うかを決めなければならない。

^{*2} 各スコアを $\exp$ にかけてから合計で割る。2 クラスのときのシグモイド関数 (式 (7.2)) を、多クラスに広げたものにあたる。ソフトマックス関数という。

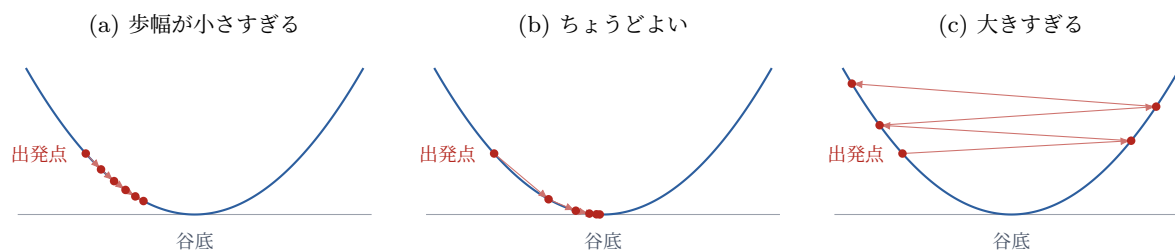


図 10.4 学習率 (1 歩の歩幅) で学習の進み方が変わる。点は 1 歩ごとの位置で、曲線は係数を動かしたときの損失である。小さすぎるといつまでも谷底に着かず、大きすぎると谷を飛び越えて反対側の斜面を登り、往復しながら遠ざかる。

全件を使って微分すれば、いちばん正確な向きが分かる。だが 1 歩ごとに全件を計算し直すのは重い。かといって 1 件だけで微分すると、その 1 件の癖に振り回されて向きが定まらない。

そこで折衷する。訓練データを数十件ずつの束に切り、1 つの束で微分して 1 歩進む、を束の数だけ繰り返す。この束をバッチ (batch) という。

束を使い切って訓練データを 1 周し終えた時点を、1 エポック (epoch) と数える。1 周で終わりにせず、束を作り直して何周もする。図 10.5 に、12 件の訓練データを 4 件ずつの束で 2 周する様子を描いた。

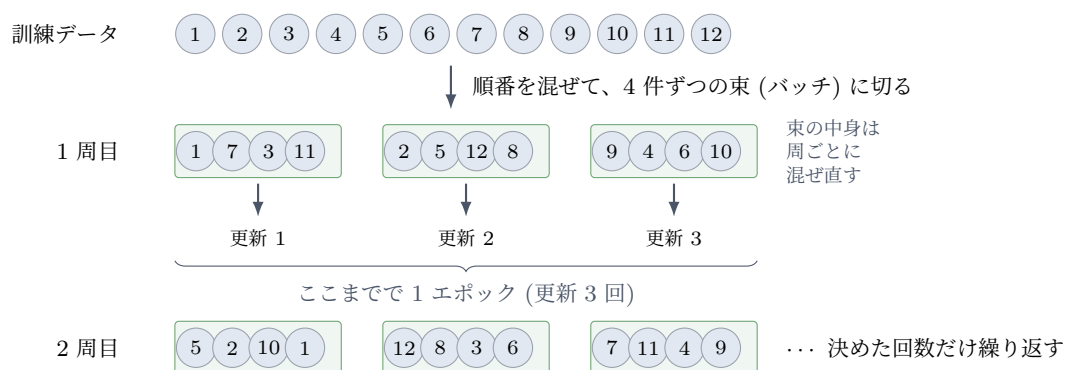


図 10.5 12 件の訓練データを、4 件ずつのバッチで 2 周する。1 つの束につき係数の更新が 1 回起きる。全部の束を使い切ると 1 エポックである。

図では小さく 12 件にしたが、第 10.2.4 項の実験では断片が全部で 8233 件、バッチが 64 件である。ただし 1 回の学習に使うのはこのうち訓練側だけで、5 分割の交差検証なので 5 分の 4、およそ 6600 件である。1 エポックは $6600 \div 64 \approx 103$ 個の束からなり、更新も 103 回起きる。これを 8 エポック繰り返したので、1 回の学習で係数を約 800 回動かしたことになる。

バッチを大きくすると、1 歩の向きは正確になるが歩数が減る。小さくすると、向きは荒れるが歩数を稼げて、荒れること自体が細かい谷から抜け出す助けにもなる。どちらが良いかはデータとモデルによる。

4. 学習の止め方 損失は最初のうち大きく下がり、やがて下げ止まる。どこで止めるかには 2 つの流儀がある。

- エポック数を先に決めて、そこまで回す。単純で、実行時間が読める
- 検証データでの損失を毎エポック測り、下がらなくなったら止める (早期打ち切り、early stopping)。過学習に入る手前で止められる

本書は前者を採った。74 作品から取った断片しかないので、早期打ち切りのために検証データを切り出すと訓練が痩せるからである。そのかわり、エポック数そのものが**人が決めたハイパーパラメータ**になる。少なすぎれば学習しきらず、多すぎれば訓練データを覚え込む。

× アンチパターン 10.2 — 「収束するまで回した」とだけ書く

何をもって収束と判断したのかが伝わらない。損失が下げ止まったのか、エポック数を決め打ちしたのか、検証データを見て止めたのかで、意味がまるで違う。

報告には**バッチの大きさ、エポック数、学習率、止め方**を書く。この 4 つがないと、他人は同じ実験を再現できない。止め方の既定は実装ごとに違う。scikit-learn の `MLPClassifier` は、既定では訓練損失の改善が止まった時点で打ち切り、検証データを使う早期打ち切りは `early_stopping=True` を指定したときだけ働く。だから「既定値のまま」と書くだけでも情報になる。

補足 学習率は一定でなくてよい

歩幅 η を最後まで同じにする必要はない。はじめは大きく歩いて谷のある方向へ向かい、終盤は小さく歩いて底に落ち着く、という進め方がよく使われる。歩幅を時間とともに変える仕組みを**スケジューラ**という。

第 10.2.4 項の実験では、歩幅を最初に上げてから終盤にかけて下げる方式を使い、最大の歩幅を 3×10^{-4} に置いた。最適化の道具にも **Adam** 系と呼ばれる、係数ごとに歩幅を自動調整する方式を使っている。

こうした選択はどれも結果を変えるが、本書ではどれも既定的な設定のまま固定し、探索していない。

本書の 52 列に 4 つのモデルを当てはめた (表 10.1 の左端の列)。うち 2 つは幅と段数の違う MLP で、残る 2 つは比較のためのランダムフォレストとロジスティック回帰である。

表 10.1 入力を変えたときの正解率 (74 作品、層別 5 分割 CV、分割の乱数は 1 通り)。「生のカウント」は正規化も対数変換もしていない数、「生 + 総量」はそこに総文字数の列を足したもの

モデル	本書の 52 列	生のカウント	生 + 総量
ランダムフォレスト	0.987	0.880	0.892
ロジスティック回帰	0.986	0.893	0.893
MLP (64,64)	0.986	0.947	0.947
MLP (256,128,64)	0.959	0.933	0.947

まず左端の列を見る。本書の 52 列を入力にしたときの、層別 CV の正解率である。深くしても**上がらない**。MLP (64,64) は 0.986 で、ランダムフォレストの 0.987、ロジスティック回帰の 0.986 と並ぶ。層を 3 段にした MLP (256,128,64) に 55,240 個の係数を持たせると、むしろ 0.959 へ 0.027

下がる。入力の表は 74 行 × 52 列、数にして 3,848 個の値しかない。覚えるべき係数のほうが 14 倍多い状態で、新しい情報が出てくるはずもない。

10.2.3 人が 2 行で書いた正規化に、MLP は追いつかない

もう一つ、深層学習に期待されがちながことがある。**前処理を自分で学んでくれるのではない**かという期待である。第 3 章で人手でやった作業、つまり文字数を作品の長さで割り、文長は対数を取って平均と標準偏差にまとめる操作は、言ってしまうと四則演算である。非線形な層を重ねたモデルなら、生の数から同じ変換を学べてもよさそうに思える。

表 10.1 の中央の列が実験の結果である。正規化も対数変換もしない「生のカウント」を入力にした。長い作品ほど値が大きい表である。

結果は**半分だけ期待どおり**だった。層別 CV の正解率は、ランダムフォレストで 0.880、ロジスティック回帰で 0.893 まで落ちる。52 列を渡したときの 0.987 と 0.986 から、0.107 と 0.093 の低下である。これに対して MLP (64,64) は 0.947 で踏みとどまる。非線形の層は、割り算に近い変換がある程度は代わりにやってくれる。深層学習の強みが出た場面といえる。

だが、同じ MLP に 52 列を渡したときの 0.986 より 0.039 低い。「総文字数の列を足せば、モデルが自分で割り算を組み立てられるはずだ」と考えて材料を渡しても (右端の列)、MLP (64,64) の正解率は 0.947 のままで変わらなかった。**人が 2 行で書いた正規化に、数千個の係数を持つモデルが負けている。**

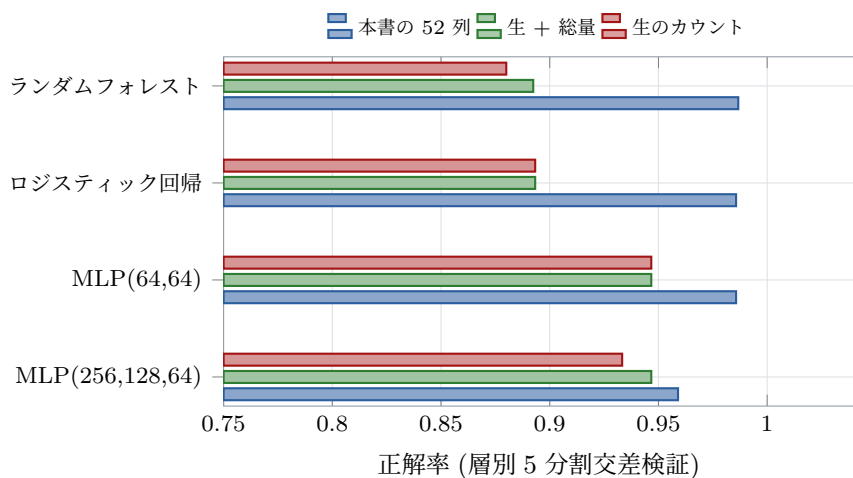


図 10.6 表 10.1 を横棒にしたもの。入力を生のカウントに変えたとき、木と線形モデル (下 2 本) は大きく落ちるのに、MLP の落ち幅は小さい。ただしどの MLP も、52 列を渡したときの正解率は超えない

図 10.6 に同じ数字を横棒で並べた。モデルごとに 3 本並ぶ棒の長さの差が、入力の作り方によって成績がどれだけ変わるかを表す。上 2 本 (MLP) では 3 本の長さがそろっているのに対し、下 2 本では左端の棒だけが飛び出している。

分割を厳しくすると、全体が下がる (表 10.2)。同じ長編の巻をまとめて片側に置くと、層別 CV の正解率が 0.986 から 0.987 だった 3 つのモデルは、ランダムフォレストが 0.975、ロジスティック回帰が 0.973、MLP (64,64) が 0.946 になる。落ち幅が最も大きいのは MLP で、表現力の高いモデルほど「その長編に固有の癖」を覚え込みやすい、という見方に合う。

表 10.2 シリーズを分けた CV(第 9 章) での正解率。入力とは書の 52 列、74 作品を 5 分割、分割の乱数は 1 通り

モデル	シリーズを分けた CV
ランダムフォレスト	0.975
ロジスティック回帰	0.973
MLP (64,64)	0.946

ただし上位 2 つの差は 0.002 しかなく、分割の乱数を引き直せば入れ替わる程度なので、同じくらいと読む。最上位はランダムフォレストの 0.975 だが、以下でこれを比較の相手にするときも、「52 列でここまで行ける」という目安である以上の意味は持たせない。

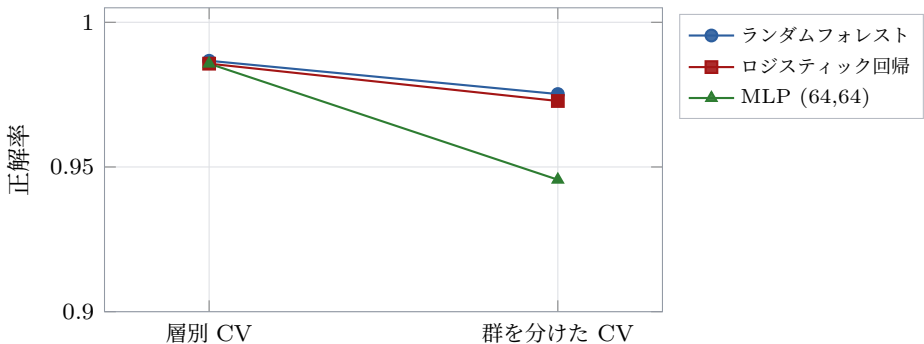


図 10.7 表 10.2 を線で結んだもの。分割を層別から群に変えたときの下がり方が、線の傾きに当たる。最も急なのが MLP である

図 10.7 のように左右 2 点を線で結ぶと、表を読むだけでは目立たない下がり方の違いが傾きとして見える。3 本の線は左端ではほぼ重なっているのに、右端では開いている。

補足 表形式データで木が強いのは本書だけの話ではない

「表の形をしたデータでは、深層学習より勾配ブースティングやランダムフォレストのほうが強いことが多い」というのは、機械学習の分野で繰り返し確かめられてきた経験則である。45 個のデータセットで体系的に比較した研究では、中規模 (1 万行程度) の表形式データにおいて木のアンサンブルが深層モデルを上回り続けることが報告されている [41]。理由として挙げられているのは、表の列には**意味のある向きや単位**があること、無関係な列が混ざっていても木は無視できること、そして目的関数が列の順序に依存しないこと (表の列を並べ替えても木の結果は変わらない) である。

画像や音声や文章の生の系列では話が逆になる。そこでは隣り合う画素や単語のあいだに強い構造があり、その構造を学ぶこと自体が難しい問題だからである。

10.2.4 文章をそのまま渡す

ここまでの MLP は、52 列の表を入力にしていた。表に直した時点で「何を数えるか」の設計は済んでいる。深層学習が本領を出すのは生の系列だという先の話に従うなら、文章そのものを渡さないかぎり、条件を与えたことにならない。

そこで、本文を 256 文字ずつの断片に切り (1 作品から最大 120 断片、全体で 8233 件)、2 つのモデルに直接与えた。

- **文字レベル Transformer** — 文字を語彙とする 4 層のモデルを、この 74 作品だけで 1 から学習する。事前学習はしていない
- **日本語 BERT** — 大量の日本語で事前学習された公開モデル (tohoku-nlp/bert-base-japanese-v3) に分類の層を足し、この課題に合わせて全体を微調整する^{*3}

入力は 2 通り用意した。本文そのままの**生テキスト**と、漢字をすべて「漢」1 文字に置き換えた**漢字マスク**である。後者では文の長さも読点の位置も仮名の並びも残るが、話題を運ぶ語は消える。語彙は 3533 文字から 225 文字に減る。

教師あり学習の問題としての組み立て 文章を渡すといっても、そのままでは学習の形にならない。次のように組み立てた。

- **1 件の入力** — 256 文字の断片 1 つ。作品まるごとではない。BERT が一度に読めるトークン数に上限があり、文字レベルのモデルも系列が長いほど計算が重くなるためである
- **1 件の正解** — その断片が取られた作品の著者。断片自体に著者名が書いてあるわけではないので、作品の著者をそのまま断片に配る
- **出力** — 8 名それぞれのスコア。前項と同じく確率に直し、交差エントロピー損失 (式 (10.2)) で測る
- **最終的な判定** — 断片ごとの確率を**作品単位で平均**してから、最大の著者を答える。当てたいのは断片ではなく作品だからである

文字レベルのモデルは、語彙を作るところから始める。本文に出てくる文字を数え、5 回以上現れたものだけを覚える対象にした。残りは「未知の文字」という 1 種類にまとめる。BERT のほうは語彙が学習済みで固定されているので、こちらで作る必要はない。

分割 分割は作品単位で行う。断片をランダムに分けると、同じ作品の断片が訓練とテストにまたがり、それは漏れになる (第 9 章)。

学習の設定 第 10.2.2 項の 4 つを、表 10.3 のとおりに決めた。どれも探索していない。既定的な値を置いて固定してある。

BERT の学習率が 2 桁小さいのは、事前学習で得た係数を**壊さない**ためである。すでに日本語について学んだ状態から出発するので、大きく動かすとその知識が失われてしまう。エポックが 3 回と少ないのも同じ理由による。

図 10.8 に 2 つの表を重ねた。標準偏差を縦線で描くと、4 条件のうち 3 つは破線 (52 列の最上位) と縦線が重なる。はっきり下回っているのは文字レベル (漢字マスク) だけである。シリーズを分けた CV では、正解率 0.913 に標準偏差 0.041 を足した縦線の上端が 0.954 で、破線の 0.975 を 0.021 下回る。残る 3 つについて、この図から読み取るべきは順位ではなく、**破線と区別がついていない**と

^{*3} **微調整** (fine-tuning) は、事前学習済みのモデルの係数を初期値として、手元の課題のデータで学習を続けることをいう。第 10.1 節の埋め込みは係数を動かさずに出力を借りただけだったが、微調整では事前学習で得た係数そのものが更新される。

表 10.3 生の文章を渡した実験の学習設定

	文字レベル Transformer	日本語 BERT
係数の初期値	乱数	事前学習済み
断片の長さ	256 文字	256 文字
バッチ	64 件	16 件
エポック	8	3
学習率 (最大)	3×10^{-4}	2×10^{-5}
止め方	エポック数を固定	エポック数を固定

表 10.4 文字レベル Transformer(この 74 作品だけで学習)。数字は断片ごとの確率を作品単位で平均してから判定した正解率で、平均と標準偏差は 5 つのフォールドにわたるもの

入力	層別 CV		シリーズを分けた CV	
	平均	標準偏差	平均	標準偏差
生テキスト	0.973	0.053	0.967	0.041
漢字をマスク	0.933	0.042	0.913	0.041

表 10.5 日本語 BERT の微調整。数字は断片ごとの確率を作品単位で平均してから判定した正解率で、平均と標準偏差は 5 つのフォールドにわたるもの

入力	層別 CV		シリーズを分けた CV	
	平均	標準偏差	平均	標準偏差
生テキスト	0.973	0.033	0.917	0.105
漢字をマスク	0.987	0.027	0.983	0.033

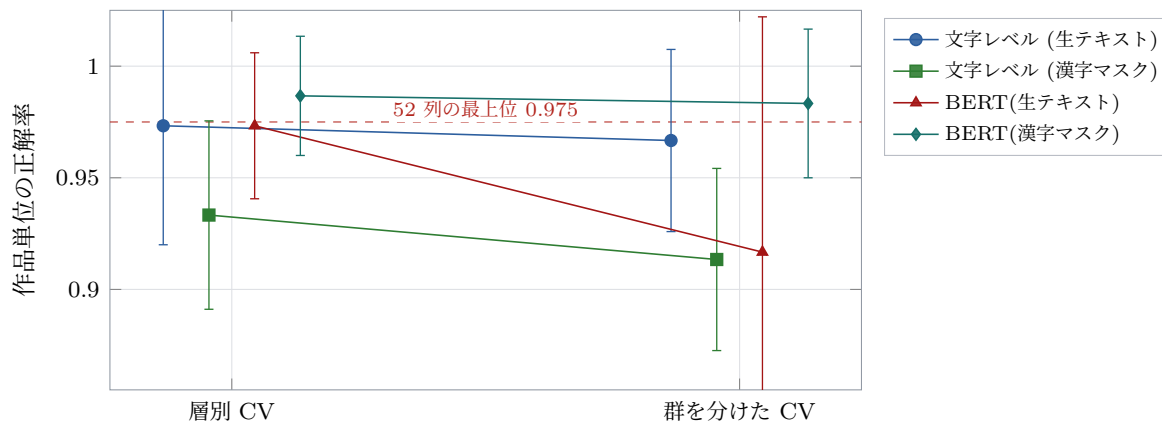


図 10.8 表 10.4 と表 10.5 をまとめたもの。縦軸は作品単位の正解率、縦線は 5 つのフォールドにわたる標準偏差である。破線は比較の相手にする 52 列の最上位 0.975。4 条件は縦線が重ならないよう左右に少しずらしてある。文字レベル (漢字マスク) だけが破線を下回る

いう一点である*4。

比べる相手を先に決めておく。表 10.2 のとおり、52 列でシリーズを分けた CV の最上位はランダムフォレストの 0.975 である。以下、層別 CV では 52 列の最上位 0.987、シリーズを分けた CV では 0.975 を相手にする。最も弱かった MLP の 0.946 と比べれば話は簡単になるが、それでは比較にならない。**比べる相手は、素朴なモデルの中で最も強かったものにする。**

なお第 10.1 節の表では、同じ 52 列・同じロジスティック回帰の正解率が、層別 CV で 0.994、群を分けた CV で 0.981 になっている。測り方が違うためである。あちらは群を 4 分割し、分割の乱数を 10 通り変えて平均したもの、こちらは深層モデルと同じ 5 分割・1 通りの分割で測ったものである。比べるときは**同じ手続きで測った数どうし**を並べる。数字が独り歩きしやすいのはここで、表をまたいで最も都合のよい値を拾ってくると比較が壊れる。

以下、正解率と断らずに書いたものは作品単位である。断片ごとの確率を作品でまとめてから判定した値で、断片単位の場合はそう明記する。

読む前に、**標準偏差を見る癖**をつけておきたい。シリーズを分けた CV の、5 つのフォールドにわたる標準偏差は、条件によって 0.033 から 0.105 まで開いている。5 分割なので 1 回のテストはおおよそ 15 作品、1 作品の当たり外れが 0.067 にあたる。0.01 や 0.02 の差を順位として読むことはできない。

そのうえで、表から読み取れるものを順に挙げる。

1 から学習しても、52 列とほぼ同じ成績になる 文字レベル Transformer に生テキストを渡したときの正解率は、層別 CV で 0.973、シリーズを分けた CV で 0.967 である。比べる相手である 52 列の 0.987 と 0.975 に対して、差は 0.014 と 0.008 しかない。どちらもこのモデルの標準偏差 (0.053 と 0.041) の中に収まっている。

8233 件の断片、文字の並びだけ、事前学習なし。それでも人が設計した 52 列と同じ水準の成績が出た。「深層学習には大量のデータが要る」という一般論は、少なくともこの課題では当てはまらなかった。

ただし**同じ水準**までである。上回ってはいない。そして手間は 1 条件あたり GPU で 90 秒、相手は CPU で数秒である。

事前学習は、生の文章では活きなかった BERT に生テキストを渡したときの正解率は、層別 CV で 0.973、シリーズを分けた CV で 0.917 である。層別 CV は文字レベルと同じ値だが、シリーズを分けた CV は文字レベルの 0.967 を 0.050 下回る。**74 作品だけで学習したモデルのほうが高い。**

もっと目を引くのは標準偏差である。シリーズを分けた CV で 0.105 あり、4 条件の中で最も大きい。分割の引き当てかたに、このモデルが最も振り回されている。

理由の候補は挙げられる。BERT は文字ではなく語 (サブワード) を見るので、「玄德」や「孔明」といった固有名詞をそのまま 1 つの単位として拾える。その巻が訓練側に残る分割では当たり、残らない分割では外す。平均が中くらいで標準偏差が大きいのは、この形の誤りと矛盾しない。文字レベルのモデルは固有名詞を語として掴みにくいぶん、文体寄りの手がかりに寄った、というのが対になる候補である。

*4 縦線が重なっていても差がないとは限らない。その扱いは第 8 章第 8.3.5 項で述べる。

ただし、ここにあるのは平均とフォールド間の標準偏差だけで、乱数の系列も 1 本である。この数字だけでは、上の説明を他の候補 (入力長さ、語彙と未知語の扱い、学習の不安定さ) から区別できない。確かめるなら、作品ごとの誤分類、入力長さ、語彙の重なり、複数の乱数系列の結果をそろえて見る必要がある。本節では仮説として記すにとどめる。

事前学習済みだから強いとは限らない。何で学習されたモデルが、いま当てたいものと合っているかによる。

漢字を消す効果は、モデルによって逆になる 本節の 3 点のうち、事前の予想から最も外れたのがこの結果である。

BERT に漢字マスクを渡すと、シリーズを分けた CV の正解率が 0.917 から 0.983 へ 0.066 上がった。標準偏差も 0.105 から 0.033 へ、3 分の 1 に縮んだ。**情報を減らしたのに成績が上がった。**

ところが文字レベル Transformer では**逆になる**。同じシリーズを分けた CV で、作品単位の正解率が 0.967 から 0.913 へ 0.054 下がった。断片単位の正解率も 0.738 から 0.705 へ 0.033 下がる。

同じ前処理が、モデルによって逆に働いている。「漢字を消すと良くなる」という規則ではない。分かれ目は**そのモデルが漢字から何を読んでいたか**にある。

- BERT は漢字を含む語を**語**として持っている。漢字を潰すと、話題を運ぶ語がまとめて消える。残るのは仮名の並びと文の切り方で、これはどの著者にも同じ条件で当てはまるから、分割の運に左右されなくなる
- 文字レベル Transformer は**文字**を語彙とする。漢字を潰すと語彙が 3533 文字から 225 文字へ、15 分の 1 に減る。消えるのは話題だけではない。どの漢字を選ぶか、旧字を使うか、熟語をどれだけ重ねるかという**文字の使い分けそのもの**も一緒に捨てている

前処理は、モデルと組にして考えるものである。

最も高かった条件と、その差の大きさ BERT に漢字マスクを渡した条件が最も高い。層別 CV の正解率 0.987 は 52 列の 0.987 と同じ値で、シリーズを分けた CV の 0.983 はランダムフォレストの 0.975 を 0.008 上回る。

ただし **0.008** である。BERT 側の標準偏差 0.033 の 4 分の 1 で、乱数しだいで順位が入れ替わりうる。GPU で 1016 秒かけて学習した 1 億パラメータのモデルが、52 個の数と直線 1 本に、条件を厳しくしても並ばれている。

上がった理由を測る BERT で上がったことについて、もっともらしい説明が 2 つ思いつく。

1 つは**未知語が減ったから**という説明である。青空文庫の底本には旧字や稀用漢字が多いから、生テキストでは相当数の字が「未知語」という同じ記号に潰れていて、それが成績を下げていたのではないか。

もう 1 つは**話題が消えたから**という説明である。漢字は話題を運ぶ。『三国志』を訓練で見たモデルは「玄德」や「孔明」で吉川英治を当てられるが、それは文体ではない。その手がかりが使える分割では高得点が出て、使えない分割では落ちる。だから平均が上下し、標準偏差が膨らむ。

前者は測れる。測ってみると、支持されなかった。生テキストで語彙から漏れる文字は全体の 0.1%、BERT のトークナイザが未知語として扱うトークンは 0.01% しかない。0.917 と 0.983 の差

を生む量ではない。

ついでに、断片の長さがそろっているかも確かめた。断片は 256 文字にそろえてあるが、BERT が読むのは文字ではなくトークンなので、条件によってトークン数が変わり、上限で切られる恐れがある。

測ってみると、心配した向きとは逆だった。1 断片あたりの平均トークン数は生テキストが 202(標準偏差 12)、漢字マスクが 223(標準偏差 11) で、**漢字を消したほうが増えている**。「漢」の連なりが細かく分割されるためである。それでも上限を超えて切り捨てられた断片は、生テキストで 0%、漢字マスクで 1% と、どちらも 1% に満たない。2 つの条件で、モデルが読んでいる範囲は実質的に変わっていない。

消去法で残るのは話題の除去である。積極的な手がかりもある。話題説が正しければ、成績は**分割の引き当てかた**に左右されるはずで、『三国志』の巻が訓練側に残る分割では高く、残らない分割では低くなる。実際、シリーズを分けた CV の標準偏差は生テキストで 0.105 と大きく、漢字を消すと 0.033 に落ちた。分割ごとの当たり外れが消えたことになる。

文字レベルで逆に下がったことも、この説明と矛盾しない。文字レベルのモデルはもともと固有名詞を語として使えていないので、消して得るものが少なく、失うもの (文字の使い分け) のほうが大きかった、という読みになる。

× アンチパターン 10.3 — 測れば確かめられる原因の説明を測らずに書く —

「漢字を消したら上がった」に対して、未知語が減ったからだ、という説明は自然に思いつく。語彙は 3533 文字から 225 文字に減っているのだから、何割かは未知語だったに違いないと考えたくなる。

実際に語彙から漏れていた文字は、全体の 0.1% だった。思いついた時点で書いていたら、そのまま間違いが残っていた。**説明のほうも仮説である**。測れる形に言い直せるなら、測ってから書く。

もう一つ、この節には**反証**が転がっていた。同じ前処理で文字レベルは下がっている。「語彙が減ると良くなる」が一般の規則なら、こちらも上がるはずである。自分の説明に合わない条件が手元にないかを、先に探す。

特徴設計というと「何を数えるか」を考えることだと思いがちだが、生の入力を受け取るモデルに対しては**何を見せないか**も同じくらい成績を左右する。本節では漢字を 1 文字に潰しただけで、BERT のシリーズを分けた CV の正解率が 0.917 から 0.983 に上がり、フォールド間のばらつきは 3 分の 1 に縮んだ。

ただしこれを一般の規則にはできない。同じ操作で文字レベル Transformer は下がっている。

○ 定石 10.1 — 前処理を 1 つ足したらモデルごとに測り直す —

前処理の良し悪しは、**そのモデルが何を手がかりにしていたか**で決まる。落とすものが「関係のない手がかり」なのか「そのモデルが頼っていた手がかり」なのかは、モデルによって違う。

本節の漢字マスクがそうだった。BERT では話題を運ぶ語だけが消えたので成績が上がり、文字レベル Transformer では文字の使い分けまで消えたので下がった。同じデータ、同じ前処

理、逆の結果である。

前処理を 1 つ足したら、使うモデルすべてで測り直す。1 つのモデルで良かったから他でも良い、とは言えない。

計算の手間には大きな差がある。文字レベル Transformer は 1 条件あたり 90 秒、BERT は 960 秒で、どちらも GPU を使った値である。52 列のランダムフォレストとロジスティック回帰は、CPU で数秒しかかからない。最も高かった BERT は、GPU を使ってなお 2 桁以上の差がある。その差を払って得られたのは、シリーズを分けた CV の正解率で 0.008 という、標準偏差に埋もれる程度の上積みだった。

補足 漢字マスクは話題だけを消したわけではない

漢字マスクは話題を消すと同時に、**漢字の位置と連続する長さ**という情報を残している。「漢漢漢」と「漢」では見え方が違うから、語の切れ目や熟語の長さは伝わっている。これは第 3 章の字種比率を、系列の形にしたものと見ることもできる。話題だけをきれいに取り除いたわけではない。

また、試したのは事前学習済みモデル 1 つ、断片の長さ 1 通り、学習の設定 1 組である。モデルを変えれば序列は変わりうる。

10.2.5 設計の仕事は「何を数えるか」から「何を見せないか」へ移る

深層学習が本領を出すのは、おおむね次の場合である。

- **データが桁違いに多い。**数万から数百万件あれば、係数の多さが表現力として働く
- **生の入力に構造がある。**画像の画素配置、音声の波形、文章の語順。構造を捉える層 (畳み込み、注意機構) が用意されている
- **良い特徴の作り方が分からない。**何を数えればよいか見当がつかない領域では、表現を学ばせるほうが速い
- **学習済みモデルを借りられる。**自分で 1 から学習する必要がなく、大量のデータで得た知識をそのまま持ち込める

本書の題材を、4 つに照らしてみる。

1 つ目の**データの量**は足りない。点は 74 個しかない。2 つ目の**生の入力の構造**はある。文章には語順があり、第 10.2.4 節ではそれをそのまま渡した。3 つ目の**良い特徴の作り方が分からない**は当てはまらない。何を数えればよいかは、読点と字種という領域の知識から決められた。4 つ目の**学習済みモデルを借りられる**は当てはまる。

条件が半分しか揃っていないのに、結果は「深層学習の出番はない」とは言えないものだった。

74 作品だけで 1 から学習した文字レベル Transformer が、シリーズを分けた CV の正解率 0.967 に達している。ランダムフォレストの 0.975 との差は 0.008 で、標準偏差の中である。1 つ目の条件 (データの量) が満たされていなくても、ここまでは来た。

最も高かったのは、事前学習済みの BERT に漢字を消した入力を渡した条件である。作品単位の正解率は、層別 CV で 0.987、シリーズを分けた CV で 0.983 である。どちらも 52 列と同等か、わ

ずかに上である。

ただし、そこまで来るのに2つのものが要った。1つは公開された学習済みモデルで、これは借り物である。もう1つは**漢字を消すという前処理**で、これは人が決めた。生の文章をそのまま渡した場合、シリーズを分けた CV の正解率は 0.917 にとどまり、自前の文字レベルモデルの 0.967 にすら負けている。設計の仕事が消えたのではなく、**何を数えるかから何を見せないか**に移っただけである。

そして、追い抜かれた側の 52 列は、CPU で数秒、係数は著者 1 人あたり 53 個 (52 列ぶんと切片 1 つ) で、8 人ぶん並べても 424 個にすぎない。どの列が判別をどちらへ動かしたかを人が読める。差が標準偏差に埋もれる程度なら、そちらを選ぶ理由は十分にある。

○ 定石 10.2 — 深層学習を使う前に素朴なモデルの成績を出す

モデルを取り替える前に、ランダムフォレストやロジスティック回帰の成績を測る。どちらも設定するものがほとんどなく、数秒で終わる。

上回れなかったときに、それが分かる。上回れたときも、**どれだけ上回ったか**を言えるのは比べる相手があるからである。

比べる相手は**素朴なモデルの中で最も強かったもの**にする。本節の実験でも、シリーズを分けた CV の相手を最上位の正解率 0.975 にするか、最下位の MLP の 0.946 にするかで、「ようやく並んだ」とも「大きく上回った」とも書けてしまう。相手を選んだ時点で結論はほとんど決まる。第 7.2 節で見た**基準線を先に置く**という手順そのものである。

× アンチパターン 10.4 — 前処理を省いてモデルに任せる

「深層学習なら前処理を自分で学んでくれる」は、半分は正しいが、そのぶんの負担をモデルとデータが払うことになる。表 10.1 では、正規化を省いた入力で MLP が木や線形モデルより粘ったものの、先に見たとおり正規化した入力に 0.039 負けている。表 10.5 では、事前学習済みの BERT ですら、漢字を消しておいたほうが成績が上がった。

領域の知識で 1 行書けば済むことを、モデルに推測させる理由はない。**知っていることは前処理で渡し、知らないことだけをモデルに学ばせるのが順序**である。

10.3 手作りの特徴が残った理由

52 列は、4096 次元の埋め込みにも、GPU で学習した 1 億パラメータのモデルにも、この課題では並ばれていない。勝った理由は、モデルの優劣ではなく**課題と特徴の噛み合い**にある。埋め込みは「何が書かれているか」に近い文書を近くに置くよう学習されており、読点をどこで打つかのような、書き手が意識せず内容とも無関係に出る癖は、その目的関数がわざわざ拾おうとする情報ではない。

裏を返せば、噛み合いが崩れれば結論も変わる。実際、ライトノベルを足して著者ごとに 1 シリーズしかない状況にすると順位が入れ替わった。第 11 章では、その「問いを変えると何が起きるか」を正面から扱う。

第 11 章

ジャンルを分ける

11.1 問いを変えてみる

ここまで、近代文学 74 作品で著者を当ててきた。最後に、別のデータを足して別の問いを立てる。ライトノベル 60 区間 (6 シリーズ) を加える。『本好きの下剋上』『薬屋のひとりごと』『Re: ゼロから始める異世界生活』『無職転生』『片田舎のおっさん、剣聖になる』『サイレント・ウィッチ』を、それぞれ 10 区間に区切ったものである (第 1.6.1 節)。

合わせて 134 点、14 名になった。

ここで立てる問いは次の 1 つである。

作品が近代文学かライトノベルかを、文体だけで判別できるか。

補足 テキストは配れないが特徴量なら配れる

第 1 章でも触れたとおり、近代文学は著作権が切れているのでテキストそのものを配布でき、ライトノベルは著作権が生きているので**特徴量** (feature)、つまり第 3 章で作った 52 個の数値しか配れない。ここでは著作権の側の詳細を補っておく。本書の 8 名で最も没年が遅いのは谷崎潤一郎 (1965 年) で、旧法の保護期間 (没後 50 年) を 2015 年末に満了している。2018 年の保護期間延長 (50 年から 70 年) は、その時点で既に消滅していた著作権を復活させないので、8 名全員がパブリックドメインである。なお青空文庫の「収録ファイルの取り扱い規準」は、著作権切れ作品の複製・再配布を有償無償を問わず認めたくて、底本・入力者・校正者などの記載を残すことと、改変した場合は作業履歴を明記することを希望している。本書の配布物もこれに従っている [74]。

特徴量なら配ってよいのは、「平仮名の比率が 0.633」「読点の前に『て』が来る割合が 0.198」といった数値からは**もとの文章を復元できない**からである。

これは実務でもよく使う考え方である。個人情報を含むデータを共有できないとき、集約統計量や埋め込み表現だけを渡す。ただし「復元できない」の保証は自明ではなく、特徴量が細かすぎると復元されうる。モデルや特徴量から入力を推定する攻撃はモデル反転攻撃 (model inversion attack) と呼ばれ、研究が続いている。

11.2 まず眺める

字種の比率を見る。第 3 章の図 3.3 と同じ特徴だが、今度は横軸を**片仮名**にする。

見ただけで分かってしまう。ほとんど重なっていない。

1 点あたりの片仮名の比率を群ごとに平均すると、近代文学 74 作品では 0.0133、ライトノベル 60 区間では 0.0748 になる。ライトノベルのほうが高く、**5 倍以上の開き**がある。

理由は次のようなことだろう。ライトノベルには「スキル」「ステータス」「ダンジョン」といった片仮名語が多く、登場人物の名前も片仮名であることが多い。一方、近代文学では片仮名が出る場面

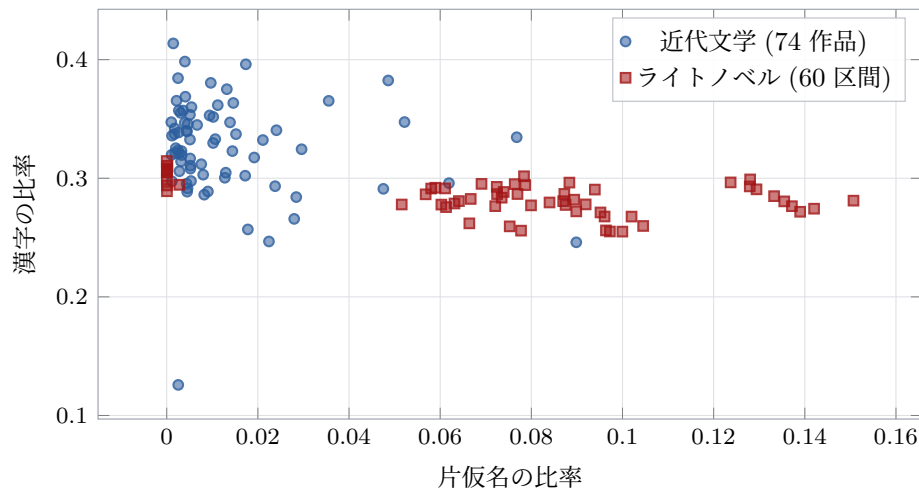


図 11.1 片仮名と漢字の比率。ほとんど重なりがない。

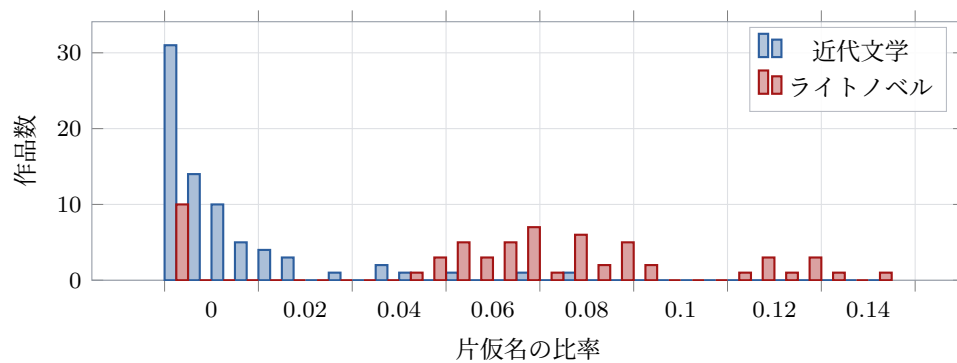


図 11.2 片仮名比率の分布。ほぼ分離している。

が外来語や擬音や強調などに限られがちなのが一因だろう。もっともこれは、数値を眺めてつけた解釈である。

11.3 分類してみる

全 134 点それぞれが近代文学かライトノベルかを、第 3 章の 52 列だけを使って 3 つのモデルで判別する。測るのは 5 分割交差検証での正解率である。作品と区間をばらばらに分ける通常の層別 CV と、同じシリーズをまとめて片側に置く CV の 2 通りで測る。基準線は、最頻クラス (近代文学) を答え続けたときの正解率 0.552 である。

ランダムフォレストは、通常の層別 CV で正解率 1.000 である。134 点すべてのジャンルを正しく当てた。基準線 0.552 を 0.448 上回っている。

11.4 喜ぶ前に

正解率 1.000 という数字が出たら、第 9 章の作法どおり、まず疑ってかかる。

表 11.1 134 点のジャンル判別の正解率

モデル	通常の層別 CV	シリーズを分けた CV
決定株 (深さ 1 の木)	0.941	0.815
k 近傍法 ($k = 5$)	0.993	0.982
ランダムフォレスト	1.000	0.978
基準線 (最頻クラス)	0.552	

11.4.1 疑い 1 — 片仮名 1 列だけで 9 割近く当たる

たった 1 つの特徴で解けるのではないか確かめる。

深さ 1 の決定木 (決定株) は、「ある特徴がある値を超えるか」という 1 回の質問しかできない。これで何割当たるかを見る。

表 11.1 の決定株は、通常の層別 CV でジャンル判別の正解率が 0.941 だった。これに対し、表 11.2 に並ぶ正解率はどれもそれより低い。前者は 52 列の中から不純度をいちばん下げる 1 列を毎回選べるのに対し、後者は使う列を 1 つに固定して測っているからである。

表 11.2 列を 1 つに固定したときのジャンル判別の正解率 (134 点、決定株、5 分割の層別 CV)。文長の 2 列は対数を取った値である

特徴	正解率	近代文学の平均	ラノベの平均
片仮名の比率	0.888	0.0133	0.0748
漢字の比率	0.806	0.3262	0.2840
文長 μ (対数)	0.718	3.3975	3.1258
文長 σ (対数)	0.612	0.7723	0.7240
平仮名の比率	0.590	0.6605	0.6412

いちばん高いのは片仮名の比率で、この 1 列だけで正解率 **0.888** である。基準線 0.552 を 0.336 上回っている。「片仮名が 5.5% を超えたらライトノベル」という 1 行のルールで、134 点のうち 9 割近くを正しく分類できている。

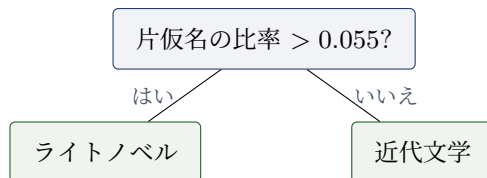


図 11.3 ジャンル判別の正解率が 0.888 になるモデルの全体像。図の木は説明用に全 134 点で学習したもので、しきい値の 5.5% はそのときの値である。0.888 は交差検証の各分割でしきい値を学び直したときの平均正解率で、この一つの木に対する値ではない。

× アンチパターン 11.1 — 最も単純なルールを試さずに高い精度を誇る

ジャンル判別の正解率 1.000 という数字だけを見ると、52 次元のランダムフォレストの働きに見える。だが片仮名の比率 1 列に固定した決定株で正解率 0.888、52 列から 1 列を選んで 1 回だけ質問する決定株で 0.941 になる。ランダムフォレストが上積みしたのは、0.941 から 1.000 までの 0.059 にすぎない。

「機械学習で 99% の精度を達成」と報告する前に、**最も単純なルールで何割当たるかを測る**。決定株、線形しきい値、多数決。これらを基準線に置く。

差がわずかなら、複雑なモデルを使う理由は**精度以外のところ**になければならない。

11.4.2 疑い 2 — ジャンルと時代が交絡している

モデルが学んだのは「ジャンルの違い」だと思いたくなるが、このデータではもう 1 つの候補がある。**時代の違い**である。

近代文学はおおむね 1890 年代から 1960 年代、ライトノベルは 2010 年代の作品である。およそ 100 年の間に日本語の表記習慣が変わった。

- 片仮名語 (外来語) の増加
- 漢字使用の減少 (当用漢字・常用漢字の制定)
- 文の短文化
- 会話文の比率の増加

決定株が選んだ片仮名の比率は、そのまま表記習慣の変化を映す列でもある。つまりこのモデルの成績は、ジャンルの違いによるものか時代の違いによるものか、**このデータでは切り分けられない**。

切り分けるには、同じ時代の作品を両側に用意するしかない。本書のコーパスにはそれがないので、「1940 年以前の文章か、2010 年以降の文章か」を見分けている可能性を排除できないままである。このモデルを「文体判別器」と呼んで配るのは、その意味で危うい。

補足 同じ時代どうしを比べれば交絡は消える

同じ時代の作品どうしを比べれば、時代の効果は消える。

たとえば「2010 年代のライトノベルと、2010 年代の一般文芸を判別する」なら、本当にジャンルの違いを見ていることになる。

問いを立てるときは、**比較したいもの以外の差**が入り込んでいないかを確認する。実験計画でいう**交絡 (confounding)**の問題である。

11.4.3 疑い 3 — シリーズを分けてもほとんど落ちない

ランダムフォレストのまま、同じシリーズをまとめて片側に置く CV にすると、ジャンル判別の正解率は 1.000 から 0.978 に落ちた。差は 0.022 である。

落ちること自体は当然である。ライトノベル側は 6 シリーズしかないので、6 シリーズを 5 つに分ければ、1 回のテストにシリーズ 1 つか 2 つが丸ごと入る。そのシリーズの文章は訓練で一度も見ていない。

それでもシリーズを分けた CV で正解率 0.978 であり、基準線 0.552 を 0.426 上回っている。ジャンル判別に関しては、シリーズ固有の癖に頼らずとも判別できているということである。近代文学とライトノベルを分ける手がかり (片仮名の多さ、文の長さ) はシリーズをまたいで共通しているからである。

次節で見る著者判別は、これとまったく対照的になる。

11.5 著者判別に戻ると

同じ 134 点、同じ 52 列のまま、ジャンルではなく 14 名の著者を当てる問題に切り替えると、事態は一変する。

表 11.3 134 点・14 名の著者判別の正解率と落差

モデル	通常の層別 CV	シリーズを分けた CV	落差
k 近傍法 ($k = 5$)	0.985	0.555	+0.430
ランダムフォレスト	0.970	0.533	+0.438

落差は、通常の層別 CV の正解率からシリーズを分けた CV の正解率を引いた差である。近代文学 74 作品だけで著者を判別したときは、ランダムフォレストの落差が 0.011 だった (第 9 章の表 9.5)。それが 134 点・14 名では、k 近傍法でもランダムフォレストでも **0.43 以上になった**。40 倍近い。跳ね上がったのは、ライトノベルには**著者ごとに 1 シリーズしかない**ためである。

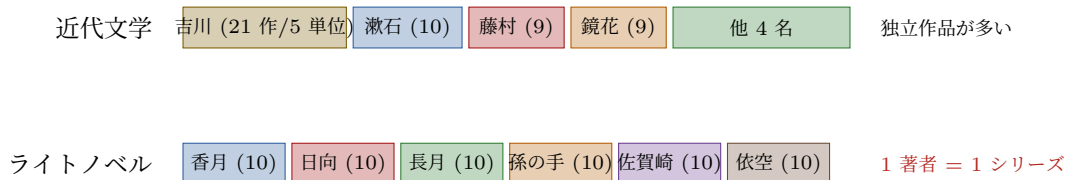


図 11.4 同じ「作品数」でも中身が違う。ライトノベルは著者とシリーズが一对一。

シリーズを分割の単位にすると、テストに現れる著者の作品が訓練データに 1 つも残らない。未知の著者を当てることはできないので、正解率は落ちる。

第 9 章で見たとおり、ライトノベル 60 区間だけで 6 名の著者を文字 3-gram で判別すると、通常の CV で正解率 1.000 だったものが、シリーズを分けた CV では 0.000 になった。極端な形で同じことが起きている。

○ 定石 11.1 — 独立した単位がいくつあるかを先に数える

「134 作品ある」という数字だけでは、何も分らない。

- 独立した単位はいくつあるか
- 各クラスに独立単位はいくつあるか
- 予測したい対象は、訓練データに現れるものか、現れないものか

本書のライトノベルは、**著者判別の教材としては使えない**。だがジャンル判別や、シリーズ内での区間の判別には使える。データが悪いのではなく、問いとの相性の問題である。

11.6 候補の外にいる著者を棄却する

もう一つ、問いの立て方に隠れている前提がある。

本書のモデルは、どの作品を入れても**必ず 8 名の誰かだと答える**。確率が最も高い著者を選ぶ仕組みなので、「この 8 名の誰でもない」という選択肢がそもそも用意されていない。これを**閉集合の仮定** (closed-set assumption) という。データベース分野の閉世界仮定 (closed-world assumption) から借りた呼び方をされることもある。

現実の著者判別で困るのは、たいていここである。匿名の文書が来たとき、それが手元の候補者の誰かととは限らない。筆跡鑑定、話者認識、不正検知も同じ形をしている。候補者リストの外にいる人物を、リストの中の誰かに割り当ててしまう。

11.6.1 未知の著者を入れてみる

実験してみる。近代文学 74 作品に戻り、夢野久作を訓練データから外して、残り 67 作品・7 名で学習する。モデルはランダムフォレスト、特徴は第 3 章の 52 列である。そのうえで、夢野久作の 7 作品を投入する。

モデルは当然、全部を 7 名の誰かだと答える。正解は 1 つもない。だが**答え方に**手がかりが残っている。ランダムフォレストが返す 7 名ぶんの確率のうちの最大値 (以下**確信度**と呼ぶ) を見ると、夢野久作の 7 作品では確信度の中央値が 0.23、最大値でも 0.36 である。学習した 7 名の作品では、確信度がこれよりずっと高い。

当たっているかどうかは分からなくても、自信がないことはモデル自身知っている。ここを使う。

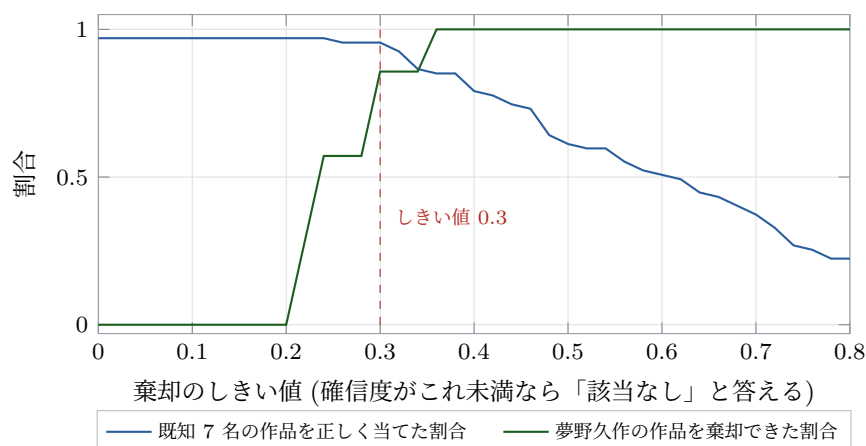


図 11.5 棄却のしきい値を動かしたときの 2 つの割合。上げるほど未知の著者を弾けるが、既知の著者も弾いてしまう。

図 11.5 が、しきい値を動かしたときの 2 本の線である。確信度がしきい値未満なら「該当なし」と答えることにする。

2本の線は別々の量である。**既知側の正解率**は、既知7名の67作品を5分割交差検証で判別したとき、正しい著者名を答えられた作品の割合である(「該当なし」と答えたものは不正解として数える)。**未知側の棄却率**は、夢野久作の7作品を「該当なし」に落とした割合である。

しきい値を0にすれば、これまでどおり必ず誰かだと答える。このとき既知側の正解率は0.9701、未知側の棄却率は0である。しきい値を0.3まで上げると、未知側の棄却率が0.8571まで上がり、既知側の正解率はそこから0.9552に下がるだけである。0.4まで上げれば未知側の棄却率が1になり、夢野久作の7作品を**すべて弾ける**。その代わり、既知側の正解率は0.7910まで下がる。

第8章のROC曲線と同じ構造である。未知側の棄却率を上げれば、既知側の正解率が下がる。どこに置くかは、**どちらの誤りが高くつくか**で決める。未知の人物を既知の誰かだと誤って名指しする害が大きいなら、しきい値は高めに置く。

補足 未知の書き手が既知の誰かに似ているとこの方法は通じない

しきい値で夢野久作を弾けたのには理由がある。訓練から外した夢野久作の文体が、残り7名のどれとも似ていなかったからである。どの著者にも当てはまらない作品を渡されたモデルは、7名のどこにも票を集められない。その結果として確信度が下がり、しきい値で拾えた。

逆の場合を考えてみる。未知の書き手が、たまたま既知の7名のうち誰かと似た書き方をしていたとする。モデルはその1名に票を集める。そのときの確信度は0.9にも0.95にもなりうる。訓練で見た著者の作品を当てたときと、数字の上で区別がつかない。しきい値をどこに置いても弾けない。

しかも、候補として並べる著者を増やすほど、「たまたま似ている誰か」が混じる見込みは高くなる。候補を網羅するほど正解を含む見込みは上がるが、「該当なし」と答えさせる目的からは逆に働く。

確信度のしきい値のほかにも方法はある。既知の著者たちの分布からどれだけ離れているかを測る**外れ値検知**や、「その他」というクラスを作って明示的に学習させるやり方があり、まとめて**オープンセット認識**(open-set recognition)と呼ばれている。ただしどの方法を使っても、**未知がどれくらい既知に似ているか**という難しさの根は変わらない。

× アンチパターン 11.2 — 答えの候補が出そろっている前提に気づかない

分類器は、訓練で見たクラスの中からしか答えを選べない。「どれでもない」と答えさせたいなら、その仕組みを**自分で足す**必要がある。

見分け方は、次の問いを自分に投げることである。**本番で来るデータは、必ず訓練で見たクラスのどれかに入るか**。入らないものが来るなら、確信度のしきい値なり外れ値検知なりを設計に組み込む。

そして棄却を入れたら、評価指標も変える。正解率だけでは「該当なしと答えた分」を数えられないので、既知側の正解率と未知側の棄却率を**並べて報告する**(図 11.5)。

11.7 2次元で眺める

最後に、134作品をPCAで2次元に落とす。

第1主成分に沿って、きれいに2つに分かれている。寄与率は第1主成分が0.181、第2主成分が0.140で、2つ合わせても3割程度でしかない。それでも分離ははっきり見える。

第6章では「寄与率が低い図で距離を語るな」と書いた。それでも図 11.6 が信用できるのは、射影は群どうしを**重ねて**見せることはあっても、**偽の分離を作る**ことはないからである。寄与率の低い

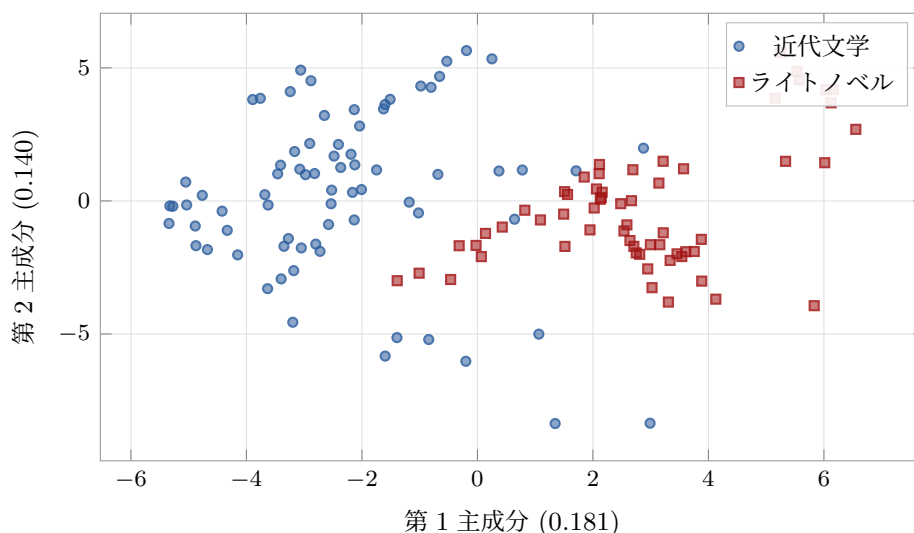


図 11.6 134 作品の PCA。2 つの群がはっきり分かれている。

射影でこれだけ分かれて見える以上、元の 52 次元の空間でも 2 つの群は分かれている。

図から読み取れるのは「分かれている」ことだけで、「どれだけ離れているか」ではない。その区別は保っておきたい。

11.8 当てているものと、当てたいものは違うことがある

本章で見たのは、当てられてはいるが、当てているものが問いとずれている、という事態だった。最後にこれを一般の形に広げておく。

本書は「著者を当てる」という問いを立てて、それに対応するラベル (作品に付いた著者名) を持っていた。問いとラベルが一致している、恵まれた設定である。

実務ではそうならないことのほうが多い。本当に知りたい量が測れないので、測れる別の量で代用する。これを**代理変数**という。

米国で数千万人に使われた医療リスクの層別アルゴリズムは、「翌年の医療費」を予測していた [62]。医療の必要度そのものは測れないので、費用で代用した。ところが同じ費用でも、医療への接し方の違いから、黒人患者のほうが実際には重症だった。代用したことで、必要な人ほど追加ケアの対象から外れる仕組みができていた。予測する対象を「費用」から「疾患の数」に変えると、追加ケアの対象に占める黒人患者の割合は 17.7% から 46.5% に変わった。

モデルの精度をいくら上げて、**代理変数と本来の目的のずれは消えない**。むしろ精度が上がるほど、ずれた目標に忠実になる。

ジャンル判別が時代判別でもありえたのと同じで、**当てているものが、当てたいものと同じかどうか**は、成績を見ても分からない。ラベルがどう作られたかを遡って確かめるしかない。

11.9 この本のまとめ

第 1 章の図 1.1 に、最後から最初へ戻る点線を描いた。その意味がここで分かる。

本書で通ったのは、次の道である。

1. テキストの性質を調べ (第 2 章)
2. 内容に依存しない特徴を設計し (第 3 章)
3. 距離を定義し (第 4 章)
4. まとまりを見つけ (第 5 章)
5. 次元を落として眺め (第 6 章)
6. 著者を当てるモデルを作り (第 7 章)
7. 正しく測る方法を整え (第 8 章)
8. 測り方が間違っていたことに気づき (第 9 章)
9. 学習された表現と比べ (第 10 章)
10. そして問いそのものを立て直した (第 11 章)

最後の 2 つが本書の主題である。

技術の習得は難しくない。scikit-learn を使えば、分類器を作るのは 3 行で済む。

難しいのは、**出てきた数字が何を意味するかを判断すること**である。近代文学 74 作品の著者判別で得た正解率 0.987 という数字が「文体を学んだ」ものなのか「長編の癖を覚えた」ものなのかは、数字を見ても分からない。データがどう作られたかを知り、評価を設計し、モデルが何を見ているかを確かめて、初めて分かる。

良い交差検証ができれば、半分は勝ったようなものだ。

モデルをきちんと評価する方法が見つからないうちは、次の工程に進まない。

— Shubin Dai (2018 年、Kaggle 世界ランキング 1 位当時のインタビューより。

原文 “A good CV is half of success.”[59])

11.10 章末の課題

1. 片仮名の比率を特徴から除いて、ジャンル判別をやり直せ。正解率はどこまで落ちるか。次に決め手となる特徴は何か。
2. 「2010 年代のライトノベルと 2010 年代の一般文芸」を判別する実験を設計せよ。どんなデータが必要か。
3. ライトノベル 6 名について、「同じシリーズの中で、前半の区間と後半の区間を判別する」問題を作れ。作家の文体は連載を追って変化するか。
4. 134 作品でクラスタリングを行い ($k = 2$)、ジャンルとの一致度 (ARI) を測れ。教師なしでもジャンルは見つかるか。
5. 本書で扱わなかった特徴を 1 つ設計し、第 3 章の 3 条件を満たすかを検討したうえで、ジャンル判別と著者判別の両方で効果を測れ。

marimo 教材の使い方

授業や自習で使うために、4 本の marimo ノートブックを用意した。スライダーや選択欄を操作すると計算と図が更新される。例えば、標準化に使うデータを変えたときに k 近傍法の予測がどう変わるかを、点の配置と近傍への線で確認できる。

A.1 教材の構成

配布されたフォルダには、次のファイルが入っている。public フォルダは、ノートブックと同じ場所に置く。

```
01_style_map.py
04_clustering.py
05_classification.py
06_leakage_and_pitfalls.py
public/
    features.csv
```

features.csv は、ノートブックが読み込む近代文学 74 作品の特徴量である。このファイルを移動したり名前を変えたりすると、ノートブックから読み込めなくなる。

表 A.1 各ノートブックで試せる操作。ファイル名の番号は本書の章番号とは異なる

ファイル	内容
01_style_map.py	次元削減による文体の地図、元の特徴空間との近傍の比較、入力文の表示 (第 3・4・6 章)
04_clustering.py	k-means の反復、初期値による違い、デンドログラムの切断 (第 5・6 章)
05_classification.py	分類器の予測確率と、判定のしきい値による誤分類の変化 (第 7・8 章)
06_leakage_and_pitfalls.py	特徴選択と標準化による漏洩、長編単位の分割、次元数による近傍の変化 (第 9 章)

A.2 自分のパソコンで実行する

実行には、Python の環境を管理する uv を使う。uv の導入方法は公式の案内を参照する。

```
https://docs.astral.sh/uv/getting-started/installation/
```

ターミナルで配布フォルダへ移動し、次のコマンドを実行する。

```
uvx marimo run --sandbox 04_clustering.py
```

初回は、必要な Python とライブラリをダウンロードするため、インターネット接続が必要になる。起動するとブラウザに教材が表示される。他の教材を開くには、末尾のファイル名を変更する。教材を終了するときは、起動に使ったターミナルで Ctrl+C を押す。

Python のコードも編集したい場合は、run を edit に変える。

```
uvx marimo edit --sandbox 04_clustering.py
```

marimo では、あるセルの値が変わると、その値を使うセルが自動で再計算される。コードを編集しなくても、画面のスライダーや選択欄だけで各実験を操作できる。

A.3 操作するときの注意

一度に変える条件は一つにすると、結果が変わった理由を調べやすい。例えば分類のしきい値を変えるときは、作家、特徴量、分類器を固定する。誤分類の件数が変わっても予測確率そのものは変わらないことを確かめる。

乱数を使う実験では、乱数の番号も変えて比較する。1 回だけ得られた高い正解率を、その手法の一般的な性能と考える。

A.4 困ったときは

データファイルが見つからない

ノートブックと同じ場所に public フォルダがあり、その中に features.csv が入っているか確認する。

初回の起動が途中で止まる

インターネット接続と、ターミナルに表示されたエラーを確認する。大学などのネットワークでダウンロードが制限されている場合は、管理者に相談する。

図の日本語が四角で表示される

ブラウザが日本語フォントを使えるか確認する。

本書と異なる数値が出る

選んだ特徴量、分類器、分割方法、乱数の番号を確認する。教材では条件を操作できるため、設定によって結果は変わる。

高次元空間の幾何

第4章と第9章で「次元の呪い」に触れた。その正体は、高次元空間の幾何が私たちの直感と大きく違うことにある。

付録では、いくつかの事実を数式で確かめる。2次元や3次元で成り立つ感覚が、なぜ通用しなくなるのかが分かる。

B.1 球の体積は消える

半径 r の d 次元球の体積は次の式で与えられる。

$$V_d(r) = \frac{\pi^{d/2}}{\Gamma(\frac{d}{2} + 1)} r^d \quad (\text{B.1})$$

Γ はガンマ関数で、 $\Gamma(n+1) = n!$ を実数に拡張したものである。

$r = 1$ として d を増やしていくと、意外なことが起きる。

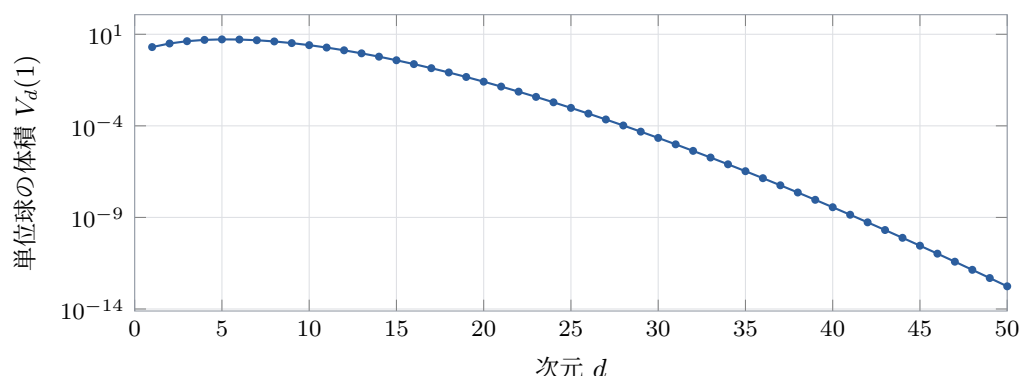


図 B.1 単位球の体積。 $d = 5$ で最大になり、その後ゼロに向かう。

半径 1 の単位球の体積 $V_d(1)$ は、 $d = 5$ のあたりで最大 (≈ 5.26) になり、そこからは急速に減っていく。 $d = 20$ では 0.026 、 $d = 50$ では 1.7×10^{-13} である。

補足 なぜ次元を上げると球の体積が減るのか

式 (B.1) の分子 $\pi^{d/2}$ も、 d とともに指数的に増える。それでも体積が減るのは、分母の $\Gamma(d/2+1)$ が**階乗の速さで増える**からである。 $d = 20$ なら分子は $\pi^{10} \approx 9.3 \times 10^4$ だが、分母は $\Gamma(11) = 10! \approx 3.6 \times 10^6$ で、すでに 2 桁離れている。 d を大きくするほど差は開く一方なので、比は 0 へ向かう。

図形として見る。半径 1 の球は、一辺 2 の立方体 $[-1, 1]^d$ にすっぽり入っている。立方体の体積は 2^d で、これも指数的に増える。球の体積は減るのだから、**立方体の体積のほとんどは、球の外側の「隅」に**

補足（続き）

あることになる。

隅がどれだけ遠いかも見ておく。立方体の中心から面までの距離は、次元がいくつでも 1 である。一方、中心から頂点までの距離は $\sqrt{d}$ で、次元とともに伸びていく。 $d = 100$ なら、中心から面までの距離は 1、中心から頂点までの距離は 10 である。球が占めるのは中心から半径 1 までなので、頂点までの残り 9 のぶんは全部、球に入らない隅ということになる。次元が上がるほど、立方体は「トゲトゲのウニ」のような形になる。

もっと直接的な比較をする。一辺 1 の立方体に内接する球（半径 $1/2$ ）の体積比は

$$\frac{V_d(1/2)}{1^d} = \frac{\pi^{d/2}}{2^d \Gamma(d/2 + 1)} \quad (\text{B.2})$$

表 B.1 一辺 1 の立方体に対する内接球の体積比

次元 d	体積比
2	0.785
3	0.524
5	0.164
10	0.0025
20	2.5×10^{-8}
50	1.5×10^{-28}

$d = 10$ では、内接球の体積は立方体の体積の 0.25% でしかない。立方体の中に一様に点を撒いても、球の中にはほぼ入らない。

立方体の中に一様に点を撒いたなら、「中心付近にデータが集まっている」という直感は通用しない。ほぼすべての点が「隅のほう」にいることになる。

実データが一様に散らばっていることはまずないので、この結論をそのまま持ち込むことはできない。ただし、次元が上がるほど中心付近が空になるという圧力は働く。どれくらい働くかは、データがどんな形に散らばっているかによる。

B.2 体積は薄い殻に集中する

もう一つの奇妙な性質がある。半径 1 の球のうち、半径 $1 - \varepsilon$ の内側が占める体積の割合は

$$\frac{V_d(1 - \varepsilon)}{V_d(1)} = (1 - \varepsilon)^d \quad (\text{B.3})$$

である。式 (B.1) で体積が r^d に比例することから直ちに従う。

$d = 100$ では、半径 1 の球の体積の 99.4% が、表面から厚み 5% の殻（半径 0.95 から 1 までの部分）に入っている。 $d = 500$ では、厚み 1% の殻に体積の 99.3% が入っている。どちらも表 B.2 の内側の割合を 1 から引いた値である。

表 B.2 半径 1 の球の体積のうち、半径 $1 - \varepsilon$ の内側が占める割合

d	$\varepsilon = 0.01$	0.05	0.1	0.2
1	0.990	0.950	0.900	0.800
10	0.904	0.599	0.349	0.107
100	0.366	0.0059	2.7×10^{-5}	2.0×10^{-10}
500	0.0066	7.3×10^{-12}	1.3×10^{-23}	3.5×10^{-49}

高次元の球は、中身のないシャボン玉のようなものである。

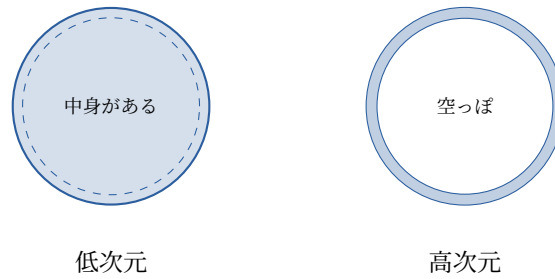


図 B.2 高次元の球では、体積のほとんどが表面近くの薄い殻にある。

B.3 距離の集中

第 4 章の図 4.18 で見た現象を、式で確かめる。

d 次元の単位立方体 $[0, 1]^d$ に一様に点を撒く。2 点 X, Y の距離の 2 乗は

$$\|X - Y\|^2 = \sum_{j=1}^d (X_j - Y_j)^2 \quad (\text{B.4})$$

各項 $(X_j - Y_j)^2$ は独立同分布である。 $X_j, Y_j \sim U(0, 1)$ のとき

$$E[(X_j - Y_j)^2] = \frac{1}{6} \quad (\text{B.5})$$

$$\text{Var}[(X_j - Y_j)^2] = \frac{7}{180} \quad (\text{B.6})$$

したがって

$$E[\|X - Y\|^2] = \frac{d}{6}, \quad \text{Var}[\|X - Y\|^2] = \frac{7d}{180} \quad (\text{B.7})$$

距離そのものについては、大数の法則から

$$\|X - Y\| \approx \sqrt{\frac{d}{6}} \quad (d \rightarrow \infty) \quad (\text{B.8})$$

ここで平均と散らばりの伸び方を比べる。平均距離は $\sqrt{d/6}$ で、 $\sqrt{d}$ に比例して伸びる。一方、距離の標準偏差は次元を上げてほとんど増えず、一定の値に近づく^{*1}。分子が一定で分母だけ伸びるので、変動係数 (標準偏差 ÷ 平均) は

$$CV = \frac{\sigma}{\mu} = O\left(\frac{1}{\sqrt{d}}\right) \rightarrow 0 \quad (\text{B.9})$$

注意すべきは、距離そのものが同じ値になるわけではないことである。最も遠い点と最も近い点の差は縮まらない。縮むのは、その差を平均距離で割った比のほうである。1 万円の買い物での 100 円の差は大きいですが、1 億円の買い物での 100 円の差は誤差に見える。高次元で起きているのは後者である。

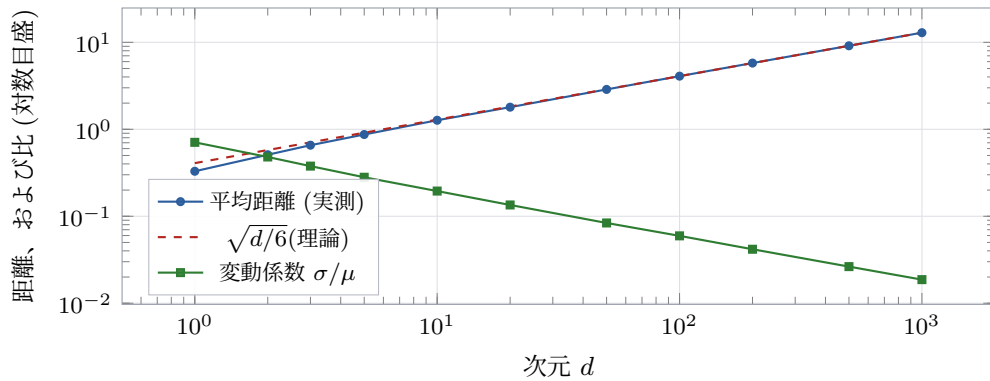


図 B.3 一様分布の点对距離。上の 2 本は距離 (単位あり)、下の 1 本は距離を平均で割った比 (単位なし) である。平均は理論値 $\sqrt{d/6}$ に一致し、変動係数は $1/\sqrt{d}$ で減る。

図 B.3 の青い線が、単位立方体に一様に撒いた点の対距離を実測した平均、赤の破線が理論値 $\sqrt{d/6}$ である。次元が上がるほど両者は重なっていく。緑の線は、距離の標準偏差を平均距離で割った変動係数で、単位のない量である。同じ図に描いてはあるが、青や赤との高さの比較に意味はない。

発展 Beyer らの定理 — 最も近い点と最も遠い点の比が 1 に近づく

Beyer ら (1999) は、より一般的な条件でこの現象を定式化した [21]。

定理 B.1 データ分布が次を満たすとする。

$$\lim_{d \rightarrow \infty} \text{Var} \left[\frac{\|X_d\|}{E[\|X_d\|]} \right] = 0 \quad (\text{B.10})$$

このとき、任意の $\varepsilon > 0$ に対して

$$\lim_{d \rightarrow \infty} P[D_{\max} \leq (1 + \varepsilon)D_{\min}] = 1 \quad (\text{B.11})$$

ここで $D_{\max}, D_{\min}$ は、基準にした 1 点 (クエリ点) から見て最も遠い点・最も近い点までの距離である。「この作品に似た作品を探す」というとき、探す起点になる作品がクエリ点にあたる。

^{*1} 2 乗距離の分散 $7d/180$ から距離そのものの標準偏差を求めると $\sqrt{7d/180} / (2\sqrt{d/6})$ となり、 d が約分されて定数になる。

発展（続き）

日本語にすると「最も遠い点までの距離と最も近い点までの距離の比が、1 にいくらでも近づく」となる。距離そのものが同じ値になるわけではない。だが比が 1 に近づけば、「いちばん近い」という言葉が意味を持たなくなる。どれも同じくらい遠いだから、最近傍探索という操作自体が空回りする。

前提条件は、各次元が独立で分散が有限なら満たされる。逆に言えば、**次元間に強い相関があるデータ**では満たされない。列どうしが連動していれば、見かけの列数ほどには自由に散らばっていないからである。このとき、データが実際に広がっている向きの数を**実効次元**という。実効次元が低ければ定理は適用されず、高次元でも近傍探索が働くことがある。本書の 52 列がそうであるように、実データの列はたいてい互いに相関している。

B.4 高次元ではベクトルがほぼ直交する

もう一つ、直感に反する事実がある。

d 次元空間で、各成分を標準正規分布から取って 2 つのベクトルを作る。そのなす角は、どんな角度も同じくらい起こりそうに思える。2 次元で試せば実際にそうなる。

ところが次元を上げると、そうならない。

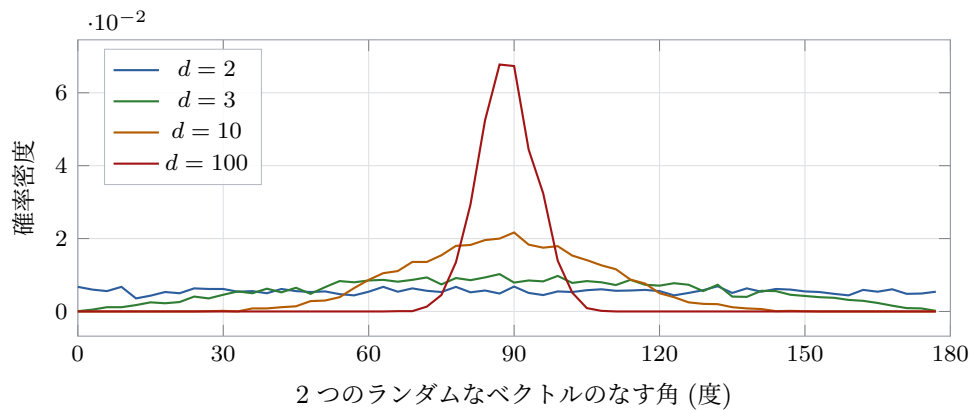


図 B.4 ランダムな 2 ベクトルのなす角の分布。次元が上がるほど 90° に集中する。

$d = 100$ では、標準正規分布から取った 2 ベクトルの組のうち 9 割以上が、なす角 $90^\circ \pm 10^\circ$ の範囲に入る (角度の分布を厳密に積分すると 91.8%)。高次元では、**無関係な 2 つのベクトルはほぼ直交する**。

理由は計算で分かる。 X, Y を標準正規分布から取ると

$$\cos \theta = \frac{X \cdot Y}{\|X\| \|Y\|} \quad (\text{B.12})$$

分子 $X \cdot Y = \sum_j X_j Y_j$ は平均 0、分散 d である。分母は $\|X\| \|Y\| \approx d$ (それぞれ $\approx \sqrt{d}$)。したがって

$$\text{Var}[\cos \theta] \approx \frac{d}{d^2} = \frac{1}{d} \implies \cos \theta = O\left(\frac{1}{\sqrt{d}}\right) \quad (\text{B.13})$$

$\cos \theta \rightarrow 0$ 、すなわち $\theta \rightarrow 90^\circ$ である。

補足 直交性がもたらす良いこと

ほぼ直交するという性質は、悪いことばかりではない。

d 次元空間には、互いにほぼ直交するベクトルを d 個よりずっと多く詰め込める。互いのなす角の余弦の絶対値が ε 以下でよいとすれば、入る本数は $e^{O(\varepsilon^2 d)}$ である。

これがランダム射影 (Johnson–Lindenstrauss の補題) や、高次元の埋め込み表現が多数の語を互いに干渉させずに置ける理由である。「単語埋め込みが 300 次元で数十万語を表現できる」のは、高次元空間が思ったより広いからである。

B.5 呪いと祝福

高次元がもたらす困難と利点をまとめる。

表 B.3 高次元の性質

呪い	祝福
距離が集中し、近傍探索が意味を失う	ランダム射影で低次元に落としても距離が保たれる
必要なサンプル数が指数的に増える	互いにほぼ直交するベクトルを大量に詰め込める
体積が殻に集中し、内挿で補えない	線形分離できる確率が上がる (Cover の定理)
無関係な特徴がノイズとして距離を歪める	多様体仮説により、実効次元は低いことが多い

発展 Cover の定理 — 次元を上げると線形分離しやすくなる

d 次元空間に n 個の点が一般の位置に置かれているとする^a。これらを 2 クラスに分ける方法は 2^n 通りあるが、そのうち 1 枚の超平面で分けられるものの割合は

$$P(n, d) = \frac{1}{2^{n-1}} \sum_{k=0}^d \binom{n-1}{k} \quad (\text{B.14})$$

$n \leq d+1$ なら和が二項係数の全体になるので、 $P = 1$ になる。点の数が次元の数を超えなければ、どんなラベルの付き方でも線形分離できるということである。

低次元では分離できないデータでも、高次元に写してしまえば分けられるようになる。この見通しがカーネル法の背景にある。第 7.4 節のサポートベクトルマシンがカーネルで曲がった境界を引けるのも、同じ考え方による^b。

ただし「分離できる」と「未知のデータで当たる」ことは別である。 $n \leq d+1$ の状況は、訓練データをそっくり暗記できてしまう状況でもある。第 7 章で見た過学習そのものである。

^a どの $d+1$ 個を取っても同じ超平面に乗らない、という条件である。点をでたらめに置けば、ほぼ確実に満たされる。

発展 (続き)

^b ただしカーネル法の要点は、高次元に写した座標を実際には計算せず、内積の値だけを別の式で求めてしまうところにある。Cover の定理はその動機を説明するが、手法そのものの中身ではない。

B.6 実務での構え方

付録で見た性質は、実務では次の形で顔を出す。

- **特徴の数と標本数の比。** 標本数 n が特徴の数 d を下回るなら、ここまでの話がそのまま当てはまる範囲に入っている
- **距離を使う手法** (k 近傍法、k-means、コサイン類似度) は実効次元が高い場合、次元が上がるほど当てにならなくなる (発展で見たとおり、実効次元が低ければ距離は働きうる)。当てはまるなら、先に次元を落とすか、距離を使わない手法にする
- **木ベースの手法**は、無関係な次元が増えても成績の落ち方が緩い。分割のたびに情報利得の高い列だけが選ばれて使われる、埋め込み型の特徴選択が働くためである
- **実効次元。** 列どうしが相関していれば、見かけの列数ほど自由には散らばっていない。PCA の寄与率や多様体学習で見当をつける

行動としてまとめると、第 9 章の定石と同じ 1 点に帰着する。**特徴を足したら必ず測り直す。** 増やして悪化するのとは普通のことである。

本書の近代文学のデータは、標本数が $n = 74$ (作品)、特徴の数が $d = 52$ (列) である。 n が d より少し大きい程度で、決して余裕のある状況ではない。第 9 章で乱数列を足したときに性能が急落したのは、もともとぎりぎりの状況だったからでもある。

B.7 章末の課題

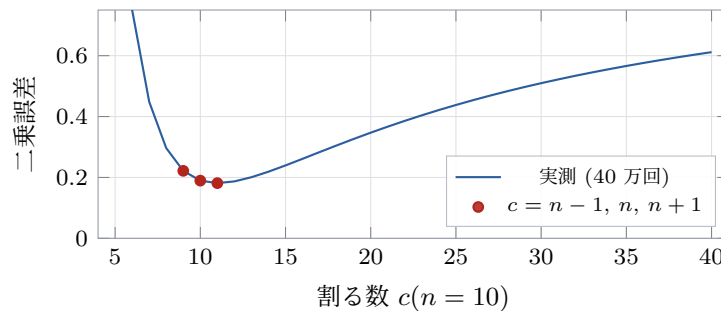
1. 式 (B.1) を使って、単位球の体積が最大になる次元を求めよ (d を実数として扱ってよい)。
2. d 次元立方体 $[-1, 1]^d$ の頂点は 2^d 個ある。中心から頂点までの距離と、中心から面までの距離の比を求め、 $d \rightarrow \infty$ での挙動を調べよ。
3. 一様分布ではなく標準正規分布から点を取ったとき、平均距離はどうなるか。 $\sqrt{2d}$ になることを確かめよ。
4. 図 B.4 を $d = 1000$ まで延長し、分布の幅が $1/\sqrt{d}$ で縮むことを数値的に確かめよ。
5. 本書の 52 次元データについて、点对距離の変動係数を計算せよ。一様分布の理論値 (図 B.3) と比べてどうか。違いがあるとすれば、それは何を意味するか。

不偏性をめぐる補足

第3章の発展「 $n-1$ で割るべきか」で触れた2つの話を、式の導出まで含めてここに置く。本文の結論（本書のデータではどちらで割っても結論は変わらない）は付録を読まなくても変わらない。

C.1 二乗誤差を最小にする割る数は $n+1$ である

ここで当然の疑問が出る。 $n+1$ が n より良いなら、 $n+10$ で割ればもっと良いはずである。だがそうはならない。割る数 c を動かして二乗誤差を測ると、 $c=n+1$ を底とする谷の形になる。



c を大きくするほど推定値のばらつきは減るが、そのぶん真の値より小さく答えるようになる。 $c=n+1$ より先では、偏りの増加がばらつきの減少を上回る。極端に大きい c は「いつもほぼ0と答える」推定量であり、二乗誤差は σ^4 に向かって戻っていく。谷の底が1か所しかないことは計算でも確かめられる。 $S = \sum (x_i - \bar{x})^2$ とおくと $E[S] = (n-1)\sigma^2$ 、 $\text{Var}[S] = 2(n-1)\sigma^4$ なので

$$\text{MSE}(c) = \frac{\text{Var}[S]}{c^2} + \left(\frac{E[S]}{c} - \sigma^2 \right)^2 = \sigma^4 \left(\frac{n^2 - 1}{c^2} - \frac{2(n-1)}{c} + 1 \right) \quad (\text{C.1})$$

となり、 c で微分してゼロと置くと $c=n+1$ が唯一の解として出る。このとき $\text{MSE} = 2\sigma^4/(n+1)$ で、 $n-1$ で割る場合の $2\sigma^4/(n-1)$ より小さい。ただしこれは正規分布を仮定したうえで、この形の推定量に限った話である。分布が違えば底の位置も変わるので、 $n+1$ を新しい既定にする話ではない。

C.2 標準偏差を不偏にする補正

第3章で見たとおり、 s^2 が不偏でも $s = \sqrt{s^2}$ は不偏にならない。どれだけずれるかを実測する。 $n-1$ で割って40万回繰り返し、 s^2 と s の平均を並べる。

n	$E[s^2]$	$E[\sqrt{S/(n-1)}]$	$E[\sqrt{S/(n-1.5)}]$
5	0.9995	0.9396	1.0045
10	1.0004	0.9728	1.0010
30	1.0005	0.9916	1.0003
100	1.0000	0.9975	1.0000

真の分散も真の標準偏差も 1 である。 s^2 の平均は 1 に一致しているのに、 s の平均は $n = 5$ で 1 より 6%、 $n = 10$ でも 2.7% 小さい。正規分布なら $E[s] = c_4(n)\sigma$ と書けることが知られていて、 $c_4(n) = \sqrt{2/(n-1)} \Gamma(n/2)/\Gamma((n-1)/2)$ である。 s を $c_4(n)$ で割れば標準偏差も不偏にできる。

ガンマ関数を持ち出さない近似もある。 $n-1$ ではなく $n-1.5$ で割ってから平方根を取るという手で、表の右端がそれである。この推定量の平均は $n = 10$ で 1.0010、 $n = 30$ で 1.0003 で、真の標準偏差 1 からのずれは $n \geq 5$ なら 0.5% 以内に収まっている。種を明かすと、 c_4 で割った推定量の実効的な割る数は $(n-1)c_4(n)^2$ で、これが $n = 10$ で 8.515、 $n = 30$ で 28.504 と、 $n-1.5$ にほぼ一致する。

ただし、こうして偏りを消せば二乗誤差は大きくなる。前と同じ話である。

ずれの大きさは標本の数で決まる。文数が数百から数千ある本書の文長 σ では、 c_4 が 1 からほとんど離れないので、この補正は要らない。補正が意味を持つのは、数十点以下の標本で標準偏差そのものを報告するときである。

参考文献

- [1] F. Mosteller and D. L. Wallace: *Inference and Disputed Authorship: The Federalist*, Addison-Wesley (1964).
- [2] 金明哲, 樺島忠夫, 村上征勝: 読点と書き手の個性, 計量国語学, 18(8), pp. 382–391 (1993).
- [3] Z. S. Harris: From Phoneme to Morpheme, *Language*, 31(2), pp. 190–222 (1955).
- [4] Z. Jin and K. Tanaka-Ishii: Unsupervised Segmentation of Chinese Text by Use of Branching Entropy, *Proc. COLING/ACL 2006 Main Conference Poster Sessions*, pp. 428–435 (2006).
- [5] 金明哲: テキストデータの統計科学入門, 岩波書店 (2009).
- [6] G. K. Zipf: *Selected Studies of the Principle of Relative Frequency in Language*, Harvard University Press (1932).
- [7] D. François, V. Wertz and M. Verleysen: The concentration of fractional distances, *IEEE Transactions on Knowledge and Data Engineering*, 19(7), 873–886 (2007).
- [8] M. Radovanović, A. Nanopoulos and M. Ivanović: Hubs in space: Popular nearest neighbors in high-dimensional data, *Journal of Machine Learning Research*, 11, 2487–2531 (2010).
- [9] G. A. Miller: Some effects of intermittent silence, *The American Journal of Psychology*, 70(2), 311–314 (1957).
- [10] W. Li: Random texts exhibit Zipf’s-law-like word frequency distribution, *IEEE Transactions on Information Theory*, 38(6), 1842–1845 (1992).
- [11] K. W. Church and W. A. Gale: Inverse Document Frequency (IDF): A Measure of Deviations from Poisson, *Proc. Third Workshop on Very Large Corpora*, pp. 121–130 (1995).
- [12] E. G. Altmann, J. B. Pierrehumbert, and A. E. Motter: Beyond Word Frequency: Bursts, Lulls, and Scaling in the Temporal Distributions of Words, *PLoS ONE*, 4(11), e7678 (2009).
- [13] K.-I. Goh and A.-L. Barabási: Burstiness and Memory in Complex Systems, *EPL*, 81, 48002 (2008).
- [14] J. Aitchison: *The Statistical Analysis of Compositional Data*, Chapman and Hall (1986).
- [15] J. A. Martín-Fernández, C. Barceló-Vidal, and V. Pawłowsky-Glahn: Dealing with Zeros and Missing Values in Compositional Data Sets Using Nonparametric Imputation, *Mathematical Geology*, 35(3), 253–278 (2003).
- [16] J. A. Martín-Fernández, K. Hron, M. Templ, P. Filzmoser, and J. Palarea-Albaladejo: Bayesian-Multiplicative Treatment of Count Zeros in Compositional Data Sets, *Statistical Modelling*, 15(2), 134–158 (2015).
- [17] G. B. Gloor, J. M. Macklaim, V. Pawłowsky-Glahn, and J. J. Egozcue: Microbiome Datasets Are Compositional: And This Is Not Optional, *Frontiers in Microbiology*, 8, 2224 (2017).

-
- [18] G. Salton, A. Wong, and C. S. Yang: A Vector Space Model for Automatic Indexing, *Communications of the ACM*, 18(11), 613–620 (1975).
 - [19] L. Lee: Measures of Distributional Similarity, *Proc. 37th Annual Meeting of the ACL*, pp. 25–32 (1999).
 - [20] H. Steck, C. Ekanadham, and N. Kallus: Is Cosine-Similarity of Embeddings Really About Similarity?, *Companion Proc. ACM Web Conference 2024 (WWW '24)*, pp. 887–890 (2024).
 - [21] K. Beyer, J. Goldstein, R. Ramakrishnan, and U. Shaft: When Is “Nearest Neighbor” Meaningful?, *Proc. ICDT 1999*, LNCS 1540, pp. 217–235 (1999).
 - [22] C. C. Aggarwal, A. Hinneburg, and D. A. Keim: On the Surprising Behavior of Distance Metrics in High Dimensional Space, *Proc. ICDT 2001*, pp. 420–434 (2001).
 - [23] R. A. Olshen and B. Rajaratnam: Successive Normalization of Rectangular Arrays, *Annals of Statistics*, 38(3), 1638–1664 (2010).
 - [24] J. Burrows: ‘Delta’: A Measure of Stylistic Difference and a Guide to Likely Authorship, *Literary and Linguistic Computing*, 17(3), 267–287 (2002).
 - [25] S. Evert, T. Proisl, F. Jannidis, I. Reger, S. Pielström, C. Schöch, and T. Vitt: Understanding and Explaining Delta Measures for Authorship Attribution, *Digital Scholarship in the Humanities*, 32(suppl. 2), ii4–ii16 (2017).
 - [26] C. Eckart and G. Young: The Approximation of One Matrix by Another of Lower Rank, *Psychometrika*, 1(3), 211–218 (1936).
 - [27] G. N. Lance and W. T. Williams: A General Theory of Classificatory Sorting Strategies: 1. Hierarchical Systems, *The Computer Journal*, 9(4), 373–380 (1967).
 - [28] L. van der Maaten and G. Hinton: Visualizing Data using t-SNE, *Journal of Machine Learning Research*, 9, 2579–2605 (2008).
 - [29] L. McInnes, J. Healy, and J. Melville: UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction, arXiv:1802.03426 (2018).
 - [30] J. B. Tenenbaum, V. de Silva, and J. C. Langford: A Global Geometric Framework for Nonlinear Dimensionality Reduction, *Science*, 290, 2319–2323 (2000).
 - [31] S. T. Roweis and L. K. Saul: Nonlinear Dimensionality Reduction by Locally Linear Embedding, *Science*, 290, 2323–2326 (2000).
 - [32] A. Coenen and A. Pearce: Understanding UMAP, Google PAIR (2019). <https://pair-code.github.io/understanding-umap/>
 - [33] D. Kobak and G. C. Linderman: Initialization is Critical for Preserving Global Data Structure in Both t-SNE and UMAP, *Nature Biotechnology*, 39(2), pp. 156–157 (2021).
 - [34] M. Wattenberg, F. Viégas, and I. Johnson: How to Use t-SNE Effectively, *Distill* (2016). <https://distill.pub/2016/misread-tsne/>
 - [35] F. Murtagh and P. Legendre: Ward’s Hierarchical Agglomerative Clustering Method: Which Algorithms Implement Ward’s Criterion?, *Journal of Classification*, 31(3), pp. 274–295 (2014).
 - [36] M. Belkin, D. Hsu, S. Ma, and S. Mandal: Reconciling Modern Machine-learning Practice

- and the Classical Bias–Variance Trade-off, *PNAS*, 116(32), pp. 15849–15854 (2019).
- [37] P. Nakkiran, G. Kaplun, Y. Bansal, T. Yang, B. Barak and I. Sutskever: Deep Double Descent: Where Bigger Models and More Data Hurt, *Journal of Statistical Mechanics: Theory and Experiment*, 2021(12), 124003 (2021). 初出は *ICLR 2020*.
- [38] J. Ho, T. Tumkaya, S. Aryal, H. Choi and A. Claridge-Chang: Moving Beyond P Values: Data Analysis with Estimation Graphics, *Nature Methods*, 16, pp. 565–566 (2019).
- [39] J. Demšar: Statistical Comparisons of Classifiers over Multiple Data Sets, *Journal of Machine Learning Research*, 7, pp. 1–30 (2006).
- [40] P. Nakkiran, P. Venkat, S. Kakade and T. Ma: Optimal Regularization Can Mitigate Double Descent, *Proc. ICLR 2021* (2021).
- [41] L. Grinsztajn, E. Oyallon, and G. Varoquaux: Why Do Tree-based Models Still Outperform Deep Learning on Typical Tabular Data?, *Proc. NeurIPS 2022 Datasets and Benchmarks Track* (2022).
- [42] D. Kobak, G. Linderman, S. Steinerberger, Y. Kluger, and P. Berens: Heavy-Tailed Kernels Reveal a Finer Cluster Structure in t-SNE Visualisations, *Proc. ECML PKDD 2019* (arXiv:1902.05804).
- [43] S. Greenland, S. J. Senn, K. J. Rothman, J. B. Carlin, C. Poole, S. N. Goodman, and D. G. Altman: Statistical Tests, P Values, Confidence Intervals, and Power: A Guide to Misinterpretations, *European Journal of Epidemiology*, 31(4), 337–350 (2016).
- [44] 岡部正隆, 伊藤啓: 色覚の多様性と色覚バリアフリーなプレゼンテーション, *細胞工学*, 21(7–9) (2002).
- [45] J. A. Nelder and R. W. M. Wedderburn: Generalized Linear Models, *Journal of the Royal Statistical Society, Series A*, 135(3), pp. 370–384 (1972).
- [46] L. Breiman: Random Forests, *Machine Learning*, 45, 5–32 (2001).
- [47] T. Chen and C. Guestrin: XGBoost: A Scalable Tree Boosting System, *Proc. KDD 2016*, pp. 785–794 (2016).
- [48] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu: LightGBM: A Highly Efficient Gradient Boosting Decision Tree, *Proc. NIPS 2017* (2017).
- [49] L. S. Shapley: A Value for n -Person Games, in *Contributions to the Theory of Games II*, Princeton University Press, pp. 307–317 (1953).
- [50] S. M. Lundberg and S.-I. Lee: A Unified Approach to Interpreting Model Predictions, *Proc. NIPS 2017* (2017).
- [51] S. M. Lundberg, G. Erion, H. Chen, et al.: From Local Explanations to Global Understanding with Explainable AI for Trees, *Nature Machine Intelligence*, 2, 56–67 (2020).
- [52] C. J. van Rijsbergen: *Information Retrieval*, 2nd ed., Butterworths (1979).
- [53] Y. Sasaki: The Truth of the F-measure, teaching materials, University of Manchester (2007).
- [54] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman: Leakage in Data Mining: Formulation, Detection, and Avoidance, *ACM Transactions on Knowledge Discovery from Data*, 6(4), 15 (2012).

-
- [55] Kaggle: TGS Salt Identification Challenge (2018). <https://www.kaggle.com/c/tgs-salt-identification-challenge>
 - [56] S. Kapoor and A. Narayanan: Leakage and the Reproducibility Crisis in Machine-learning-based Science, *Patterns*, 4(9), 100804 (2023).
 - [57] M. Roberts, D. Driggs, M. Thorpe, et al.: Common Pitfalls and Recommendations for Using Machine Learning to Detect and Prognosticate for COVID-19 Using Chest Radiographs and CT Scans, *Nature Machine Intelligence*, 3, 199–217 (2021).
 - [58] D. Trehan and P. Chopra: Why LLMs Aren’t Scientists Yet: Lessons from Four Autonomous Research Attempts, arXiv:2601.03315 (2026).
 - [59] Kaggle Team: Profiling Top Kagglers: Bestfitting, Currently #1 in the World, Kaggle Blog (2018).
 - [60] V. C. Bradley, S. Kuriwaki, M. Isakov, D. Sejdinovic, X.-L. Meng and S. Flaxman: Unrepresentative Big Surveys Significantly Overestimated US Vaccine Uptake, *Nature*, 600, pp. 695–700 (2021).
 - [61] R. van den Goorbergh, M. van Smeden, D. Timmerman and B. Van Calster: The Harm of Class Imbalance Corrections for Risk Prediction Models, *Journal of the American Medical Informatics Association*, 29(9), pp. 1525–1534 (2022).
 - [62] Z. Obermeyer, B. Powers, C. Vogeli and S. Mullainathan: Dissecting Racial Bias in an Algorithm Used to Manage the Health of Populations, *Science*, 366(6464), pp. 447–453 (2019).
 - [63] F. J. Anscombe: Graphs in Statistical Analysis, *The American Statistician*, 27(1), pp. 17–21 (1973).
 - [64] J. Matejka and G. Fitzmaurice: Same Stats, Different Graphs: Generating Datasets with Varied Appearance and Identical Statistics through Simulated Annealing, *Proceedings of CHI 2017*, pp. 1290–1294 (2017).
 - [65] T. O. Kvålseth: Cautionary Note about R^2 , *The American Statistician*, 39(4), pp. 279–285 (1985).
 - [66] A.-N. Spiess and N. Neumeyer: An Evaluation of R^2 as an Inadequate Measure for Nonlinear Models in Pharmacological and Biochemical Research: A Monte Carlo Approach, *BMC Pharmacology*, 10, 6 (2010).
 - [67] D. Chicco, M. J. Warrens and G. Jurman: The Coefficient of Determination R-squared is More Informative than SMAPE, MAE, MAPE, MSE and RMSE in Regression Analysis Evaluation, *PeerJ Computer Science*, 7, e623 (2021).
 - [68] J. F. Pimentel, L. Murta, V. Braganholo and J. Freire: A Large-Scale Study About Quality and Reproducibility of Jupyter Notebooks, *Proceedings of MSR 2019*, pp. 507–517 (2019).
 - [69] J. F. Pimentel, L. Murta, V. Braganholo and J. Freire: Understanding and Improving the Quality and Reproducibility of Jupyter Notebooks, *Empirical Software Engineering*, 26, 65 (2021).
 - [70] M. Ziemann, Y. Eren and A. El-Osta: Gene Name Errors are Widespread in the Scientific

- Literature, *Genome Biology*, 17, 177 (2016).
- [71] M. Abeysooriya, M. Soria, M. S. Kasu and M. Ziemann: Gene Name Errors: Lessons Not Learned, *PLOS Computational Biology*, 17(7), e1008984 (2021).
- [72] J. Clark and D. Amodei: Faulty Reward Functions in the Wild, OpenAI Blog (2016). <https://openai.com/index/faulty-reward-functions/>
- [73] Y. Bengio and Y. Grandvalet: No Unbiased Estimator of the Variance of K -Fold Cross-Validation, *Journal of Machine Learning Research*, 5, pp. 1089–1105 (2004).
- [74] 青空文庫: 青空文庫収録ファイルの取り扱い規準 (2022 年改訂). <https://www.aozora.gr.jp/guide/kijyun.html>

アンチパターンと定石の一覧

本書に散らばる ×(アンチパターン) と○(定石) を、登場順に集めた。分析の途中で「この操作、何かまずくなかったか」と思ったら、ここから引いてほしい。

× 2.1 両対数の直線を言語の証拠として使う	11
× 2.2 頻出する内容語で著者を判別する	12
○ 2.1 著者を当てるなら文書全体に様に散らばる単語を選ぶ	13
× 2.3 ワードクラウドや共起ネットワークを分析の結論として出す	14
○ 2.2 刻み幅を自分で決めたら刻み幅のいらない見方でも確かめる	17
× 3.1 ルビや注記を取り除かずに文字を数える	21
○ 3.1 捨てるものはその情報が誰に由来するかで決める	21
× 3.2 分析に使う CSV を表計算ソフトで開く	21
× 3.3 正規化と標準化を混同する	23
○ 3.2 散布図は全体図と拡大図の 2 枚を描く	28
× 3.4 分散は必ず $n - 1$ で割ると覚える	30
× 3.5 分布を見ずに平均だけを取る	31
× 3.6 連続した値を中央値で 2 つに割ってから分析する	32
○ 3.3 著者を当てる特徴に求める 3 条件	34
× 3.7 比率どうしの相関をそのまま解釈する	36
○ 3.4 ゼロを埋める前にそのゼロの由来を確かめる	40
× 3.8 対数比を保ちたいのに、比率へ一律に 1 を足す	41
○ 4.1 長さの違いを消したいときにコサイン距離を選ぶ	48
× 4.1 定番だからという理由で距離を選ぶ	49
× 4.2 単位の違う列を混ぜたまま距離を測る	50
× 4.3 順序を考えずに正規化と標準化を混ぜる	51
× 4.4 標準化すれば組成データの制約が消えると思う	52
○ 4.2 標準化の要否を 1 行で判定する	56
× 4.5 距離を正解ラベルで選ぶ	59
× 4.6 散布図の見た目の細さから相関の強さを判断する	63
○ 4.3 要約した数値を報告する前に必ず散布図を描く	64
× 4.7 相関を連鎖させて推論する	68
× 4.8 集計した単位の相関をその中の個体の話として読む	69
× 4.9 列の数だけを見て次元の呪いを心配する	73
○ 4.4 距離を使う手法にける前に次元を落とす	74
× 5.1 シルエット係数が最大の k を採用する	83

○ 5.1 前処理は結果ではなく測り方の理屈で選ぶ	87
× 5.2 合計が 1 に固定された比率の表を列ごとに標準化する	87
○ 5.2 Ward 法はユークリッド距離と組み合わせる	87
○ 5.3 k-means は初期値を変えて複数回試す	90
× 5.3 k-means を 1 回だけ実行する	90
× 5.4 エルボー法だけでクラスタ数を決める	91
× 6.1 寄与率を見ずに 2 次元の図で「近い」と判断する	100
○ 6.1 分かれた理由を説明したいときはパイプロットを使う	102
× 6.2 t-SNE や UMAP が見せた図を、そのまま信じる	111
○ 6.2 t-SNE や UMAP の図には、必ずパラメータを書き添える	114
× 6.3 t-SNE や UMAP の軸に意味を読む	116
× 6.4 片方の手法だけ調整して次元削減を比べる	119
× 6.5 t-SNE や UMAP の図で距離を語る	120
○ 6.3 次元削減は複数の手法を並べて見る	121
× 7.1 基準線を置かずに精度を報告する	124
× 7.2 相関の強い列を並べたまま係数の大小を比べる	128
× 7.3 ブースティングの木をランダムフォレストと同じつもりで増やす	135
○ 7.1 ブースティングのパラメータは 3 手に分けて触る	137
○ 7.2 分類手法はデータの規模と目的で選ぶ	138
× 7.4 訓練データで性能を報告する	140
○ 7.3 設定値を探索したかどうかをはっきり書く	142
○ 7.4 成績が並んだときは境界の形か誤りの型で選ぶ	143
× 7.5 不純度重要度だけを見る	145
× 7.6 特徴重要度を因果や本質と読む	145
× 7.7 L1 が残した列を「重要な列」と読む	146
○ 8.1 分割数は 5 か 10 から始めてデータが少ないときだけ増やす	151
○ 8.2 成績を報告するときはばらつきの幅を添える	156
× 8.1 エラーバーが何かを書かずに図を出す	157
× 8.2 並べた区間の重なりを見て手法に順位をつける	160
○ 8.3 比べたいものが差なら差を軸にした図を描く	162
○ 8.4 報告する指標と同じ指標で基準線を測る	165
× 8.3 別々のデータで測った決定係数を並べて優劣を論じる	166
○ 8.5 回帰は誤差と基準線を組にして報告する	167
× 8.4 全員が満点近くになる課題で手法の優劣を論じる	168
× 8.5 不均衡データで AUC だけを見る	169
○ 8.6 しきい値は誤りのコストで決める	170
○ 8.7 確率の値を使うなら較正まで確かめる	172
× 8.6 不均衡を「補正」して確率を壊す	172
× 8.7 「パラメータを大きくすると複雑になる」と覚える	174

○ 8.8 モデルをいじる前に学習曲線を描く	174
○ 9.1 前処理は Pipeline に入れて分割の内側で fit する	180
× 9.1 特徴選択を前処理として分割の前で済ませる	182
× 9.2 落差が小さいことを「漏れがない」の証明として使う	183
× 9.3 行数をデータ量だと思う	184
× 9.4 何を当てたいかを考えずに n-gram を入れる	186
× 9.5 試行回数を報告しない	190
× 9.6 未来で訓練して過去を予測する	192
× 9.7 訓練データの範囲の外を平均を返すモデルに予測させる	193
○ 9.2 成績が悪いときは偏りとばらつきのどちらが原因かを先に見る	197
× 9.8 二重降下を「モデルは大きいほどよい」の根拠にする	198
× 9.9 距離を使う手法にとりあえず特徴を全部入れる	199
○ 9.3 列を足して成績が落ちたらまず列を減らす	199
× 9.10 AI が当てられたことを「この特徴で判別できる」証拠にする	200
× 9.11 AI の相づちを裏付けに数える	200
○ 9.4 AI が出した数字は 1 つ選んで自分で再計算する	201
× 10.1 分割の乱数を 1 つ選んで手法を比べる	207
× 10.2 「収束するまで回した」とだけ書く	212
× 10.3 測れば確かめられる原因の説明を測らずに書く	219
○ 10.1 前処理を 1 つ足したらモデルごとに測り直す	219
○ 10.2 深層学習を使う前に素朴なモデルの成績を出す	221
× 10.4 前処理を省いてモデルに任せる	221
× 11.1 最も単純なルールを試さずに高い精度を誇る	225
○ 11.1 独立した単位がいくつあるかを先に数える	226
× 11.2 答えの候補が出そろっている前提に気づかない	228

索引

- Adam.....212
- Adjusted Rand Index.....40, 80
- Aitchison 37
- Aitchison 距離 **37**
- Anscombe の四重奏 **63**
- AUC.....167

- bagging **132**
- benchmark contamination **200**
- BERT.....**215**
- bias **195**
- biplot **101**
- boosting.....**132**
- bootstrap.....**131**
- Brier score **170**
- Brier スコア **170**
- burstiness.....**16**

- calibration.....**170**
- CD ダイアグラム.....**162**
- ceiling effect.....**168**
- chaining effect **79**
- Chebyshev distance 46
- closed-set assumption.....**227**
- clr 変換.....**37**
- clustering.....**76**
- coefficient of determination **55**
- coefficient of variation **16**
- compositional data.....**35**
- concentration of distances.....**71**
- confounding.....**225**
- confusion matrix **154**
- correlation coefficient.....61
- cosine distance.....47
- covariance matrix.....**96**

- coverage probability.....155
- Cover の定理 238
- critical difference **163**
- critical difference diagram **162**
- cross entropy **125**
- cross-validation **151**
- curse of dimensionality.....**71**

- data leakage.....148
- decision tree.....**130**
- Delta 59
- dendrogram.....**76**
- dimensionality reduction.....**93**
- double descent.....**197**
- drift.....**191**

- effect size.....**57**
- eigenvalue problem **97**
- embedding **203**
- entropy.....**131**
- estimation plot **162**
- Euclidean distance 46
- eureka instinct **200**
- explained variance ratio.....**98**
- exponential distribution.....**14**

- F-statistic **86**
- feature **6, 222**
- feature engineering.....**43**
- fold **206**
- forest plot **159**
- function word.....**11**
- F 値.....**86**

- Gardner–Altman プロット 162
- generalized linear model **128**

- generative AI.....200
- geodesic distance.....106
- gradient boosting.....134
- Gram matrix.....97
- Group Lasso.....146
- GroupKFold.....182
- hypercube.....106
- hyperparameter.....173
- ilr 变换.....38
- information gain.....130
- intrinsic dimension.....105
- Isomap.....106, 117
- Johnson–Lindenstrauss.....238
- k-means.....6, 48, 88, 239
- k-means++.....90
- k-nearest neighbors.....124
- Kaggle.....135
- kernel.....129
- Lance–Williams.....79
- lasso.....53
- Lasso 回归.....53
- lazy learning.....124
- learning curve.....174
- learning rate.....134
- LightGBM.....135
- LLE.....117
- log-normal distribution.....29
- logistic regression.....125
- macro-average.....153
- Manhattan distance.....46
- manifold hypothesis.....96
- margin.....129
- marimo.....i, 3, 232
- maximum likelihood.....30
- memorylessness.....15
- micro-average.....153
- Minkowski distance.....45
- multilayer perceptron.....210
- mutual information.....131
- n-gram.....18, 185
- Nemenyi 検定.....163
- normalization.....22
- overfitting.....139
- partial correlation.....69
- permutation importance.....144
- Poisson distribution.....12
- Poisson process.....15
- Polars.....30
- power law.....9
- precision.....153
- principal component analysis.....93
- PR 曲线.....169
- random forest.....131
- recall.....153
- regularization term.....53
- residual IDF.....13
- ridge regression.....53
- Ridge 回归.....53
- ROC 曲线.....7, 168, 228
- scikit-learn.....4, 53, 90, 125, 180, 212, 230
- SHAP.....147
- Shapley 值.....147
- sigmoid function.....125
- silhouette coefficient.....81
- standardization.....23, 50
- StratifiedKFold.....151
- subcompositional incoherence.....36
- supervised learning.....123
- support vector machine.....54, 129
- Swiss roll.....105

- t-SNE 104, 173, 204
 ternary plot 25
 TF-IDF 14

 UMAP 104, 199, 204
 unbiasedness 29
 uv 231

 variance 195
 VIF 127
 viridis 100

 Ward 法 41, 48, 78

 XGBoost 135

 z-score 50
 Zipf の法則 6, 9

 一般化線形モデル 128
 入れ子交差検証 137, 188

 埋め込み 203

 エポック 211
 ℓ_1 正規化 22
 ℓ_2 正規化 22
 ℓ_2 正規化 48
 エルボー法 90

 オートエンコーダ 121

 カーネル 129
 階層クラスタリング 6, 76
 過学習 7, 53, 139, 173, 176, 212, 238
 学習曲線 174, 197
 学習率 134, 173, 210
 過剰適合 139
 数え上げゼロ 40
 偏り 195
 完全連結 78
 ガンマ分布 15

 幾何分布 17
 基準線 4, 52, 123, 175, 189, 221, 223
 機能語 11, 145, 186
 教師あり学習 123, 215
 凝集型 76
 寄与率 25, 55, 98, 228, 239
 距離の公理 60, 103
 距離の集中 7, 71, 108, 235

 鎖効果 79
 クラスタリング 2, 42, 54, 76, 230
 グラム行列 97
 グループの漏れ 177
 群平均法 77

 形態素解析 8
 計量文献学 59
 計量文体論 1
 k 近傍法 .. 7, 52, 124, 169, 178, 224, 226, 239
 決定株 133, 224
 決定木 7, 41, 52, 130, 173, 193, 210, 224
 決定係数 55, 191

 効果量 57
 交差エントロピー 125
 交差エントロピー損失 210
 交差検証 ... 7, 31, 52, 137, 140, 151, 203, 230
 較正 7, 170
 構造ゼロ 40
 勾配降下法 91, 210
 勾配ブースティング 7, 53, 134, 193, 209
 交絡 194, 225
 コサイン距離 6, 23, 47, 83, 206
 コサイン類似度 48, 239
 古典的 MDS 97
 固有値問題 97
 混同行列 7, 75, 92, 143, 154, 201

 再現率 7, 153
 最尤推定 30, 125

- サポートベクトルマシン.....54, **129**, 238
三角図.....**25**
三角不等式.....45, 103
残差 IDF.....**13**

色覚多様性.....101
シグモイド関数.....**125**
時系列の漏れ.....165, 177
次元削減.....74, **93**, 199
次元の呪い.....7, **71**, 124, 198, 233
指数分布.....**14**
主成分分析.....6, 25, 55, 93, 209
情報エントロピー.....**131**
乗法的置換.....41
情報利得.....**130**, 239
シルエット係数.....**81**, **204**
信用区間.....156
信頼度曲線.....**170**

スイスロール.....**105**
推定プロット.....**162**
スクリープロット.....99
スペクトル埋め込み.....117

正規化.....**22**, 50, 90, 96, 185, 212
生成 AI.....**200**
正則化項.....**53**
生存者バイアス.....**5**
生態学的誤謬.....**69**
 z 得点.....50
前処理の漏れ.....177

相関係数.....35, 61, 95
早期打ち切り.....135
相互情報量.....131
層別化.....7, 140, 152
層別交差検証.....31, 52, 140, 151
測地距離.....**106**
組成データ.....ii, 6, **35**, 52, 87, **115**, 128
ターゲットの漏れ.....177

対数正規分布.....**29**
対数損失.....125
怠惰学習.....**124**
多次元尺度構成法.....6, 93
多重共線性.....**127**
多重テストの漏れ.....177
多層パーセプトロン.....**210**
多様体仮説.....74, **96**, 238
単体.....24
単連結.....78

チェビシェフ距離.....46
置換重要度.....**144**
超立方体.....**106**

追従性.....200

データ漏洩.....7, **148**, 176
適合率.....7, **153**
天井効果.....**168**
デンドログラム.....**76**

特徴選択の漏れ.....177
特徴量.....i, **6**, 20, 48, 86, 180, 208, 222
特徴量エンジニアリング.....**43**
ドリフト.....**191**

内在次元.....73, **105**
内容語.....i, **11**, 44, 185

二重降下.....**197**

バースト性.....**16**
ハイパーパラメータ.....**173**, 187
バイプロット.....**101**
バギング.....**132**
バッチ.....**211**
ハブ.....73
ぼらつき.....**195**

1 つ抜き交差検証.....**151**
被覆確率.....155

- 標準化.....6, **23**, **50**, 80, 93, 126, 177, 206
- ブースティング.....7, 53, **132**, **197**, 209
- ブートストラップ.....**131**
- フォールド.....140, **162**, 191, **206**
- フォレストプロット.....**159**
- 部分組成の非整合性.....**36**
- 不偏性.....**29**
- 分割型.....76
- 分岐エントロピー.....18
- 分散共分散行列.....**96**
- 閉集合の仮定.....**227**
- べき乗則.....**9**
- 偏差ベクトル.....**61**
- 偏相関係数.....**69**
- ベンチマーク汚染.....**200**
- 変動係数.....**16**
- ポアソン過程.....**15**
- ポアソン分布.....6, **12**, 128
- ホールドアウト.....**150**
- ボンフェローニ補正.....**160**
- マイクロ平均.....**153**
- マクロ平均.....**153**
- マージン.....**129**
- マンハッタン距離.....6, 46, **85**
- ミンコフスキー距離.....45, 85
- 無記憶性.....**15**
- ユークリッド距離.6, 23, 46, 79, 103, 130, 191
- ユーレカ本能.....**200**
- ラグランジュ未定乗数法.....97
- ランダムフォレスト.7, 42, 53, **131**, 152, 176,
210, 223, 224
- 臨界差.....**163**
- 臨界差図.....162
- レインボー.....100
- ロジスティック回帰...53, **125**, 138, 169, 206

アンチパターンで学ぶデータ分析

作家の判別を題材に

著者	久保山 哲二 (くぼやま てつじ) 学習院大学 計算機センター
版	バージョン 1.2.1
初版発行	2026 年 8 月 24 日
本版発行	2026 年 9 月 13 日
組版	LuaLaTeX(ltjsbook + luatexja) 本文フォント: 原ノ味明朝・原ノ味ゴシック 図版: TikZ / PGFPlots
データ	近代文学 74 作品 (青空文庫) ライトノベル 60 作品 (特徴量のみ)
付属教材	marimo ノートブック 4 本
使い方	付録 A を参照
ライセンス	本文・図版: CC BY 4.0 / コード: MIT

近代文学の本文は青空文庫 (<https://www.aozora.gr.jp/>) に拠る。ライトノベルについては、著作権に配慮し、本文からは復元できない要約統計量 (特徴量) のみを扱った。